# Release notes

`disabled` property
Buttons
Event handlers
Textbox
Checkbox
Dropdown
Server-Driven UI
    Flet approach
Roadmap
    Flet widget for Flutter
    Flet Studio for iOS and Android
    White-labeled Flet mobile app
    Standalone mobile package for Flet app
    Embedding Flet into native apps
Customizing package icon
Packaging assets
Customizing macOS bundle
Customizing Windows executable metadata
Using CI for multi-platform packaging
Flet CLI
    Examples
    Examples
Ads
    Platform Support
    Usage
        Android
        iOS
        Test Values
    Consent (UMP)
        Testing the consent form
        Interpreting consent status
        Under age of consent
    Example
    Examples
    Examples
    Examples
    Examples
    Examples
    Examples
    Examples
    Examples
    Examples
    Examples

Camera
Platform Support
Usage
Requirements
Android
iOS
Cross-platform
Description
Canvas
Examples
Examples
Examples
Examples
Charts
Platform Support
Usage
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Usage
Example
Example
Example
Color pickers
Usage
Example
Example
Example
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples

Examples
Examples
Examples
Examples
Examples
DataTable2
Platform Support
Usage
Examples
Description
Examples
Examples
Examples
Examples
Remove `Dismissible` `on_dismiss` inside component
Examples
Examples
Examples
Dropdown
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Lottie
Platform Support
Usage
Description
Platform Support
Usage
Examples
Reference
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
NavigationDrawer
Examples
Examples

- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- `RawImage` vs `Image`
- Examples
  - Photo viewer
  - Plasma animation
  - Pillow paint
  - Mandelbrot explorer
  - Game of Life
- Examples
- Examples
- Examples

Rive

- Platform Support
- Usage
- Example
- Description
- Example
- Examples
- Examples
- Example
- Examples
- Examples

SegmentedButton

- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples

SpinKit

- Usage
  - Boot screen
- Examples
- Available controls
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Example
- Basic Example

Examples
Platform Support
Usage
    Android
    Linux
Examples
Description
Platform Support
Usage
Troubleshooting
    NET::ERR_CLEARTEXT_NOT_PERMITTED Error
Examples
Screen readers
    Text
    Buttons
    `TextField` and `Dropdown`
    Custom semantics
    Debugging semantics
Material and Cupertino controls
Custom adaptive controls
Implicit animations
    Opacity animation
    Rotation animation
    Scale animation
    Offset animation
    Position animation
    Animate
    Animated content switcher
    Animation end callback
Example: Displaying a Local Image
Accessing Asset Files in Production
Getting started with async
Control event handlers
    Async lambdas
Sleeping
Threading
Login process overview
Configuring OAuth provider
Sign in with OAuth provider
    Redirect URL
    Scope
    Checking authentication results
Accessing user details
Using OAuth token
Saving and restoring an auth token
Signing out
Customizing authorization flow
Configuring a custom OAuth provider
Disabling auto-update
Hexadecimal Values
Named Colors
    Theme Colors
    Color Palettes
    Color opacity

Defining colors for Flet controls
Control Level
Control Theme Level
Ancestor Theme Level
`Ref` class
Composite controls
Life-cycle methods
`build()`
`did_mount()`
`will_unmount()`
`before_update()`
Isolated controls
Basic pattern
Updating dialog content from state
Chaining dialogs
`on_dismiss` timing
When to use `page.show_dialog()` instead
Imperative
Declarative
Observables
Components
Hooks
Example
Conclusion
Secret key
Encrypting data
Decrypting data
`expand`
Example 1
Example 2
Example 3
`expand_loose`
Example 1
Installation
Running the server
Configuring an AI client
Tool groups
What the tools do
Using Flet MCP from your own agent
System fonts
Usage Example
Importing Fonts
Usage Example
Static and Variable Fonts
ListView
GridView
Batch updates
Python
Built-in Web Server
How does it work?
Guidelines
Always guard your entry point
Use importable, picklable worker functions
Don’t touch the GUI from workers

- Others
- Examples
  - Parallel sort with live progress
  - Stream progress from a worker
  - Keep a persistent worker
  - Cancel a runaway task
- Routing model in Flet
- Route basics
- Handling route changes
- Building views from route
- Programmatic navigation
- Back navigation and pop confirmation
- Pop multiple views with a result
- Navigation UI patterns
- Route templates (parameterized routes)
- Web URL strategy
- Declarative Router
- Practical recommendations
  - Storage Paths
  - Writing to a File
  - Reading from a File
- Example: Counter App
- Why Router?
- Quick start
- Defining routes
  - Flat routes
  - Nested routes
  - Index routes
  - Prefix routes
- Layout routes and `use_route_outlet()`
- Dynamic segments, optional segments, and splats
- Navigation
- Active links with `is_route_active()`
- Data loading
- Authentication
- 404 handling
- Multi-view navigation
  - Components return Views
  - Nested route hierarchy
  - Shared layouts with `outlet=True`
  - Avoiding transition animation
  - Back navigation
- Modal routes
  - Global modals — reachable from anywhere
  - Local modals — pinned to a specific page
- Recursive routes
  - Sibling-wins-over-recursion rule
- Hooks reference
  - Write data
  - Read data
  - Check key existence
  - Get all keys
- Examples
  - Launching another Android app

App-wide themes
Nested themes
Prerequisites
Create Flet extension from template
Integrate 3rd-party Flutter package
Flet extension structure
Package
Example app
Docs
Customize properties
Common properties
Control-specific properties
Dedicated data channels
How it works
Python-side API
Dart-side API
Multiple channels per widget
Threading + backpressure
See also
Boot screen
The `createBootScreen` hook
Example widget
Examples
Your first app
Basic controls
Text
Row and Column
Stack
Container
Structuring the page
Handling events
Example: Product catalog
What’s next
Prerequisites
Python version
Operating System
Creating a virtual environment (venv)
Upgrade Flet
Where tests live
Enabling tests in an existing app
Writing a test
The tester API
Finding controls
Interacting
Pumping
Screenshot testing
Running tests
On desktop
On a mobile emulator/simulator or device
Useful options
Running specific tests
Running with pytest directly
Desktop app
Web app

- Passing arguments to your app
- Flet app example
- Prerequisites
  - Android SDK
  - Android wheels for binary Python packages
- `flet build apk`
  - Split APK per ABI
- Native library packaging (modern vs legacy)
  - How each mode works
  - “Why did my APK get bigger?”
  - Trade-offs
  - When to use which
  - Accessing a custom native library by file path
  - Resolution order
  - Example
- Signing an Android bundle
  - Create an upload keystore
  - Key alias
  - Application attributes
  - Meta-data
  - Providers
  - Features
  - Permissions
  - Minimum SDK version
  - Target SDK version
  - Adaptive icon background
- ProGuard / R8 rules
  - Dropping the defaults
  - When you need this
  - Example
- Extract packages
  - Symptoms
  - Resolution order
  - Example
  - Wildcards
  - Affected packages
- ADB Tips

Publishing a Flet app

- Prerequisites
  - Platform matrix
  - Flutter SDK
- Project structure
- Choosing a Python version
- How it works
- Configuration options
  - App path
  - Entry point
  - Project name
  - Product name
  - Artifact name
  - Organization name
  - Bundle ID
  - Company Name
  - Copyright

- Versioning
- Output directory
- App dependencies
- Source packages
- Icons
- Splash screen
- Boot screen
- Hidden app window on startup
- Deep linking
- Target Architecture
- Excluding files and directories
- Compilation and cleanup
- Permissions
- Build template
- Additional `flutter build` Arguments
- Flutter dependencies
- Verbose logging

Console output

Continuous Integration/Continuous Deployment (CI/CD)

- GitHub Actions

Troubleshooting

- Prerelease compatibility
- Solution

Prerequisites

- Rosetta 2
- Xcode
- CocoaPods
- iOS wheels for binary Python packages

`flet build ipa`

`flet build ios-simulator`

Configuration options

- Application Identifier (App ID)

Deploying an App to an Apple Device for Testing

Uploading app to App Store Connect for distribution

Prerequisites

`flet build linux`

Window positioning on Wayland

Prerequisites

- Rosetta 2
- Xcode
- CocoaPods

`flet build macos`

Target architecture

Permissions

- Info.plist
- Entitlements

Prerequisites

Project files

- `requirements.txt`
- `fly.toml`

Deploy

Setup Flet Environment

- `requirements.txt` and virtualenv
- Sample Flet app

- Automatically start Flet app
  - Create `systemd` unit file
  - Enable Flet app service
- NGINX Proxy Setup
  - Sync and async handlers
- Running the app locally
- Running the app in production
  - ASGI web server
- Assets
  - Customizing web app
- Environment variables
- Advanced FastAPI scenarios
  - Flet FastAPI app
  - Hosting multiple Flet apps under the same domain
  - Adding Flet to the existing FastAPI app
  - Configuring individual Flet endpoints

Publish Flet web app

- Connecting to a Git provider
- Direct Upload
- Troubleshooting
- Setup
- Workflow
- Differences
  - Sdist-only dependencies
- `flet publish`
  - Testing the site
  - Assets
- `flet build web`
  - Testing the site
- Configuration options
  - Base URL
- Prerequisites
  - Visual Studio
- `flet build windows`
- Troubleshooting
  - Developer mode
  - `FLET_APP_CONSOLE`
  - `FLET_APP_STORAGE_DATA`
  - `FLET_APP_STORAGE_CACHE`
  - `FLET_APP_STORAGE_TEMP`
  - `FLET_APP_USER_MODEL_ID`
  - `FLET_ASSETS_DIR`
  - `FLET_ANDROID_SIGNING_KEY_ALIAS`
  - `FLET_ANDROID_SIGNING_KEY_PASSWORD`
  - `FLET_ANDROID_SIGNING_KEY_STORE`
  - `FLET_ANDROID_SIGNING_KEY_STORE_PASSWORD`
  - `FLET_CLI_NO_RICH_OUTPUT`
  - `FLET_PLATFORM`
  - `FLET_CLI_SKIP_FLUTTER_DOCTOR`
  - `FLET_HIDE_WINDOW_ON_START`
  - `FLET_FORCE_WEB_SERVER`
  - `FLET_OAUTH_CALLBACK_HANDLER_ENDPOINT`
  - `FLET_OAUTH_STATE_TIMEOUT`
  - `FLET_MAX_UPLOAD_SIZE`

FLET_SECRET_KEY
FLET_SERVER_IP
FLET_SERVER_PORT
FLET_SERVER_UDS_PATH
FLET_SESSION_TIMEOUT
FLET_UPLOAD_DIR
FLET_UPLOAD_HANDLER_ENDPOINT
FLET_WEB_APP_PATH
FLET_WEB_NO_CDN
FLET_WEBSOCKET_HANDLER_ENDPOINT
FLET_WEB_RENDERER
FLET_WEB_USE_COLOR_EMOJI
FLET_WEB_ROUTE_URL_STRATEGY
API Reference
Examples
Audio
Platform Support
Usage
Examples
Description
Audio Recorder
Platform Support
Usage
Android
iOS
macOS
Linux
Cross-platform
Examples
Stream chunks and save/download {#stream-chunks-and-save-download}
Streaming upload
Description
Examples
Examples
Examples
Examples
Examples
Usage
Uploading files
Upload storage
Examples
Pick and upload files
Pick text content and save/download it
Platform Support
Usage
Description
Platform Support
Usage
Android
iOS
macOS
Cross-platform
Troubleshooting
macOS: web browser returns POSITION_UNAVAILABLE or never resolves

- Example
- Description
- Examples
- Examples
- Examples
- Platform Support
- Usage
- Requirements
  - Android
  - iOS
- Example
- Description
- Examples

Secure Storage
- Platform Support
- Usage
- Description
- Usage example
- Disabling Auto Backup
- Encryption Options
  - Default
  - Optional Biometrics
  - Required Biometrics
- Custom Cipher Combinations
- Biometric Authentication
  - Required Permissions
  - Using Biometric Authentication
  - Requirements
- Usage Example
  - Usage with accessibility control
  - Biometric authentication:
- Important Security Considerations
- Application-Specific Key Wrapping
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples
- Examples

Flet Studio
- Why it exists
- How it works
- Limitations
- Does it replace local Flet development?

Flet Studio Privacy Policy
- 1. Introduction
  - 1.1 Who we are
  - 1.2 Our role under GDPR
  - 1.3 Scope
- 2. Information We Collect
  - 2.1 Information you provide
  - 2.2 Information collected automatically

2.3 Information from third parties
3. How We Use Information
4. Legal Bases for Processing (GDPR)
5. How We Share Information
5.1 Service providers (subprocessors)
5.2 Legal and safety disclosures
5.3 Business transfers
6. Cookies and Similar Technologies
7. Data Retention
8. Data Security
9. International Data Transfers
10. Your Rights (EEA, UK, and Swiss Residents)
11. Children’s Privacy
12. Changes to This Policy
13. Contact Us
Flet Studio Terms of Service
1. Acceptance of Terms
2. The Service
3. Accounts and Eligibility
3.1 Eligibility
3.2 Account creation
3.3 Account security
3.4 One account per person
3.5 Privacy
3.6 Communications
4. Acceptable Use
5. User Content and License Grant
5.1 Ownership
5.2 License to AppVeyor
5.3 No publication or sharing without your instruction
5.4 Your responsibility
5.5 Copyright complaints
6. AI Features
7. Plans, Billing, and Payments
7.1 Plans
7.2 AI credit packs
7.3 Payment processing
7.4 Taxes
7.5 Refunds
7.6 Non-payment
8. Service Availability and Modifications
9. Suspension and Termination
9.1 Termination by you
9.2 Suspension or termination by AppVeyor
9.3 Effect of termination
9.4 Survival
10. Disclaimers
11. Limitation of Liability
12. Indemnification
13. Governing Law and Dispute Resolution
14. General Provisions
14.1 Entire agreement
14.2 Severability
14.3 No waiver

14.4 Changes to these Terms
14.5 Assignment
14.6 Relationship of the parties
14.7 Force majeure
14.8 Notices
14.9 Contact
What’s new in Flet Studio
July 14, 2026
May 29, 2026
May 26, 2026
Getting started with Flet
Styled Controls
Reusable UI components
Handling events
Publishing your app
Summary
Getting started with Flet
User name dialog
Enhancing user interface
Reusable user controls
Laying out controls
Keyboard support
Animated scrolling to the last message
Page title
Deploying the app
What’s next
Summary
Getting started with Flet
Step 2: Drop the card in the slot or bounce it back
Step 3: Adding a second card
Step 4: Adding more slots
Fanned card piles
Slot, Card and Solitaire classes
Placing card with offset
Drag pile of cards
Solitaire setup
Create card deck
Create slots
Deal cards
Reveal top cards in tableau piles
Solitaire rules
General rules
Foundations rules
Tableau rules
Stock and waste
Winning the game
Deploying the app
Summary
Getting started with Flet
Adding page controls and handling events
Page layout
Reusable UI components
View, edit and delete list items
Filtering list items

Examples
Examples
Examples
Examples
Example 1
Examples
Usage example
Various button shapes example
Styled button example
Examples
Examples
@component
Examples
@control
Usage Example
1. Single Value for All States
2. Specific Values for Each State
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Mobile device orientation configuration
Examples
Examples
Examples
Examples
Examples
Containers with gradients
Examples
Examples
Types
Predefined filters
Usage example
Examples
Examples
Examples
Container with linear gradient
Examples
Examples
Examples
Examples
Example 1
Examples
Examples
Examples
@observable
Examples
Examples
Examples
Example 1

Examples
Canvas paint
Examples
Examples
Canvas paint
Examples
Canvas paint
Examples
Canvas paint
Examples
Examples
Examples
Container with radial gradient
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Container with sweep gradient
Examples
Examples
Examples
Example 1
Examples
Examples
Examples
Usage Example
Examples
Examples
Examples
Examples
Examples
Examples
Examples
Tooltip with decoration
Examples
@value
Examples
Usage Example
Examples
Breaking changes and deprecations
By release
Released in Flet 0.86.0
Released in Flet 0.85.0
`DragTargetEvent` coordinate fields deprecated
Summary
Context
Migration guide
Timeline

References
`Video` control APIs deprecated
Summary
Context
Migration guide
Hiding controls
Updating the playlist
Timeline
References
Deprecated spacing and border helper functions removed
Summary
Background
Migration guide
Timeline
References
Android: site-packages ship zipped; some packages need `extract_packages`
Summary
Symptoms
Migration guide
No action needed for
Timeline
References
Android: `x86` removed from target architectures
Summary
Background
Migration guide
Timeline
References
App files ship unpacked in a read-only bundle; storage dirs reworked
Summary
Background
Migration guide
Write data to the storage dirs, not next to your code
Read shipped files via `__file__`, not relative paths
Storage environment variables
Physical locations
`flet run` dev storage
Timeline
References
App and packages are compiled to `.pyc` by default
Summary
Background
Migration guide
Opt out via the CLI flag
Opt out in `pyproject.toml`
Timeline
References
Flet protocol framing upgraded for DataChannel support
Summary
Background
Migration guide
Most users — nothing to change
Users running `flet-cli` and the `flet` Python package from different installs
Users with custom backends or sidecars speaking the Flet protocol

Code that depended on `StreamingMsgpackDeserializer`
Custom widgets that override `MatplotlibChartCanvas`
Timeline
References
Default bundled Python version is now 3.14
Summary
Background
Migration guide
Pin in `pyproject.toml` (recommended)
Pin via the CLI flag
Pin via environment variable
Timeline
References
`flet build --clear-cache` flag deprecated
Summary
Context
Migration guide
Timeline
References
`flet.version.pyodide_version` and `PYODIDE_VERSION` removed
Summary
Background
Migration guide
Timeline
References
Compatibility policy
Breaking changes
Deprecation policy
Where changes are documented
Stay up to date
Release notes
Stable releases
0.86.x
0.85.x
0.84.x
0.83.x
0.82.x
0.81.x
0.80.x
0.70.x
0.28.x
0.27.x
0.26.x
0.25.x
0.24.x
0.23.x
0.22.x
0.21.x
0.20.x
0.19.x
0.18.x
0.17.x
0.16.x
0.15.x

0.14.x
0.13.x
0.12.x
0.11.x
0.10.x
0.9.x
0.8.x
0.7.x
0.6.x
0.5.x
0.4.x
0.3.x
0.2.x
0.1.x

User interface is made of **Controls** (aka widgets).

To make controls visible to a user they must be added to a Page (the top-most control in the tree) or inside other controls. Nesting controls into each other could be represented as a tree with Page as a root.

Controls are just regular Python classes. Create control instances via constructors with parameters matching their properties, for example:

```
ft.Text(value="Hello, world!", color="green")
```

To display control on a page add it to `controls` list of a Page and call `page.update()` to send page changes to a browser or desktop client:

```
import flet as ft

def main(page: ft.Page):
    t = ft.Text(value="Hello, world!", color="green")
    page.controls.append(t)
    page.update()

ft.run(main)
```



In the following examples we will be showing just the contents of `main` function.

You can modify control properties and the UI will be updated on the next `page.update()`:

```
t = ft.Text()
page.add(t) # it's a shortcut for page.controls.append(t) and
then page.update()

for i in range(10):
    t.value = f"Step {i}"
    page.update()
    time.sleep(1)
```

Some controls are “container” controls (like Page) which could contain other controls. For example, `Row` control allows arranging other controls in a row one-by-one:

```
page.add(
    ft.Row(
        controls=[
            ft.Text("A"),
            ft.Text("B"),
            ft.Text("C")
        ]
    )
)
```

or `TextField` and `Button` next to it:

```
page.add(
    ft.Row(
        controls=[
            ft.TextField(label="Your name"),
            ft.Button(text="Say my name!")
        ]
    )
)
```

`page.update()` is smart enough to send only the changes made since its last call, so you can add a couple of new controls to the page, remove some of them, change other controls’ properties and then call `page.update()` to do a batched update, for example:

```
for i in range(10):
    page.controls.append(ft.Text(f"Line {i}"))
    if i > 4:
        page.controls.pop(0)
    page.update()
    time.sleep(0.3)
```

Some controls, like buttons, could have event handlers reacting on a user input, for example `Button.on_click`:

```
def button_clicked(e):
    page.add(ft.Text("Clicked!"))

page.add(ft.Button(text="Click me", on_click=button_clicked))
```

and more advanced example for a simple To-Do:

```
import flet as ft

def main(page):
    async def add_clicked(e):
        page.add(ft.Checkbox(label=new_task.value))
        new_task.value = ""
        await new_task.focus()
        new_task.update()
```

```
            new_task = ft.TextField(hint_text="What's needs to be
done?", width=300)
            page.add(ft.Row([new_task, ft.Button("Add",
on_click=add_clicked)]))

        ft.run(main)
```



Flet implements *imperative* UI model where you “manually” build application UI with stateful controls and then mutate it by updating control properties. Flutter implements *declarative* model where UI is automatically re-built on application data changes. Managing application state in modern frontend applications is inherently complex task and Flet’s “old-school” approach could be more attractive to programmers without frontend experience.

### `disabled` property

Every control has `disabled` property which is `false` by default - control and all its children are enabled. `disabled` property is mostly used with data entry controls like `TextField`, `Dropdown`, `Checkbox`, buttons. However, `disabled` could be set to a parent control and its value will be propagated down to all children recursively.

For example, if you have a form with multiple entry control you can set `disabled` property for each control individually:

```
        first_name = ft.TextField()
        last_name = ft.TextField()
        first_name.disabled = True
        last_name.disabled = True
        page.add(first_name, last_name)
```

or you can put form controls into container, e.g. `Column` and then set `disabled` for the column:

```
        first_name = ft.TextField()
        last_name = ft.TextField()
        c = ft.Column(controls=[
            first_name,
            last_name
        ])
        c.disabled = True
        page.add(c)
```

## Buttons

`Button` is the most essential input control which generates `click` event when pressed:

```
        btn = ft.Button("Click me!")
        page.add(btn)
```



All events generated by controls on a web page are continuously sent back to your script, so how do you respond to a button click?

# Event handlers

Buttons with events in “Counter” app:

```
        import flet as ft

        def main(page: ft.Page):
            page.title = "Flet counter example"
            page.vertical_alignment = ft.MainAxisAlignment.CENTER

            txt_number = ft.TextField(value="0", text_align="right",
width=100)

            def minus_click(e):
                txt_number.value = str(int(txt_number.value) - 1)
                page.update()

            def plus_click(e):
                txt_number.value = str(int(txt_number.value) + 1)
                page.update()

            page.add(
                ft.Row(
                    [
                        ft.IconButton(ft.Icons.REMOVE,
on_click=minus_click),
                        txt_number,
                        ft.IconButton(ft.Icons.ADD,
on_click=plus_click),
                    ],
                    alignment=ft.MainAxisAlignment.CENTER,
                )
            )

        ft.run(main)
```



# Textbox

Flet provides a number of controls for building forms: `TextField`, `Checkbox`, `Dropdown`, `Button`.

Let’s ask a user for a name:

```
        import flet as ft

        def main(page):
            def btn_click(e):
                if not txt_name.value:
                    txt_name.error_text = "Please enter your name"
                    page.update()
                else:
                    name = txt_name.value
                    page.clean()
                    page.add(ft.Text(f"Hello, {name}!"))

            txt_name = ft.TextField(label="Your name")

            page.add(txt_name, ft.Button("Say hello!",
on_click=btn_click))

        ft.run(main)
```



# Checkbox

The Checkbox control provides you with various properties and events emmiters for ease of use.

Let's create a one checkbox ToDo:

```
        import flet as ft

        def main(page):
            def checkbox_changed(e):
                output_text.value = (
                    f"You have learned how to ski :
{todo_check.value}."
                )
                page.update()

            output_text = ft.Text()
            todo_check = ft.Checkbox(label="ToDo: Learn how to use
ski", value=False, on_change=checkbox_changed)
            page.add(todo_check, output_text)

        ft.run(main)
```



# Dropdown

```
        import flet as ft

        def main(page: ft.Page):
            def button_clicked(e):
                output_text.value = f"Dropdown value is:
{color_dropdown.value}"
                page.update()

            output_text = ft.Text()
            submit_btn = ft.Button(text="Submit",
on_click=button_clicked)
            color_dropdown = ft.Dropdown(
                width=100,
                options=[
                    ft.dropdown.Option("Red"),
                    ft.dropdown.Option("Green"),
                    ft.dropdown.Option("Blue"),
                ],
            )
            page.add(color_dropdown, submit_btn, output_text)

        ft.run(main)
```



This article covers Flet's vision for going mobile and provides a roadmap.

# Server-Driven UI

Flet is a Server-driven UI (SDUI) framework. SDUI is an emerging technology which is the best described in Technology Radar post:

> Server-driven UI separates the rendering into a generic container in the mobile app while the structure and data for each view is provided by the server. This means that changes that once required a round trip to an app store can now be accomplished via simple changes to the responses the server sends.

Companies like DoorDash, Airbnb, Lyft and others have been successfully implementing Server-driven UI in their mobile apps to reduce time-to-market.

## Flet approach

Flet is going to implement Server-Driven UI approach where program written in Python or other language is running on the server and only a thin client - either standalone Flutter app (`.apk` or `.ipa` package) in app store or a Flutter widget as a part of another app - is delivered to a mobile:

> Missing image omitted: Flet highlevel diagram

Once SDUI experience is ready we'll start working on a standalone mobile package.

# Roadmap

To provide the best experience for Flet apps on mobile platforms, we plan to release the following items by the end of this year:

## Flet widget for Flutter

The first step we are going to do is to separate Flet client into a Flutter widget and publish the package at https://pub.dev. Flet widget could be then integrated by mobile developers into existing or new Flutter apps for adding dynamic server-driven UI experiences to the core app functionality. A new Flutter app could be also created with a single Flet widget just for the purpose of hosting a complete Flet app.

Developers will follow Flutter guide for packaging, signing and distributing their apps to Android, iOS, Linux, macOS or Windows platforms.

Flet team will provide sample CI pipelines to automate packaging, signing and publishing of Flutter apps.

## Flet Studio for iOS and Android

The next step is a standalone "Flet Studio" app (the name is not final) in App Store and Google Play for "testing mobile experiences developed with Flet framework". Developers or beta testers will be able to "register" URL of their hosted Flet app within Flet Studio and instantly see how it performs on a mobile device.

## White-labeled Flet mobile app

We are going to provide a guide and CI pipeline for automatic publishing of white-labeled Flet app to a user App Store or Google Play account. This app will be "pinned" to a specific app URL and could additionally bundle app assets (media, fonts) to minimize network usage.

## Standalone mobile package for Flet app

We are going to investigate the way and develop a prototype for bundling together Flet framework, user program, language runtime and all dependencies into a standalone mobile package (`.apk` or `.ipa` package), so Flet program does not require a web server.

## Embedding Flet into native apps

We are going to provide a guide, sample apps and CI pipeline to integrate Flet widget into existing native Android and iOS apps (not developed with Flutter) using Flutter Add-to-App feature. Put Flutter to work article gives a real-world example on how to integrate Flutter into existing mobile app.

:::danger[A better way to package is here]

You can now use `flet build` command to package your Flet app into executable or install bundle for macOS, Windows and Linux.

`flet build` does not longer rely on PyInstaller like `flet pack` does, but uses Flutter SDK to produce a fast, offline, fully customizable (your own icons, about dialog and metadata) executable for Windows, Linux and macOS with Python runtime embedded into executable and running in-process.

Follow this guide for desktop packaging :::

Flet Python app and all its dependencies can be packaged into an executable and user can run it on their computer without installing a Python interpreter or any modules.

Flet wraps PyInstaller API to package Flet Python app and all its dependencies into a single package for Windows, macOS and Linux. To create Windows package, PyInstaller must be run on Windows; to build Linux app, it must be run on Linux; and to build macOS app - on macOS.

Start from installing PyInstaller:

```
pip install pyinstaller
```

Navigate to the directory where your `.py` file is located and build your app with the following command:

```
flet pack your_program.py
```

Your bundled Flet app should now be available in `dist` folder. Try running the program to see if it works.

On macOS:

```
open dist/your_program.app
```

on Windows:

```
dist\your_program.exe
```

on Linux:

```
dist/your_program
```

Now you can just zip the contents of `dist` folder and distribute to your users! They don’t need Python or Flet installed to run your packaged program - what a great alternative to Electron!

By default, an executable/bundle has the same name as a Python script. You can change it with `--name` argument:

```
flet pack your_program.py --name bundle_name
```

# Customizing package icon

Default bundle app icon is diskette which might be confusing for younger developers missed those ancient times when floppy disks were used to store computer data.

You can replace the icon with your own by adding `--icon` argument:

```
flet pack your_program.py --icon <your-image.png>
```

PyInstaller will convert provided PNG to a platform specific format (`.ico` for Windows and `.icns` for macOS), but you need to install Pillow module for that:

```
pip install pillow
```

# Packaging assets

Your Flet app can include assets. Provided app assets are in `assets` folder next to `your_program.py` they can be added to an application package with `--add-data` argument, on macOS/Linux:

```
flet pack your_program.py --add-data "assets:assets"
```

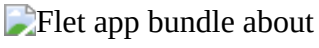
On Windows `assets;assets` must be delimited with `;`:

```
flet pack your_program.py --add-data "assets;assets"
```

# Customizing macOS bundle

macOS bundle details can be customized with the following `flet pack` macOS-specific arguments:

- `--product-name` - display name of macOS bundle, shown in Dock, Activity Monitor, About dialog.
- `--product-version` - bundle version shown in “About” dialog.
- `--copyright` - copyright notice shown in “About” dialog.
- `--bundle-id` unique bundle ID.




# Customizing Windows executable metadata

Windows executable “Details” properties dialog can be customized with the following `flet pack` arguments:

- `--product-name` - “Product name” field.
- `--product-version` - “Product version” field.
- `--file-version` - “File version” field.
- `--file-description` - “File description” field, also program display name in Task Manager.
- `--copyright` - “Copyright” field.

# Using CI for multi-platform packaging

To create an app package with PyInstaller for specific OS it must be run on that OS.

If you don't have an access to Mac or PC you can bundle your app for all three platforms with AppVeyor - Continuous Integration service for Windows, Linux and macOS. In short, Continuous Integration (CI) is an automated process of building, testing and deploying (Continuous Delivery - CD) application on every push to a repository.

AppVeyor is free for open source projects hosted on GitHub, GitLab and Bitbucket. To use AppVeyor, push your app to a repository within one of those source-control providers.

AppVeyor is the company behind Flet.

To get started with AppVeyor sign up for a free account.

Click "New project" button, authorize AppVeyor to access your GitHub, GitLab or Bitbucket account, choose a repository with your program and create a new project.

Now, to configure packaging of your app for Windows, Linux and macOS, add file with the following contents into the root of your repository `appveyor.yml`. `appveyor.yml` is a build configuration file, or CI workflow, describing build, test, packaging and deploy commands that must be run on every commit.

You can just fork flet-dev/python-ci-example repository and customize it to your needs.

When you push any changes to GitHub repository, AppVeyor will automatically start a new build:

> Missing image omitted: AppVeyor CI Flet Python project

What that CI workflow does on every push to the repository:

- Clones the repository to a clean virtual machine.
- Installs app dependencies using `pip`.
- Runs `flet pack` to package Python app into a bundle for **Windows**, **macOS** and **Ubuntu**.
- Zip/Tar app bundles and uploads them to "Artifacts".
- Uploads app bundles to **GitHub releases** when a new tag is pushed. Just push a new tag to make a release!

`GITHUB_TOKEN` in `appveyor.yml` is a GitHub Personal Access Token (PAT) used by AppVeyor to publish created packages to repository "Releases". You need to generate your own token and replace it in `appveyor.yml`. Login to your GitHub account and navigate to Personal access token page. Click "Generate new token" and select "public_repo" or "repo" scope for public or private repository respectively. Copy generated token to a clipboard and return to AppVeyor Portal. Navigate to Encrypt configuration data page and paste token to "Value to encrypt" field, click "Encrypt" button. Put encrypted value under `GITHUB_TOKEN` in your `appveyor.yml`.

Configure AppVeyor for your Python project, push a new tag to a repository and "automagically" get desktop bundle for all three platforms in GitHub releases! 🎉

> Missing image omitted: AppVeyor CI Flet GitHub releases

In addition to GitHub Releases, you can also configure releasing of artifacts to Amazon S3 bucket or Azure Blob storage.

import CliBuild from '@site/.crocodocs/cli-build.mdx';

import CliClean from '@site/.crocodocs/cli-clean.mdx';

import CliCreate from '@site/.crocodocs/cli-create.mdx';

import CliDebug from '@site/.crocodocs/cli-debug.mdx';

import CliDevices from '@site/.crocodocs/cli-devices.mdx';

import CliDoctor from '@site/.crocodocs/cli-doctor.mdx';

import CliEmulators from '@site/.crocodocs/cli-emulators.mdx';

import CliMcp from '@site/.crocodocs/cli-mcp.mdx';

import CliPack from '@site/.crocodocs/cli-pack.mdx';

import CliPublish from '@site/.crocodocs/cli-publish.mdx';

:::note[`--pre` vs `--python-version`] `--pre` allows **micropip** to resolve pre-release versions of *Python packages* at runtime. It is unrelated to which CPython/Pyodide release is bundled. To choose a pre-release Python interpreter (e.g. 3.15 once it appears in the supported list), name it explicitly via `--python-version` or `[project].requires-python` in `pyproject.toml` — see Choosing a Python version. :::

import CliRun from '@site/.crocodocs/cli-run.mdx';

import CliServe from '@site/.crocodocs/cli-serve.mdx';

import CliTest from '@site/.crocodocs/cli-test.mdx';

import CliRoot from '@site/.crocodocs/cli-root.mdx';

# Flet CLI

Flet command-line interface (CLI) provides an easy way to create, run, test and package Flet apps.

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/banner_ad_and_interstitial_ad/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.gif'} width="55%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/consent/main.py'} language="python" />

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {CodeExample, Image} from '@site/src/components/crocodocs';

# Ads

Displaying Google Ads in Flet apps.

It is powered by the google_mobile_ads Flutter package.

## Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ❌ | ❌ | ❌ | ✅ | ✅ | ❌ |

## Usage

To use ads controls add `flet-ads` package to your project dependencies:

```
uv add flet-ads

pip install flet-ads  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Requirements

The below sections show the required configurations for each platform.

### Android

A valid AdMob app ID is required to be specified, otherwise the app might crash on launch or behave unexpectedly.

```
flet build apk \
  --android-meta-data
```

```
com.google.android.gms.ads.APPLICATION_ID="ca-app-pub-
xxxxxxxxxxxxxxx~yyyyyyyyyy"
```

```
        [tool.flet.android.meta_data]
        "com.google.android.gms.ads.APPLICATION_ID" = "ca-app-pub-
xxxxxxxxxxxxxxx~yyyyyyyyyy"
```

See also:

- setting Android permissions
- AdMob app ID for testing purposes

### iOS

A valid AdMob app ID is required to be specified, otherwise the app might crash on launch or behave unexpectedly.

```
        flet build ipa \
          --info-plist GADApplicationIdentifier="ca-app-pub-
xxxxxxxxxxxxxxx~yyyyyyyyyy"
```

```
        [tool.flet.ios.info]
        GADApplicationIdentifier = "ca-app-pub-
xxxxxxxxxxxxxxx~yyyyyyyyyy"
```

See also:

- setting iOS permissions
- AdMob app ID for testing purposes

### Test Values

AdMob provides app and ad unit IDs for testing purposes.

- **AdMob app ID**: `"ca-app-pub-3940256099942544~3347511713"`
- `BannerAd.unit_id`
  - **Android**: `"ca-app-pub-3940256099942544/9214589741"`
  - **iOS**: `"ca-app-pub-3940256099942544/2435281174"`
- `InterstitialAd.unit_id`
  - **Android**: `"ca-app-pub-3940256099942544/1033173712"`
  - **iOS**: `"ca-app-pub-3940256099942544/4411468910"`

**They are not meant to be used in production.**

## Consent (UMP)

[`ConsentManager`][flet_ads.ConsentManager] wraps Google's User Messaging Platform (UMP) to gather user consent (for example, the GDPR/EEA consent form) before requesting ads.

The consent message itself is created and published in the AdMob console, not in code: Google serves the region-appropriate message (EEA/GDPR, a regulated US state, and so on) based on the user's location. Your app only requests an update and shows the form when required.

## Testing the consent form

Force a geography during development with [`DebugGeography`] [flet_ads.DebugGeography] on [`ConsentDebugSettings`] [flet_ads.ConsentDebugSettings]:

- **Simulators and emulators are automatically registered as test devices**, so the debug geography applies on them **without** any [`test_identifiers`] [flet_ads.ConsentDebugSettings.test_identifiers].
- On a **physical device**, register it first: call [`request_consent_info_update()`] [flet_ads.ConsentManager.request_consent_info_update] once, then read the device's hashed ID from the **native** device log. Add that ID to `test_identifiers` and re-run.

The message carrying the hashed ID differs per platform:

**Android** (`adb logcat`):

```
Use new
ConsentDebugSettings.Builder().addTestDeviceHashedId("33BE2250B43518CCI
to set this as a debug device.
```

**iOS** (Xcode console / Console.app):

```
<UMP SDK>To enable debug mode for this device,
set: UMPDebugSettings.testDeviceIdentifiers =
@[2077ef9a63d2b398840261c8221a0c9b]
```

## Interpreting consent status

[`ConsentStatus`][flet_ads.ConsentStatus] reflects whether a decision was *collected*, not what the user chose — it becomes `OBTAINED` whether they consent **or** decline. To gate ad loading, use [`can_request_ads()`][flet_ads.ConsentManager.can_request_ads]; when consent is declined the SDK simply serves non-personalized ads.

Furthermore, the consent status is cached across sessions, so the form is shown only once. When in testing/debug mode, you can call [`ConsentManager.reset()`] [flet_ads.ConsentManager.reset] to replay the flow as a first-time user. The test AdMob app ID provided above already has a sample consent message, so it shows the form during testing; in production your own app ID needs a message **published** in the AdMob console.

## Under age of consent

Setting [`tag_for_under_age_of_consent`] [flet_ads.ConsentRequestParameters.tag_for_under_age_of_consent] to `True` suppresses the consent form — users under the age of consent cannot be asked to consent. Leave it

unset (or `False`) while testing the form.

## Example

<CodeExample path={frontMatter.examples + '/banner_ad_and_interstitial_ad/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.gif'} width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/banner_ad_and_interstitial_ad/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.gif'} width="55%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="AlertDialog" />

## Examples

<CodeExample path={frontMatter.examples + '/modal_and_non_modal/main.py'} language="python" />

<Image src={frontMatter.example_images + '/alert_dialog_flow.gif'} alt="Modal and non-modal dialogs" width="50%" caption="Modal and non-modal dialogs" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/scale_effect/main.py'} language="python" />

<Image src={frontMatter.example_images + '/scale_effect.gif'} alt="scale-effect" width="25%" />

<CodeExample path={frontMatter.examples + '/image_switch/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/image_switch_buffered/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="AppBar" imageWidth="65%" />

# Examples

<CodeExample path={frontMatter.examples + '/actions_and_popup_menu/main.py'} language="python" />

<Image src={frontMatter.example_images + '/app_bar_flow.gif'} alt="actions-and-popup-menu" width="55%" />

<CodeExample path={frontMatter.examples + '/theme_mode_toggle/main.py'} language="python" />

<Image src={frontMatter.example_images + '/theme_mode_toggle_flow.gif'} alt="theme-and-material-mode-toggles" width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/auto_complete/main.py'} language="python" />

<Image src={frontMatter.example_images + '/auto_complete_basic_flow.gif'} alt="auto-complete-basic-example" width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + ‘/autofill_group/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.gif’} alt=“basic” width=“40%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Banner” />

# Examples

<CodeExample path={frontMatter.examples + ‘/banner/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/banner_flow.gif’} alt=“basic” width=“50%” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“BottomAppBar” imageWidth=“50%” />

# Examples

<CodeExample path={frontMatter.examples + ‘/notched_fab/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/notched_fab.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/border_radius/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/border_radius.png’} width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“BottomSheet” imageWidth=“30%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/bottom_sheet/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.gif’} width=“40%” />

<CodeExample path={frontMatter.examples + ‘/fullscreen/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/fullscreen.gif’} width=“40%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Enabled and disabled buttons” imageWidth=“25%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/button/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“Basic button” width=“25%” />

<CodeExample path={frontMatter.examples + ‘/icons/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/icons.png’} alt=“Basic button” width=“35%” />

<CodeExample path={frontMatter.examples + ‘/handling_clicks/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/handling_clicks.png’} alt=“Handling clicks” width=“35%” />

<CodeExample path={frontMatter.examples + ‘/custom_content/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/custom_content.png’} alt=“Buttons with custom content” width=“35%” />

<CodeExample path={frontMatter.examples + ‘/button_shapes/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/button_shapes.png’} alt=“Buttons with different shapes” width=“35%” />

<CodeExample path={frontMatter.examples + ‘/styling/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/styled_initial.png’} alt=“Styled button - default state” width=“25%” caption=“Default state” />

<Image src={frontMatter.example_images + '/styled_hovered.png'} alt="Styled button - hovered state" width="25%" caption="Hovered state" />

<CodeExample path={frontMatter.examples + '/animate_on_hover/main.py'} language="python" />

<Image src={frontMatter.example_images + '/animate_on_hover_initial.png'} alt="Unhovered button" width="40%" caption="Normal button" />

<Image src={frontMatter.example_images + '/animate_on_hover_hovered.png'} alt="Hovered button" width="40%" caption="Hovered button" />

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

# Camera

Display a live camera preview, capture photos and videos, and stream camera frames directly in your Flet apps.

Powered by the camera Flutter package.

## Platform Support

| Platform | iOS | Android | Web | Windows | macOS | Linux |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ❌ | ❌ | ❌ |

## Usage

Add the `flet-camera` package to your project dependencies:

```
uv add flet-camera

pip install flet-camera  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. :::tipPermissions Request camera (and microphone if recording video with audio) permissions on mobile platforms before initializing the control. You can use `PermissionHandler` to prompt the user. You can also use predefined cross-platform permission bundles for camera and microphone permissions. :::

## Requirements

The below sections show the required configurations for each platform.

## Android

Configuration to be made to access the camera and optionally the microphone:

- `android.permission.CAMERA`: Allows camera usage.
- `android.permission.RECORD_AUDIO` (optional): Allows video recording with audio.

```
flet build apk \
  --android-permissions android.permission.CAMERA=true \
  --android-permissions android.permission.RECORD_AUDIO=true
```

```
[tool.flet.android.permission]
"android.permission.CAMERA" = true
"android.permission.RECORD_AUDIO" = true
```

See also:

- setting Android permissions

## iOS

Configuration to be made to access the camera and optionally the microphone:

- `NSCameraUsageDescription`: Required for camera usage.
- `NSMicrophoneUsageDescription` (optional): Required only for video recording with audio. For example, when `enable_audio` parameter of `Camera.initialize` is set to `True` (default).

```
flet build ipa \
  --info-plist NSCameraUsageDescription="Some message to
describe why you need this permission..." \
  --info-plist NSMicrophoneUsageDescription="Some message to
describe why you need this permission..."
```

```
[tool.flet.ios.info]
NSCameraUsageDescription = "Some message to describe why you
need this permission..."
NSMicrophoneUsageDescription = "Some message to describe why
you need this permission..."
```

See also:

- setting iOS permissions

## Cross-platform

Additionally/alternatively, you can make use of our predefined cross-platform `camera` (and optionally `microphone`) permission bundles:

```
flet build <target_platform> --permissions camera microphone
```

```
[tool.flet]
permissions = ["camera", "microphone"]
```

## Example

<CodeExample path={frontMatter.examples + '/camera_playground/main.py'} language="python" />

# Description

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Canvas

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Canvas" imageWidth="25%"/>

# Examples

<CodeExample path={frontMatter.examples + ‘/smiling_face/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/smiling_face.png’} alt=“smiling-face” width=“25%” />

<CodeExample path={frontMatter.examples + ‘/flet_logo/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/flet_logo.png’} alt=“flet-logo” width=“35%” />

<CodeExample path={frontMatter.examples + ‘/triangles/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/triangles.png’} alt=“triangles” width=“25%” />

<CodeExample path={frontMatter.examples + ‘/bezier_curves/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/bezier_curves.png’} alt=“bezier-curves” width=“25%” />

<CodeExample path={frontMatter.examples + ‘/text/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/text.png’} alt=“text” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/brush/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/gradients/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Card” />

## Examples

<CodeExample path={frontMatter.examples + ‘/music_info/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/music_info.png’} alt=“music-info” width=“50%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/interactive_bar_chart/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/example_1.png’} width=“65%” />

<CodeExample path={frontMatter.examples + ‘/bar_chart_with_hover/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/example_2.png’} width=“65%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Candlestick chart” imageWidth=“65%” />

## Examples

<CodeExample path={frontMatter.examples + ‘/candlestick_chart/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/example_1.png’} width=“65%” />

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’;

# Charts

Interactive chart controls powered by flet-charts let you display data as bar, line, pie, scatter and plotly visualisations directly in your Flet apps.

It is built on top of the fl_chart Flutter package and ships with helper classes for axis labels, tooltips and more.

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

# Usage

Add `flet-charts` to your project dependencies:

```
uv add flet-charts

pip install flet-charts  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Available Charts

- `BarChart`
- `CandlestickChart`
- `LineChart`
- `MatplotlibChart`
- `PieChart`
- `PlotlyChart`
- `RadarChart`
- `ScatterChart`

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/multi_series_line_chart/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/line_chart_with_custom_axes/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_2.png'} width="65%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/bar_chart/main.py'} language="python" />

<Image src={frontMatter.example_images + '/bar_chart.png'} width="65%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/toolbar.png'} imageWidth="65%" />

# Examples

<CodeExample path={frontMatter.examples + '/toolbar/main.py'} language="python" />

<Image src={frontMatter.example_images + '/toolbar.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/three_d/main.py'} language="python" />

<Image src={frontMatter.example_images + '/three_d.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/handle_events/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handle_events.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/animate/main.py'} language="python" />

<Image src={frontMatter.example_media + '/animate.png'} width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/pie_chart_with_hover_borders/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/pie_chart_with_hover_sections/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_2.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/pie_chart_with_icon_badges/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_3.png'} width="65%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Bar chart based on official Plotly example" imageWidth="65%" />

# Examples

<CodeExample path={frontMatter.examples + '/plotly_line_chart/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/plotly_bar_chart/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_2.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/plotly_pie_chart/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_3.png'} width="65%" />

<CodeExample path={frontMatter.examples + '/plotly_box_plot/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_4.png'} width="65%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Radar chart" imageWidth="65%" />

# Examples

<CodeExample path={frontMatter.examples + '/radar_chart/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.png'} width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Scatter chart" imageWidth="55%" />

# Examples

<CodeExample path={frontMatter.examples + '/scatter_chart/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_1.png'} width="65%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

```
import {ClassAll} from '@site/src/components/crocodocs';
import {ClassAll} from '@site/src/components/crocodocs';
import {ClassAll} from '@site/src/components/crocodocs';
```

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Checkbox” imageWidth=“12%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/checkbox/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“50%” caption=“After clicking Submit” />

<CodeExample path={frontMatter.examples + ‘/handling_events/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/handling_events.png’} alt=“handling-events” width=“35%” caption=“After three clicks” />

<CodeExample path={frontMatter.examples + ‘/styled/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/styled_checkboxes_selected.png’} alt=“Styled checkboxes” width=“35%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Chip” imageWidth=“20%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/assist_chips/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/assist_chips.png’} alt=“assist-chips” width=“50%” />

<CodeExample path={frontMatter.examples + ‘/filter_chips/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/filter_chips.png’} alt=“filter-chips” width=“80%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CircleAvatar” imageWidth=“5%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/user_avatars/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/user_avatars.png’} alt=“user-avatars” width=“20%” />

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’; import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CodeEditor” imageWidth=“60%”/>

# Usage

Add `flet-code-editor` to your project dependencies:

```
uv add flet-code-editor

pip install flet-code-editor  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Examples

<CodeExample path={frontMatter.examples + ‘/code_editor/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/example_1.png’} alt=“code-editor-example-1” width=“60%” />

<CodeExample path={frontMatter.examples + ‘/selection_handling/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/example_2.png’} alt=“code-editor-example-2” width=“65%” />

<CodeExample path={frontMatter.examples + '/folding_and_initial_selection/main.py'} language="python" />

<Image src={frontMatter.example_images + '/example_3.png'} alt="code-editor-example-3" width="65%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/block_picker.png'} imageCaption="BlockPicker" imageWidth="35%"/>

# Example

<CodeExample path={frontMatter.examples + '/block_picker/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="ColorPicker" imageWidth="55%"/>

# Example

<CodeExample path={frontMatter.examples + '/color_picker/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/hue_ring_picker.png'} imageCaption="HueRingPicker" imageWidth="55%"/>

# Example

<CodeExample path={frontMatter.examples + '/hue_ring_picker/main.py'} language="python" />

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs';

# Color pickers

Multiple color picker controls built on Flutter's `flutter_colorpicker` package.

## Usage

Add `flet-color-pickers` to your project dependencies:

```
uv add flet-color-pickers

pip install flet-color-pickers  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Available color pickers

- ColorPicker
- BlockPicker
- HueRingPicker
- MaterialPicker
- MultipleChoiceBlockPicker
- SlidePicker

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/material_picker.png'} imageCaption="MaterialPicker" imageWidth="55%"/>

## Example

<CodeExample path={frontMatter.examples + '/material_picker/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/multiple_choice_block_picker.png'} imageCaption="MultipleChoiceBlockPicker" imageWidth="35%"/>

## Example

<CodeExample path={frontMatter.examples + '/multiple_choice_block_picker/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/slide_picker.png’} imageCaption=“SlidePicker” imageWidth=“35%”/>

# Example

<CodeExample path={frontMatter.examples + ‘/slide_picker/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Column” imageWidth=“25%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/spacing/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/spacing.gif’} alt=“spacing” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/wrap/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/wrap.gif’} alt=“wrap” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/alignment/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/alignment.png’} alt=“alignment” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/horizontal_alignment/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/horizontal_alignment.png’} alt=“horizontal-alignment” width=“40%” />

<CodeExample path={frontMatter.examples + ‘/infinite_scrolling/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/programmatic_scroll/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/programmatic_scroll.png’} alt=“programmatic-scroll” width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_media + ‘/overview_padding_margin_border.png’} imageCaption=“Container explained” />

# Examples

<CodeExample path={frontMatter.examples + '/clickable/main.py'} language="python" />

<Image src={frontMatter.example_media + '/clickable.gif'} width="65%" />

<CodeExample path={frontMatter.examples + '/handling_clicks/main.py'} language="python" />

<Image src={frontMatter.example_media + '/handling_clicks.gif'} width="40%" />

<CodeExample path={frontMatter.examples + '/handling_hovers/main.py'} language="python" />

<Image src={frontMatter.example_media + '/handling_hovers.gif'} width="15%" />

<CodeExample path={frontMatter.examples + '/animate_size_and_color/main.py'} language="python" />

<Image src={frontMatter.example_media + '/animate_1.gif'} width="25%" />

<CodeExample path={frontMatter.examples + '/animate_gradient_and_shape/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/animated_slide_in_menu/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/inherited_and_overridden_theme/main.py'} language="python" />

<Image src={frontMatter.example_images + '/nested_themes_1.png'} width="40%" />

<CodeExample path={frontMatter.examples + '/page_dark_and_light_themes/main.py'} language="python" />

<Image src={frontMatter.example_images + '/nested_themes_2.png'} width="60%" />

<CodeExample path={frontMatter.examples + '/theme_mode_toggle/main.py'} language="python" />

<Image src={frontMatter.example_media + '/nested_themes_3.gif'} width="40%" />

<CodeExample path={frontMatter.examples + '/size_aware/main.py'} language="python" />

<Image src={frontMatter.example_images + '/size_aware.png'} width="40%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="ContextMenu" imageWidth="30%"/>

# Examples

<CodeExample path={frontMatter.examples + ‘/triggers/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/triggers_flow.gif’} width=“30%” />

<CodeExample path={frontMatter.examples + ‘/programmatic_open/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/programmatic_open.png’} width=“30%” />

<CodeExample path={frontMatter.examples + ‘/custom_trigger/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/custom_trigger_flow.gif’} width=“30%” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CupertinoActionSheet” />

# Examples

<CodeExample path={frontMatter.examples + ‘/cupertino_action_sheet/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/basic.png’} alt=“basic” width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CupertinoActivityIndicator” imageWidth=“8%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/cupertino_activity_indicator/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/basic.png’} alt=“basic” width=“25%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + '/file_deletion_confirmation/main.py'} language="python" />

<Image src={frontMatter.example_images + '/file_deletion_confirmation.png'} alt="file-deletion-confirmation" width="40%" />

<CodeExample path={frontMatter.examples + '/cupertino_material_and_adaptive/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_app_bar/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="55%" />

<CodeExample path={frontMatter.examples + '/theme_mode_toggle/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_action_sheet/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="cupertinoactionsheet" width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="CupertinoButton" imageWidth="10%"/>

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_button/main.py'} language="python" />

<Image src={frontMatter.example_media + '/basic.png'} alt="basic" width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="CupertinoCheckbox" imageWidth="10%"/>

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_material_and_adaptive/main.py'} language="python" />

<Image src={frontMatter.example_media + '/cupertino_material_and_adaptive.png'} alt="cupertino-material-and-adaptive" width="80%" />

<CodeExample path={frontMatter.examples + '/styled/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_context_menu/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.gif'} alt="basic" width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_date_picker/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="55%" />

import {ClassAll, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/file_deletion_confirmation/main.py'} language="python" />

<Image src={frontMatter.example_images + '/file_deletion_confirmation.png'} alt="file-deletion-confirmation" width="40%" />

<CodeExample path={frontMatter.examples + '/cupertino_material_and_adaptive/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CupertinoFilledButton” imageWidth=“10%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/cupertino_filled_button/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/basic.png’} alt=“basic” width=“25%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CupertinoListTile” />

# Examples

<CodeExample path={frontMatter.examples + ‘/notched/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/notched.png’} alt=“notched” width=“60%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/cupertino_navigation_bar/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/wired/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/fruit_selection/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/fruit_selection.gif’} alt=“fruit-selection” width=“35%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CupertinoRadio” imageWidth=“10%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/cupertino_material_and_adaptive/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/cupertino_material_and_adaptive.png’} alt=“cupertino-material-and-adaptive” width=“40%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CupertinoSegmentedButton” imageWidth=“20%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/cupertino_segmented_button/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/basic.gif’} alt=“basic” width=“40%” />

<CodeExample path={frontMatter.examples + ‘/segments_padding/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“CupertinoSlider” imageWidth=“20%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/handling_events/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/handling_events.gif’} alt=“handling-events” width=“40%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="CupertinoSlidingSegmentedButton" imageWidth="25%"/>

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_sliding_segmented_button/main.py'} language="python" />

<Image src={frontMatter.example_media + '/basic.gif'} alt="basic" width="30%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="CupertinoSwitch" imageWidth="8%"/>

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_material_and_adaptive/main.py'} language="python" />

<Image src={frontMatter.example_media + '/cupertino_material_and_adaptive.gif'} alt="cupertino-material-and-adaptive" width="70%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="CupertinoTextField" imageWidth="40%"/>

# Examples

<CodeExample path={frontMatter.examples + '/cupertino_material_and_adaptive/main.py'} language="python" />

<Image src={frontMatter.example_media + '/cupertino_material_and_adaptive.png'} alt="cupertino-material-and-adaptive" width="70%" />

<CodeExample path={frontMatter.examples + '/selection_change/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/background_image/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Simple Cupertino timer picker" imageWidth="70%"/>

## Examples

<CodeExample path={frontMatter.examples + '/cupertino_timer_picker/main.py'} language="python" />

<Image src={frontMatter.example_media + '/basic.gif'} alt="basic" width="35%" />

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="CupertinoTintedButton" imageWidth="10%"/>

import {ClassAll, CodeExample, Image} from '@site/src/components/crocodocs';

import {ClassAll, CodeExample, Image} from '@site/src/components/crocodocs';

import {ClassAll, CodeExample, Image} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample, Image} from '@site/src/components/crocodocs';

# DataTable2

Enhanced data table for Flet that adds sticky headers, fixed rows/columns, and other UX improvements via the `flet-datatable2` extension.

It wraps the Flutter `data_table_2` package.

## Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

## Usage

Add `flet-datatable2` to your project dependencies:

```
uv add flet-datatable2

pip install flet-datatable2  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`.

# Examples

<CodeExample path={frontMatter.examples + ‘/empty_state/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/empty_state.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/sortable_and_selectable/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/sortable_and_selectable.gif’} width=“65%” />

<CodeExample path={frontMatter.examples + ‘/column_widths/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/column_widths.png’} width=“65%” />

# Description

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“DataTable” imageWidth=“25%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/data_table/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} width=“40%” />

<CodeExample path={frontMatter.examples + ‘/spacing/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/spacing_flow.gif’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/adaptive_row_heights/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/adaptive_row_heights.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/sortable_and_selectable/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/sortable_and_selectable.png’} width=“55%” />

<CodeExample path={frontMatter.examples + '/handling_events/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handling_events_flow.gif'} width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Date picker" imageWidth="45%"/>

## Examples

<CodeExample path={frontMatter.examples + '/date_picker/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="45%" />

<CodeExample path={frontMatter.examples + '/custom_locale/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_locale.png'} alt="custom-locale" width="45%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Date range picker" imageWidth="55%"/>

## Examples

<CodeExample path={frontMatter.examples + '/date_range_picker/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="55%" />

<CodeExample path={frontMatter.examples + '/custom_locale/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_locale.png'} alt="custom-locale" width="55%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/dismissible_list_tiles/main.py'} language="python" />

<Image src={frontMatter.example_images + '/dismissible_list_tiles.gif'} alt="dismissible-list-tiles" width="30%" />

### Remove `Dismissible` `on_dismiss` inside component

Always specify a key for `Dismissible` when using inside Flet component.

The issue you may encounter here is specific to the `Dismissible` control used inside Flet component (declarative UI).

When a user swipes (dismisses) an item, that widget is marked as "dismissed" on the Flutter side and effectively removed from the UI. However, when Flet recalculates the UI diff on the Python side, it may attempt to reuse widgets in the list based on their order rather than their identity.

If no key is provided, Flet's diffing algorithm can't tell that a particular `Dismissible` corresponds to a specific item — so it assumes the items have merely shifted. That leads to update commands like:

> "Update text in items 0…N-1, then delete the last item (N)."

On Flutter's side, though, the already-dismissed `Dismissible` widget in the middle of the list can't be updated — it's gone — causing runtime errors.

**Always assign a stable, unique key to each `Dismissible`, typically based on the item's identifier or index.**

<CodeExample displayTitle={false} path={frontMatter.examples + '/remove_on_dismiss_declarative/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Divider" imageWidth="30%"/>

# Examples

<CodeExample path={frontMatter.examples + '/divider/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="20%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/drag_and_drop_containers/main.py'} language="python" />

<Image src={frontMatter.example_images + '/drag_and_drop_containers.gif'} alt="drag-and-drop-containers" width="40%" />

<CodeExample path={frontMatter.examples + '/drag_and_drop_containers_declarative/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/drag_and_drop_containers/main.py'} language="python" />

<Image src={frontMatter.example_images + '/drag_and_drop_containers.gif'} alt="drag-and-drop-containers" width="40%" />

<CodeExample path={frontMatter.examples + '/drag_and_drop_containers_declarative/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Dropdown

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Dropdown" imageWidth="25%"/>

## Examples

<CodeExample path={frontMatter.examples + '/color_selection_with_filtering/main.py'} language="python" />

<Image src={frontMatter.example_images + '/color_selection_with_filtering_flow.gif'} alt="color-selection-with-filtering" width="35%" />

<CodeExample path={frontMatter.examples + '/icon_selection/main.py'} language="python" />

<Image src={frontMatter.example_images + '/icon_selection_flow.gif'} alt="icon-selection" width="35%" />

<CodeExample path={frontMatter.examples + '/declarative/main.py'} language="python" />

<Image src={frontMatter.example_images + ‘/declarative_flow.gif’} alt=“declarative-dropdown” width=“35%” />

<CodeExample path={frontMatter.examples + ‘/select_and_change_events/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/select_and_change_events_flow.gif’} alt=“select-and-change-events” width=“35%” />

<CodeExample path={frontMatter.examples + ‘/styled/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/styled_flow.gif’} alt=“styled-dropdowns” width=“35%” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“ExpansionPanelList” imageWidth=“45%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/expansion_panel_list/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic_flow.gif’} width=“40%” />

<CodeExample path={frontMatter.examples + ‘/scrollable/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/scrollable.png’} width=“40%” />

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“ExpansionPanel” imageWidth=“45%”/>

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“ExpansionTile” />

# Examples

<CodeExample path={frontMatter.examples + ‘/expansion_tile/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} width=“30%” />

<CodeExample path={frontMatter.examples + ‘/programmatic_expansion/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/programmatic_expansion_flow.gif’} width=“50%” />

<CodeExample path={frontMatter.examples + ‘/custom_animations/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/custom_animations_default.gif’} width=“50%” />

<CodeExample path={frontMatter.examples + ‘/theme_mode_toggle/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/borders/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/borders_flow.gif’} width=“50%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“FilledButton” imageWidth=“13%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/filled_button/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“25%” />

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“FilledIconButton” imageWidth=“5%”/>

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“FilledTonalButton” imageWidth=“13%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/filled_tonal_button/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“25%” />

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="FilledTonalIconButton" imageWidth="5%"/>

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="FloatingActionButton" imageWidth="7%"/>

# Examples

<CodeExample path={frontMatter.examples + '/handling_clicks/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handling_clicks.gif'} alt="handling-clicks" width="40%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

Solitaire game tutorial

<CodeExample path={frontMatter.examples + '/handling_events/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/draggable_containers/main.py'} language="python" />

<Image src={frontMatter.example_images + '/draggable_containers.gif'} alt="draggable-containers" width="40%" />

<CodeExample path={frontMatter.examples + '/window_drag_area/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/mouse_cursors/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="GridView" imageWidth="20%"/>

# Examples

<CodeExample path={frontMatter.examples + ‘/photo_gallery/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/photo_gallery.png’} alt=“photo-gallery” width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/hero/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.gif’} width=“40%” />

<CodeExample path={frontMatter.examples + ‘/gallery/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“IconButton” imageWidth=“6%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/handling_clicks/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/handling_clicks.gif’} alt=“handling-clicks” width=“40%” />

<CodeExample path={frontMatter.examples + ‘/selected_icon/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/selected_icon.gif’} alt=“selected-icon” width=“20%” />

<CodeExample path={frontMatter.examples + ‘/variants/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/variants.png’} alt=“variants” width=“30%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Icon” imageWidth=“5%”/>

# Examples

To browse and visualize all available icons, visit our icons browser

<CodeExample path={frontMatter.examples + '/icon/main.py'} language="python" />

<Image src={frontMatter.example_media + '/basic.png'} alt="basic" width="25%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Image" imageWidth="10%" />

# Examples

<CodeExample path={frontMatter.examples + '/gallery/main.py'} language="python" />

<Image src={frontMatter.example_media + '/gallery.gif'} width="45%" />

<CodeExample path={frontMatter.examples + '/fade_in/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/src_base64_and_bytes/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/static_svg/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/dynamic_svg/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/lucide_icons/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/gapless_playback/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/handling_events/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/transformations/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + ‘/detect_keys/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/flip/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/flip.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/rotate/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/rotate.png’} width=“55%” />



<CodeExample path={frontMatter.examples + ‘/scale/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/scale.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/offset/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/offset.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/matrix4_transform/main.py’} language=“python” />

<Image src={frontMatter.example_images_examples + ‘/matrix4_transform.png’} width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“ListTile” />

# Examples

<CodeExample path={frontMatter.examples + ‘/list_tile/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“ListView” imageWidth=“10%”/>

## Examples

<CodeExample path={frontMatter.examples + ‘/autoscroll_and_dynamic_items/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/autoscroll_and_dynamic_items.gif’} alt=“autoscroll-and-dynamic-items” width=“40%” />

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’; import {ClassAll, CodeExample} from ‘@site/src/components/crocodocs’;

# Lottie

Render rich Lottie animations inside your Flet apps with a simple control.

It is backed by the lottie Flutter package.

## Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

## Usage

Add the `flet-lottie` package to your project dependencies:

```
uv add flet-lottie

pip install flet-lottie  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Example

<CodeExample path={frontMatter.examples + ‘/lottie_animations/main.py’} language=“python” />

## Description

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’; import {CodeExample} from ‘@site/src/components/crocodocs’;

Display interactive maps in your Flet apps with markers, overlays, and rich attributions provided by the `flet-map` extension. The control is built on top of `flutter_map` and supports multiple tile providers and layers.

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

# Usage

Add the `flet-map` package to your project dependencies:

```
uv add flet-map

pip install flet-map  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. :::danger[Important] Different tile providers have their own usage policies. Make sure you fully comply with their requirements (ex: attribution, rate limits) when using them in your app, to avoid being blocked or facing legal issues. More details here. :::

# Examples

<CodeExample path={frontMatter.examples + ‘/map/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/attribution/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/camera_controls/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/idle_camera/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/interaction_flags/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/multi_layers/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/overlay_images/main.py’} language=“python” />

# Reference

- Map
  - Layers: TileLayer, MarkerLayer, OverlayImageLayer, CircleLayer, PolygonLayer, PolylineLayer
  - Markers and overlays: Marker, CircleMarker, PolygonMarker, PolylineMarker, OverlayImage, RotatedOverlayImage
  - Attributions: SimpleAttribution, RichAttribution, SourceAttribution

See the types section for additional configuration helpers.

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/overlay_images/main.py'} language="python" />

import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/overlay_images/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/overlay_images/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

```
import {ClassAll} from '@site/src/components/crocodocs';
import {ClassAll} from '@site/src/components/crocodocs';
import {ClassAll} from '@site/src/components/crocodocs';
```

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Markdown" imageWidth="30%"/>

# Examples

<CodeExample path={frontMatter.examples + '/markdown/main.py'} language="python" />

<Image src={frontMatter.example_media + '/basic.gif'} alt="basic" width="55%" />

<CodeExample path={frontMatter.examples + '/code_syntax_highlight/main.py'} language="python" />

<Image src={frontMatter.example_media + '/code_syntax_highlight.png'} alt="code-syntax-highlight" width="65%" />

<CodeExample path={frontMatter.examples + '/custom_text_theme/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="MenuBar" imageWidth="18%"/>

# Examples

<CodeExample path={frontMatter.examples + '/nested_submenus/main.py'} language="python" />

<Image src={frontMatter.example_images + '/nested_submenus.gif'} width="45%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Menu item buttons outside of menubar" imageWidth="25%"/>

# Examples

<CodeExample path={frontMatter.examples + '/menu_item_button/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.gif'} alt="basic" width="45%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Simple navigation bar" imageWidth="35%"/>

## Examples

<CodeExample path={frontMatter.examples + '/navigation_bar/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="40%" />

Adaptive navigation example switching between a navigation bar and a navigation rail with an end drawer

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# NavigationDrawer

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Navigation drawer extended" />

## Examples

<CodeExample path={frontMatter.examples + '/position_start/main.py'} language="python" />

<Image src={frontMatter.example_images + '/position_start.gif'} alt="Start-aligned navigation drawer example" width="40%" />

<CodeExample path={frontMatter.examples + '/position_end/main.py'} language="python" />

<Image src={frontMatter.example_images + '/position_end.gif'} alt="End-aligned navigation drawer example" width="40%" />

<CodeExample path={frontMatter.examples + '/theming/main.py'} language="python" />

<Image src={frontMatter.example_images + '/theming.gif'} alt="Themed navigation drawer example" width="40%" />

<CodeExample path={frontMatter.examples + '/adaptive_navigation/main.py'} language="python" />

<Image src={frontMatter.example_images + '/adaptive_navigation.gif'} alt="Adaptive navigation example switching between a navigation bar and a navigation rail with an end drawer" width="55%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Navigation rail extended" imageWidth="12%"/>

# Examples

<CodeExample path={frontMatter.examples + '/navigation_rail/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="45%" />

 Adaptive navigation example switching between a navigation bar and a navigation rail with an end drawer

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Simple Outlined Button" imageWidth="25%"/>

# Examples

<CodeExample path={frontMatter.examples + '/outlined_button/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="25%" />

<CodeExample path={frontMatter.examples + '/handling_clicks/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handling_clicks.gif'} alt="handling-clicks" width="40%" />

<CodeExample path={frontMatter.examples + '/icons/main.py'} language="python" />

<Image src={frontMatter.example_images + '/icons.png'} alt="icons" width="35%" />

<CodeExample path={frontMatter.examples + '/custom_content/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_content.png'} alt="custom-content" width="35%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Pagelet" />

## Examples

<CodeExample path={frontMatter.examples + '/pagelet/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} width="55%" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/keyboard_events/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/device_orientation/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/app_exit_confirm_dialog/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/window_hidden_on_start/main.py'} language="python" />

If you need this feature when packaging a desktop app using `flet build`, see this.

<CodeExample path={frontMatter.examples + '/semantics_debugger/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/device_locale/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/page_view/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/programmatic_swipe/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Placeholder" />

# Examples

<CodeExample path={frontMatter.examples + '/placeholder/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="30%" />

import {ClassAll, ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Opened popup menu under button" imageWidth="11%"/>

# Examples

<CodeExample path={frontMatter.examples + '/popup_menu_button/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.gif'} alt="basic" width="35%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Fixed progress bar" />

# Examples

<CodeExample path={frontMatter.examples + '/determinate_and_indeterminate/main.py'} language="python" />

<Image src={frontMatter.example_media + '/determinate_and_indeterminate.gif'} alt="determinate-and-indeterminate" width="40%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Fixed progress ring” imageWidth=“10%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/determinate_and_indeterminate/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/determinate_and_indeterminate.gif’} alt=“determinate-and-indeterminate” width=“40%” />

<CodeExample path={frontMatter.examples + ‘/gauge_with_progress/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/gauge_with_progress.png’} alt=“determinate-and-indeterminate” width=“13%” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Simple radio buttons” imageWidth=“80%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/radio/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.gif’} alt=“basic” width=“40%” />

<CodeExample path={frontMatter.examples + ‘/handling_selection_changes/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/handling_selection_changes.gif’} alt=“handling-selection-changes” width=“40%” />

<CodeExample path={frontMatter.examples + ‘/styled/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/styled.gif’} alt=“styled” width=“50%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Range Slider” />

# Examples

<CodeExample path={frontMatter.examples + '/range_slider/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.gif'} alt="basic" width="55%" />

<CodeExample path={frontMatter.examples + '/handling_change_events/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handling_events.gif'} alt="handling_events" width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="RawImage" imageWidth="30%" />

## `RawImage` vs `Image`

Use `Image` for pictures that come from a file, URL, asset or a one-off byte string. Use `RawImage` when your Python code *produces* pixels — Pillow drawings, NumPy arrays, camera frames, plots, procedural animations — and you want to push them to the screen repeatedly and fast.

Every update of `Image.src` travels through the regular Flet protocol and is decoded from scratch on the client. `RawImage` instead streams frames over a dedicated data channel: bytes skip the protocol entirely and, when the client runs on the same machine (desktop app, `flet run`, Pyodide), frames are transferred as raw RGBA pixels and uploaded straight to a GPU texture — no image encoding or decoding on either side. Remote web clients automatically receive compact PNG frames instead.

The `render` methods are awaitable and resolve when the client has displayed the frame, so a plain loop self-paces to display speed:

```
raw_image = ft.RawImage(expand=True)
page.add(raw_image)

while True:
    await raw_image.render(produce_pil_image())
```

# Examples

### Photo viewer

Regular PNG/JPEG/WebP bytes — downloaded, read from a file or pulled from a database — are displayed with `render_encoded`; the client decodes them with its image codecs.

<CodeExample path={frontMatter.examples + '/photo_viewer/main.py'} language="python" />

### Plasma animation

Streams a procedurally generated plasma effect with a live FPS counter and a render-resolution slider.

<CodeExample path={frontMatter.examples + '/plasma/main.py'} language="python" />

### Pillow paint

An interactive paint app: pan gestures draw brush strokes onto a Pillow image that is streamed to the screen through a dirty-flag render loop.

<CodeExample path={frontMatter.examples + '/paint/main.py'} language="python" />

### Mandelbrot explorer

Click to zoom into the Mandelbrot set — every zoom is a burst of NumPy-rendered frames computed in a background thread.

<CodeExample path={frontMatter.examples + '/mandelbrot/main.py'} language="python" />

### Game of Life

Conway's Game of Life on a tiny grid upscaled with crisp nearest-neighbor filtering; draw cells with the pointer while the simulation runs.

<CodeExample path={frontMatter.examples + '/game_of_life/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Reorderable list with custom drag handles" imageWidth="40%"/>

## Examples

<CodeExample path={frontMatter.examples + '/custom_drag_handle/main.py'} language="python" />

<Image src={frontMatter.example_media + '/basic.gif'} alt="basic" width="55%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Reorderable list view" imageWidth="70%"/>

## Examples

<CodeExample path={frontMatter.examples + '/horizontal_and_vertical/main.py'} language="python" />

<Image src={frontMatter.example_media + '/horizontal_and_vertical.gif'} alt="horizontal-and-vertical" width="65%" />

 custom-drag-handle

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Responsive row" imageWidth="80%"/>

## Examples

<CodeExample path={frontMatter.examples + '/responsive_row/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.gif'} width="80%" />

<CodeExample path={frontMatter.examples + '/custom_breakpoint/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_breakpoint.gif'} width="70%" />

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

# Rive

Render Rive animations in your Flet app with the `flet-rive` extension.

## Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ (x64 only) | ✅ | ✅ | ✅ |

# Usage

Add `flet-rive` to your project dependencies:

```
uv add flet-rive

pip install flet-rive  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. :::tip[Hosting Rive files] Host `.riv` files locally or load them from a CDN. Use `placeholder` to keep layouts responsive while animations load. :::

# Example

<CodeExample path={frontMatter.examples + '/rive_animations/main.py'} language="python" />

# Description

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# Example

<CodeExample path={frontMatter.examples + '/rotated_box/main.py'} language="python" />

<Image src={frontMatter.example_images + '/rotated_box.png'} width="55%" />

import {ClassSummary, ClassMembers, CodeExample} from '@site/src/components/crocodocs';

`Router` matches the current page route against a tree of Route definitions and renders the matched component chain with nested outlet contexts.

Navigation is done via page.navigate() or page.push_route().

# Examples

<CodeExample path={frontMatter.examples + '/routing/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/layout_outlet/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/dynamic_segments/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/loaders/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/active_links/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/featured/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/nested_routes/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/nested_outlet_views/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/featured_views/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/modal_routes/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/recursive_routes/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Row" imageWidth="60%"/>

# Examples

<CodeExample path={frontMatter.examples + '/spacing/main.py'} language="python" />

<Image src={frontMatter.example_images + '/row_spacing_adjustment.gif'} alt="spacing" width="70%" />

<CodeExample path={frontMatter.examples + '/wrap/main.py'} language="python" />

<Image src={frontMatter.example_images + '/wrap_adjustment.gif'} alt="wrap" width="70%" />

<CodeExample path={frontMatter.examples + '/alignment/main.py'} language="python" />

<Image src={frontMatter.example_images + '/alignment.png'} alt="alignment" width="28%" />

<CodeExample path={frontMatter.examples + '/vertical_alignment/main.py'} language="python" />

<Image src={frontMatter.example_images + '/vertical_alignment.png'} alt="vertical-alignment" width="23%" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Example

<CodeExample path={frontMatter.examples + '/safe_area/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/taking_screenshot/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="SearchBar" imageWidth="70%"/>

## Examples

<CodeExample path={frontMatter.examples + '/search_bar/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="45%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

# SegmentedButton

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="SegmentedButton" />

## Examples

<CodeExample path={frontMatter.examples + '/single_multiple_selection/main.py'} language="python" />

<Image src={frontMatter.example_images + '/single_multiple_selection.png'} width="55%" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Selectable and unselectable text” imageWidth=“15%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/selection_area/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/basic.gif’} alt=“basic” width=“30%” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/semantics/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Linear gradient mask” />

# Examples

<CodeExample path={frontMatter.examples + ‘/pink_radial_glow/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/pink_radial_glow.png’} alt=“pink-radial-glow” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/fade_out_image_bottom/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/fade_out_image_bottom.png’} alt=“fade-out-image-bottom” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/linear_and_radial_gradients/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/linear_and_radial_gradients.png’} alt=“fade-out-image-bottom” width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.gif'} imageCaption="Shimmer" imageWidth="30%"/>

# Examples

<CodeExample path={frontMatter.examples + '/shimmer/main.py'} language="python" />

<Image src={frontMatter.example_images + '/image_for_docs.gif'} alt="custom-label" width="30%" />

<CodeExample path={frontMatter.examples + '/shimmer_basic_placeholder/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic_placeholder.png'} alt="custom-label" width="30%" />

<CodeExample path={frontMatter.examples + '/custom_gradient/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_gradient.png'} alt="custom-label" width="30%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Slider" imageWidth="70%"/>

# Examples

<CodeExample path={frontMatter.examples + '/slider/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="20%" />

<CodeExample path={frontMatter.examples + '/custom_label/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_label.png'} alt="custom-label" width="30%" />

<CodeExample path={frontMatter.examples + '/handling_events/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handling_events.png'} alt="handling-events" width="25%" />

<CodeExample path={frontMatter.examples + '/random_values/main.py'} language="python" />

<Image src={frontMatter.example_media + ‘/random_values.gif’} alt=“random-values” width=“65%” />

import {ClassAll, ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Opened snack bar” imageWidth=“30%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/snack_bar/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“30%” />

<CodeExample path={frontMatter.examples + ‘/counter/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/snack_bar_flow.gif’} alt=“Snack bar with counter” width=“30%” caption=“Snack bar with counter” />

<CodeExample path={frontMatter.examples + ‘/action/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/action_simple.png’} alt=“Snack bar with a simple action” width=“50%” caption=“Snack bar with a simple action” />

<Image src={frontMatter.example_images + ‘/action_custom.png’} alt=“Snack bar with a custom action” width=“50%” caption=“Snack bar with a custom action” />

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {CodeExample} from '@site/src/components/crocodocs';

# SpinKit

30 animated loading spinner controls built on Flutter's `flutter_spinkit` package.

## Usage

Add `flet-spinkit` to your project dependencies:

```
uv add flet-spinkit

pip install flet-spinkit  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`.

Then import the package in your Flet app:

```
import flet_spinkit as spins
```

### Boot screen

`flet-spinkit` also provides a boot screen named `spinkit` — an animated loader shown while your packaged app is starting up. Once `flet-spinkit` is a dependency, select it in `pyproject.toml`:

```
[tool.flet.boot_screen]
name = "spinkit"

[tool.flet.boot_screen.spinkit]
spinner = "WanderingCubes"          # any SpinKit animation (without the "SpinKit" prefix)
spinner_color_light = "#7c4dff"
spinner_color_dark = "#b388ff"
spinner_size = 60
bgcolor_light = "#ffffff"
bgcolor_dark = "#0d0d12"
prepare_message = "Preparing your app…"
startup_message = "Starting up…"
```

All options are optional:

| Option | Default | Description |
|---|---|---|
| `spinner` | `WanderingCubes` | Any SpinKit animation, named without the `SpinKit` prefix (e.g. `Wave`, `FadingCube`, |

| Option | Default | Description |
|---|---|---|
| | | `PouringHourGlass`). Case-insensitive. |
| `theme_mode` | `auto` | `auto` (follow device), `light`, or `dark`. |
| `bgcolor_light` / `bgcolor_dark` | Flet theme background | Background color. |
| `spinner_color_light` / `spinner_color_dark` | Flet theme primary | Spinner color. |
| `spinner_size` | `60` | Spinner size in logical pixels. |
| `text_color_light` / `text_color_dark` | Flet theme on-surface | Message text color. |
| `prepare_message` | none | Text shown while unpacking the app (Android only). |
| `startup_message` | none | Text shown while the Python runtime and app start. |
| `fade_out_duration` | `0` | Fade-out duration in milliseconds when the app becomes ready; `0` removes it instantly. |

This boot screen is a complete, real-world example of the `createBootScreen` extension hook — see its source under `flet-spinkit`'s `src/flutter/flet_spinkit/lib/src/boot_screen.dart`.

# Examples

<CodeExample path={frontMatter.examples + '/spinkit_showcase/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/spinkit_props/main.py'} language="python" />

# Available controls

- ChasingDots
- Circle
- CubeGrid
- DancingSquare
- DoubleBounce
- DualRing
- FadingCircle
- FadingCube
- FadingFour
- FadingGrid
- FoldingCube
- HourGlass
- PianoWave

- PouringHourGlass
- PouringHourGlassRefined
- Pulse
- PulsingGrid
- PumpingHeart
- Ring
- Ripple
- RotatingCircle
- RotatingPlain
- SpinningCircle
- SpinningLines
- SquareCircle
- ThreeBounce
- ThreeInOut
- WanderingCubes
- Wave
- WaveSpinner

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Stack” />

# Examples

<CodeExample path={frontMatter.examples + ‘/online_avatar/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/online_avatar.png’} alt=“online-avatar” width=“8%” />

<CodeExample path={frontMatter.examples + ‘/absolute_positioning/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/absolute_positioning.png’} alt=“absolute-positioning” width=“20%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Activated submenu button” imageWidth=“25%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/submenu_button/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/standalone/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/standalone.gif’} alt=“standalone” width=“35%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Switch” imageWidth=“20%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/switch/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“40%” />

<CodeExample path={frontMatter.examples + ‘/handling_events/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/handling_events.gif’} alt=“handling-events” width=“40%” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Tabs” imageWidth=“70%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/tabs/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/basic.gif’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/nested/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/dynamic_tab_addition/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/custom_indicator/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/move_to/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Simple text button” imageWidth=“20%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/text_button/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/basic.png’} alt=“basic” width=“20%” />

<CodeExample path={frontMatter.examples + ‘/icons/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/icons.png’} alt=“icons” width=“30%” />

<CodeExample path={frontMatter.examples + '/handling_clicks/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handling_clicks.png'} alt="handling-clicks" width="30%" />

<CodeExample path={frontMatter.examples + '/custom_content/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_content.png'} alt="custom-content" width="30%" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="TextField" imageWidth="30%"/>

## Examples

<CodeExample path={frontMatter.examples + '/text_field/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="30%" />

<CodeExample path={frontMatter.examples + '/handling_change_events/main.py'} language="python" />

<Image src={frontMatter.example_images + '/handling_change_events.gif'} alt="handling-change-events" width="55%" />

<CodeExample path={frontMatter.examples + '/selection_change/main.py'} language="python" />

<Image src={frontMatter.example_images + '/selection_change.gif'} alt="selection-change" width="65%" />

<CodeExample path={frontMatter.examples + '/password/main.py'} language="python" />

<Image src={frontMatter.example_images + '/password.gif'} alt="password" width="45%" />

<CodeExample path={frontMatter.examples + '/multiline/main.py'} language="python" />

<Image src={frontMatter.example_images + '/multiline.gif'} alt="multiline" width="55%" />

<CodeExample path={frontMatter.examples + '/underlined_and_borderless/main.py'} language="python" />

<Image src={frontMatter.example_images + '/underlined_and_borderless.gif'} alt="underlined-and-borderless" width="55%" />

<CodeExample path={frontMatter.examples + ‘/prefix_and_suffix/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/prefix_and_suffix.gif’} alt=“prefix-and-suffix” width=“55%” />

<CodeExample path={frontMatter.examples + ‘/styled/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/styled.gif’} alt=“styled” width=“45%” />

<CodeExample path={frontMatter.examples + ‘/label_hint_helper_counter/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/label_hint_helper_counter.gif’} alt=“label-hint-helper-counter” width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + ‘/image_for_docs.png’} imageCaption=“Text” imageWidth=“30%”/>

# Examples

<CodeExample path={frontMatter.examples + ‘/custom_styles/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/custom_styles.gif’} width=“40%” />

<CodeExample path={frontMatter.examples + ‘/text_theme_styles/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/text_theme_styles.png’} width=“40%” />

<CodeExample path={frontMatter.examples + ‘/variable_font_weight/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/variable_font_weight.gif’} width=“40%” />

<CodeExample path={frontMatter.examples + ‘/rich_text/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/rich_text_basic.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/rich_text_border_stroke/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/rich_text_border_stroke.png’} width=“55%” />

<CodeExample path={frontMatter.examples + ‘/rich_text_gradient/main.py’} language=“python” />

<Image src={frontMatter.example_media + ‘/rich_text_gradient.png’} width=“55%” />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Time picker" imageWidth="60%"/>

## Examples

<CodeExample path={frontMatter.examples + '/time_picker/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.gif'} width="55%" />

<CodeExample path={frontMatter.examples + '/hour_formats/main.py'} language="python" />

<Image src={frontMatter.example_images + '/hour_formats.gif'} width="55%" />

<CodeExample path={frontMatter.examples + '/custom_locale/main.py'} language="python" />

<Image src={frontMatter.example_images + '/custom_locale.png'} width="55%" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Example

## Basic Example

<CodeExample path={frontMatter.examples + '/transparent_pointer/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Vertical divider" imageWidth="30%"/>

## Examples

<CodeExample path={frontMatter.examples + '/vertical_divider/main.py'} language="python" />

<Image src={frontMatter.example_images + '/basic.png'} alt="basic" width="45%" />

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

Embed a full-featured video player in your Flet app with playlist support, hardware acceleration controls, and subtitle configuration.

It is powered by the media_kit Flutter package.

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

# Usage

Add the `flet-video` package to your project dependencies:

```
uv add flet-video

pip install flet-video  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Requirements

The below sections show the required configurations for each platform.

## Android

**No permissions are required** to play media from a URL, from a bundled asset, or from a file the user picked with `FilePicker` — the system picker grants access to the picked file.

You only need media permissions if your app reads media **directly from the device's shared media library**, without the user picking each file:

- `android.permission.READ_MEDIA_AUDIO`: Allows to read audio files from external storage. Android 13 or higher.
- `android.permission.READ_MEDIA_VIDEO`: Allows to read video files from external storage. Android 13 or higher.
- `android.permission.READ_EXTERNAL_STORAGE`: Allows reading from external storage. Android 12 or lower — bound with `maxSdkVersion` so that it is not requested on Android 13 or higher.

:::danger[Google Play policy] Google Play's Photo and Video Permissions policy **rejects** apps targeting Android 13 (API 33) or higher that declare `READ_MEDIA_IMAGES` or `READ_MEDIA_VIDEO` when the system pickers are sufficient. Declaring them in *any* active version code — including internal and closed testing tracks — is enough to block a release.

Use `FilePicker` unless you genuinely need library-wide access; in that case you must also submit the photo picker declaration form in the Play Console. :::

```
        flet build apk \
          --android-permissions
android.permission.READ_MEDIA_AUDIO=true \
          --android-permissions
android.permission.READ_MEDIA_VIDEO=true

        [tool.flet.android.permission]
        "android.permission.READ_MEDIA_AUDIO" = true
        "android.permission.READ_MEDIA_VIDEO" = true
        "android.permission.READ_EXTERNAL_STORAGE" = { maxSdkVersion =
"32" }
```

Use `PermissionHandler` to **request** permissions at runtime.

See also:

- setting Android permissions

### Linux

`libmpv` libraries must be installed and present on the machine running the app.

On Ubuntu/Debian, this can be done with:

```
        sudo apt install libmpv-dev mpv
```

If you encounter `libmpv.so.1` load errors, run:

```
        sudo apt update
        sudo apt install libmpv-dev libmpv2
        sudo ln -s /usr/lib/x86_64-linux-gnu/libmpv.so
/usr/lib/libmpv.so.1
```

## Examples

<CodeExample path={frontMatter.examples + '/video/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/playback/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/screenshot/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/playlist/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/events/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/subtitles/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/controls/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/controls_mode/main.py'} language="python" />

<CodeExample path={frontMatter.examples + '/button_bars/main.py'} language="python" />

# Description

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

`VideoVolumeButton` is currently rendered only by `MaterialDesktopVideoControls`.

import {ClassAll} from '@site/src/components/crocodocs';

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

Display web content in a WebView to be shown in your Flet apps.

It is powered by the webview_flutter and webview_flutter_web Flutter packages.

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ❌ | ✅ | ❌ | ✅ | ✅ | ✅ |

# Usage

Add `flet-webview` to your project dependencies:

```
uv add flet-webview

pip install flet-webview  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Example

<CodeExample path={frontMatter.examples + '/web_view/main.py'} language="python" />

# Troubleshooting

## NET::ERR_CLEARTEXT_NOT_PERMITTED Error

If you run into the NET::ERR_CLEARTEXT_NOT_PERMITTED error in Android, then the app you're using is trying to access a web page that wants to transmit cleartext or unsecured information. Android blocks apps from doing this in order to avoid compromising user data.

For more details, see this and this.

To fix it, your app's configuration (precisely, the manifest application attributes) needs to be modified as follows:

```
[tool.flet.android.manifest_application]
usesCleartextTraffic = "true"
```

## Description

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample, Image} from '@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image={frontMatter.example_images + '/image_for_docs.png'} imageCaption="Simple Window Drag Area" imageWidth="70%"/>

# Examples

<CodeExample path={frontMatter.examples + '/no_frame_window/main.py'} language="python" />

<Image src={frontMatter.example_images + '/no_frame_window.png'} alt="no-frame-window" width="55%" />

import {CodeExample, Image} from '@site/src/components/crocodocs';

Flet is based on Flutter which includes first-class framework support for accessibility in addition to that provided by the underlying operating system.

# Screen readers

For mobile, screen readers (TalkBack, VoiceOver) enable visually impaired users to get spoken feedback about the contents of the screen and interact with the UI via gestures on mobile and keyboard shortcuts on desktop. Turn on VoiceOver or TalkBack on your mobile device and navigate around your app.

For web, the following screen readers are currently supported:

Mobile Browsers:

- iOS - VoiceOver
- Android - TalkBack

Desktop Browsers:

- MacOS - VoiceOver
- Windows - JAWs & NVDA

Screen Readers users on web will need to toggle "Enable accessibility" button to build the semantics tree.

## Text

Use `Text.semantics_label` property to override default Text control semantics.

## Buttons

All buttons with text on them generate proper semantics.

Use `tooltip` property to add screen reader semantics for `IconButton`, `FloatingActionButton` and `PopupMenuButton` buttons.

### `TextField` and `Dropdown`

Use `TextField.label` and `Dropdown.label` properties to add screen reader semantics to those controls.

## Custom semantics

For any specific requirements use `Semantics` control.

## Debugging semantics

Set `Page.show_semantics_debugger` to `True` to show an overlay that shows the accessibility information reported by the framework.

You can implement a specific keyboard shortcut to conveniently toggle semantics debugger during app development:



import {Image} from '@site/src/components/crocodocs';

Flet framework allows you to develop adaptive apps, which means having a single codebase that will deliver a different look depending on the device's platform.

Below is an example of a very simple app that has a different look on iOS and Android platforms:

```
        import flet as ft

        def main(page):

            page.adaptive = True

            page.appbar = ft.AppBar(
                leading=ft.TextButton("New",
style=ft.ButtonStyle(padding=0)),
                title=ft.Text("Adaptive AppBar"),
                actions=[
                    ft.IconButton(ft.CupertinoIcons.ADD,
style=ft.ButtonStyle(padding=0))
                ],
                bgcolor=ft.Colors.with_opacity(0.04,
ft.CupertinoColors.SYSTEM_BACKGROUND),
            )

            page.navigation_bar = ft.NavigationBar(
                destinations=[
                    ft.NavigationBarDestination(icon=ft.Icons.EXPLORE,
label="Explore"),
                    ft.NavigationBarDestination(icon=ft.Icons.COMMUTE,
label="Commute"),
                    ft.NavigationBarDestination(
```

```
                        icon=ft.Icons.BOOKMARK_BORDER,
                        selected_icon=ft.Icons.BOOKMARK,
                        label="Bookmark",
                    ),
                ],
                border=ft.Border(

top=ft.BorderSide(color=ft.CupertinoColors.SYSTEM_GREY2, width=0)
                ),
            )

            page.add(
                ft.SafeArea(
                    ft.Column(
                        [
                            ft.Checkbox(value=False, label="Dark
Mode"),
                            ft.Text("First field:"),

ft.TextField(keyboard_type=ft.KeyboardType.TEXT),
                            ft.Text("Second field:"),

ft.TextField(keyboard_type=ft.KeyboardType.TEXT),
                            ft.Switch(label="A switch"),
                            ft.FilledButton(content=ft.Text("Adaptive
button")),
                            ft.Text("Text line 1"),
                            ft.Text("Text line 2"),
                            ft.Text("Text line 3"),
                        ]
                    )
                )
            )

        ft.run(main)
```

By setting just `page.adaptive = True` you can make your app look awesome on both iOS and Android devices:

| iPhone | Android |
| --- | --- |
|  |  |

# Material and Cupertino controls

Most of Flet controls are based on Material design.

There is also a number of iOS-style controls in Flet that are called Cupertino controls.

Cupertino controls usually have a matching Material control that has `adaptive` property which defaults to`False`. When using a Material control with `adaptive` property set to `True`, a different control will be created depending on the platform, for example:

```
ft.Checkbox(adaptive=True, value=True, label="Adaptive Checkbox")
```

Flet checks the value of `Page.platform` property and if it is `PagePlatform.IOS` or `ft.PagePlatform.MACOS`, Cupertino control will be created; in all other cases Material control will be created.

`adaptive` property can be set for an individual control or a container-controls (ex: `Row`, `Column`) that has children controls. If a container-control is made adaptive, all its children will be adaptive too, unless `adaptive` property is explicitly set to `False` for a child control.

▶ Material vs Cupertino

## Custom adaptive controls

While Flet offers a number of controls that will be adapted to a platform

automatically using their `adaptive` property, there will be cases when you need more specific adaptive UI presentation, for example, using different icon, background color, padding, etc., depending on the platform.

With Flet, you can create your own reusable custom controls in Python that will inherit from a Flet control and implement specific properties you need.

In the example below, we are creating a new `AdaptiveNavigationBarDestination` custom control that will be displaying different icon on iOS and Android, and use it as destination for the `NavigationBar`:

```
import flet as ft

class AdaptiveNavigationBarDestination(ft.NavigationBarDestination):
    def __init__(self, ios_icon, android_icon, label):
        super().__init__()
        self._ios_icon = ios_icon
        self._android_icon = android_icon
        self.label = label

    def build(self):
        # we can check for platform in build method because self.page is known
        self.icon = (
            self._ios_icon
            if self.page.platform == ft.PagePlatform.IOS
            or self.page.platform == ft.PagePlatform.MACOS
            else self._android_icon
        )
```

```
def main(page: ft.Page):
    page.adaptive = True

    page.navigation_bar = ft.NavigationBar(
        selected_index=2,
        destinations=[
            AdaptiveNavigationBarDestination(
                ios_icon=ft.CupertinoIcons.PERSON_3_FILL,
                android_icon=ft.Icons.PERSON,
                label="Contacts",
            ),
            AdaptiveNavigationBarDestination(
                ios_icon=ft.CupertinoIcons.CHAT_BUBBLE_2,
                android_icon=ft.Icons.CHAT,
                label="Chats",
            ),
            AdaptiveNavigationBarDestination(
                ios_icon=ft.CupertinoIcons.SETTINGS,
                android_icon=ft.Icons.SETTINGS,
                label="Settings",
            ),
        ],
    )

    page.update()

ft.run(main)
```

Now the `NavigationBar` and icons within it will look like different on Android and iOS:

| iOS | Android |
| --- | --- |
|  |  |

You may utilise reusable controls approach to adapt your app not only depending on the `Page.platform`, but also use `Page.web` property to have different UI depending on whether the app is running in a browser or not, or even combine the usage of both properties to have specific UI for your apps.

import {CodeExample, Image} from '@site/src/components/crocodocs';

# Implicit animations

With implicit animations, you can animate a control property by setting a target value; whenever that target value changes, the control animates the property from the old value to the new one.

Animation produces interpolated values between the old and the new value over the given *duration*.

By default, the animation is *linearly* increasing the animation value, however, a *curve* can be applied to the animation which changes the value according to the provided curve. For example, `AnimationCurve.EASE_OUT_CUBIC` curve increases the animation value quickly at the beginning of the animation and then slows down until the target value is reached:




`LayoutControl` (and its subclasses) provides a number of `animate_{something}` properties, described below, to enable implicit animation of its appearance:

- `animate_opacity`
- `animate_rotation`
- `animate_scale`
- `animate_offset`
- `animate_position`
- `animate` (`Container`)

`animate_*` properties could have one of the following values:

- Instance of `Animation` - allows configuring the duration and the curve of the
- animation, for example `animate_rotation=Animation(duration=300, curve=AnimationCurve.BOUNCE_OUT)`. See this Flutter docs on animation curves for possible values. Default is `AnimationCurve.LINEAR`.
- `int` value - enables animation with specified duration in milliseconds and `AnimationCurve.LINEAR` curve.
- `bool` value - enables animation with the duration of 1000 milliseconds and `AnimationCurve.LINEAR` curve.

## Opacity animation

Setting control's `animate_opacity` to either `True`, number or an instance of `Animation` class (see above) enables implicit animation of `Control.opacity` property.


animate-opacity

## Rotation animation

Setting control's `animate_rotation` to either `True`, number or an instance of `Animation` class (see above) enables implicit animation of `LayoutControl.rotate` property.


animate-rotation

## Scale animation

Setting control's `animate_scale` to either `True`, number or an instance of `Animation` class (see above) enables implicit animation of `LayoutControl.scale` property.

 animate-scale

## Offset animation

Setting control's `animate_offset` to either `True`, number or an instance of `Animation` class (see above) enables implicit animation of `LayoutControl.offset` property.

`offset` property is an instance of `Offset` class which specifies horizontal `x` and vertical `y` offset of a control scaled to control's size. For example, an offset `Offset(-0.25, 0)` will result in a horizontal translation of one quarter the width of the control.

Offset animation is used for various sliding effects:

 animate-offset

## Position animation

Setting control's `animate_position` to either `True`, number or an instance of `Animation` class (see above) enables implicit animation of the following `LayoutControl` properties: `left`, `right`, `bottom`, `top`.

Note: Positioning is effective only if the control is a descendant of one of the following:

```
- [`Stack`](../controls/stack.md) control
- [`Page.overlay`](../controls/page.md) list
```

 animate-position

## Animate

Setting `Container.animate` to `AnimationValue` enables implicit animation of container properties such as size, background color, border style, gradient.

 animate

## Animated content switcher

`AnimatedSwitcher` allows animated transition between two controls ('new' and 'old').

```
        import time

        import flet as ft

        def main(page: ft.Page):
            i = ft.Image(src="https://picsum.photos/150/150",
width=150, height=150)
```

```
            def animate(e):
                sw.content = ft.Image(
                    src=f"https://picsum.photos/150/150?
{time.time()}", width=150, height=150
                )
                page.update()

            sw = ft.AnimatedSwitcher(
                i,
                transition=ft.AnimatedSwitcherTransition.SCALE,
                duration=500,
                reverse_duration=500,
                switch_in_curve=ft.AnimationCurve.BOUNCE_OUT,
                switch_out_curve=ft.AnimationCurve.BOUNCE_IN,
            )

            page.add(
                sw,
                ft.Button("Animate!", on_click=animate),
            )

        ft.run(main)
```




## Animation end callback

`LayoutControl` also has an `on_animation_end` event handler, which is called when an animation is complete. It can be used to chain multiple animations.

Event's `data` field/property contains the name of animation:

- `"opacity"`
- `"rotation"`
- `"scale"`
- `"offset"`
- `"position"`
- `"container"`

For example:

```
        ft.Container(
            content=ft.Text("Animate me!"),
            animate=ft.Animation(1000, ft.AnimationCurve.BOUNCE_OUT),
            on_animation_end=lambda e: print("Container animation
end:", e.data)
        )
```

Flet apps can include both code and assets/resources. An asset is a file that is bundled and deployed with your app and is accessible at runtime. Common types of assets include static data (e.g., JSON files), configuration files, icons, images, videos, etc.

To use relative paths for your asset files, you need to provide a path to your assets directory when launching your app with the `ft.run()` function. The parameter for this is called `assets_dir`, which defaults to `"assets"`. This parameter specifies the folder where local assets are stored and can be either an absolute path or a path relative to the app's entry point file, such as `main.py`.

## Example: Displaying a Local Image

Suppose you have a folder named `assets` in the same directory as your `main.py` file, and this folder contains an image file named `sample.png` in a subfolder called `images`:

```
assets
    images
        sample.png
main.py
```

To display this image in your app, you can do the following:

```python
import flet as ft

def main(page: ft.Page):
    page.add(
        ft.Image(src="images/sample.png")
    )

ft.run(main, assets_dir="assets")
```

The same approach applies to other asset types like fonts, Lottie animations, Rive files, etc.

## Accessing Asset Files in Production

For UI controls such as Image, you usually pass a path relative to `assets_dir`:

```python
ft.Image(src="images/sample.png")
```

For non-UI file operations (ex: reading JSON, opening a bundled SQLite database, loading ML models, etc.), it is often more convenient to use an absolute path to your assets directory. For this reason, Flet sets the FLET_ASSETS_DIR environment variable (in production apps built with `flet build`,excluding `web`) to an absolute path to your assets directory.

You can use it with a local-development fallback, as shown in the below example (which assumes the asset folder is named `"assets"`):

```python
import json
import os
from pathlib import Path

import flet as ft
```

```
        def get_assets_dir() -> Path:
            default_assets_dir = Path(__file__).parent / "assets"   #
fallback for local runs
            return Path(os.environ.get("FLET_ASSETS_DIR",
str(default_assets_dir))).resolve()

        def main(page: ft.Page):
            assets_dir = get_assets_dir()

            # load a JSON file
            with (assets_dir / "data" / "some_config.json").open() as
f:
                config = json.load(f)

            page.add(ft.Text(f"Loaded profile:
{config['profile_name']}"))

        ft.run(main, assets_dir="assets")
```

This gives you a single way to resolve bundled files across local runs and production builds. See FLET_ASSETS_DIR for details.

Flet app can be written as an async app and use `asyncio` and other Python async libraries. Calling coroutines is naturally supported in Flet, so you don't need to wrap them to run synchronously.

By default, Flet executes control event handlers in separate threads, but sometimes that could be an ineffective usage of CPU or it does nothing while waiting for a HTTP response or executing `sleep()`.

Asyncio, on the other hand, allows implementing concurrency in a single thread by switching execution context between "coroutines". This is especially important for apps that are going to be published as static websites using Pyodide. Pyodide is a Python runtime built as a WebAssembly (WASM) and running in the browser. At the time of writing it doesn't support threading yet.

# Getting started with async

You could mark `main()` method of Flet app as `async` and then use any asyncio API inside it:

```
        import flet as ft

        async def main(page: ft.Page):
            await asyncio.sleep(1)
            page.add(ft.Text("Hello, async world!"))

        ft.run(main)
```

You can use `await ft.run_async(main)` if Flet app is part of a larger app and called from `async` code.

# Control event handlers

Control event handlers could be both sync and `async`.

If a handler does not call any async methods it could be a regular sync method:

```
        def page_resize(e):
            print("New page size:", page.window.width,
page.window.height)

        page.on_resize = page_resize
```

However, if a handler calls async logic it must be async too:

```
        async def main(page: ft.Page):

            async def button_click(e):
                await some_async_method()
                page.add(ft.Text("Hello!"))

            page.add(ft.Button("Say hello!", on_click=button_click))

        ft.run(main)
```

### Async lambdas

There are no async lambdas in Python. It's perfectly fine to have a lambda event handler in async app for simple things:

```
        page.on_error = lambda e: print("Page error:", e.data)
```

but you can't have an async lambda, so an async event handler must be used.

# Sleeping

To delay code execution in async Flet app you should use `asyncio.sleep()` instead of `time.sleep()`, for example:

```
        import asyncio
        import flet as ft

        def main(page: ft.Page):
            async def button_click(e):
                await asyncio.sleep(1)
                page.add(ft.Text("Hello!"))

            page.add(
                ft.Button("Say hello with delay!",
on_click=button_click)
            )
```

```
ft.run(main)
```

# Threading

To run something in the background use page.run_task(). For example, “Countdown” custom control which is self-updating on background could be implemented as following:

```
import asyncio
import flet as ft

class Countdown(ft.Text):
    def __init__(self, seconds):
        super().__init__()
        self.seconds = seconds

    def did_mount(self):
        self.running = True
        self.page.run_task(self.update_timer)

    def will_unmount(self):
        self.running = False

    async def update_timer(self):
        while self.seconds and self.running:
            mins, secs = divmod(self.seconds, 60)
            self.value = "{:02d}:{:02d}".format(mins, secs)
            self.update()
            await asyncio.sleep(1)
            self.seconds -= 1

def main(page: ft.Page):
    page.add(Countdown(120), Countdown(60))

ft.run(main)
```



import {Image} from ‘@site/src/components/crocodocs’;

You can implement user authentication (“Login with X” buttons) in your Flet app using 3rd-party identity providers such as GitHub, Google, Azure, Auth0, LinkedIn and others.

Identity provider must support OAuth 2.0 Authorization Code Flow to retrieve API access token.

Built-in Flet login with credentials and user management is planned for future releases. If you have a requirement to create and manage user accounts in your app you can implement it by your own or use Auth0 identity provider which provides a generous free tier.

Flet authentication features:

- Works with Flet desktop, web and mobile apps.
- Using multiple authentication providers in one app.
- Built-in OAuth providers with automatic user details fetching:
  - GitHub
  - Azure
  - Google
  - Auth0
- Optional groups fetching.
- Automatic token refresh.
- Login with a saved token (“Remember me”).
- Custom OAuth providers.

## Login process overview

- Configure OAuth provider (built-in or generic) with Client ID, Client secret, Redirect URL.
- Call `page.login(provider)` to initiate OAuth web flow.
- User is being redirected to OAuth provider website.
- On provider website user signs in and gives consent to access service API with requested scopes.
- Provider website redirects to Flet’s OAuth callback URL with authorization code.
- Flet exchanges authorization code for a token and calls `page.on_login` event handler.
- Flet app can retrieve API token from `page.auth.token` property and user details from `page.auth.user`.

## Configuring OAuth provider

Flet has the following built-in OAuth providers:

- GitHub
- Azure
- Google
- Auth0

Additionally, you can configure a generic OAuth provider and provide authorization, token and user info endpoints.

In this guide we will configure Flet login page with GitHub account.

To integrate Flet authentication with GitHub a new GitHub OAuth app should be registered first (**Profile settings → Developer settings → OAuth Apps**).

**Authorization callback URL** should be in the format:

```
{application-url}/oauth_callback
```




On OAuth app details page click “Generate a new client secret” button. Copy “Client ID” and “Client secret” values to a safe place - you’ll need them in a Flet app.




# Sign in with OAuth provider

```
        import os

        import flet as ft
        from flet.auth.providers import GitHubOAuthProvider

        GITHUB_CLIENT_ID = os.getenv("GITHUB_CLIENT_ID")
        assert GITHUB_CLIENT_ID, "set GITHUB_CLIENT_ID environment
variable"
        GITHUB_CLIENT_SECRET = os.getenv("GITHUB_CLIENT_SECRET")
        assert GITHUB_CLIENT_SECRET, "set GITHUB_CLIENT_SECRET
environment variable"

        def main(page: ft.Page):
            provider = GitHubOAuthProvider(
                client_id=GITHUB_CLIENT_ID,
                client_secret=GITHUB_CLIENT_SECRET,
                redirect_url="http://localhost:8550/oauth_callback",
            )

            def login_click(e):
                page.login(provider)

            def on_login(e):
                print("Login error:", e.error)
                print("Access token:", page.auth.token.access_token)
                print("User ID:", page.auth.user.id)

            page.on_login = on_login
            page.add(ft.Button("Login with GitHub",
on_click=login_click))

        ft.run(main, port=8550, view=ft.WEB_BROWSER)
```

Notice, we are fetching OAuth app client ID and client secret from an environment variables. Do not embed any secrets into source code to avoid accidental exposure to a public!

Before running the app set the secret environment variables in a command line:

```
$ export GITHUB_CLIENT_ID="<client_id>"
$ export GITHUB_CLIENT_SECRET="<client_secret>"
```

Run the program and click “Login with GitHub” button. GitHub authorize app page will be opened in:

- **Desktop** app - a new browser window or tab.
- **Web** app - a new popup window (make sure popup blocker is off).

- **Mobile** app - an in-app web browser.




## Redirect URL

We used `http://localhost:8550/oauth_callback` as a redirect URL while registering GitHub OAuth app. Notice it has a fixed port `8550`. To run your Flet app on a fixed port use `port` argument in `flet.app` call:

```
ft.run(main, port=8550)
```

## Scope

Most of OAuth providers allows applications to request one or more scopes to limit application's access to a user's account.

Built-in Flet providers, by default, request scopes to access user profile, but you can request additional scopes in login method, like `public_repo` in the example above:

```
page.login(
    provider,
    scope=["public_repo"]
)
```

`page.login()` method has a number of arguments to control authentication process:

- `fetch_user` (bool) - whether to fetch user details into `page.auth.user`. Default is `True`.
- `fetch_groups` (bool) - whether to fetch user groups into `page.auth.user.groups`. Default is `False`.
- `scope` - a list of scopes to request.
- `saved_token` - a JSON snapshot of `page.auth.token` to restore authorization from. Token can be serialized with `page.auth.token.to_json()`, encrypted and saved in `page.client_storage`. See below.
- `on_open_authorization_url` - a callback to open a browser with authorization URL. See below.
- `complete_page_html` - a custom HTML contents of "You've been successfully authenticated. Close this page now" page.
- `redirect_to_page` (bool) - used with Flet web app only when authorization page is opened in the same browser tab.

The result of `page.login()` call is an instance of `Authorization` class with the following fields:

- **`token`** - OAuth token used to access provider's API. See below.
- **`user`** - user details with a mandatory `id` field and other fields specific to OAuth provider.
- **`provider`** - an instance of OAuth provider used for authorization.

A reference to the last authorization is saved in `page.auth` property.

If your app allows authorizations with multiple OAuth providers you can save authorizations in a session, for example:

```
page.session["github_auth"] = page.login(github_provider)
page.session["google_auth"] = page.login(google_provider)
```

## Checking authentication results

Upon successful or failed authorization `page.on_login` event handler is called.

Event handler argument `e` is an instance of `LoginEvent` class with the following properties:

- `error` (str) - OAuth error.
- `error_description` (str) - OAuth error description.

Authorization was successful if `error` is an empty string.

You can use this event handler to toggle signed in/out UI, for example:

```
import os

import flet as ft
from flet.auth.providers import GitHubOAuthProvider

def main(page: ft.Page):
    provider = GitHubOAuthProvider(
        client_id=os.getenv("GITHUB_CLIENT_ID"),
        client_secret=os.getenv("GITHUB_CLIENT_SECRET"),
        redirect_url="http://localhost:8550/oauth_callback",
    )

    def login_button_click(e):
        page.login(provider, scope=["public_repo"])

    def on_login(e: ft.LoginEvent):
        if not e.error:
            toggle_login_buttons()

    def logout_button_click(e):
        page.logout()

    def on_logout(e):
        toggle_login_buttons()

    def toggle_login_buttons():
        login_button.visible = page.auth is None
        logout_button.visible = page.auth is not None
        page.update()

    login_button = ft.Button("Login with GitHub",
on_click=login_button_click)
```

```
            logout_button = ft.Button("Logout",
on_click=logout_button_click)
            toggle_login_buttons()

            page.on_login = on_login
            page.on_logout = on_logout

            page.add(login_button, logout_button)

        ft.run(main, port=8550, view=ft.AppView.WEB_BROWSER)
```

## Accessing user details

If `page.login()` method is called with `fetch_user=True` (default) a user profile will be assigned to `page.auth.user`.

All built-in OAuth providers implement `user.id` property - unique user identifier - which value depends on the provider (a number, Guid or email) and can be used in your app as a user key.

The rest of user profile properties depends on provider and can be accessed with an indexer. For example, to print some properties of GitHub user:

```
        print("Name:", page.auth.user["name"])
        print("Login:", page.auth.user["login"])
        print("Email:", page.auth.user["email"])
```

## Using OAuth token

Upon successful authorization `page.auth.token` will contain OAuth token that can be used to access providers's API. Token object has the following properties:

- `access_token` - access token used as an authorization token in API request header.
- `scope` - token's scope.
- `token_type` - access token type, e.g. `Bearer`.
- `expires_in` - optional number of seconds when access token expires.
- `expires_at` - optional time (`time.time()` + `expires_in`) when access token expires.
- `refresh_token` - optional refresh token which is used to get a new access token, when the old one expires.

Usually, only `page.auth.token.access_token` is needed to call provider's API, for example to list user's GitHub repositories:

```
        import requests
        headers = {"Authorization": "Bearer
{}".format(page.auth.token.access_token)}
        repos_resp = requests.get("https://api.github.com/user/repos",
headers=headers)
        user_repos = json.loads(repos_resp.text)
```

```
        for repo in user_repos:
            print(repo["full_name"])
```

Do not save a reference to `page.auth.token` somewhere in your code, but rather call `page.auth.token` every time you need to grab access token. `page.auth.token` is a property which automatically refreshes OAuth token if/when it expires.

Correct code:

```
        access_token = page.auth.token.access_token
```

Wrong code:

```
        token = page.auth.token
        # some other code
        access_token = token.access_token # token could expire by this
moment
```

## Saving and restoring an auth token

To implement persistent login (“Remember me” checkbox on login page) you can save auth token in a client storage and use it to login next time a user opens your Flet app.

To serialize auth token to JSON:

```
        jt = page.auth.token.to_json()
```

Encrypt sensitive data before sending it to a client storage.

Flet includes utility methods for encrypting text data using symmetric algorithm (where the same key is used for encryption and decryption). They use Fernet implementation from cryptography package, which is AES 128 with some additional hardening, plus PBKDF2 to derive encryption key from a user passphrase.

To encrypt JSON token:

```
        import os
        from flet.security import encrypt, decrypt

        secret_key = os.getenv("MY_APP_SECRET_KEY")
        # returns base64-encoded string
        ejt = encrypt(jt, secret_key)
```

Notice, we are fetching a secret key (aka passphrase, password, etc.) from an environment variable. Do not embed any secrets into source code to avoid accidental exposure to a public!

Before running the app set the secret in a command line:

```
$ export MY_APP_SECRET_KEY="<secret>"
```

Now, encrypted value can be stored in a client storage:

```
await page.shared_preferences.set("myapp.auth_token", ejt)
```

Next time a user opens the app you can read encrypted token from a client storage and, if it exists, decrypt it and use in `page.login()` method:

```
ejt = await page.shared_preferences.get("myapp.auth_token")
if ejt:
    jt = decrypt(ejt, secret_key)
    page.login(provider, saved_token=jt)
```

See complete app example.

## Signing out

Calling `page.logout()` resets `page.auth` reference and triggers `page.on_logout` event handlers.

You can remove saved token in logout method, for example:

```
async def logout_button_click(e):
    await page.shared_preferences.remove(AUTH_TOKEN_KEY)
    page.logout()
```

See complete app example.

## Customizing authorization flow

By default, OAuth authorization flow is happening in either new browser window/tab (desktop app), browser popup window (web) or in-app web view (mobile).

Upon completion of authorization flow a user is redirected to Flet's OAuth callback page (`/oauth_callback`) which tries to close a browser window/tab with JavaScript and provides user with instructions to close the window manually if JavaScript close didn't work.

This section is applicable to Flet desktop and web apps only as in-app web view in mobile apps can be closed by Flet without relying on JavaScript.

You can customize the contents of "Authorization complete" page in `page.login()` method, for example:

```
complete_page_html = """
<!DOCTYPE html>
<html>
  <head>
    <title>Signed in to MyApp</title>
  </head>
<body>
  <script type="text/javascript">
      window.close();
  </script>
```

```
            <p>You've been successfully signed in! You can close this
tab or window now.</p>
        </body>
        </html>
        """

        page.login(
            provider,
            complete_page_html=complete_page_html,
        )
```

You can also change web app to open provider's authorization page in the same tab which might be more familiar to your users and save them from dealing with popup blockers:

```
        page.login(
            provider,
            on_open_authorization_url=lambda url:
asyncio.create_task(page.launch_url(url, web_window_name="_self")),
            redirect_to_page=True
        )
```

To open flow in a new tab (notice `_self` replaced with `_blank`):

```
        page.login(
            provider,
            on_open_authorization_url=lambda url:
asyncio.create_task(page.launch_url(url, web_window_name="_blank"))
        )
```

# Configuring a custom OAuth provider

You can configure any OAuth-compatible authentication provider in your app with `flet.auth.oauth_provider.OAuthProvider` class.

Following the instructions in LinkedIn Authorization Code Flow guide we are able to get all required parameters to configure LinkedIn OAuth provider to allow users of your Flet app to login with their LinkedIn accounts:

```python {9-16} import os

import flet from flet import Button, Page from flet.auth import OAuthProvider

def main(page: Page): provider = OAuthProvider(
client_id=os.getenv("LINKEDIN_CLIENT_ID"),
client_secret=os.getenv("LINKEDIN_CLIENT_SECRET"),
authorization_endpoint="https://www.linkedin.com/oauth/v2/authorization",
token_endpoint="https://www.linkedin.com/oauth/v2/accessToken",
user_endpoint="https://api.linkedin.com/v2/me", user_scopes=["r_liteprofile",
"r_emailaddress"], user_id_fn=lambda u: u["id"],
redirect_url="http://localhost:8550/oauth_callback", )
```

```
def login_click(e):
    page.login(provider)

def on_login(e):
    if e.error:
        raise RuntimeError(e.error)
    print("User ID:", page.auth.user.id)
    print("Access token:", page.auth.token.access_token)

page.on_login = on_login
page.add(Button("Login with LinkedIn", on_click=login_click))
```

flet.app(main, port=8550, view=flet.WEB_BROWSER)

```
Mandatory provider settings:

* `client_id`
* `client_secret`
* `authorization_endpoint`
* `token_endpoint`
* `redirect_url`

Similar to other examples client ID and client secret are fetched from
environment variables.

---
title: "Auto-update"
---

Flet automatically calls `page.update()` (or `.update()` on the
nearest isolated ancestor) at the end of every event handler and
`main()` function. This means you don't need to call `.update()`
yourself in most cases:

```python
import flet as ft

def main(page: ft.Page):
    def button_click(e):
        page.controls.append(ft.Text("Clicked!"))
        # no need to call page.update() — it happens automatically

    page.controls.append(ft.Button("Click me", on_click=button_click))
    # no need to call page.update() here either

ft.run(main)
```

If your event handler already calls `.update()` explicitly (e.g. code written for Flet 0.x), the automatic update is skipped to avoid a redundant double update.
```

# Disabling auto-update

You can disable auto-update for fine-grained control over when updates are sent to the client. Use `ft.context.disable_auto_update()` and `ft.context.enable_auto_update()` to toggle the behavior.

When called inside a handler, the setting applies to the current handler context only:

```
        import flet as ft

        def main(page: ft.Page):
            def add_many_items(e):
                ft.context.disable_auto_update()
                for i in range(100):
                    page.controls.append(ft.Text(f"Item {i}"))
                page.update()  # single update for all 100 items

            page.controls.append(ft.Button("Add items",
on_click=add_many_items))

        ft.run(main)
```

When called outside of event handlers (e.g. at the module level), it controls the global default for the entire app:

```
        import flet as ft

        # disable auto-update globally
        ft.context.disable_auto_update()

        def main(page: ft.Page):
            def button_click(e):
                page.controls.append(ft.Text("Clicked!"))
                page.update()  # must call explicitly since auto-
update is off

            page.controls.append(ft.Button("Click me",
on_click=button_click))
            page.update()

        ft.run(main)
```

Flet's client storage API allows storing key-value data on a client side in a persistent storage. Flet implementation uses `shared_preferences` Flutter package.

The actual storage mechanism depends on a platform where Flet app is running:

- Web - Local storage.
- Desktop - JSON file.
- iOS - NSUserDefaults.
- Android - SharedPreferences.

Writing data to the storage:

```
        # strings
        await page.shared_preferences.set("key", "value")

        # numbers, booleans
        await page.shared_preferences.set("number.setting", 12345)
        await page.shared_preferences.set("bool_setting", True)

        # lists
        await page.shared_preferences.set("favorite_colors", ["red",
"green", "blue"])
```

Each Flutter application using `shared_preferences` plugin has its own set of preferences. As the same Flet client (which is a Flutter app) is used to run UI for multiple Flet apps any values stored in one Flet application are visible/available to another Flet app running by the same user.

To distinguish one application settings from another it is recommended to use some unique prefix for all storage keys, for example `{company}.{product}.`. For example to store auth token in one app you could use `acme.one_app.auth_token` key and in another app use `acme.second_app.auth_token`.

It is responsibility of Flet app developer to encrypt sensitive data before sending it to a client storage, so it's not read/tampered by another app or an app user.

Reading data:

```
        # The value is automatically converted back to the original
type
        value = await page.shared_preferences.get("key")

        colors = await page.shared_preferences.get("favorite_colors")
        # colors = ["red", "green", "blue"]
```

Check if a key exists:

```
        await page.shared_preferences.contains_key("key") # True if
the key exists
```

Get all keys:

```
        await page.shared_preferences.get_keys("key-prefix.")
```

Remove a value:

```
        await page.shared_preferences.remove("key")
```

Clear the storage:

```
        await page.shared_preferences.clear()
```

`clear()` is a dangerous function that removes all preferences of all Flet apps ever run by the same user and serves as a heads-up that permanent application data shouldn't be stored in the client storage.



Flet supports two primary ways of defining colors:

1. Hexadecimal values
2. Named colors

# Hexadecimal Values

Hex color values follow either of these formats:

- `#aarrggbb` or `0xaarrggbb`
- `#rrggbb` or `0xrrggbb` (opacity defaults to `ff` if `aa` is omitted)

```
>>> ft.Container(bgcolor="#ff0000")
```

# Named Colors

Flet uses Material Design theme colors and color palettes.

You can define named colors in two ways:

- As a **string** (e.g., `"blue"`, `"redAccent100"`)
- Using the `Colors` or `CupertinoColors` **enums** for better type safety and autocompletion

When using strings, use only Flet named color identifiers (the same set exposed by `Colors` / `CupertinoColors`), not arbitrary CSS-like color names.

```
>>> ft.Container(bgcolor="yellow")
>>> ft.Container(bgcolor=ft.Colors.YELLOW)
>>> ft.Container(bgcolor=ft.CupertinoColors.DESTRUCTIVE_RED)
```

## Theme Colors



There are 30 named theme colors in `Theme.color_scheme` that are generated based on the `Theme.color_scheme_seed`, which defaults to `Colors.BLUE`.

```
>>> page.theme = ft.Theme(color_scheme_seed=ft.Colors.GREEN)
```

Any of the 30 theme scheme colors can be overridden, in which case they will have an absolute value that will no longer be dependent on the seed color.

```
page.theme = ft.Theme(
    color_scheme=ft.ColorScheme(
        primary=ft.Colors.GREEN,
        error=ft.Colors.RED,
    ),
)
```



Theme colors serve as fallback values for most Flet controls.

## Color Palettes



Originally introduced by Material Design in 2014, **color palettes** are collections of coordinated colors designed to work together harmoniously.

Each **color swatch** (palette) includes multiple shades of a specific color, where lower numbers represent lighter shades and higher numbers represent darker ones. Most palettes range from `50` to `900`, in increments of `100`, while **accent swatches** (e.g., `Colors.RED_ACCENT`) have only `100`, `200`, `400`, and `700`.

In addition to color swatches, Flet provides named black and white variants with built-in opacities, such as: - `Colors.BLACK_54` → black at 54% opacity - `Colors.WHITE_70` → white at 70% opacity

These palette colors can be used: - directly as values for control color properties (e.g., `bgcolor`, `color`) - as seed colors for generating dynamic theme color schemes using `Theme.color_scheme_seed`

## Color opacity

Flet provides multiple ways to set opacity:

### Using `with_opacity`

This method is present in both `Colors` and `CupertinoColors` enums. It takes an opacity and a color, and returns a string in the format `"color,opacity"`, understood by Flet.

```
>>> ft.Colors.with_opacity(0.5, ft.Colors.RED)
"red,0.5"
>>> ft.CupertinoColors.with_opacity(0.8, ft.CupertinoColors.LINK)
"link,0.8"
```

### Embedding Opacity in Hex Code

For colors in hex format, you can specify the `aa` channel with values between `00` and `ff`.

```
>>> "#7fff6666"  # (1)!
```

1. Red color at 50% opacity (`7f`).

# Defining colors for Flet controls

Most Flet controls have default colors defined by the `Theme.color_scheme`, and these can be overridden at various levels.



## Control Level

If a color is provided as value for a control's color-like property, it will be used directly.

```
>>> ft.Card(bgcolor=ft.Colors.GREEN_200)
```

Note: Not every Flet control has a color property that can be set on the control level. For example, `FilledButton` always has a default "primary" color that is defined by the nearest ancestor's `theme`.

## Control Theme Level

The `Theme` object has a lot of properties that can be used to override default colors for Flet controls.

For example, the nearest `Theme.card_bgcolor` will be used for the `Card` control.

Note: If you need to change theme for particular descendants, you can wrap them in a `Container`, for example, and customize its `theme` property, which will be applied to all its descendants.

## Ancestor Theme Level

Flet searches upward in the widget tree to find the nearest ancestor with a defined `theme`, and will take color from its `Theme.color_scheme`. In the example below, the nearest ancestor for the `FilledButton` is `Container`, and the `primary` color that is used for the button will be taken from the `Container.theme`.

```
        import flet as ft

        def main(page: ft.Page):
            page.add(
                ft.Container(
                    width=200,
                    height=200,
                    border=ft.Border.all(1, ft.Colors.BLACK),
                    content=ft.FilledButton("Primary color"),

theme=ft.Theme(color_scheme=ft.ColorScheme(primary=ft.Colors.YELLOW))
                )
            )

        ft.run(main)
```

If no theme is defined for a control, its parent, or its parent's ancestors, the control defaults to using the uppermost ancestor's theme, which is `Page.theme` (or `Page.dark_theme` in dark mode).

Flet controls are objects and to access their properties we need to keep references (variables) to those objects.

Consider the following example:

```python {6-8,18,19,21} import flet as ft

def main(page): first_name = ft.TextField(label="First name", autofocus=True)
last_name = ft.TextField(label="Last name") greetings = ft.Column()

async def btn_click(e):
    greetings.controls.append(ft.Text(f"Hello, {first_name.value}
{last_name.value}!"))
    first_name.value = ""
    last_name.value = ""
    page.update()
    await first_name.focus()

page.add(
    first_name,
    last_name,
    ft.Button("Say hello!", on_click=btn_click),
    greetings,
)

ft.run(main)


In the very beginning of `main()` method we create three controls
which we are going to use in button's
`on_click` handler: two `TextField` for first and last names and a
`Column` - container for greeting messages.
We create controls with all their properties set and in the end of
`main()` method, in `page.add()` call,
we use their references (variables).

When more and more controls and event handlers are added it becomes
challenging to keep all control
definitions in one place, so they become scattered across `main()`
body. Glancing at `page.add()` parameters it's
hard to imagine (without constant jumping to variable definitions in
IDE) what would the end form look like:

```python {2-5}
    page.add(
        first_name,
        last_name,
        ft.Button("Say hello!", on_click=btn_click),
        greetings,
    )
```

Is `first_name` a TextField, does it have autofocus set? Is greetings a `Row` or a `Column`?
```

# `Ref` class

Flet provides `Ref` utility class which allows to define a reference to the control, use that reference in event handlers and set the reference to a real control later, while building a tree. The idea comes from React.

To define a new typed control reference:

```
first_name = ft.Ref[ft.TextField]()
```

To access referenced control (control de-reference) use `Ref.current` property:

```
# empty first name
first_name.current.value = ""
```

To assign control to a reference set `Control.ref` property to a reference:

```
python {2} page.add(     ft.TextField(ref=first_name, label="First
name", autofocus=True) )
```

All Flet controls have `ref` property.

We could re-write our program to use references:

```python {7-9,21-24} import flet as ft

def main(page):

first_name = ft.Ref[ft.TextField]()
last_name = ft.Ref[ft.TextField]()
greetings = ft.Ref[ft.Column]()

async def btn_click(e):
    greetings.current.controls.append(
        ft.Text(f"Hello, {first_name.current.value}
{last_name.current.value}!")
    )
    first_name.current.value = ""
    last_name.current.value = ""
    page.update()
    await first_name.current.focus()

page.add(
    ft.TextField(ref=first_name, label="First name", autofocus=True),
    ft.TextField(ref=last_name, label="Last name"),
    ft.Button("Say hello!", on_click=btn_click),
    ft.Column(ref=greetings),
)

ft.run(main)


Now we can clearly see in `page.add()` the structure of the page and
```

````
all the controls it's built of.

Yes, the logic becomes a little bit more verbose as you need to add
`.current.` to access ref's control, but it's a matter of personal
preference :)

---
title: "Custom Controls"
---

import {Image} from '@site/src/components/crocodocs';

While Flet provides 140+ built-in controls that can be used on their
own, the real beauty of programming with Flet is that all those
controls can be utilized for creating your own reusable UI components
using Python object-oriented programming concepts.

You can create custom controls in Python by styling and/or combining
existing Flet controls.

## Styled controls

The most simple custom control you can create is a styled control, for
example, a button of a certain color and behaviour that will be used
multiple times throughout your app.

To create a styled control, you need to create a new dataclass in
Python that inherits from the Flet control you are going to customize,
`Button` in this case:

```python
@ft.control
class MyButton(ft.Button):
    bgcolor: ft.Colors = ft.Colors.ORANGE_300
    color: ft.Colors = ft.Colors.GREEN_800
    style: ft.ButtonStyle = field(
        default_factory=lambda: ft.ButtonStyle(
            shape=ft.RoundedRectangleBorder(radius=10)
        )
    )
    expand: int = 1
````

You can define either `@dataclass` or `@ft.control` decorator on the inherited class - both methods works the same:

```python
@dataclass
class MyButton(ft.Button):
    bgcolor: ft.Colors = ft.Colors.ORANGE_300
    color: ft.Colors = ft.Colors.GREEN_800
    style: ft.ButtonStyle = field(
        default_factory=lambda: ft.ButtonStyle(
            shape=ft.RoundedRectangleBorder(radius=10)
```
```

```
            )
        )
        expand: int = 1
```

You can also set properties in `init()` method:

```
@ft.control
class MyButton(ft.Button):
    def init(self):
        self.bgcolor = ft.Colors.ORANGE_300
        self.color = ft.Colors.GREEN_800
        self.style = ft.ButtonStyle(
            shape=ft.RoundedRectangleBorder(radius=10)
        )
        self.expand = 1
```

- You should define either @dataclass or @ft.control decorator on the inherited class - both methods works the same.
- A field, to be a class field, must have a type annotation, so `expand: int = 1` will override inherited property, but `expand = 1` won't. Not sure which type to use? Just `Any` will work, for example `expand: Any = 1`.
- Use literal default value for simple data types, such as `int`, `bool`, `str` and `field(default_factory=lambda: <new_value>)` for immutable types such as `class`, `list`, `dict`.
- You can set property values in `init()` method, but you won't be able to override if write `MyButton3(expand=False)`.

Now you can use your brand-new control in your app:

```
import flet as ft

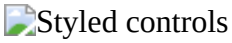
@ft.control
class MyButton(ft.Button):
    bgcolor: ft.Colors = ft.Colors.ORANGE_300
    color: ft.Colors = ft.Colors.GREEN_800

def main(page: ft.Page):
    def ok_clicked(e):
        print("OK clicked")

    def cancel_clicked(e):
        print("Cancel clicked")

    page.add(
        MyButton(content="OK", on_click=ok_clicked),
        MyButton(content="Cancel", on_click=cancel_clicked),
    )

ft.run(main)
```




See example of using styled controls in Calculator App tutorial.

# Composite controls

Composite custom controls inherit from container controls such as `Column`, `Row`, `Stack` or even `View` to combine multiple Flet controls. The example below is a `Task` control that can be used in a To-Do app:

```
        import flet as ft

        @ft.control
        class Task(ft.Row):
            text: str = ""

            def init(self):
                self.text_view = ft.Text(value=self.text)
                self.text_edit = ft.TextField(value=self.text,
visible=False)
                self.edit_button = ft.IconButton(icon=ft.Icons.EDIT,
on_click=self.edit)
                self.save_button = ft.IconButton(
                    visible=False, icon=ft.Icons.SAVE,
on_click=self.save
                )
                self.controls = [
                    ft.Checkbox(),
                    self.text_view,
                    self.text_edit,
                    self.edit_button,
                    self.save_button,
                ]

            def edit(self, e):
                self.edit_button.visible = False
                self.save_button.visible = True
                self.text_view.visible = False
                self.text_edit.visible = True
                self.update()

            def save(self, e):
                self.edit_button.visible = True
                self.save_button.visible = False
                self.text_view.visible = True
                self.text_edit.visible = False
                self.text_view.value = self.text_edit.value
                self.update()

        def main(page: ft.Page):
            page.add(
```

```
                Task(text="Do laundry"),
                Task(text="Cook dinner"),
            )

    ft.run(main)
```




# Life-cycle methods

Custom controls provide life-cycle “hook” methods that you may need to use for different use cases in your app.

### `build()`

`build()` method is called when the control is being created and assigned its `self.page`.

Override `build()` method if you need to implement logic that cannot be executed in control’s constructor because it requires access to the `self.page`. For example, choose the right icon depending on `self.page.platform` for your adaptive app.

### `did_mount()`

`did_mount()` method is called after the control is added to the page and assigned transient `uid`.

Override `did_mount()` method if you need to implement logic that needs to be executed after the control was added to the page, for example Weather widget which calls Open Weather API every minute to update itself with the new weather conditions.

### `will_unmount()`

`will_unmount()` method is called before the control is removed from the page.

Override `will_unmount()` method to execute clean-up code.

### `before_update()`

`before_update()` method is called every time when the control is being updated.

Make sure not to call `update()` method within `before_update()`.

# Isolated controls

Custom control has `is_isolated` property which defaults to `False`.

If you set `is_isolated` to `True`, your control will be isolated from outside layout, i.e. when `update()` method is called for the parent control, the control itself will be updated but any changes to the controls’ children are not included into the update digest.

Isolated controls should call `self.update()` to push its changes to a Flet page.

As a best practice, any custom control that calls `self.update()` inside its class methods should be isolated.

In the above examples, simple styled `MyButton` doesn't need to be isolated, but the `Task` should be:

```
        class Task(ft.Row):
            def __init__(self, text):
                super().__init__()

            def is_isolated(self):
                return True
```

`ft.use_dialog()` lets a component show and update dialogs declaratively. Instead of imperatively calling [`page.show_dialog()`][flet.BasePage.show_dialog] and later remembering to close or remove the dialog, you render a [`DialogControl`][flet.DialogControl] from component state:

- pass a dialog instance to show it;
- pass `None` to hide it.

This keeps dialog logic in the same state flow as the rest of a declarative app:

- state decides whether the dialog is visible;
- dialog content updates when state changes;
- there is no `page.update()` call in the component.

## Basic pattern

Call `ft.use_dialog()` on every render. When the dialog should be open, return a dialog control; otherwise return `None`.

```
        import flet as ft


        @ft.component
        def App():
            show, set_show = ft.use_state(False)

            ft.use_dialog(
                ft.AlertDialog(
                    modal=True,
                    title=ft.Text("Delete report.pdf?"),
                    content=ft.Text("This cannot be undone."),
                    actions=[
                        ft.TextButton("Delete", on_click=lambda:
set_show(False)),
                        ft.TextButton("Cancel", on_click=lambda:
set_show(False)),
                    ],
```

```
                    on_dismiss=lambda: set_show(False),
                )
                if show
                else None
            )

            return ft.Column(
                controls=[
                    ft.TextButton("Open dialog", on_click=lambda:
set_show(True)),
                ]
            )


    ft.run(lambda page: page.render(App))
```

The important part is that `show` is the source of truth. The dialog is not opened by mutating the page tree directly; it appears because the component renders it.

## Updating dialog content from state

Because the dialog is declarative, its content can react to state changes while it is open. This is useful for confirmations, form validation, and async workflows:

```
    import asyncio
    import flet as ft


    @ft.component
    def App():
        show, set_show = ft.use_state(False)
        deleting, set_deleting = ft.use_state(False)

        async def handle_delete():
            set_deleting(True)
            await asyncio.sleep(2)
            set_deleting(False)
            set_show(False)

        ft.use_dialog(
            ft.AlertDialog(
                modal=True,
                title=ft.Text("Delete report.pdf?"),
                content=ft.Text(
                    "Deleting, please wait..." if deleting else
"This cannot be undone."
                ),
                actions=[
                    ft.Button(
                        "Deleting..." if deleting else "Delete",
```

```
                            disabled=deleting,
                            on_click=handle_delete,
                        ),
                        ft.TextButton(
                            "Cancel",
                            disabled=deleting,
                            on_click=lambda: set_show(False),
                        ),
                    ],
                    on_dismiss=lambda: set_show(False),
                )
                if show
                else None
            )

            return ft.TextButton("Delete file", on_click=lambda:
set_show(True))

        ft.run(lambda page: page.render(App))
```

This pattern works well with `asyncio` and other async APIs in Flet apps. For more background, see Async apps.

## Chaining dialogs

You can call `ft.use_dialog()` more than once in the same component. That makes follow-up flows straightforward, for example:

- confirmation dialog;
- async action;
- success dialog after the first dialog fully closes.

```
        import flet as ft


        @ft.component
        def App():
            show_confirm, set_show_confirm = ft.use_state(False)
            show_success, set_show_success = ft.use_state(False)
            should_chain = ft.use_ref(False)

            def confirm_delete():
                should_chain.current = True
                set_show_confirm(False)

            def on_confirm_dismiss():
                if should_chain.current:
                    should_chain.current = False
                    set_show_success(True)

            ft.use_dialog(
```

```
                    ft.AlertDialog(
                        title=ft.Text("Delete file?"),
                        actions=[
                            ft.TextButton("Delete",
on_click=confirm_delete),
                            ft.TextButton("Cancel", on_click=lambda:
set_show_confirm(False)),
                        ],
                        on_dismiss=on_confirm_dismiss,
                    )
                    if show_confirm
                    else None
                )

                ft.use_dialog(
                    ft.AlertDialog(
                        title=ft.Text("Done"),
                        content=ft.Text("The file was deleted."),
                        actions=[
                            ft.TextButton("OK", on_click=lambda:
set_show_success(False)),
                        ],
                    )
                    if show_success
                    else None
                )

                return ft.TextButton("Open", on_click=lambda:
set_show_confirm(True))

        ft.run(lambda page: page.render(App))
```

`ft.use_ref()` is helpful here because the value survives re-renders without causing another render by itself.

## `on_dismiss` timing

[`DialogControl.on_dismiss`][flet.DialogControl.on_dismiss] fires after the dialog close animation completes, not immediately when `open` changes to `False`. This makes it safe to start follow-up UI after the dialog has actually finished closing.

Use `on_dismiss` for logic that should happen after the dialog is fully gone, such as:

- opening the next dialog in a chain;
- resetting temporary dialog-local state;
- starting a follow-up animation or toast.

# When to use `page.show_dialog()` instead

`ft.use_dialog()` is a better fit inside `@ft.component` functions and other declarative flows.

[`page.show_dialog()`][flet.BasePage.show_dialog] is still a good option when:

- the app is written imperatively;
- dialog lifecycle is handled outside the component tree;
- you need to trigger a dialog from existing page-level event code and do not want to convert that part of the app to declarative style yet.

In practice, the two APIs serve different styles:

- use declarative dialogs when UI should follow component state;
- use imperative dialogs when UI is managed by direct page mutation.

import {CodeExample} from '@site/src/components/crocodocs';

By default, a Flet app follows an **imperative** approach: you directly change control properties like `visible` and `value`, add or remove controls, and Flet pushes the change to the page for you. That's fine for small apps, but it doesn't scale well — as a screen grows, the logic for keeping related controls in sync gets duplicated across event handlers, and it's easy to miss one and end up with an inconsistent UI.

Flet also supports a **declarative** approach: the state of your app is the single source of truth, and the UI is derived from it. You don't change controls or their properties directly; instead, only change application state, by assigning to an observable field or calling a hook's setter. Flet components detect the change and re-render UI that depends on it.

This article shows the same simple "User Manager" app built both ways — imperative and declarative — so you can compare the two approaches directly. It's a classic CRUD app: list users, add a new user, edit inline, and delete:

<img alt="view" className="doc-screenshot" src="/docs/assets/cookbook/declarative-vs-imperative-crud-app/crud1.png" style={{width: "45%"}} />

<img alt="inline edit" className="doc-screenshot" src="/docs/assets/cookbook/declarative-vs-imperative-crud-app/crud2.png" style={{width: "55%"}} />

# Imperative

In this example, clicking a button changes control properties directly (`visible`, `value`) and adds or removes controls from the page's control list — that's what makes it imperative. Flet pushes each change to the page automatically.

- **Add** creates a new `Item` row from the `first_name`/`last_name` `TextField` values, appends it to `page.controls`, and clears both fields.
- **Edit** resets the `TextFields` to the row's `first_name`/`last_name`, sets `visible=False` on `text`, `edit_button`, and `delete_button`, and `visible=True`

on `edit_text`, `save_button`, and `cancel_button` — switching the row into edit mode.
- **Save** copies the two `TextField` values into `first_name`/`last_name` and `text.value`, then sets `visible=True` on `text`, `edit_button`, and `delete_button`, and `visible=False` on `edit_text`, `save_button`, and `cancel_button` — returning to read-only mode with the new values.
- **Cancel** sets `visible=True` on `text`, `edit_button`, and `delete_button`, and `visible=False` on `edit_text`, `save_button`, and `cancel_button` — returning to read-only mode without saving.
- **Delete** removes the row's `Item` instance from `page.controls`.

<CodeExample path={frontMatter.examples + '/imperative/main.py'} language="python" displayTitle={false} />

# Declarative

In this example, clicking a button doesn't change controls directly. Instead, it changes application state that exists separately from UI. As soon as the state changes, Flet detects it and re-renders UI.

To understand how Flet stores state and detects when it changes, you need to understand the concepts that lay in the declarative approach: Observables, Components, and Hooks.

## Observables

Observables (`@ft.observable`) are classes whose instances hold your application's state. In this "User Manager" app, each `User` instance holds one person's name, and a single `App` instance holds the list of users. The `@ft.observable` decorator makes their attribute and collection changes trackable: assigning to a field like `user.first_name`, or changing a list field like `app.users.append(...)`, notifies the components subscribed to it, triggering them to re-render.

```
        @ft.observable
        @dataclass
        class User:
            first_name: str
            last_name: str

            def update(self, first_name: str, last_name: str):
                self.first_name = first_name  # notifies subscribed
components
                self.last_name = last_name
```

## Components

Components (`@ft.component`) are functions that take arguments — like `user` and `delete_user` in `UserView(user, delete_user)` — and return the controls describing the UI for its current state. Only the arguments that are themselves `@ft.observable` instances, like `user`, get subscribed to; a plain callback like `delete_user` is just passed through.

```
        @ft.component
        def UserView(user: User, delete_user) -> ft.Control:
            return ft.Row([
                ft.Text(f"{user.first_name} {user.last_name}"),
                ft.Button("Delete", on_click=lambda:
delete_user(user)),
            ])
```

Unlike the imperative example, a component doesn't change an existing control's properties or modify `page.controls`. It just returns a new set of controls each render, and Flet reconciles that against what's already on screen, patching only what changed.

A component renders once when it's first created, and again whenever an observable it's subscribed to — received as an argument, or held via a hook — is changed, or a hook's setter replaces its value; only that component re-renders, not the whole app. On each render, it subscribes again to every such observable. The subscription is to the whole object, not individual fields — which is why, in the example below, editing one user only re-renders its `UserView` (it is subscribed to that one `user`), while adding or deleting re-renders all of `AppView` (it is subscribed to `app`, and a user list change is a change to `app`).

## Hooks

Hooks (`ft.use_state`) hold local state scoped to one component instance — like a row's "editing" flag. A plain local variable won't do the job: it resets on every render, and changing it doesn't trigger one. Calling a hook's setter re-renders that component, using the same mechanism as observables — just scoped to one component.

```
        # Broken: a plain local resets on every render, and changing
it doesn't trigger one
        @ft.component
        def UserView(user: User, delete_user) -> ft.Control:
            is_editing = False
            if not is_editing:
                return ft.Row([
                    ft.Text(f"{user.first_name} {user.last_name}"),
                    ft.Button("Edit", on_click=lambda: (is_editing :=
True)),  # no re-render
                ])
            ...

        # Correct: hook state survives across renders and triggers one
when set
        @ft.component
        def UserView(user: User, delete_user) -> ft.Control:
            is_editing, set_is_editing = ft.use_state(False)
            if not is_editing:
                return ft.Row([
                    ft.Text(f"{user.first_name} {user.last_name}"),
                    ft.Button("Edit", on_click=lambda:
set_is_editing(True)),
                ])
            ...
```

### Example

The state lives in instances of two `@ft.observable` classes: `User` (`first_name`, `last_name`) and `App` (`users: list[User]`). `@ft.component` functions read that state and return controls — `UserView` renders one row, read-only or editing; `AddUserForm` renders the add form. Each row's "editing" flag and input buffers are local `ft.use_state` hooks: they're view-only and don't belong on `User`.

- **Add** (in `AddUserForm`) calls `add_user_and_clear()`, which calls `app.add_user(...)` — appending a `User` to `app.users` — then clears its own local `use_state` buffers. `AppView` holds `app` via `use_state`, so this re-renders `AppView`, regenerating the whole user list.
- **Edit** calls `start_edit()`, which resets the local buffer hooks to the user's current values and calls `set_is_editing(True)` — re-rendering just that row's `UserView` into its editing form.
- **Save** calls `save()`, which calls `user.update(...)` and `set_is_editing(False)` — `user` was passed directly into this `UserView`, so only this one row re-renders with the new values.
- **Cancel** calls `cancel()`, which calls `set_is_editing(False)` — re-rendering that row back to read-only without touching `user`.
- **Delete** calls `app.delete_user(user)`, removing it from `app.users` — like Add, this re-renders all of `AppView`, not just the one row.

<img alt="declarative data flow diagram" className="doc-screenshot" src="/docs/assets/cookbook/declarative-vs-imperative-crud-app/crud-declarative.drawio.png" style={{width: "55%"}} />

<CodeExample path={frontMatter.examples + '/declarative/main.py'} language="python" displayTitle={false} />

## Conclusion

Imperative and declarative aren't just two ways to write the same code — they're two different ways to think about UI. Imperative asks "what should change on screen right now?" — you list, step by step, exactly what to change for each interaction. Declarative asks "what does my data look like right now?" — you describe the UI as a function of that data, and let Flet figure out what changed on screen.

That shift pays off most as an app gets more interactive:

- **Debugging** — since the UI is derived from state, a bug is either "the state is wrong" or "the render is wrong," not "some event handler forgot to update a flag." You can inspect the state directly instead of tracing through a chain of mutations.
- **Testing** — `app.add_user(...)` and `user.update(...)` are just regular Python methods with no Flet controls involved. You can call them directly in a test and check the result, with no page or rendering required.
- **Multiple views of the same data** — if two parts of the UI need to reflect the same state (a badge count, a list, a chart), each just reads from that state instead of being kept in sync by hand.
- **Complex, multi-step UIs** — wizards, filters, undo/redo — where many pieces of UI depend on overlapping state, declarative code scales by adding more state and components, not more handler-to-handler coordination.

For more declarative examples, see the declarative examples collection — including a Counter, a To-Do app, and games like Tic-Tac-Toe and Minesweeper. You can also try them interactively in the Flet Studio.

import {Image} from '@site/src/components/crocodocs';

The mechanics of drag-and-drop in Flet is pretty simple - a user starts dragging `Draggable` control and "drops" it on `DragTarget`. If both draggable and drag target has the same `group` a drag target will call `on_accept` event handler and pass draggable control ID as event data. In this case draggable serves as a source "data" for drag-and-drop operation.

Let's take a look at the following example. In the program below you can drag left control displaying "1" on top of the right control displaying "0" and when drag operation completes left control is replaced with "0" and the right control becomes "1":

```
import flet as ft

def main(page: ft.Page):
    page.title = "Drag and Drop example"

    def drag_accept(e):
        # update text inside draggable (source) control
        e.src.content.content.value = "0"
        # update text inside drag target control
        e.control.content.content.value = "1"
        page.update()

    page.add(
        ft.Row(
            controls=[
                ft.Draggable(
                    group="number",
                    content=ft.Container(
                        width=50,
                        height=50,
                        bgcolor=ft.Colors.CYAN_200,
                        border_radius=5,
                        content=ft.Text("1", size=20),
                        alignment=ft.Alignment.CENTER,
                    ),
                ),
                ft.Container(width=100),
                ft.DragTarget(
                    group="number",
                    content=ft.Container(
                        width=50,
                        height=50,
                        bgcolor=ft.Colors.PINK_200,
                        border_radius=5,
                        content=ft.Text("0", size=20),
                        alignment=ft.Alignment.CENTER,
```

```
                        ),
                        on_accept=drag_accept,
                    ),
                ]
            )
        )

    ft.run(main)
```



So, it's developer's responsibility to determine what happens with "source" (draggable) and "destination" (drag target) controls when `on_accept` event occurs.

:::note[Try something] Change DragTarget's group property to `number1` and note `on_accept` is not called any more when you drop "1" on the target. :::

There are additional properties and event handlers to make drag-and-drop operation even more interactive. For example, draggable has `content_when_dragging` property to display a different control instead of `content` when drag operation is under way. There is also `content_feedback` property to show a different control under the pointer. By default, the same `content` control, but with 50% opacity is displayed under cursor when dragging.

Let's modify Draggable in our example to display a "hole" in place of dragged control and just "1" under cursor while dragging:

```
    ...
                ft.Draggable(
                    group="number",
                    content=ft.Container(
                        width=50,
                        height=50,
                        bgcolor=ft.Colors.CYAN_200,
                        border_radius=5,
                        content=ft.Text("1", size=20),
                        alignment=ft.alignment.center,
                    ),
                    content_when_dragging=ft.Container(
                        width=50,
                        height=50,
                        bgcolor=ft.Colors.BLUE_GREY_200,
                        border_radius=5,
                    ),
                    content_feedback=ft.Text("1"),
                ),
    ...
```



Drag target control additionally has `on_will_accept` and `on_leave` event handlers which help better visualize when it's a good time to "drop" something on the target. Let's modify DragTarget in our example to draw a border around target control when it's ready

to accept incoming drag:

```
        import flet as ft

        def main(page: ft.Page):
            page.title = "Drag and Drop example 2"

            def drag_accept(e):
                # update text inside draggable (source) control
                e.src.content.content.value = "0"
                # reset source group, so it cannot be dropped to a
target anymore
                e.src.group = ""
                # update text inside drag target control
                e.control.content.content.value = "1"
                # reset border
                e.control.content.border = None
                page.update()

            def drag_will_accept(e):
                # black border when it's allowed to drop and red when
it's not
                e.control.content.border = ft.Border.all(
                    2, ft.Colors.BLACK_45 if e.data == "true" else
ft.Colors.RED
                )
                e.control.update()

            def drag_leave(e):
                e.control.content.border = None
                e.control.update()

            page.add(
                ft.Row(
                    controls=[
                        ft.Draggable(
                            group="number",
                            content=ft.Container(
                                width=50,
                                height=50,
                                bgcolor=ft.Colors.CYAN_200,
                                border_radius=5,
                                content=ft.Text("1", size=20),
                                alignment=ft.Alignment.CENTER,
                            ),
                            content_when_dragging=ft.Container(
                                width=50,
                                height=50,
                                bgcolor=ft.Colors.BLUE_GREY_200,
                                border_radius=5,
                            ),
```

```
                            content_feedback=ft.Text("1"),
                        ),
                        ft.Container(width=100),
                        ft.DragTarget(
                            group="number",
                            content=ft.Container(
                                width=50,
                                height=50,
                                bgcolor=ft.Colors.PINK_200,
                                border_radius=5,
                                content=ft.Text("0", size=20),
                                alignment=ft.Alignment.CENTER,
                            ),
                            on_accept=drag_accept,
                            on_will_accept=drag_will_accept,
                            on_leave=drag_leave,
                        ),
                    ]
                )
            )

        ft.run(main)
```


Drag and drop number 3

Sensitive data such as tokens, keys, credit card numbers and other “secrets” must be stored at rest (database, files, client storage) in encrypted form to avoid data breaches.

Flet includes utility methods to encrypt and decrypt sensitive text data using symmetric algorithm (where the same key is used for encryption and decryption). They use Fernet implementation from cryptography package, which is AES 128 with some additional hardening, plus PBKDF2 to derive encryption key from a user passphrase.

## Secret key

Encryption secret key (aka password, or passphrase) is an arbitrary password-like string configured by a user and used for encrypting and decrypting data. Crypto algorithm uses secret key to “derive” encryption key (32 bytes).

Do not embed any secrets into the source code to avoid accidental exposure to the public!

You can provide a secret to your app via environment variable:

```
import os
secret_key = os.getenv("MY_APP_SECRET_KEY")
```

Before running the app set the secret in a command line:

```
$ export MY_APP_SECRET_KEY="<secret>"
```

While passing secrets via environment variables is a common practice amongst developers and service providers it does not fully prevent secrets leaking in some environments. Other mechanisms can be used to inject secrets to your application such as mounting secret files or vault services.

## Encrypting data

Use `encrypt()` method to encrypt a string:

```
import os
from flet.security import encrypt, decrypt

secret_key = os.getenv("MY_APP_SECRET_KEY")
plain_text = "This is a secret message!"
encrypted_data = encrypt(plain_text, secret_key)
```

`encrypted_data` is a URL-safe base64-encoded string.

`encrypt` accepts strings only, so any objects must be serialized to JSON, XML or other text-based format before encryption.

## Decrypting data

Use `decrypt()` method to decrypt the data:

```
import os
from flet.security import encrypt, decrypt

secret_key = os.getenv("MY_APP_SECRET_KEY")
encrypted_data =
"601llp2zpPp4QjBWe2cOwGdBQUFBQUJqTTFJbmgyWU5jblVp..."
plain_text = decrypt(encrypted_data, secret_key)
print(plain_text)
```

import {CodeExample, Image} from '@site/src/components/crocodocs';

## expand

When a child control is placed into a Row, Column, View or Page you can "expand" it to fill the available space.

When adding child controls to a Row, you can make them automatically grow to fill available horizontal space using the expand property, which all Flet controls (inheriting from Control) have.

It lets you control how they use free space inside a layout like Row or Column.

You can set `expand` to one of the following values:

- a boolean — Whether the control should take all the available space.

- an integer — Used to proportionally divide free space among multiple expanding controls (useful when you want more control over sizing).

### Example 1

In this example, a `TextField` stretches to fill all remaining space in the row, while the `Button` stays sized to its content:

<CodeExample path={frontMatter.examples + '/expand_textfield_in_row/main.py'} language="python" />

<Image src={frontMatter.example_images + '/expand_textfield_in_row.png'} alt="expand textfield in row" width="45%" />

### Example 2

In this example, we create a `Row` with three `Container`s, distributed like 20% / 60% / 20%:

<CodeExample path={frontMatter.examples + '/expand_row_proportional_1_3_1/main.py'} language="python" />

Here, the available space is split into 5 total parts (1+3+1). The first and third containers get 1 part each (20%), and the middle one gets 3 parts (60%).

<Image src={frontMatter.example_images + '/expand_row_proportional_1_3_1.png'} alt="expand row proportional 1 3 1" width="45%" />

### Example 3

This example demonstrates how two controls inside a `Row` can each expand to fill half of the available horizontal space using expand=True.

The layout uses a parent `Container` and a nested row, where both controls are expanded equally, resulting in a 50/50 split.

<CodeExample path={frontMatter.examples + '/expand_row_equal_split/main.py'} language="python" />

<Image src={frontMatter.example_images + '/expand_row_equal_split.png'} alt="expand row equal split" width="45%" />

## `expand_loose`

The `expand_loose` property allows a control to *optionally* expand and fill the available space along the main axis of its parent container (e.g., horizontally in a `Row`, or vertically in a `Column`).

Unlike `expand`, which *forces* the control to occupy all available space, `expand_loose` gives the control **flexibility** to grow **if needed**, but it's not required to fill the space.

For this property to have effect:

- it must be used on child controls within the following layout containers, or any of their subclasses: `Row`, `Column`, `View`, `Page`
- the control using this property must have a non-none value for `expand`.

### Example 1

In this example, `Container`s being placed in `Row`s with `expand_loose = True`:

<CodeExample path={frontMatter.examples + '/expand_loose_chat_messages/main.py'} language="python" />

<Image src={frontMatter.example_images + '/expand_loose_chat_messages.png'} alt="expand loose chat messages" width="45%" />

`flet-mcp` is an MCP (Model Context Protocol) server that gives LLM agents and AI coding assistants accurate, version-specific knowledge about Flet. Instead of relying on the model's training data — which drifts out of date and leads to hallucinated controls, properties, and enum members — the agent can look up the real Flet API, search example projects, find icons, and inspect CLI options on demand.

This is useful both inside AI-powered editors (Claude Desktop, Cursor, VS Code, …) and when building your own agents that generate or refactor Flet code.

## Installation

```
pip install flet-mcp
```

This registers a `flet mcp` command (the `flet` CLI must be installed, which it is in any normal Flet project). The package ships with a pre-built API index, so no extra build step is required to start using it.

## Running the server

Start the server over the default `stdio` transport (used by most desktop AI clients):

```
flet mcp
```

Or expose it over HTTP:

```
flet mcp --transport streamable-http --port 8000
```

## Configuring an AI client

Most MCP-aware clients are configured with a small JSON snippet. For example, in Claude Desktop's `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "flet": {
      "command": "flet",
```

```
        "args": ["mcp"]
      }
    }
  }
```

Cursor, VS Code, and other clients use the same `command` / `args` shape in their respective MCP settings.

# Tool groups

Tools are organized into groups that you toggle with `FLET_MCP_ENABLE_*` environment variables read at server startup. The defaults focus on the hallucination-reduction starter set — **API** and **icons** are on, everything else is off:

| Group | Variable | Default | Tools |
|---|---|---|---|
| API | `FLET_MCP_ENABLE_API` | on | `list_controls`, `get_api`, `get_enum`, `search_enum_members`, `enum_has_member` |
| Icons | `FLET_MCP_ENABLE_ICONS` | on | `find_icon` |
| Examples | `FLET_MCP_ENABLE_EXAMPLES` | off | `search_examples`, `get_example` |
| CLI | `FLET_MCP_ENABLE_CLI` | off | `get_cli_help` |

Accepted truthy values are `1`, `true`, and `yes` (case-insensitive). To enable the example search tools, for instance:

```
FLET_MCP_ENABLE_EXAMPLES=1 flet mcp
```

The active groups are also surfaced in the server's startup instructions, so clients that forward those instructions to the model keep its guidance in sync with what is actually registered.

:::note[Documentation search] A documentation search group (`search_docs` / `get_doc`) is planned but not yet enabled — it is being reworked to index the current Docusaurus-based docs site. :::

# What the tools do

| Tool | Description |
|---|---|
| `get_api` | The primary verifier. Look up any Flet symbol by name — control, service, dataclass type (`ButtonStyle`, `Padding`, …), event, or enum. A "not found" result is definitive for the installed Flet version. Async methods are flagged `"async": true`, and every entry carries a `"package"` field naming the pip package it lives in (`"flet"` for core, otherwise an extension like `"flet-audio"`). |

| Tool | Description |
|---|---|
| `list_controls` | Browse available controls and services, with optional category/kind filtering. |
| `get_enum` | Get an enum's members. |
| `search_enum_members` | Search large enums (`Icons`, `CupertinoIcons`) by substring. |
| `enum_has_member` | Verify a specific enum member exists before using it. |
| `find_icon` | Search Material and Cupertino icons by keyword, with synonym matching (e.g. "user" finds `account_circle`). |
| `search_examples` | Search Flet example projects by keyword, optionally filtered by platform. |
| `get_example` | Fetch the full source and metadata for an example returned by `search_examples`. |
| `get_cli_help` | Get structured help for the `flet` CLI commands and their options. |

# Using Flet MCP from your own agent

The server is also importable as a FastMCP instance, so a custom agent can consume it directly.

With Pydantic AI:

```
        from pydantic_ai import Agent
        from pydantic_ai.toolsets import MCPToolset
        from flet_mcp import mcp

        agent = Agent("anthropic:claude-sonnet-4-6", toolsets=
[MCPToolset(mcp)])
        result = agent.run_sync("Create a Flet app with a login form")
```

Or in-process via a FastMCP client — no subprocess, no transport. Set the `FLET_MCP_ENABLE_*` variables before importing `flet_mcp` so the desired tool groups register; the client deserializes structured results onto `.data`:

```
        import asyncio
        from fastmcp import Client
        from flet_mcp import mcp

        async def main():
            async with Client(mcp) as client:
                api = (await client.call_tool("get_api", {"name":
"TextField"})).data
                print(api["kind"], api["package"],
len(api["properties"]))

        asyncio.run(main())
```

# System fonts

You can use the (system) fonts installed on your computer, e.g. “Consolas”, “Arial”, “Verdana”, “Tahoma”, etc.

System fonts cannot be used in Flet web apps rendered with canvas kit web renderer.

### Usage Example

The following example demonstrates how to use the “Consolas” font in a Flet application.

```
import flet as ft

def main(page: ft.Page):
    page.add(
        ft.Text(
            value="This text is rendered with Consolas font",
            font_family="Consolas"
        )
    )

ft.run(main)
```

# Importing Fonts

Font can be imported from external resource by providing an absolute URL or from application assets directory (see Assets Guide).

This is done by setting the page’s `fonts` property.

To apply one of the imported fonts, you can: - Use `Theme.font_family` to set a default/fallback app-wide font family. - Specify a font for individual controls. For example, `Text.font_family`.

### Usage Example

The example below loads the “Kanit” font from GitHub and `"Open Sans"` from local assets. “Kanit” is set as the default app font, while `"Open Sans"` is applied to a specific `Text` control.

```
import flet as ft

def main(page: ft.Page):
    page.fonts = {
        "Kanit":
"https://raw.githubusercontent.com/google/fonts/master/ofl/kanit/Kanit-
Bold.ttf",
        "Open Sans": "/fonts/OpenSans-Regular.ttf"
    }
```

```
            page.theme = Theme(font_family="Kanit")  # Default app
font

            page.add(
                ft.Text("This text uses the Kanit font"),
                ft.Text("This text uses the Open Sans font",
font_family="Open Sans")
            )

        ft.run(main, assets_dir="assets")
```

## Static and Variable Fonts

Currently, only static fonts are supported. These fonts have a specific width, weight, or style combination (e.g., “Open Sans Regular”).

Support for variable fonts is in progress.

However, to use variable fonts, you can create static instances at specific weights using fonttools, e.g.:

```
        fonttools varLib.mutator ./YourVariableFont-VF.ttf wght=140
wdth=85
```

To explore available font features (e.g. possible options for `wght`) use Wakamai Fondue online tool.

import {CodeExample, Image} from ‘@site/src/components/crocodocs’;

A solid keyboard support is a key for user productivity while using your web and, especially, desktop app. Indeed, it could be really annoying to constantly switch between mouse and keyboard.

In addition to form controls’ `.autofocus` property and `TextField.focus()` method Flet allows handling “global” keyboard events.

To capture all keystrokes implement `page.on_keyboard_event` handler. Event handler parameter `e` is an instance of `KeyboardEvent` class with the following properties:

- `key` - a textual representation of a pressed key, e.g. `A`, `Enter` or `F5`.
- `shift` - `True` if “Shift” key is pressed.
- `ctrl` - `True` if “Control” key is pressed.
- `alt` - `True` if “Alt” (“Option”) key is pressed.
- `meta` - `True` if “Command” key is pressed.

This is a simple usage example:

```
        import flet as ft

        def main(page: ft.Page):
            def on_keyboard(e: ft.KeyboardEvent):
                page.add(
                    ft.Text(
```

```
                        f"Key: {e.key}, Shift: {e.shift}, Control:
{e.ctrl}, Alt: {e.alt}, Meta: {e.meta}"
                    )
                )

            page.on_keyboard_event = on_keyboard
            page.add(
                ft.Text("Press any key with a combination of CTRL,
ALT, SHIFT and META keys...")
            )

        ft.run(main)
```

 Keyboard shortcuts

Below is a more advanced example:

You can use `Column` and `Row` controls to display lists in the most cases, but if the list contains hundreds or thousands of items `Column` and `Row` will be ineffective with lagging UI as they render all items at once even they are not visible at the current scrolling position.

In the following example we are adding 5,000 text controls to a page. Page uses `Column` as a default layout container:

```
        import flet as ft

        def main(page: ft.Page):
            for i in range(5000):
                page.controls.append(ft.Text(f"Line {i}"))
            page.scroll = "always"
            page.update()

        ft.run(main, view=ft.AppView.WEB_BROWSER)
```

Run the program and notice that it's not just it takes a couple of seconds to initially load and render all text lines on a page, but scrolling is slow and laggy too:

 Scroll column

For displaying lists with a lot of items use `ListView` and `GridView` controls which render items on demand, visible at the current scrolling position only.

# ListView

`ListView` could be either vertical (default) or horizontal. ListView items are displayed one after another in the scroll direction.

ListView already implements effective on demand rendering of its children, but scrolling performance could be further improved if you can set the same fixed height or width (for `horizontal` ListView) for all items ("extent"). This could be done by either setting

absolute extent with `item_extent` property or making the extent of all children equal to the extent of the first child by setting `first_item_prototype` to `True`.

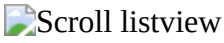
Let's output a list of 5,000 items using ListView control:

```
import flet as ft

def main(page: ft.Page):
    lv = ft.ListView(expand=True, spacing=10)
    for i in range(5000):
        lv.controls.append(ft.Text(f"Line {i}"))
    page.add(lv)

ft.run(main, view=ft.AppView.WEB_BROWSER)
```

Now the scrolling is smooth and fast enough to follow mouse movements:




We used `expand=True` in ListView constructor. In order to function properly, ListView must have a height (or width if `horizontal`) specified. You could set an absolute size, e.g. `ListView(height=300, spacing=10)`, but in the example above we make ListView to take all available space on the page, i.e. expand. Read more about `Control.expand` property.

# GridView

`GridView` allows arranging controls into a scrollable grid.

You can make a "grid" with `ft.Column(wrap=True)` or `ft.Row(wrap=True)`, for example:

```
import os
import flet as ft

os.environ["FLET_WS_MAX_MESSAGE_SIZE"] = "8000000"

def main(page: ft.Page):
    r = ft.Row(wrap=True, scroll="always", expand=True)
    page.add(r)

    for i in range(5000):
        r.controls.append(
            ft.Container(
                ft.Text(f"Item {i}"),
                width=100,
                height=100,
                alignment=ft.Alignment.CENTER,
                bgcolor=ft.Colors.AMBER_100,
                border=ft.Border.all(1, ft.Colors.AMBER_400),
                border_radius=ft.BorderRadius.all(5),
            )
```

```
                )
            page.update()

        ft.run(main, view=ft.AppView.WEB_BROWSER)
```



Try scrolling and resizing the browser window - everything works, but very laggy.

At the start of the program we are setting the value of `FLET_WS_MAX_MESSAGE_SIZE` environment variable to `8000000` - this is the maximum size of WebSocket message in bytes that can be received by Flet Server rendering the page. Default size is 1 MB, but the size of JSON message describing 5,000 container controls would exceed 1 MB, so we are increasing allowed size to 8 MB.

Squeezing large messages through WebSocket channel is, generally, not a good idea, so use batched updates approach to control channel load.

GridView, similar to ListView, is very effective to render a lot of children. Let's implement the example above using GridView:

```
        import os
        import flet as ft

        os.environ["FLET_WS_MAX_MESSAGE_SIZE"] = "8000000"

        def main(page: ft.Page):
            gv = ft.GridView(expand=True, max_extent=150,
child_aspect_ratio=1)
            page.add(gv)

            for i in range(5000):
                gv.controls.append(
                    ft.Container(
                        ft.Text(f"Item {i}"),
                        alignment=ft.Alignment.CENTER,
                        bgcolor=ft.Colors.AMBER_100,
                        border=ft.Border.all(1, ft.Colors.AMBER_400),
                        border_radius=ft.BorderRadius.all(5),
                    )
                )
            page.update()

        ft.run(main, view=ft.AppView.WEB_BROWSER)
```



With GridView scrolling and window resizing are smooth and responsive!

You can specify either fixed number of rows or columns (runs) with `runs_count` property or the maximum size of a "tile" with `max_extent` property, so the number of runs can vary automatically. In our example we set the maximum tile size to 150 pixels and set its shape to "square" with `child_aspect_ratio=1`. `child_aspect_ratio` is the ratio of the cross-axis to the main-axis extent of each child. Try changing it to `0.5` or `2`.

# Batch updates

When `page.update()` is called a message is being sent to Flet server over WebSockets containing page updates since the last `page.update()`. Sending a large message with thousands of added controls could make a user waiting for a few seconds until the messages is fully received and controls rendered.

To increase usability of your program and present the results to a user as soon as possible you can send page updates in batches. For example, the following program adds 5,100 child controls to a ListView in batches of 500 items:

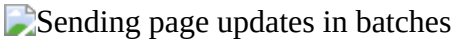
```
import flet as ft

def main(page: ft.Page):

    # add ListView to a page first
    lv = ft.ListView(expand=1, spacing=10, item_extent=50)
    page.add(lv)

    for i in range(5100):
        lv.controls.append(ft.Text(f"Line {i}"))
        # send page to a page
        if i % 500 == 0:
            page.update()
    # send the rest to a page
    page.update()

ft.run(main, view=ft.AppView.WEB_BROWSER)
```


Sending page updates in batches

This article explains how to control level of the logs messages generated by Flet library and its underlying components. You may need to enable detailed logging to troubleshoot Flet library or when submitting a new Flet issue.

# Python

Flet Python uses the following named loggers:

- `flet` - general framework and transport logging.
- `flet_object_patch` - detailed control tree diff/patch logging.
- `flet_components` - declarative component lifecycle logging.

For normal use, configure logging before calling `ft.run()`:

```
import logging

logging.basicConfig(level=logging.INFO)
```

This gives you the usual `flet` logs without too much noise.

To see more detail from the main `flet` logger, either raise the root logging level:

```
import logging

logging.basicConfig(level=logging.DEBUG)
```

or set the `flet` logger explicitly:

```
import logging

logging.basicConfig(level=logging.INFO)
logging.getLogger("flet").setLevel(logging.DEBUG)
```

`flet_object_patch` and `flet_components` set their own level to `INFO`, so they do not inherit the root `DEBUG` level unless you enable them explicitly.

To enable the most verbose diagnostics, set the corresponding loggers to `DEBUG`:

```
import logging

logging.basicConfig(level=logging.DEBUG)
logging.getLogger("flet_object_patch").setLevel(logging.DEBUG)
logging.getLogger("flet_components").setLevel(logging.DEBUG)
```

Use this level mostly for troubleshooting and issue reports.

## Built-in Web Server

When running a web app, Flet starts its built-in web transport on top of FastAPI and Uvicorn.

The effective level of the `flet` Python logger is passed to the built-in web server, so configure `flet` logging before starting the app:

```
import logging

logging.basicConfig(level=logging.INFO)
logging.getLogger("flet").setLevel(logging.DEBUG)
```

If you host a Flet ASGI app with your own server process, such as `uvicorn` or `gunicorn`, configure that server's logging separately using its own options.

import {CodeExample} from '@site/src/components/crocodocs';

In this cookbook recipe, you'll learn how to use Python's built-in `multiprocessing` module — including `concurrent.futures.ProcessPoolExecutor` — from a Flet app, for true CPU parallelism across processes.

For I/O-bound work, or work that just needs to stay off the UI thread, prefer async or threads — they are lighter and work on every platform. Reach for `multiprocessing` when you need multiple CPU cores doing Python work at the same time (number crunching, batch processing, ML inference, etc.), or when you need process isolation for work that may fail or need to be stopped.

:::important[Platform and Flet version support] `multiprocessing` works in Flet desktop apps during development (`flet run`) and in packaged **desktop** apps built with `flet build {macos,windows,linux}` or `flet debug {macos,windows,linux}` when using Flet v0.86.0 or newer.

It is **not supported on iOS and Android** (mobile operating systems don't allow apps to spawn arbitrary child processes) or **in the browser**. On those platforms, prefer threads or `asyncio` instead. :::

# How does it work?

In a desktop app packaged with `flet build`, there is no separate `python` executable — the interpreter is embedded inside your app's binary. When `multiprocessing` spawns a worker, it re-executes that binary with a CPython helper command line; the binary recognizes that shape and services it as a plain, windowless Python interpreter. This also covers multiprocessing's helper processes (the resource tracker and the `forkserver`).

# Guidelines

These are standard Python `multiprocessing` guidelines — but in a packaged Flet app they are **mandatory**, not just good style.

## Always guard your entry point

Start your app only under the `if __name__ == "__main__":` guard. For example:

```
import flet as ft

def main(page: ft.Page):
    ...

if __name__ == "__main__":
    ft.run(main)
```

With the `spawn` and `forkserver` start methods, worker/helper processes need to safely import your main module. `spawn` is the default on macOS and Windows; `forkserver` is the default on Linux starting with Python 3.14. Without the guard, a child process can try to start your whole app again.

## Use importable, picklable worker functions

Worker targets, arguments, and return values must be picklable so Python can send them between processes. In practice:

- define worker functions at module top level, not inside `main()` or inside a button handler
- pass plain data such as numbers, strings, lists, dicts, or dataclasses
- do not pass Flet controls, `page`, database connections, open files, lambdas, or nested functions

Good:

```
def sort_chunk(chunk):
    return sorted(chunk)
```

Avoid:

```
def main(page: ft.Page):
    def sort_chunk(chunk):
        return sorted(chunk)
```

The nested version is not reliably picklable because worker processes need to import the function by name from a module.

### Don’t touch the GUI from workers

Worker processes run in a separate interpreter with no connection to your app’s page. Pass data back through `multiprocessing.Queue`, `Pipe`, or pool futures, and update the UI from the main process.

### Others

- `sys.executable` in a packaged app points at your app’s binary, not a `python` executable. That is intentional — don’t override it with `multiprocessing.set_executable()`.
- You usually do not need `multiprocessing.freeze_support()` in Flet apps. Calling it inside the `if __name__ == "__main__":` block is harmless, but Flet does not rely on PyInstaller-style frozen-executable bootstrapping.
- Worker `print()` output is not connected to your app’s console log; use a `Queue` or file-based logging if you need worker diagnostics.
- On Linux, avoid forcing the `fork` start method: your app’s process runs the Flutter engine with many active threads, and forking it is unsafe. Prefer the platform default (usually `forkserver` or `spawn`), or request one explicitly with `multiprocessing.set_start_method()` or `multiprocessing.get_context()`.
- Starting a worker costs more in a packaged app than in plain Python — each spawned child loads your app binary’s libraries before Python takes over. Create pools and long-lived workers once and reuse them, rather than spawning per button click (see Keep a persistent worker example).

## Examples

### Parallel sort with live progress

Sort chunks of data across all CPU cores and stream progress to the page:

Note how the long-running orchestration is moved off the UI event handler with `page.run_thread`, while the CPU-heavy work runs in the process pool. The worker function may live in your main module (as above) or in a separate importable module — both work.

### Stream progress from a worker

To show fine-grained progress from inside a single long-running job, pass a `multiprocessing.Queue` to the worker and drain it on a background thread. The worker `put()`s progress values and a `None` sentinel when it is done:

### Keep a persistent worker

Starting a process is not free — especially in a packaged app, where each child loads your app binary's libraries first. For repeated jobs, start one worker that stays alive and serves requests over a `Pipe`, paying the startup cost once. Expensive setup (loading a model, opening a dataset) can then also happen once, in the worker:

The worker is started with `daemon=True`, so it is terminated automatically when your app exits.

### Cancel a runaway task

Threads cannot be forcefully stopped from the outside — a worker `Process` can, at any time, with `terminate()`. This makes processes the right tool for jobs you may need to abort, such as long calls into external libraries:

A background thread `join()`s the worker and reports how it ended — normally (exit code `0`) or via cancellation (a negative exit code).

Python's multiprocessing guidelines recommend avoiding process termination or doing it only for processes which never use any shared resources.

import {CodeExample} from '@site/src/components/crocodocs';

Navigation and routing is the core of building multi-screen Flet apps. It lets you organize your UI into virtual pages (`View` objects), keep URL/history in sync, and support deep links to specific app states.

This page focuses on the current routing model and maintained examples.

## Routing model in Flet

A `Page` is a container of views (`page.views`), where each view represents one route-level screen.

- `page.route` is the current route string (for example `/`, `/store`, `/settings/mail`).
- `page.views` is the active navigation stack.
- `page.on_route_change` rebuilds the stack when route changes.
- `page.on_view_pop` handles Back navigation (system Back, AppBar Back, browser Back).

A reliable setup uses a single source of truth: derive `page.views` from `page.route`.

# Route basics

The default route is `/` when no route is provided.

All routes should start with `/`, for example `/store`, `/products/42`, `/settings/mail`.

# Handling route changes

Whenever route changes (URL edit, browser Back/Forward, or app navigation), `page.on_route_change` event is triggered. Use this event as the place where you decide which views must exist for the current route.

# Building views from route

The pattern below is the baseline for most apps:

1. Clear `page.views`.
2. Add root view (`/`).
3. Add extra views conditionally based on `page.route`.
4. Handle Back in `page.on_view_pop` and navigate to the new top view.

:::tip[Why is this pattern important?] - Keeps URL, history stack, and visible UI synchronized. - Supports deep links and reloads naturally. - Makes navigation deterministic and easier to debug. :::

# Programmatic navigation

Use `page.navigate()` to navigate from synchronous callbacks (e.g. `on_click`), or `page.push_route()` in async contexts:

```
# Sync (on_click, etc.)
page.navigate("/search", q="flet", page=2)

# Async
await page.push_route("/search", q="flet", page=2)
```

# Back navigation and pop confirmation

When users go back, Flet triggers `page.on_view_pop`. For flows requiring confirmation (for example, unsaved changes), disable automatic pop and confirm manually with `View.can_pop` + `View.on_confirm_pop`.

# Pop multiple views with a result

When a multi-step flow finishes deep in the view stack, use `page.pop_views_until()` to jump back to a target route and deliver a result to the destination view via `page.on_views_pop_until`.

# Navigation UI patterns

Routing composes well with navigation controls such as drawer, rail, and tabs. This example shows route-driven drawer navigation with multiple top-level destinations:

# Route templates (parameterized routes)

Use `TemplateRoute` to match and parse route parameters, for example `/books/:id`. Template syntax is provided by repath.

```
        import flet as ft

        troute = ft.TemplateRoute(page.route)

        if troute.match("/books/:id"):
            print("Book ID:", troute.id)
        elif troute.match("/account/:account_id/orders/:order_id"):
            print("Account:", troute.account_id, "Order:",
troute.order_id)
        else:
            print("Unknown route")
```

# Web URL strategy

Flet web apps support two URL strategies :

- `"path"` (default): in the form `https://myapp.dev/store`
- `"hash"`: `https://myapp.dev/#/store`

It can be set via `route_url_strategy` in `ft.run()`

```
        import flet as ft
        ft.run(main, route_url_strategy="hash")
```

For Flet server deployments, you can also set the `FLET_ROUTE_URL_STRATEGY` environment variable.

# Declarative Router

For declarative apps using `@component`, Flet provides a **Router** system inspired by React Router. It handles nested routes, layout routes with outlets, dynamic segments, data loaders, and active link detection — all without manual route matching.

With `manage_views=True`, Router can also manage the view stack directly, enabling swipe-back gestures and AppBar back buttons on mobile. Route components return `View` objects with their own `AppBar`, and navigating deeper pushes views onto the stack.

See the Router cookbook for a complete guide.

# Practical recommendations

- Always keep a root `/` view in `page.views`.
- Keep route handling centralized in `page.on_route_change`; avoid mutating `page.views` from many places.
- When adding new routes, test these cases: direct deep link, browser Back/Forward, app Back button, and reload.
- Use route templates for dynamic segments instead of manual string splitting.

import {Image} from '@site/src/components/crocodocs';

If you build a chat app using Flet you need somehow to pass user messages between sessions. When a user sends a message it should be broadcasted to all other app sessions and displayed on their pages.

Flet provides a simple built-in PubSub mechanism for asynchronous communication between page sessions.

Flet PubSub allows broadcasting messages to all app sessions or sending only to specific "topic" (or "channel") subscribers.

A typical PubSub usage would be:

- `subscribe()` to broadcast messages or `subscribe_topic()` on app session start.
- `send_all()` broadcast message or `send_all_on_topic()` on some event, like "Send" button click.
- `unsubscribe()` from broadcast messages or `unsubscribe_topic()` on some event, like "Leave" button click.
- `unsubscribe_all()` from everything on `page.on_close`.

This is an example of a simple chat application that uses PubSub:

```
import flet as ft

def main(page: ft.Page):
    page.title = "Flet Chat"

    # subscribe to broadcast messages
    def on_message(msg):
        messages.controls.append(ft.Text(msg))
        page.update()
```

```
            page.pubsub.subscribe(on_message)

            def send_click(e):
                page.pubsub.send_all(f"{user.value}: {message.value}")
                # clean up the form
                message.value = ""
                page.update()

            messages = ft.Column()
            user = ft.TextField(hint_text="Your name", width=150)
            message = ft.TextField(hint_text="Your message...",
expand=True)  # fill all the space
            send = ft.Button("Send", on_click=send_click)
            page.add(messages, ft.Row(controls=[user, message, send]))

        ft.run(main, view=ft.AppView.WEB_BROWSER)
```


Chat app

In some cases, you need to read and write files to disk. For example, you might need to persist data across app launches, or download data from the internet and save it for later offline use.

Flet makes it easy to work with files and directories on the mobile/desktop device, as seen in the following example.

## Storage Paths

Flet provides three pre-created, app-private directory paths as environment variables:

- `FLET_APP_STORAGE_DATA` — **durable** data (databases, state, config). Preserved across app updates and backed up; the OS never deletes it. This is also the program's current working directory, so relative writes land here.
- `FLET_APP_STORAGE_CACHE` — **regenerable** cache. The OS may purge it; only store things you can rebuild.
- `FLET_APP_STORAGE_TEMP` — **throwaway** scratch (the OS temp dir). May vanish between launches.

Use `FLET_APP_STORAGE_DATA` for anything you must keep — putting a database in cache or temp risks silent data loss.

Their values can be read as follows:

```
        import os

        app_data_path = os.getenv("FLET_APP_STORAGE_DATA")
        app_cache_path = os.getenv("FLET_APP_STORAGE_CACHE")
        app_temp_path = os.getenv("FLET_APP_STORAGE_TEMP")
```

Because `FLET_APP_STORAGE_DATA` is the working directory, a relative path like `"counter.txt"` resolves inside it; use `os.path.join(app_data_path, ...)` if you want to be explicit. The same behavior applies under `flet run` (the dev working

directory is `<project>/.flet/storage/data`).

### Writing to a File

To write data to a new/existing file, you can use the built-in open function.

For example:

```
import os

app_data_path = os.getenv("FLET_APP_STORAGE_DATA")
my_file_path = os.path.join(app_data_path, "test_file.txt")

with open(my_file_path, "w") as f:
    f.write("Some file content...")
```

### Reading from a File

To read data from an existing file, you can equally use the built-in open function.

For example:

```
import os

app_data_path = os.getenv("FLET_APP_STORAGE_DATA")
my_file_path = os.path.join(app_data_path, "test_file.txt")

with open(my_file_path, "r") as f:
    file_content = f.read()
    print(file_content)
```

Also, you can use the os module (or any other out there) to perform various file operations like renaming, deleting, listing files present in the directory, etc.

## Example: Counter App

Below is an example that showcases a basic Counter application, whose value persists across app launches. This is made possible by writing the counter value to a file in the app's data storage directory and reading it when the app launches.



```
        import os
        from datetime import datetime
        import flet as ft

        # constants
        FLET_APP_STORAGE_DATA = os.getenv("FLET_APP_STORAGE_DATA")
        COUNTER_FILE_PATH = os.path.join(FLET_APP_STORAGE_DATA,
"counter.txt")
        FLET_APP_CONSOLE = os.getenv("FLET_APP_CONSOLE")

        class Counter(ft.Text):
            def __init__(self, storage_path=COUNTER_FILE_PATH):

super().__init__(theme_style=ft.TextThemeStyle.HEADLINE_LARGE)
                self.storage_path = storage_path
                self.count = self.__read_from_storage()

            def increment(self):
                """Increment the counter, store the new value, and
return it."""
                self.count += 1
                self.update()
                self.__write_to_storage()

            def before_update(self):
                super().before_update()
                self.value = f"Button tapped {self.count} time{'' if
self.count == 1 else 's'}"

            def __log(self, action: str, value: int = None):
                """Log executed action."""
                if value is None:
                    value = self.count
                print(f"{datetime.now().strftime('%Y/%m/%d %H:%M:%S')}
- {action} = {value}")

            def __read_from_storage(self):
                """Read counter value. If an error occurs, use 0."""
                try:
                    with open(self.storage_path, "r") as f:
                        value = int(f.read().strip())
                except (FileNotFoundError, ValueError):
                    # file does not exist or int parsing failed
                    value = 0

                self.__log("READ", value)
                return value

            def __write_to_storage(self):
                """Write current counter value to storage."""
                with open(self.storage_path, "w") as f:
```

```
                    f.write(str(self.count))
                self.__log("WRITE")

        def main(page: ft.Page):
            page.theme_mode = ft.ThemeMode.LIGHT
            page.vertical_alignment = ft.MainAxisAlignment.CENTER
            page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

            def show_logs(e: ft.ControlEvent):
                if FLET_APP_CONSOLE is not None:
                    with open(FLET_APP_CONSOLE, "r") as f:
                        dlg = ft.AlertDialog(
                            title=ft.Text("App Logs"),
                            content=ft.Text(f.read()),
                            scrollable=True,
                        )
                        page.open(dlg)

            counter = Counter()
            page.appbar = ft.AppBar(
                title=ft.Text("Storage Playground",
weight=ft.FontWeight.BOLD),
                center_title=True,
                bgcolor=ft.Colors.BLUE,
                color=ft.Colors.WHITE,
                adaptive=True,
                actions=[
                    ft.IconButton(
                        icon=ft.Icons.REMOVE_RED_EYE,
                        tooltip="Show logs",
                        visible=FLET_APP_CONSOLE is not None,
                        on_click=show_logs,
                    ),
                ],
            )
            page.floating_action_button = ft.FloatingActionButton(
                icon=ft.Icons.ADD,
                text="Increment Counter",
                foreground_color=ft.Colors.WHITE,
                bgcolor=ft.Colors.BLUE,
                on_click=lambda e: counter.increment(),
            )
            page.floating_action_button_location =
ft.FloatingActionButtonLocation.CENTER_FLOAT

            page.add(ft.SafeArea(counter))

        ft.run(main)
```

- `Counter` class is a custom control, which is a subclass/extension of the `Text` control. More information here.

- FLET_APP_CONSOLE is an environment variable that points to the application's console log file (`console.log`) which contains the app's console output (ex: `print()` statements). Its value is set in production mode.
- If you have an android emulator or physical device, you can download and install this apk.
- Follow this guide to package your app for all platforms.

Router is a declarative routing system for Flet apps, inspired by React Router. It handles nested route matching, layout routes with outlets, dynamic segments, data loading, view-stack navigation, and provides hooks for accessing route state.

# Why Router?

Without Router, multi-page Flet apps require manual route matching, conditional rendering, and parameter extraction. Router automates all of this:

- **Nested routes** — define parent/child hierarchies
- **Layout routes** — share layouts across pages using use_route_outlet()
- **Dynamic segments** — `:paramName` in paths, accessed via use_route_params()
- **Data loading** — fetch data before rendering with `loader`
- **Active link detection** — highlight nav items with is_route_active()
- **View-stack navigation** — swipe-back and AppBar back button with `manage_views=True`

# Quick start

```
@ft.component
def Home():
    return ft.Text("Welcome home!")

@ft.component
def About():
    return ft.Text("About us")

@ft.component
def App():
    return ft.Router([
        ft.Route(index=True, component=Home),
        ft.Route(path="about", component=About),
    ])

ft.run(lambda page: page.render(App))
```

See full example.

# Defining routes

Routes are defined as a tree of Route objects.

### Flat routes

All routes at the top level:

```
ft.Router([
    ft.Route(index=True, component=Home),
    ft.Route(path="about", component=About),
    ft.Route(path="contact", component=Contact),
])
```

### Nested routes

Child routes are nested under parent routes. The parent's path is automatically prepended:

```
ft.Route(
    path="products",
    component=ProductsList,
    children=[
        ft.Route(path=":pid", component=ProductDetails),
    ],
)
# /products → ProductsList
# /products/42 → ProductDetails (params: {"pid": "42"})
```

See full example.

### Index routes

An index route (`index=True`) matches when the parent path matches exactly, with no further segments:

```
ft.Route(path="settings", children=[
    ft.Route(index=True, component=SettingsHome),   # /settings
    ft.Route(path="profile", component=Profile),     # /settings/profile
])
```

### Prefix routes

**Pathless layout routes** — group children under a shared layout without adding a path segment:

```
ft.Route(component=AdminLayout, children=[
    ft.Route(path="users", component=Users),       # /users
    ft.Route(path="settings", component=Settings), # /settings
])
```

**Path-only routes** — add a path prefix without rendering a component:

```
        ft.Route(path="api", children=[
            ft.Route(path="users", component=ApiUsers),        #
/api/users
            ft.Route(path="products", component=ApiProducts),   #
/api/products
        ])
```

## Layout routes and `use_route_outlet()`

A layout route has both a `component` and `children`. Its component renders shared UI (header, sidebar, footer) and calls use_route_outlet() to place the matched child:

```
        @ft.component
        def AppLayout():
            outlet = ft.use_route_outlet()
            return ft.Column([
                ft.AppBar(title=ft.Text("My App")),
                ft.Container(content=outlet, expand=True),
                ft.Text("Footer"),
            ])

        ft.Router([
            ft.Route(component=AppLayout, children=[
                ft.Route(index=True, component=Home),
                ft.Route(path="about", component=About),
            ]),
        ])
```

Layout routes can be nested to any depth. Each level calls `use_route_outlet()` to get its immediate child. See full example.

## Dynamic segments, optional segments, and splats

Route paths support Express-style patterns via the `repath` library.

**Dynamic segments** — `:paramName` matches a single path segment:

```
        ft.Route(path="users/:userId", component=UserProfile)
        # /users/42 → use_route_params() returns {"userId": "42"}
```

**Optional segments** — `:paramName?` may be absent:

```
        ft.Route(path="users/:userId?", component=UserProfile)
        # /users → {"userId": None}
        # /users/42 → {"userId": "42"}
```

**Splats (catch-all)** — `:paramName*` matches zero or more segments:

```
        ft.Route(path="files/:path*", component=FileBrowser)
        # /files/a/b/c → {"path": "a/b/c"}
```

**Custom regex** — `:paramName(\\d+)` constrains a segment:

```
ft.Route(path="item/:id(\\d+)", component=ItemDetails)
# /item/42 → matches
# /item/abc → does NOT match
```

## Navigation

Use page.navigate() from synchronous callbacks:

```
ft.Button("About", on_click=lambda:
ft.context.page.navigate("/about"))
```

For async contexts, use page.push_route() with `await`. The Router automatically re-renders when the route changes.

## Active links with `is_route_active()`

is_route_active(path) returns `True` if the given path matches the current location:

```
ft.Button(
    "Products",
    style=ft.ButtonStyle(
        bgcolor=ft.Colors.PRIMARY_CONTAINER
        if ft.is_route_active("/products")
        else None,
    ),
    on_click=lambda: ft.context.page.navigate("/products"),
)
```

By default it uses prefix matching — `is_route_active("/products")` returns `True` for `/products/42`. Pass `exact=True` for exact matching. See full example.

## Data loading

Routes can define a `loader` function. It runs when the route matches and its return value is available via use_route_loader_data():

```
def load_user(params):
    return db.get_user(params["userId"])

ft.Route(path="users/:userId", component=UserProfile,
loader=load_user)

@ft.component
def UserProfile():
    user = ft.use_route_loader_data()
    return ft.Text(f"Hello, {user.name}")
```

See full example.

# Authentication

Auth is implemented using layout routes as guards — no special Router API needed.

**Page redirect** — redirect to `/login` if not authenticated:

```
@ft.component
def ProtectedRoute():
    outlet = ft.use_route_outlet()
    if not auth.is_authenticated:
        ft.context.page.navigate("/login")
        return ft.Text("Redirecting...")
    return outlet
```

**Role-based access** — nest guard routes for layered access control:

```
ft.Router([
    ft.Route(component=ProtectedRoute, children=[
        ft.Route(index=True, component=Home),
        ft.Route(component=AdminOnly, children=[
            ft.Route(path="admin", component=AdminPanel),
        ]),
    ]),
])
```

# 404 handling

Pass a `not_found` component to render when no route matches:

```
@ft.component
def NotFound():
    location = ft.use_route_location()
    return ft.Text(f"Page not found: {location}")

ft.Router(routes, not_found=NotFound)
```

# Multi-view navigation

By default, Router renders all route content within a single View. With `manage_views=True`, the Router returns a list of Views — one per path level. This enables:

- Swipe-back gesture on mobile
- System back button
- AppBar implicit back button
- Slide transition between route levels

Use page.render_views() instead of `page.render()`.

### Components return Views

Each route component returns a View with `route`, `appbar`, and `controls`:

```
        @ft.component
        def ProductDetails():
            params = ft.use_route_params()
            return ft.View(
                route=f"/products/{params['pid']}",
                appbar=ft.AppBar(title=ft.Text(f"Product #
{params['pid']}")),
                controls=[
                    ft.Text(f"Details for product #{params['pid']}"),
                ],
            )
```

### Nested route hierarchy

Structure routes as a nested tree. A pathless root ensures it is always in the view stack:

```
        ft.Router([
            ft.Route(component=Home, children=[
                ft.Route(path="products", component=ProductsList,
children=[
                    ft.Route(path=":pid", component=ProductDetails),
                ]),
            ]),
        ], manage_views=True)
```

:::tip[View stack] - `/` — 1 view (Home) - `/products` — 2 views (Home, Products) — back button to Home - `/products/1` — 3 views (Home, Products, Product Details) — back button to Products :::

See full example.

### Shared layouts with `outlet=True`

Set `outlet=True` on a route to make it a layout that wraps child routes via use_route_outlet(). The layout returns a View; child components return regular controls.

Use use_view_path() for `View.route` — it returns the resolved URL for each child view level, so Flutter's Navigator gets a unique key per view in the stack:

```
        @ft.component
        def ProductsLayout():
            outlet = ft.use_route_outlet()
            return ft.View(
                route=ft.use_view_path(),
                appbar=ft.AppBar(title=ft.Text("Products")),
```

```
                controls=[
                    ft.Container(content=outlet, expand=True),
                    ft.Text("Footer"),
                ],
            )

        ft.Route(path="products", component=ProductsLayout,
outlet=True, children=[
            ft.Route(component=ProductsList, children=[
                ft.Route(path=":pid", component=ProductDetails),
            ]),
        ])
```

:::note[`use_route_location()` vs `use_view_path()`] `use_route_location()` always returns the full current URL. `use_view_path()` returns the URL for the *current view level only* — essential as `View.route` when a single layout component wraps multiple child views, so the Flutter Navigator can distinguish them. :::

The layout's shared UI appears on every child route's View, and each child is still a separate View in the stack — back navigation works between them. See full example.

## Avoiding transition animation

When switching between top-level NavigationRail destinations, set a fixed `route` value on the root layout's View to prevent slide transitions:

```
        @ft.component
        def RootLayout():
            outlet = ft.use_route_outlet()
            return ft.View(
                route="/",  # fixed key — no animation between top-
level pages
                can_pop=False,
                controls=[
                    ft.Row([
                        ft.NavigationRail(...),
                        ft.Container(content=outlet, expand=True),
                    ], expand=True),
                ],
            )
```

See full example.

## Back navigation

When the user pops the topmost View (close X on a modal, AppBar back arrow, iOS swipe-back, system back button) the Router by default navigates to the **parent of the current route in the route tree** — i.e. `chain[-2].resolved_path` of the matched chain, not `views[-2].route`. This is robust against shared `View.route` values (the "avoid transition animation" trick above) — a deep-Gallery pop still goes to `/gallery` even if `RootLayout` emits `route="/"` for both the Apps and Gallery tab roots.

If you need fully custom pop behavior, install your own `page.on_view_pop` handler **before** `page.render_views(App)`. The Router detects a pre-existing handler and delegates to it instead of running the default.

# Modal routes

Set `modal=True` on a route to render its View as a fullscreen-dialog overlay that sits on top of the previous (non-modal) location's view stack. Closing the modal pops it without rebuilding the underlying stack — the views underneath stay mounted, so there's no rebuild flash.

The modal's component still returns a `View` like any other route — set `fullscreen_dialog=True` on the View so Flutter renders the slide-up presentation and the close (X) leading icon.

## Global modals — reachable from anywhere

Declared at the top level of the route tree. The base stack comes from the last non-modal location the Router saw (defaults to `/` on first render):

```
        ft.Router([
            ft.Route(component=Home),
            ft.Route(component=Profile, path="profile"),
            ft.Route(component=SettingsModal, path="settings",
modal=True),
        ], manage_views=True)
```

- `/` → stack `[Home]`
- `/profile` → stack `[Profile]`
- Navigate from `/profile` to `/settings` → stack `[Profile, SettingsModal]` (Profile still mounted underneath the modal)
- Close → back to `/profile`, stack `[Profile]` (Profile not rebuilt)
- Deep-link `/settings` directly → stack `[Home, SettingsModal]` (base defaults to `/`)

## Local modals — pinned to a specific page

Declared as a child of a non-modal parent. The base stack is the chain above the modal in the route tree, so the URL itself encodes which item the modal applies to and deep-link works without any state:

```
        ft.Router([
            ft.Route(component=ItemList, path="items", children=[
                ft.Route(component=ItemDetails, path=":id", children=[
                    ft.Route(component=EditModal, path="edit",
modal=True),
                ]),
            ]),
        ], manage_views=True)
```

- `/items/42` → stack `[ItemList, ItemDetails(42)]`

- `/items/42/edit` → stack `[ItemList, ItemDetails(42), EditModal]`
- Close → `/items/42`

See full example.

# Recursive routes

A route marked `recursive=True` can match itself as its own descendant — one `View` is emitted per consumed URL segment. Use this for tree-shaped URLs with unbounded depth without writing nested duplicates:

```
ft.Router([
    ft.Route(path="folder", component=Folders, children=[
        ft.Route(
            path=":name",
            component=Folder,
            recursive=True,
            children=[
                ft.Route(path="search", component=Search),
            ],
        ),
    ]),
], manage_views=True)
```

- `/folder/a/b/c` → stack `[Folders, Folder(a), Folder(a/b), Folder(a/b/c)]`
- Back-swipe walks one segment at a time

Inside the component, `use_route_params()` returns just the **current segment** (e.g. `{"name": "b"}` at depth 2). Use `use_view_path()` for the accumulated URL up to this level — useful when the component needs to know the full path.

## Sibling-wins-over-recursion rule

A non-recursive child of the recursive route is tried **before** self-recursion at every depth. In the example above, `/folder/x/search` matches as `[Folders, Folder(x), Search]` — the `Search` sibling wins over `Folder("search")`, so you only need to declare the special child once, not at every depth.

This makes the recursive route ideal for replacing N-level fan-outs like:

```
# Without recursive=True — declared N times for max depth N:
ft.Route(path=":s1", component=Folder, children=[
    ft.Route(path="search", component=Search),
    ft.Route(path=":s2", component=Folder, children=[
        ft.Route(path="search", component=Search),
        ft.Route(path=":s3", component=Folder, children=[
            ft.Route(path="search", component=Search),
            ft.Route(path=":s4", component=Folder, children=
[...]),
        ]),
```

```
            ]),
        ]),

        # With recursive=True — one declaration, unbounded depth:
        ft.Route(path=":name", component=Folder, recursive=True,
children=[
            ft.Route(path="search", component=Search),
        ]),
```

See full example.

# Hooks reference

| Hook | Returns | Description |
|---|---|---|
| use_route_params() | `dict[str, str]` | All dynamic segment values from the matched route chain |
| use_route_location() | `str` | Current URL pathname |
| use_view_path() | `str` | Resolved URL for the current view level (unique per view in `manage_views` mode) |
| use_route_outlet() | component | Matched child route component (for layout routes) |
| use_route_loader_data() | `Any` | Return value of the current route's `loader` |
| is_route_active(path) | `bool` | Whether `path` matches the current location |

Flet provides an API for storing key-value data in user's session on a server side.

In the current Flet implementation the data stored in a session store is transient and is not preserved between app restarts.

## Write data

```
        # strings
        page.session.store.set("key", "value")

        # numbers
        page.session.store.set("number.setting", 12345)

        # booleans
        page.session.store.set("bool_setting", True)

        # lists
        page.session.store.set("favorite_colors", ["red", "green",
"blue"])
```

### Read data

```
        # The value is automatically converted back to the original
type
        value = page.session.store.get("key")

        colors = page.session.store.get("favorite_colors")
        # colors = ["red", "green", "blue"]
```

### Check key existence

```
        page.session.store.contains_key("key") # True if the key
exists
```

### Get all keys

```
        page.session.store.get_keys()
```

#### Remove data by key

```
        page.session.store.remove("key")
```

#### Clear all data

```
        page.session.store.clear()
```

In this cookbook recipe, you'll learn how to run external system commands from your Flet app using Python's built-in `subprocess` module.

This approach is useful when you need to interact with platform tools or system utilities that are not exposed through Flet APIs.

Running external commands is **not supported when your app runs in a browser**.

# Examples

## Launching another Android app

Sometimes your Flet app may need to open another Android application. For example, you might want to open the **Settings** app so the user can adjust system or app settings.

On Android, this can be done using the `am start` command, which launches activities from the Android shell.

The command requires the **component name** (`-n`) of the target activity in the form: `package.name/.ActivityName`.

For example, the Android Settings app uses:

```
com.android.settings/.Settings
```

It is also recommended to pass the `--user` flag. Without it, the command may fail with an error such as:

```
        Security exception: Permission Denial: startActivityAsUser
asks to run as
        user -2 but is calling from uid u0a164; this requires
android.permission.INTERACT_ACROSS_USERS_FULL
```

This happens because Android may attempt to launch the activity for a different user than the one your app runs under. Setting `--user 0` ensures the command runs for the primary user.

The full command therefore becomes:

```
        am start -n com.android.settings/.Settings --user 0
```

And can be run from a Flet app like this:

```
        import subprocess
        import flet as ft

        def main(page: ft.Page):
            def open_settings(e):
                result = subprocess.run(
                    ["am", "start", "-n",
"com.android.settings/.Settings", "--user", "0"],
                    shell=False, # default
                    capture_output=True,
                    text=True,
                )

                print("STDOUT:", result.stdout)
                print("STDERR:", result.stderr)

            page.add(
                ft.SafeArea(
                    content=ft.Button("Open Settings app",
on_click=open_settings)
                )
            )

        ft.run(main)
```

Running this app on an Android device, you should see the **Settings** app launch when the button is clicked.

**Common Pitfall**

When using `subprocess.run()`, be careful with the `shell` argument.

This will **not work correctly**:

```
        subprocess.run(
            ["am", "start", "-n", "com.android.settings/.Settings", "-
-user", "0"],
            shell=True
        )
```

Because when `shell=True` is used, the command must be passed as a **single string**, not a list.

Correct usage:

```
        subprocess.run(
            "am start -n com.android.settings/.Settings --user 0",
            shell=True
        )
```

However, when possible, prefer **`shell=False` with a list of arguments**, as shown in the main example. This is generally safer and avoids shell parsing issues.

It is possible to configure your application and/or the containing controls to follow a particular themes.

## App-wide themes

The `Page` control (uppermost control in the tree) has two useful properties for this: `theme` and `dark_theme` properties to configure the appearance/theme of the entire app in light and dark theme modes respectively.

Both of type `Theme`, they represent the default/fallback themes to be used app-wide, except explicitly modified/overriden in the tree.

```
        page.theme = ft.Theme(color_scheme_seed=ft.Colors.GREEN)
        page.dark_theme = ft.Theme(color_scheme_seed=ft.Colors.BLUE)
```

## Nested themes

You can have a part of your app to use a different theme or override some theme styles for specific controls.

Some container-like controls have `theme` and `theme_mode` properties of type `Theme` and `ThemeMode` respectively.

Specifying `theme_mode` in the `Container` means you don't want to inherit parent theme mode, but want a completely new, unique scheme for all controls inside the container. However, if the container does not have `theme_mode` property set then the styles from its theme property will override the ones from the parent inherited theme:

```
        import flet as ft

        def main(page: ft.Page):
            # Yellow page theme with SYSTEM (default) mode
            page.theme = ft.Theme(
                color_scheme_seed=ft.Colors.YELLOW,
```

```
            )

            page.add(
                # Page theme
                ft.Container(
                    content=ft.Button("Page theme button"),
                    bgcolor=ft.Colors.SURFACE_CONTAINER_HIGHEST,
                    padding=20,
                    width=300,
                ),

                # Inherited theme with primary color overridden
                ft.Container(

theme=ft.Theme(color_scheme=ft.ColorScheme(primary=ft.Colors.PINK)),
                    content=ft.Button("Inherited theme button"),
                    bgcolor=ft.Colors.SURFACE_CONTAINER_HIGHEST,
                    padding=20,
                    width=300,
                ),

                # Unique always DARK theme
                ft.Container(

theme=ft.Theme(color_scheme_seed=ft.Colors.INDIGO),
                    theme_mode=ft.ThemeMode.DARK,
                    content=ft.Button("Unique theme button"),
                    bgcolor=ft.Colors.SURFACE_CONTAINER_HIGHEST,
                    padding=20,
                    width=300,
                ),
            )

        ft.run(main)
```




Flet controls based on 3rd-party Flutter packages that used to be a part of Flet repository, now have been moved to separate repos and published on pypi:

- flet-ads
- flet-audio
- flet-audio-recorder
- flet-camera
- flet-charts
- flet-datatable2
- flet-flashlight
- flet-geolocator
- flet-lottie
- flet-map
- flet-permission-handler
- flet-rive
- flet-secure-storage

- flet-video
- flet-webview

To use a built-in Flet extension in your project, add it to the `dependencies` section of your `pyproject.toml` file, for example:

```
dependencies = [
  "flet-audio",
  "flet>=0.26.0",
]
```

While Flet controls leverage many built-in Flutter widgets to enable the creation of complex applications, not all Flutter widgets or third-party packages can be directly supported by the Flet team or included in the core Flet framework. At the same time, the Flutter ecosystem is vast and offers developers a wide range of possibilities to extend functionality beyond the core.

To address this, the Flet framework provides an extensibility mechanism. This allows you to incorporate widgets and APIs from your own custom Flutter packages or third-party libraries directly into your Flet application.

In this guide, you will learn how to create Flet extension from template and then customize it to integrate 3rd-party Flutter package into your Flet app.

### Prerequisites

To integrate custom Flutter package into Flet you need to have basic understanding of how to create Flutter apps and packages in Dart language and have Flutter development environment configured. See Flutter Getting Started for more information about Flutter and Dart.

## Create Flet extension from template

Flet now makes it easy to create and build projects with your custom controls based on Flutter widgets or Flutter 3rd-party packages. In the example below, we will be creating a custom Flet extension based on the flutter_spinkit package.

**Step 1.** Create new virtual environment and install Flet there.

**Step 2.** Create new extension project from template.

```
flet create --template extension --project-name flet-spinkit
```

A project with new FletSpinkit control will be created. The control is just a Flutter Text widget with text property, which we will customize later.

**Step 3.** Build example app.

Flet project created from extension template has `examples/flet_spinkit_example` folder with the example app.

When in the folder where your `pyproject.toml` for the app is (`examples/flet_spinkit_example`), run `flet build` command, for example, for macos:

```
flet build macos -v
```

Open the app and see the new custom Flet Control:

```
open build/macos/flet-spinkit-example.app
```



**Change Python files**

Once the project was built for desktop once, you can make changes to your python files and run it without rebuilding.

First, if you are not using uv, install dependencies from pyproject.toml:

```
pip install .
```

or

```
poetry install
```

Now you can make changes to your example app main.py:

```
        import flet as ft

        from flet_spinkit import FletSpinkit

        def main(page: ft.Page):
            page.vertical_alignment = ft.MainAxisAlignment.CENTER
            page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

            page.add(
                ft.Container(
                    height=150,
                    width=300,
                    alignment=ft.Alignment.CENTER,
                    bgcolor=ft.Colors.PINK_200,
                    content=FletSpinkit(
                        tooltip="My new PINK FletSpinkit Control
tooltip",
                        value="My new PINK FletSpinkit Flet Control",
                    ),
                ),
            )

        ft.run(main)
```

and run:

```
flet run
```



**Change Flutter package**

When you make any changes to your flutter package, you need to rebuild:

```
flet build macos -v
```

If you need to debug, run this command:

```
build/macos/flet-spinkit-example.app/Contents/MacOS/flet-spinkit-example --debug
```

# Integrate 3rd-party Flutter package

Let's integrate flutter_spinkit package into our Flet app.

**Step 1.** Add dependency

Go to `src/flutter/flet_spinkit` folder and run this command to add dependency to `flutter_spinkit` to `pubspec.yaml`:

```
flutter pub add flutter_spinkit
```

Read more information about using Flutter packages here.

**Step 2.** Modify `dart` file

In the `src/flutter/flet_spinkit/lib/src/flet_spinkit.dart` file, add import statement and replace Text widget with `SpinKitRotatingCircle` widget:

```
import 'package:flet/flet.dart';
import 'package:flutter/material.dart';
import 'package:flutter_spinkit/flutter_spinkit.dart';

class FletSpinkitControl extends StatelessWidget {
  final Control control;

  const FletSpinkitControl({
    super.key,
    required this.control,
  });

  @override
  Widget build(BuildContext context) {
    Widget myControl = SpinKitRotatingCircle(color:
Colors.red, size: 100.0);

    return LayoutControl(control: control, child: myControl);
```

```
            }
        }
```

**Step 3.** Rebuild example app

Go to `examples/flet_spinkit_example`, clear cache and rebuild your app:

```
flet build macos -v
```

**Step 4.** Run your app



# Flet extension structure

After creating new Flet project from extension template, you will see the following folder structure:

```
├── LICENSE
├── mkdocs.yml
├── README.md
├── docs
│   └── index.md
│   └── FletSpinkit.md
├── examples
│   └── flet_spinkit_example
│       ├── README.md
│       ├── pyproject.toml
│       └── src
│           └── main.py
├── pyproject.toml
└── src
    ├── flet_spinkit
    │   ├── __init__.py
    │   └── flet_spinkit.py
    └── flutter
        └── flet_spinkit
            ├── CHANGELOG.md
            ├── LICENSE
            ├── README.md
            ├── lib
            │   ├── flet_spinkit.dart
            │   └── src
            │       ├── create_control.dart
            │       └── flet_spinkit.dart
            └── pubspec.yaml
```

Flet extension consists of: * **package**, located in `src` folder * **example app**, located in `examples/flet-spinkit_example` folder * **docs**, located in `docs` folder

## Package

Package is the component that will be used in your app. It consists of two parts: Python and Flutter.

### Python

#### *flet_spinkit.py*

Defines the Python-side Flet control. `FletSpinkit` is registered with `@ft.control("flet_spinkit")` and inherits from `ft.LayoutControl`, which ties it to the Flutter `Control.type` handled in the extension. The class currently includes a value: str property and a placeholder docstring.

### Flutter

#### *pubspec.yaml*

Flutter package manifest for the extension. Declares SDK constraints and dependencies. Notable deps:

- `flet` for Flet extension APIs
- `flutter_spinkit` for the spinner widgets used by the control

#### *flet_spinkit.dart*

Library entrypoint. Exports the public `Extension` class from `extension.dart`.

#### *src/extension.dart*

Registers the extension with Flet. `Extension.createWidget` maps `Control.type` to the Flutter widget; currently maps “flet_spinkit” to FletSpinkitControl.

#### *src/flet_spinkit.dart*

Flutter wrapper widget for the control. `FletSpinkitControl` builds a `SpinKitRotatingCircle` and wraps it with `LayoutControl` so layout/state from the Python control are applied.

## Example app

#### *src/main.py*

Python program that uses Flet Python control.

#### *pyproject.toml*

Here you specify dependency to your package, which can be:

- **Path dependency**

Absolute path to your Flet extension folder, for example:

```
dependencies = [
  "flet-spinkit @ file:///Users/user-name/projects/flet-spinkit",
  "flet>=0.80.2",
]
```

- **Git dependency**

Link to git repository, for example:

```
dependencies = [
  "flet-ads @ git+https://github.com/flet-dev/flet-ads.git",
  "flet>=0.80.2",
]
```

- **PyPi dependency**

Name of the package published on pypi.org, for example:

```
dependencies = [
  "flet-ads",
  "flet>=0.80.2",
]
```

## Docs

If you are planning to share your extension with community, you can easily generate documentation from your source code using mkdocs.

Flet extension comes with a `docs` folder containing initial files for your documentation and a `mkdocs.yml` file at the project root.

From the folder that contains `mkdocs.yml`, run the following command to see how your docs look locally:

```
mkdocs serve
```

Open http://127.0.0.1:8000 in your browser:



Once your documentation is ready, if your package is hosted on GitHub, your can run the following command to host your documentation on GitHub pages:

```
mkdocs gh-deploy
```

You may find this guide helpful to get started with mkdocs.

# Customize properties

In the example above, Spinkit control creates a hardcoded Flutter widget. Now let's customize its properties.

## Common properties

Generally, there are three types of controls in Flet:

1. Visual controls that are added to the app/page surface, such as FletSpinkit.
2. Dialog and other popup controls (dialogs, pickers, panels, etc.) that are opened from the page (for example, `page.open(dlg)`).
3. Services (Clipboard, Battery, Video, Audio, etc.) that are created as standalone instances and automatically registered with the page.

When creating a visual control that should participate in layout (size, position, transforms, margin, etc.), define a dataclass-based control annotated with `@ft.control("control_name")` and inherit from `LayoutControl`. In its Dart counterpart (`src/flet_spinkit.dart`), wrap your Flutter widget with `LayoutControl(...)`.

When creating a dialog or other popup control (opened with `page.open(...)`), define a dataclass-based control annotated with `@ft.control("control_name")` and inherit from `DialogControl`. In its Dart counterpart, show the dialog/popup (for example, `showDialog` or `showModalBottomSheet`) and return a placeholder widget like `SizedBox.shrink()` instead of wrapping with `LayoutControl(...)` or `BaseControl(...)`.

When creating a service control (Clipboard, Battery, Video, Audio, etc.), define a dataclass-based control annotated with `@ft.control("control_name")` and inherit from `Service`. In its Dart counterpart, implement `FletService` and register it via `FletExtension.createService` (no widget wrapper).

You can use all `LayoutControl`, `DialogControl`, and `Service` properties inherited by your dataclass-based control without re-declaring them as fields (unless you want to override defaults or metadata).

If you have created your extension project from Flet extension template, your Python Control is already inherited from `LayoutControl` and you can use its properties in your example app:

```
import flet as ft

from flet_spinkit import FletSpinkit

def main(page: ft.Page):
    page.vertical_alignment = ft.MainAxisAlignment.CENTER
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

    page.add(
        ft.Stack(
```

```
                    [
                        ft.Container(height=200, width=200,
bgcolor=ft.Colors.BLUE_100),
                        FletSpinkit(opacity=0.5, tooltip="Spinkit
tooltip", top=0, left=0),
                    ]
                )
            )

        ft.run(main)
```



## Control-specific properties

Now that you have taken full advantage of the properties Flet `LayoutControl` offer, let's define the properties that are specific to the new Control you are building.

In the FletSpinkit example, let's define its `color` and `size`.

In Python class, define new `color` and `size` properties:

```
from typing import Optional

import flet as ft

@ft.control("flet_spinkit")
class FletSpinkit(ft.LayoutControl):
    """
    FletSpinkit Control description.
    """

    color: Optional[ft.ColorValue] = None
    size: float = 100.00
```

In `src/flet_spinkit.dart` file, use helper methods `getColor` and `getDouble` to access color and size values:

```
import 'package:flet/flet.dart';
import 'package:flutter/material.dart';
import 'package:flutter_spinkit/flutter_spinkit.dart';

class FletSpinkitControl extends StatelessWidget {
  final Control control;

  const FletSpinkitControl({
    super.key,
    required this.control,
  });

  @override
  Widget build(BuildContext context) {
```

```
            Widget myControl = SpinKitRotatingCircle(
              color: control.getColor("color", context),
              size: control.getDouble("size") ?? 100.0,
            );

            return LayoutControl(control: control, child: myControl);
          }
        }
```

Use `color` and `size` properties in your app:

```
        import flet as ft

        from flet_spinkit import FletSpinkit

        def main(page: ft.Page):
            page.vertical_alignment = ft.MainAxisAlignment.CENTER
            page.horizontal_alignment = ft.CrossAxisAlignment.CENTER

            page.add(
                ft.Stack(
                    controls=[
                        ft.Container(height=200, width=200,
bgcolor=ft.Colors.BLUE_100),
                        FletSpinkit(
                            color=ft.Colors.YELLOW,
                            size=150,
                            opacity=0.5,
                            tooltip="Spinkit tooltip",
                            top=0,
                            left=0,
                        ),
                    ]
                )
            )

        ft.run(main)
```

Rebuild and run:



:::warning[Important: Default values must match on both sides]

Properties with default values on the Python side are **not sent to Flutter** when the user hasn't changed them from the default. This means your Dart code **must provide the same default value** for every property that has one in Python.

For example, if your Python control declares:

```
        size: float = 100.0
        animate: bool = True
```

Then your Dart code **must** use matching defaults:

```
        // Correct - defaults match Python
        final size = control.getDouble("size", 100.0)!;
        final animate = control.getBool("animate", true)!;

        // Wrong - no default, will be null when user doesn't set the
property
        final size = control.getDouble("size");      // returns null!
        final animate = control.getBool("animate");  // returns null!
```

This also applies to `@ft.value` types parsed with helper functions. If a value type field has a default, the corresponding `parseDouble()`, `parseBool()`, `parseDuration()`, etc. call on the Dart side must provide the same default.

Common pitfalls:

- **Missing defaults**: `control.getDouble("prop")` instead of `control.getDouble("prop", 0.0)!`
- **Mismatched defaults**: Python has `True` but Dart defaults to `false`
- **Unit mismatches**: Python uses `Duration(milliseconds=150)` but Dart uses `Duration(microseconds: 150)`
- **Empty collections**: `field(default_factory=list)` means an empty list won't be sent; Dart must handle null with `?? const []`

:::

You can find source code for this example here.

# Dedicated data channels

Property updates and method calls between Python and Dart go over Flet's MsgPack-framed control protocol. That works well for widget state and small events but caps at ~1 GB/s with several allocations per frame. For widgets that need to move **bulk binary data** — image frames, audio buffers, file blobs, ML inference tensors — Flet provides **dedicated `DataChannels`** that bypass MsgPack entirely and reach memory-bandwidth-class throughput on every supported platform.

Examples of widgets that benefit:

- A chart receiving a stream of bitmaps from Python (1080p RGBA at 60 fps ≈ 480 MB/s — far above what the control protocol can sustain)
- A camera widget pushing frames Dart → Python for ML inference
- A microphone widget streaming PCM samples to a Python DSP pipeline

For sub-KB widget state, you don't need this — the regular property protocol is faster end-to-end at that size. Reach for `DataChannel` when payloads start at a few KB and especially when they stream at high rates.

## How it works

Allocation lives on the **Dart side**: the widget calls `FletBackend.of(context).openDataChannel()` in `initState`, then **fires a control event named `data_channel_open`** carrying `{channel_name, channel_id}`. The Python side declares an `on_data_channel_open` handler, retrieves the matching channel via `Control.get_data_channel(channel_id)`, and starts sending/receiving bytes.

Transport is chosen per deployment mode automatically — dedicated `PythonBridge` per channel in embedded native (`flet build` for desktop/mobile), raw-byte frames muxed over `postMessage` with Transferable ArrayBuffer in Pyodide, muxed over the protocol socket in `flet run` dev mode, muxed over WebSocket with a Python server. **Widget code is identical across all modes.**

## Python-side API

```
        from typing import Callable, Optional

        import flet as ft


        @ft.control("MyImageChart")
        class MyImageChart(ft.LayoutControl):
            on_data_channel_open:
Optional[ft.EventHandler[ft.DataChannelOpenEvent]] = None

            def init(self) -> None:
                # `init` is the @ft.control post-construct hook (runs
before `did_mount`).
                # Wire up the channel-capture handler here.
                self._frames: Optional[ft.DataChannel] = None
                if self.on_data_channel_open is None:
                    self.on_data_channel_open = self._capture_channel

            def _capture_channel(self, e: ft.DataChannelOpenEvent) ->
None:
                # `e.channel_name` is the label the Dart side put in
the event
                # payload — dispatch on it when a widget opens
multiple channels.
                # Single-channel widgets can ignore it.
                self._frames = self.get_data_channel(e.channel_id)
                # Optional: subscribe to bytes flowing Dart → Python.
                self._frames.on_bytes(self._on_frame_from_dart)

            def push_frame(self, rgba_bytes: bytes) -> None:
                """Python → Dart — fire-and-forget byte send."""
                if self._frames is not None:
                    self._frames.send(rgba_bytes)

            def _on_frame_from_dart(self, payload: bytes) -> None:
```

```
                # Called from the transport's delivery thread (under
the GIL in
                # embedded native mode). Push to a `queue.Queue` and
let a worker
                # drain — don't do heavy CPU work here, it'll starve
the transport.
                ...
```

`ft.DataChannel` is an abstract class — instances come back from `Control.get_data_channel(channel_id)`. Its surface:

- `send(payload: bytes)` — Python → Dart, fire-and-forget
- `on_bytes(callback: Callable[[bytes], None] | None)` — register a handler for bytes pushed from Dart; pass `None` to clear
- `close()` — release the channel (idempotent; the framework auto-closes on control unmount, you rarely need to call it explicitly)

The `ft.DataChannelOpenEvent` fields are `channel_name: str` and `channel_id: int`. The field is `channel_name`, not `name`, because `name` is reserved on the base `Event` class for the event's own name (`"data_channel_open"`).

## Dart-side API

```dart
import 'package:flet/flet.dart';

class MyImageChartState extends State<MyImageChartWidget> {
  late final DataChannel _frames;
  StreamSubscription<Uint8List>? _sub;

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    if (_sub != null) return;  // initialise lazily, once
    _frames = FletBackend.of(context).openDataChannel();
    _sub = _frames.messages.listen(_onFrameFromPython);
    // Tell Python about the channel via a regular control
event.
    widget.control.triggerEvent("data_channel_open", {
      "channel_name": "frames",
      "channel_id": _frames.id,
    });
  }

  void _onFrameFromPython(Uint8List bytes) {
    // hand to a Texture, dart:ui.Image.fromPixels, etc.
  }

  void _pushFrameToPython(Uint8List bytes) {
    _frames.send(bytes);
  }

  @override
```

```
  void dispose() {
    _sub?.cancel();
    _frames.close();
    super.dispose();
  }
}
```

`DataChannel` Dart-side surface:

- `int get id` — the channel identifier that goes into the `data_channel_open` event payload
- `Stream<Uint8List> get messages` — bytes pushed from Python
- `bool send(Uint8List bytes)` — Dart → Python, fire-and-forget
- `void close()` — release the channel (idempotent)

**Allocate the channel in `didChangeDependencies`, not `initState`** — `FletBackend.of(context)` needs an active `BuildContext` and that's the first lifecycle hook where it's safely available.

Neither side imports `serious_python` or `dart_bridge`. The DataChannel API surface lives entirely in `package:flet` / `flet`, so your extension's dependencies stay the same.

## Multiple channels per widget

A control can open as many channels as it needs — each `openDataChannel()` call mints a unique id. Disambiguate them by `channel_name` in the event:

```
// Dart — open two channels, label each
_frames = FletBackend.of(context).openDataChannel();
_audio  = FletBackend.of(context).openDataChannel();
widget.control.triggerEvent("data_channel_open", {
  "channel_name": "frames",
  "channel_id": _frames.id,
});
widget.control.triggerEvent("data_channel_open", {
  "channel_name": "audio",
  "channel_id": _audio.id,
});

# Python — dispatch on channel_name
def _on_data_channel_open(self, e: ft.DataChannelOpenEvent) ->
None:
    match e.channel_name:
        case "frames":
            self._frames = self.get_data_channel(e.channel_id)
        case "audio":
            self._audio = self.get_data_channel(e.channel_id)
```

### Threading + backpressure

The `on_bytes` handler runs **synchronously under the GIL** on whatever OS thread the transport delivered from. For anything CPU-heavy (PNG decode, ML inference) push the payload onto a `queue.Queue` or `asyncio.Queue` and let a worker drain — blocking the delivery thread will starve the transport.

The receiving side's queue is **unbounded by default**. If a producer outpaces the consumer (camera frames into a slow decoder, matplotlib rotation into Flutter paint), memory grows. For media-streaming widgets, implement backpressure or a drop-old policy on the producer side. A reference example is `MatplotlibChartCanvas` — it uses a 1-byte ack frame from Dart so the producer waits per-frame, mirroring matplotlib WebAgg's `waiting` flag pattern.

### See also

- **First-party reference widget**: `MatplotlibChartCanvas` — a complete migration from `_invoke_method` PNG dispatch to a 1-byte-opcode data channel, including backpressure ack and the GPU / CPU rendering strategies. Python · Dart
- **Full design / performance notes** (wire format, cross-mode operation, concurrency model, empirical numbers): `dedicated-data-channels.md` in `flet-dev/serious-python`.
- **Wire-format protocol upgrade** for anyone implementing a custom Flet backend or sidecar: Flet protocol framing upgraded for DataChannel support.

## Boot screen

An extension can provide a custom **boot screen** — the screen shown while your packaged app is starting, before the first page appears (see Boot screen in the publishing guide for the end-user/configuration side). This lets you fully brand the launch experience, including custom layouts and animations, instead of using the built-in `flet` screen.

### The `createBootScreen` hook

Override `createBootScreen` on your `FletExtension` (in `src/extension.dart`). A single extension can serve multiple named screens:

```
        import 'package:flet/flet.dart';
        import 'package:flutter/material.dart';

        import 'my_boot_screen.dart';

        class Extension extends FletExtension {
          @override
          Widget? createBootScreen(String name, Map<String, dynamic>
options,
              ValueListenable<BootStatus> status) {
            switch (name) {
              case "my_screen":
```

```
            return MyBootScreen(options: options, status: status);
          default:
            return null; // not handled — Flet tries the next
extension, then the built-in "flet" screen
        }
      }
    }
```

The hook receives:

- **`name`** — the configured screen name (from `[tool.flet.boot_screen].name`). Return `null` for names you don't handle.
- **`options`** — the `[tool.flet.boot_screen.<name>]` table from `pyproject.toml` as a `Map<String, dynamic>`. Put whatever options your screen needs there.
- **`status`** — a `ValueListenable<BootStatus>` that reports the current boot state. Watch it with a `ValueListenableBuilder` to update the UI live.

`BootStatus` carries:

| Field | Type | Description |
|---|---|---|
| `stage` | `BootStage` | `BootStage.preparing` (unpacking the app bundle — Android only) or `BootStage.startingUp` (Python runtime + app starting). |
| `error` | `String?` | Non-null when startup failed; the message is ready for display. |

## Example widget

```
    import 'package:flet/flet.dart';
    import 'package:flutter/material.dart';

    class MyBootScreen extends StatelessWidget {
      final Map<String, dynamic> options;
      final ValueListenable<BootStatus> status;

      const MyBootScreen({super.key, required this.options,
required this.status});

      @override
      Widget build(BuildContext context) {
        // Read options from pyproject.toml. Colors accept hex
(#rrggbb) or named
        // colors via Flet's HexColor helper.
        final bgcolor =
            HexColor.fromString(null, options["bgcolor"] as
String?) ?? Colors.black;

        return ValueListenableBuilder<BootStatus>(
          valueListenable: status,
          builder: (context, boot, _) {
```

```
                if (boot.error != null) {
                  return Scaffold(
                    backgroundColor: bgcolor,
                    body: Center(
                      child: Text(boot.error!,
                          style: const TextStyle(color:
Colors.redAccent)),
                    ),
                  );
                }
                final message = boot.stage == BootStage.preparing
                    ? "Preparing…"
                    : "Starting up…";
                return Scaffold(
                  backgroundColor: bgcolor,
                  body: Center(
                    child: Column(
                      mainAxisAlignment: MainAxisAlignment.center,
                      children: [
                        const CircularProgressIndicator(),
                        const SizedBox(height: 16),
                        Text(message, style: const TextStyle(color:
Colors.white)),
                      ],
                    ),
                  ),
                );
              },
            );
          }
        }
```

Then select your screen by name in the app's `pyproject.toml`:

```
[tool.flet.boot_screen]
name = "my_screen"

[tool.flet.boot_screen.my_screen]
bgcolor = "#101020"
```

In a packaged app, the boot screen is rendered by a persistent host overlay so your widget is **not** remounted between the prepare and startup phases — the same instance (and its animations) is reused, and the host fades it out when the app is ready. Drive your UI from `status`, and don't rely on the app's `Theme` for colors during the prepare phase — read them from `options` so the screen looks consistent throughout. The host also reads `fade_out_duration` (ms) from your `options` to control that fade-out.

:::tip[Real-world example] The `flet-spinkit` extension ships a complete `spinkit` boot screen built exactly this way — see `boot_screen.dart` and its `extension.dart`. :::

# Examples

Flet has controls that are implemented as built-in extensions and could serve as a starting point for your own controls.

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {CodeExample, Image} from '@site/src/components/crocodocs';

A Flet app's UI is made up of controls, arranged on the page. Controls can be styled, nested inside each other to build layouts, and respond to events like clicks and taps.

This page walks through creating your first app, then the controls and events you'll use to build almost any UI, tying them together into a small example app.

# Your first app

Create a new directory (or directory with `pyproject.toml` already exists if initialized with a project manager) and switch into it.

To create a new "minimal" Flet app run the following command:

```
uv run flet create

flet create
```

:::danger[Important] Any existing `README.md` or `pyproject.toml` (for example, created by `uv init`) will be replaced by the one created by `flet create` command. :::

The command will create the following directory structure:

```
README.md
pyproject.toml
src
    assets
        icon.png
        splash_android.png
    main.py # (1)!
tests
    test_main.py # (2)!
```

1. Contains a simple Flet program. It has `main()` function where you would add UI elements (controls) to a page or a window. The application ends with a `ft.run()` function which initializes the Flet app and runs `main()`.
2. A sample integration test for the app, ready to run with `flet test`.

You can find more information about `flet create` command here.

`src/main.py` already contains a small working counter app with a button that increments it:


Counter app running on macOS

- `page` is the top-level container for everything in the app window (or browser tab).
- `page.add()` appends controls to the page.
- `page.floating_action_button` sets the round action button in the bottom-right corner.
- `increment_click` is an event handler (see Handling events below); Flet renders the changes it makes as soon as the handler returns.

See Running a Flet app to launch it as a desktop window or in a browser.

# Basic controls

You'll most likely need controls for showing text, laying out other controls, and adding a background, border, or padding.

## Text

`Text` displays a string, with optional styling:

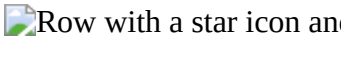
```
        ft.Text("Flet is fun to build with!", size=20,
weight=ft.FontWeight.BOLD, color=ft.Colors.BLUE)
```


Bold blue Text control

## Row and Column

`Row` and `Column` lay out their `controls` horizontally and vertically, respectively. Both accept `alignment` (main axis) and `vertical_alignment`/`horizontal_alignment` (cross axis) to control spacing and positioning:

```
        ft.Row(
            alignment=ft.MainAxisAlignment.CENTER,
            controls=[
                ft.Icon(ft.Icons.STAR),
                ft.Text("Featured"),
            ],
        )
```


Row with a star icon and Featured text

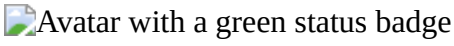
You can nest a `Column` inside a `Row` (or vice versa) to build more complex layouts.

### Stack

Stack overlaps its children instead of laying them out in a line. Children are positioned with `top`, `bottom`, `left`, and `right`, which makes `Stack` useful for badges, overlays, and anything else that needs to sit on top of another control:

```
        ft.Stack(
            controls=[

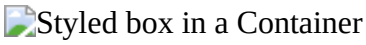
ft.CircleAvatar(foreground_image_src="https://picsum.photos/100"),
                ft.Container(
                    width=14,
                    height=14,
                    bgcolor=ft.Colors.GREEN,
                    border_radius=7,
                    right=0,
                    bottom=0,
                ),
            ],
        )
```




### Container

Container wraps a single control and adds visual styling around it: background color, border, border radius, padding, margin, and fixed width/height:

```
        ft.Container(
            content=ft.Text("Styled box"),
            bgcolor=ft.Colors.AMBER_100,
            padding=12,
            border_radius=8,
        )
```




`Container` also accepts `on_click`, which makes it a handy way to make a control clickable when that control doesn't have an `on_click` of its own: just wrap it in a `Container`. See Handling events below.

## Structuring the page

Beyond individual controls, `page` itself has properties that shape the whole app: `page.title` sets the window/tab title, and `page.appbar` puts an AppBar (the header row with a title and actions) at the top of the page:

```
        def main(page: ft.Page):
            page.title = "My App"
            page.appbar = ft.AppBar(
                title=ft.Text("My App"),
```

```
            bgcolor=ft.Colors.SURFACE_TINT,
            actions=[ft.IconButton(ft.Icons.SETTINGS)],
        )
        page.add(ft.Text("Body content goes here"))

ft.run(main)
```

`page.theme` and `page.theme_mode` control the color scheme (light/dark and a seed color) applied across all Material controls on the page (see Theming for more).

`AppBar`, `Button`, and most other controls follow Material Design. For an iOS-style look, Flet also ships a parallel set of Cupertino controls (`ft.Cupertino*`). See Adaptive apps for building a single app that looks native on both platforms.

## Handling events

Interactive controls like `Button`, `IconButton`, and `Container` accept event handlers, plain functions that run when the user interacts with the control:

```
def main(page: ft.Page):
    def handle_click(e: ft.Event[ft.Button]):
        page.show_dialog(ft.SnackBar(ft.Text("Button
clicked!")))

    page.add(ft.Button("Click me", on_click=handle_click))

ft.run(main)
```

Every event handler takes a single argument, conventionally named `e`, of type `Event`. `e.control` is the control that triggered the event, so you can read its current state from there (a `TextField`'s `on_change` handler, for example, reads the new value via `e.control.value`). Typing the handler as `ft.Event[ft.Button]`, as above, tells your editor that `e.control` is specifically a `Button`.

Anything you change inside a handler (a control property, or, as above, calling a page method like `show_dialog()`) is picked up automatically, so most handlers don't need to manage updates by hand.

For lower-level pointer interactions (taps, drags, hover, scroll), wrap a control in `GestureDetector`.

## Example: Product catalog

Here's a small product catalog that uses everything above: an `AppBar` for the page header, `Container` and `Column`/`Row` for layout and styling, a `Stack` to draw a "Sale" badge on one item, and `on_click` handlers to react to taps:


Product catalog app running on macOS

Run it with `flet run` and you'll get a scrollable list of product cards, each with a "Buy" button that pops up a confirmation.

# What’s next

- Running a Flet app: see the apps above running as a desktop window or in a browser.
- Controls reference: the full list of controls available in Flet.
- Auto-update: how and when Flet sends control changes to the client.
- Declarative vs. imperative: once your UI needs to manage evolving state (like a shopping cart or a to-do list), this is the next thing to read.
- Theming: customize colors, fonts, and light/dark mode across the app.

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’;

# Prerequisites

## Python version

Flet requires Python 3.10 or later. (1)

1. Check your Python version using `python --version`.

## Operating System

### macOS

Flet supports macOS 12 (Monterey) or later.

### Windows

Flet supports 64-bit version of Microsoft Windows 10 and Windows 11.

### Linux

Flet supports Debian 10, 11 and 12 and Ubuntu 20.04, 22.04 and 24.04 LTS.

▶ Desktop flavor (audio & video support)
▶ Windows Subsystem for Linux (WSL)

# Creating a virtual environment (venv)

We recommend using a virtual environment for your Flet projects to keep dependencies isolated and avoid conflicts with your other Python projects.

First, create a new directory for your Flet project and switch into it:

```
mkdir my-app
cd my-app
```

Next, create and activate a virtual environment (we recommend using `uv` as package manager):

**uv** is “An extremely fast Python package and project manager, written in Rust”.

Install uv if you haven’t already, then run the following commands:

```
uv init --python='>=3.10'
uv venv
source .venv/bin/activate # (1)!
```

1. If you are on Windows, use `.venv\Scripts\activate` instead. Using Python’s built-in venv module:

```
python -m venv .venv  # (1)!
source .venv/bin/activate # (2)!
```

1. On Unix-like systems (Linux, macOS), use `python3 -m venv .venv` if `python` points to Python 2.x.
2. If you are on Windows, use `.venv\Scripts\activate` instead. ## Install Flet

To install Flet and add it to your project dependencies, do the following depending on your package manager:

```
uv add 'flet[all]'

pip install 'flet[all]' # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Verify installation

To make sure Flet has been installed correctly, we can check its version using the `--version` (or `-V`) flag or the `doctor` command:

```
uv run flet --version
# or
uv run flet doctor

flet --version
# or
flet doctor
```

Now you are ready to create your first Flet app.

# Upgrade Flet

To upgrade Flet to its latest version:

```
uv add 'flet[all]' --upgrade

pip install 'flet[all]' --upgrade
```

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’;

Flet lets you write **integration tests** for your app and run them with the `flet test` command. Tests drive your app the same way a user would — finding controls by key or text, tapping buttons, entering text and asserting the resulting UI — while the app runs **on**

**the target device** exactly as it ships: a built monolithic app with embedded Python.

Tests are written with `pytest`, so everything you already know about pytest (fixtures, parametrization, markers, `-k` filtering) just works.

`flet test` builds and runs your app the same way `flet build` does, so the Flutter SDK and build prerequisites must be installed. The first run provisions a test host (and downloads the SDK if needed), which is slow; subsequent runs are cached and fast.

## Where tests live

Put your tests in a `tests/` directory at the **root of your app** — a sibling of `src/`, not inside it (`src/` is what gets packaged into the on-device app; your test code stays on the host and drives the app):

```
my_app/
├── pyproject.toml
├── src/
│   └── main.py          # your app
└── tests/
    └── test_main.py     # your tests
```

A new app created with `flet create` already includes a `tests/` folder with a sample test and the required pytest configuration in `pyproject.toml`:

```toml
[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
```

## Enabling tests in an existing app

If your app wasn't created with `flet create`, enable testing by editing its `pyproject.toml`.

**1. Add the test dependencies** to your development dependencies. The `flet[test]` extra brings in `pytest`, `pytest-asyncio` and the screenshot-comparison libraries:

```toml
[dependency-groups]
dev = [
    # ...your existing dev dependencies...
    "flet[test]",
]

[project.optional-dependencies]
dev = [
    # ...your existing dev dependencies...
    "flet[test]",
]
```

**2. Configure pytest.** `asyncio_mode = "auto"` is **required** — it runs each async test on the same event loop as the `flet_app` fixture:

```
[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
```

**3. Create a `tests/` directory** and add your first test (see Writing a test below).

# Writing a test

Test functions are `async` and receive the [`flet_app`][flet.testing.FletTestApp] fixture, which starts your app and exposes a [tester][flet.testing.Tester] to drive it. Each test gets a **fresh app instance**, so tests are independent and can run in any order.

Here is the counter sample (`tests/test_main.py`) that `flet create` generates:

```python title="tests/test_main.py"
import flet.testing as ftt

async def test_increment(flet_app: ftt.FletTestApp):
    tester = flet_app.tester

    await tester.pump_and_settle()

    # Initial state
    assert (await tester.find_by_text("0")).count == 1

    # Tap the increment button (found by its key) and let the UI update
    await tester.tap(await tester.find_by_key("increment"))
    await tester.pump_and_settle()

    # New state
    assert (await tester.find_by_text("1")).count == 1
```

The matching app gives the button a `key` so the test can find it reliably:

```python title="src/main.py"
import flet as ft


def main(page: ft.Page):
    counter = ft.Text("0", size=50, data=0)

    def increment_click(e):
        counter.data += 1
        counter.value = str(counter.data)

    page.floating_action_button = ft.FloatingActionButton(
        icon=ft.Icons.ADD, key="increment", on_click=increment_click
    )
    page.add(
        ft.SafeArea(
            expand=True,
```

```
            content=ft.Container(content=counter,
alignment=ft.Alignment.CENTER),
        )
    )


ft.run(main)
```

Give controls you want to test a stable key and find them with [`find_by_key()`] [flet.testing.Tester.find_by_key]. It's more robust than matching on text, which can change with localization or formatting.

# The tester API

[`flet_app.tester`][flet.testing.FletTestApp.tester] finds controls and drives interactions. **Finder** methods return a [`Finder`][flet.testing.Finder]; **action** methods take a `Finder`; and **pump** methods let the UI advance. All methods are awaitable.

## Finding controls

| Method | Finds controls by |
|---|---|
| [`find_by_key(key)`] [flet.testing.Tester.find_by_key] | their `key` |
| [`find_by_text(text)`] [flet.testing.Tester.find_by_text] | exact text |
| [`find_by_text_containing(pattern)`] [flet.testing.Tester.find_by_text_containing] | a regular-expression match on text |
| [`find_by_icon(icon)`] [flet.testing.Tester.find_by_icon] | their icon (e.g. `ft.Icons.ADD`) |
| [`find_by_tooltip(value)`] [flet.testing.Tester.find_by_tooltip] | tooltip text |

A [`Finder`][flet.testing.Finder] reports how many controls matched (via [`count`] [flet.testing.Finder.count]) and lets you pick one with [`first`][flet.testing.Finder.first], [`last`][flet.testing.Finder.last] or [`at()`][flet.testing.Finder.at]:

```
finder = await tester.find_by_text("Item")
assert finder.count == 3          # number of matches
await tester.tap(finder.first)    # first match
await tester.tap(finder.last)     # last match
await tester.tap(finder.at(1))    # match at index 1
```

## Interacting

| Method | Action |
|---|---|
| [`tap(finder)`][flet.testing.Tester.tap] | tap a control |

| Method | Action |
| --- | --- |
| [`long_press(finder)`] [flet.testing.Tester.long_press] | long-press a control |
| [`enter_text(finder, text)`] [flet.testing.Tester.enter_text] | type text into a field |
| [`mouse_hover(finder)`] [flet.testing.Tester.mouse_hover] | hover the mouse over a control |

### Pumping

The UI doesn't update instantly after an interaction. Call [`pump_and_settle()`] [flet.testing.Tester.pump_and_settle] to let the app process events and render the result before asserting:

```
await tester.tap(await tester.find_by_key("submit"))
await tester.pump_and_settle()
assert (await tester.find_by_text("Done")).count == 1
```

Use [`pump(duration=...)`][flet.testing.Tester.pump] to advance by a fixed amount when you don't want to wait for everything to settle.

## Screenshot testing

On **Android and iOS** you can capture a full-screen screenshot of the running app and compare it against a committed *golden* (reference) image — useful for catching visual regressions. Full-screen capture is a device feature, so this is **not available on desktop**.

[`tester.take_screenshot(name)`][flet.testing.Tester.take_screenshot] captures the screen as PNG bytes, and [`flet_app.assert_screenshot(name, bytes)`] [flet.testing.FletTestApp.assert_screenshot] compares them against the golden image, failing the test if they differ beyond a similarity threshold (≈99% by default):

```
async def test_home_screen(flet_app: ftt.FletTestApp):
    tester = flet_app.tester
    await tester.pump_and_settle()

    flet_app.assert_screenshot("home", await
tester.take_screenshot("home"))
```

Golden images are **platform-specific** and stored next to your tests, under `tests/golden/<platform>/<test_file>/<name>.png` — commit them to your repository.

To record the goldens the first time (or update them after an intentional UI change), run with `-u` (`--update-goldens`). This writes the captured screenshots as the new reference **instead of** comparing:

```
uv run flet test android --device-id emulator-5554 -u

flet test android --device-id emulator-5554 -u
```

Render each screenshot on the same device/emulator you record its golden on — different screen sizes and densities produce different pixels.

# Running tests

## On desktop

From your app directory, run `flet test`. With no arguments it targets the **current desktop** platform:

```
uv run flet test

flet test
```

## On a mobile emulator/simulator or device

First, make sure a device is running. Use `flet emulators` to list available emulators and start one, then `flet devices` to get the id of a running device:

```
uv run flet emulators              # list available emulators
uv run flet emulators start <id>   # start an emulator
uv run flet devices                # list running devices and
their ids

flet emulators              # list available emulators
flet emulators start <id>   # start an emulator
flet devices                # list running devices and their
ids
```

Then pass the platform as the first argument and the device id with `--device-id` (`-d`):

```
uv run flet test android --device-id emulator-5554
uv run flet test ios --device-id <simulator-id>

flet test android --device-id emulator-5554
flet test ios --device-id <simulator-id>
```

## Useful options

| Option | Description |
|---|---|
| `[platform]` | `macos`, `linux`, `windows`, `ios`, `android` (defaults to the current desktop) |
| `-d`, `--device-id` | Target device/emulator id (required for `ios`/`android`) |
| `-k <expr>` | Only run tests matching a pytest keyword expression |
| `--tests-dir <dir>` | Directory containing the tests (default: `tests`) |

| Option | Description |
|---|---|
| `-v` | Verbose — stream the underlying Flutter build/launch output |

## Running specific tests

Use `-k` to run only the tests matching a pytest keyword expression — handy while iterating on a single test:

```
uv run flet test -k test_screenshot

flet test -k test_screenshot
```

`-k` accepts the full pytest expression syntax, e.g. `-k screenshot`, `-k "increment or screenshot"`, or `-k "not slow"`.

When running with pytest directly you can also select a test by its node id:

```
uv run pytest tests/test_main.py::test_increment   # a single test
uv run pytest tests/test_main.py                   # one file
```

## Running with pytest directly

Because tests are plain pytest, you can also run them with `pytest`. The Flet pytest plugin provisions the test host on demand, so this works without running `flet test` first (it targets the current desktop platform):

```
uv run pytest

pytest
```

To see the live build/launch output (and the app's `debugPrints`) while running under pytest, enable CLI logging at debug level:

```
uv run pytest -s -o log_cli=true -o log_cli_level=DEBUG
```

`pytest` has no options of its own for the device target or goldens, so the `flet test` options map to environment variables:

| `flet test` option | Environment variable |
|---|---|
| `[platform]` | `FLET_TEST_PLATFORM` (e.g. `ios`, `android`) |
| `-d`, `--device-id` | `FLET_TEST_DEVICE` |
| `-u`, `--update-goldens` | `FLET_TEST_GOLDEN=1` |

```
# run on an iOS simulator and (re)record golden screenshots
FLET_TEST_PLATFORM=ios FLET_TEST_DEVICE=<simulator-id>
FLET_TEST_GOLDEN=1 uv run pytest
```

When unset, `pytest` targets the current desktop platform and compares (rather than records) screenshots.

:::noteHow it works `flet test` provisions a Flutter test host from your app (the same pipeline as `flet build`), embeds your Python code, and runs it on the device. The test code runs on your computer and drives the on-device app over an independent channel — so you're testing your app exactly as it ships, including the embedded Python runtime, not a simulated approximation. :::

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {Image} from '@site/src/components/crocodocs';

Flet apps can be executed as either desktop or web applications using the `flet run` command. Doing so will start the app in a native OS window or a web browser, respectively, with hot reload enabled to view changes in real-time.

# Desktop app

To run Flet app as a desktop app, use the following command:

```
uv run flet run

flet run
```

When you run the command without any arguments, `main.py` script in the current directory will be executed, by default.

If you need to provide a different path, use the following command:

```
uv run flet run [script]

flet run [script]
```

Where `[script]` is a relative (ex: `counter.py`) or absolute (ex: `/Users/john/projects/flet-app/main.py`) path to the Python script you want to run.

The app will be started in a native OS window:

- **macOS**

  ---

   macOS

- **Windows**

  ---

   windows

# Web app

To run Flet app as a web app, use the `--web` (or `-w`) option:

```
uv run flet run --web [script]  # (1)!
```

1. A fixed port can be specified using `--port` ( or `-p`) option, followed by the port number.

```
flet run --web [script]  # (1)!
```

1. A fixed port can be specified using `--port` ( or `-p`) option, followed by the port number. A new browser window/tab will be opened and the app will be using a random TCP port:




## Passing arguments to your app

Anything after a `--` separator is passed through to your app script instead of being interpreted by Flet, and shows up there as `sys.argv[1:]`:

```
uv run flet run --web [script] -- --dataset big.csv --verbose
```

```
flet run --web [script] -- --dataset big.csv --verbose
```

Your app reads them as it would when started with `python`:

```python title="main.py"
import argparse
import sys

import flet as ft

parser = argparse.ArgumentParser()
parser.add_argument("--dataset", default="small.csv")
parser.add_argument("--verbose", action="store_true")
options = parser.parse_args(sys.argv[1:])

def main(page: ft.Page):
    page.add(ft.Text(f"Dataset: {options.dataset}"))

ft.run(main)
```

The `--` separator is what keeps your app's arguments from being read as Flet's own -
without it, `flet run main.py --verbose` fails because `--verbose` is not a
[`flet run`](../cli/flet-run.md) option. Arguments that don't start with a `-` can be
passed without a separator, e.g. `flet run main.py big.csv`.

:::note
The same arguments are re-applied every time the app is hot reloaded, and they are
passed through in all run modes: desktop, `--web`, `--ios` and `--android`.
:::

## Watching for changes

By default, Flet will watch the script file that was run and reload

```
the app whenever the contents
of this file are modified+saved, but will **not** watch for changes in
other files.

To modify this behavior, you can use one or more of these [`flet run`]
(../cli/flet-run.md) options:

* `-d` or `--directory` to watch for changes in the `[script]`s
directory only
* `-r` or `--recursive` to watch for changes in the `[script]`s
directory and all sub-directories recursively

:::note[Example]

<Tabs groupId="uv--pip">
<TabItem value="uv" label="uv">
```bash
uv run flet run --recursive [script]
```

```
        flet run --recursive [script]
```

:::

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {Image} from '@site/src/components/crocodocs';

Start building awesome mobile apps in Python using just your computer and mobile phone!

- Make sure your computer fulfills these requirements.
- Your device and computer must be connected to the same Wi-Fi or local network.
- We recommend starting with the creation of a new virtual environment.

Install the Flet iOS app on your iOS device. You will be using this app to see how your Flet project is working on iPhone or iPad.




Run the following command to start Flet development server with your app:

```
        uv run flet run --ios [script]  # (1)!
```

1. `flet run` starts your app in hot reload mode. More info here.

```
        flet run --ios [script]  # (1)!
```

1. `flet run` starts your app in hot reload mode. More info here.

Install the Flet Android app on your Android device. You will be using this app to see how your Flet project is working on Android device.




Run the following command to start Flet development server with your app:
```

```
uv run flet run --android [script]  # (1)!
```

1. `flet run` starts your app in hot reload mode. More info here.

```
flet run --android [script]  # (1)!
```

1. `flet run` starts your app in hot reload mode. More info here. A QR code with encoded project URL will be displayed in the terminal:

 app-qr-code

Open **Camera** app on your Android device, point to QR code you got and click URL to open it in Flet app.

Try updating your `[script]` - the app will be instantly refreshed on your Android device.

To return to “Home” tab either:

- Long-press anywhere on the screen with 3 fingers or
- Shake your Android device.

You can also “manually” add a new project by clicking **“+”** floating action button in the app and typing in its URL.

- Below is a URL to a “Counter” Flet app that we hosted for testing purposes: `https://flet-counter-test-ios.fly.dev`
- The “Gallery” tab of the app has some more examples that you can try out.
- Explore Flet examples for even more examples.

Flet is a framework that allows building web, desktop and mobile applications in Python without prior experience in frontend development.

# Flet app example

Below is a simple “Counter” app, with a text field and two buttons to increment and decrement the counter value:

```python title=“counter.py” import flet as ft

def main(page: ft.Page): page.title = “Flet counter example” page.vertical_alignment = ft.MainAxisAlignment.CENTER

input = ft.TextField(value="0", text_align=ft.TextAlign.RIGHT, width=100)

def minus_click(e):
    input.value = str(int(input.value) - 1)

def plus_click(e):
    input.value = str(int(input.value) + 1)

page.add(
    ft.Row(
```

```
            alignment=ft.MainAxisAlignment.CENTER,
            controls=[
                ft.IconButton(ft.Icons.REMOVE, on_click=minus_click),
                input,
                ft.IconButton(ft.Icons.ADD, on_click=plus_click),
            ],
        )
    )

    ft.run(main)


    To run the app, [install `flet`](getting-started/installation.md):

    ```bash
    pip install 'flet[all]'
```

then launch the app:

```
        flet run counter.py
```

This will open the app in a native OS window - what a nice alternative to Electron! 🙂



To run the same app as a web app use `--web` option with `flet run` command:

```
        flet run --web counter.py
```



import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs';

Instructions for packaging a Flet app into an Android APK and Android App Bundle (AAB).

This guide provides detailed Android-specific information. Complementary and more general information is available here.

# Prerequisites

## Android SDK

The build process requires both **Java** (JDK 17) and the **Android SDK**.

If either component is missing or an incompatible version is detected, the required tools will be **automatically installed** during the first run of the `flet build` command.

- The JDK is installed in `$HOME/java/&lt;version&gt;` (for example, `17.0.13+11`).
- If **Android Studio** is installed, Flet CLI will use its SDK:
  - macOS: `~/Library/Android/sdk`
  - Windows: `%LOCALAPPDATA%\Android\Sdk`
```

    - Linux: `~/Android/Sdk`
- Otherwise, a standalone Android SDK is installed in:
    - macOS/Linux: `~/Android/sdk`
    - Windows: `%USERPROFILE%\Android\sdk`

`ANDROID_HOME` and `ANDROID_SDK_ROOT` are also respected if set.

### Android wheels for binary Python packages

Binary Python packages (in contrast to "pure" Python packages written in Python only) are packages that are partially written in C, Rust, or other languages producing native code. Example packages are `numpy`, `cryptography`, or `pydantic-core`.

Make sure all non-pure (binary) packages used in your Flet app have pre-built wheels for Android.

## `flet build apk`

This command can be run on a **macOS**, **Linux**, or **Windows**.

Builds a **release** Android APK.

Release builds are optimized for production, meaning they **don't support debugging** and are intended for publishing to app stores such as Google Play.

For Play Store deployment, it's recommended to:

- Use an **Android App Bundle (AAB)** for more efficient delivery and smaller install size
- Or **split the APK by ABI** to reduce the APK size

:::tip[APK looks large?] If the generated `.apk` file seems surprisingly big, that's expected — see Native library packaging for why, and when legacy packaging produces a smaller file for side-loading. :::

### Split APK per ABI

Android devices use different CPUs, so APKs can target different Application Binary Interfaces (ABIs).

By default, Flet builds a single "fat" APK that contains native binaries for all supported ABIs. This maximizes device compatibility but increases APK size.

Enabling ABI splits produces one APK per ABI, which reduces file size but requires distributing the correct APK for each device.

#### Supported target architectures

The following target architectures are supported, for every bundled Python version:

- `arm64-v8a` (64-bit)
- `x86_64` (64-bit)

- `armeabi-v7a` (32-bit)

The 32-bit `x86` ABI is not supported — specifying it fails the build since Flet 0.86.0 (see the migration guide).

Flet bundles its own CPython builds for Android, published for all three ABIs on every supported Python version — including 32-bit `armeabi-v7a`, which upstream CPython dropped in 3.13 (PEP 738). By default, an app is built for all supported architectures.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--split-per-abi`
2. `[tool.flet.android].split_per_abi`
3. `false`

When enabled, one APK is produced per ABI — by default, one for each architecture the bundled Python version supports (see above). These can be customized by setting `target architectures`.

**Example**

```
flet build apk --split-per-abi
```

```
[tool.flet.android]
split_per_abi = true
```

## `flet build aab`

This command can be run on a **macOS**, **Linux**, or **Windows**.

Builds a **release** Android App Bundle (AAB) file.

Release builds are optimized for production, meaning they **don't support debugging** and are intended for publishing to app stores such as the Google Play Store.

It is recommended to use this AAB format (instead of APK) for publishing to the Google Play Store due to its optimized app size.

If you need to limit the ABIs included in the bundle, use `--arch` / `[tool.flet.android].target_arch` while `split_per_abi` is `false`.

# Native library packaging (modern vs legacy)

Flet apps bundle a native Python runtime and native extension modules (`.so` files) for each ABI. How those `.so` files are stored in the APK — and whether they are copied to disk when the app is installed — is controlled by Android's `useLegacyPackaging` setting (the `android:extractNativeLibs` manifest attribute).

### How each mode works

- **Modern packaging (default, `useLegacyPackaging = false`).** Native `.so` files are stored **uncompressed and page-aligned** inside the APK, and the OS maps them **directly from the installed APK** at runtime — no second copy of the native libraries on disk. Pure Python code ships in stored (uncompressed) zip assets; on first launch, the runtime copies the standard library and site-packages zips to app-private storage and imports from those copied zips. The application payload and explicitly extracted packages are unpacked there. Modern native-library packaging is the default since Flet re-designed Android packaging in v0.86.
- **Legacy packaging (opt-in, `useLegacyPackaging = true`).** Native `.so` files are stored **compressed** inside the APK, and the installer **extracts a second copy** to the app's `nativeLibraryDir` on install. The linker then loads the libraries from that extracted directory.

### "Why did my APK get bigger?"

A common surprise is that the raw `.apk` **file** looks larger with modern packaging — sometimes roughly double. The contents are almost identical; the file is only bigger because the native libraries are stored **uncompressed** (so they can be memory-mapped). Uncompressed libraries are actually the recommended, more efficient choice for your users. From the Android Gradle Plugin release notes:

> When you build your app, the plugin now sets `extractNativeLibs` to `"false"` by default. That is, your native libraries are page aligned and packaged uncompressed. While this results in a larger upload size, your users benefit from the following: - **Smaller app install size** because the platform can access the native libraries directly from the installed APK, without creating a copy of the libraries. - **Smaller download size** because Play Store compression is typically better when you include uncompressed native libraries in your APK or Android App Bundle.
>
> — AGP 3.6.0 release notes

In other words: when you publish to Google Play, the store applies additional download compression, which is typically more effective when native libraries are stored uncompressed. Users therefore typically get a **smaller** download and a **smaller** install than they would with legacy packaging — even though the uncompressed `.apk` you upload is larger. The raw file size primarily matters when you hand the `.apk` to users directly (side-loading).

### Trade-offs

| Aspect | Modern (default) | Legacy (`--android-legacy-packaging`) |
|---|---|---|
| Raw `.apk` file size | Larger (uncompressed `.so`) | Smaller (compressed `.so`) |
| Play Store download size | **Typically smaller** | Typically larger |
| On-device install size | **Smaller** (no extra copy) | Larger (extracted 2nd copy) |
| App load / startup | Faster (mmap) | Slightly slower |

### When to use which

- **Publishing to Google Play** (recommended via AAB): **keep the default (modern)**. Play serves an optimized, compressed download regardless of the upload size.
- **Distributing a raw `.apk` for side-loading** and you want the smallest file: use **legacy** packaging. Alternatively, compress the modern `.apk` before sharing it; recipients must unpack it before installation.

Legacy packaging can also be useful as a diagnostic fallback for packaging-related native-library loading or installation failures. It is not a substitute for using 16 KB-compatible native libraries.

### Accessing a custom native library by file path

Serious Python exposes `ANDROID_NATIVE_LIBRARY_DIR` to the embedded Python process. Its value is Android's app-specific native-library directory.

With modern packaging, the variable is still available, but the `.so` files are not extracted into that directory; they remain inside the installed APK. Code that can load a packaged library through Android's linker should use its name, for example `ctypes.CDLL("libxxx.so")`.

With legacy packaging, Android extracts every packaged `.so` for the device's ABI into `ANDROID_NATIVE_LIBRARY_DIR`. This makes the option useful for code that requires a real filesystem path to a library, including custom `ctypes` loaders or `ctypes.util.find_library` patches:

```
import ctypes
import os

native_library_dir = os.environ["ANDROID_NATIVE_LIBRARY_DIR"]
ctypes.CDLL(os.path.join(native_library_dir, "libxxx.so"))
```

The library must already be packaged under `lib/<abi>/` for the device's ABI, and all of its native dependencies must also be available. Legacy packaging changes only how those libraries are stored and loaded; it does not make an incompatible library usable.

### Resolution order

Its value is determined in the following order of precedence:

1. `--android-legacy-packaging` / `--no-android-legacy-packaging`
2. `[tool.flet.android].legacy_packaging`
3. `false` (modern packaging)

### Example

```
flet build apk --android-legacy-packaging
```

```
[tool.flet.android]
legacy_packaging = true
```

▶ Template translation

# Signing an Android bundle

Android requires that all APKs be digitally signed with a certificate before they are installed on a device or updated. When releasing using Android App Bundles, you need to sign your app bundle with an upload key before uploading it to the Play Console, and Play App Signing takes care of the rest. For apps distributing using APKs on the Play Store or on other stores, you must manually sign your APKs for upload.

For detailed information, see this guide.

To publish on the Play Store, you need to sign your app with a digital certificate.

Android uses two signing keys: upload and app signing.

- Developers upload an `.aab` or `.apk` file signed with an upload key to the Play Store.
- The end-users download the `.apk` file signed with an app signing key.

To create your app signing key, use Play App Signing as described in the official Play Store documentation.

To sign your app, use the following instructions.

If you don’t provide an upload keystore, release builds are signed with the debug key. This is fine for local testing but cannot be uploaded to the Play Store.

## Create an upload keystore

If you have an existing keystore, skip to the next step. If not, create one using one of the following methods:

1. Follow the Android Studio key generation steps.

2. Run the following command at the command line: On macOS or Linux, use the following command:

   ```
   keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA \
       -keysize 2048 -validity 10000 -alias upload
   ```

   On Windows, use the following command in PowerShell:

   ```
   keytool -genkey -v -keystore $env:USERPROFILE\upload-keystore.jks `
       -storetype JKS -keyalg RSA -keysize 2048 -validity 10000 `
       -alias upload
   ```

   You will be prompted for several details, such as a keystore password, a key alias, your names, and location. Remember the password and alias for use in the configuration steps below.

A file named `upload-keystore.jks` will be created in your home directory. If you want to store it elsewhere, change the argument passed to the `-keystore` parameter. The location of the keystore file is equally important for the key store step below.

```
- The `keytool` command might not be in your path—it's part of Java,
which is installed as part of Android Studio.
For the concrete path, run `flutter doctor -v` and locate the path
printed after 'Java binary at:'.
Then use that fully qualified path replacing `java` (at the end) with
`keytool`.
If your path includes space-separated names, such as Program Files,
use platform-appropriate notation
for the names. For example, on macOS and Linux use `Program\ Files`,
and on Windows use `"Program Files"`.

- The `-storetype JKS` tag is only required for Java 9 or newer.
As of the Java 9 release, the keystore type defaults to PKCS12.
```

Keep your `keystore` file private; never check it into public source control!

## Key alias

An alias name for the key within the keystore.

#### Resolution order

Its value is determined in the following order of precedence:

1. `--android-signing-key-alias`
2. `FLET_ANDROID_SIGNING_KEY_ALIAS`
3. `[tool.flet.android.signing].key_alias`
4. `"upload"`

#### Example

```
flet build aab --android-signing-key-alias value
```

```
[tool.flet.android.signing]
key_alias = "value"
```

```
FLET_ANDROID_SIGNING_KEY_ALIAS="value"
```

### Key store

The path to the keystore file (with extension `.jks`).

If you used the CLI commands above as-is, this file might be located at `/Users/<user name>/upload-keystore.jks` on macOS or `C:\Users\<user name>\upload-keystore.jks` on Windows.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--android-signing-key-store`
2. `[tool.flet.android.signing].key_store`
3. `FLET_ANDROID_SIGNING_KEY_STORE`

**Example**

```
flet build aab --android-signing-key-store path/to/store.jks
```

```
[tool.flet.android.signing]
key_store = "path/to/store.jks"
```

```
FLET_ANDROID_SIGNING_KEY_STORE="path/to/store.jks"
```

### Key store password

A password to unlock the keystore file (can contain multiple key entries).

**Resolution order**

Its value is determined in the following order of precedence:

1. `--android-signing-key-store-password`
2. `FLET_ANDROID_SIGNING_KEY_STORE_PASSWORD`
3. key password

**Example**

```
flet build aab --android-signing-key-store-password value
```

For security reasons, the keystore password is not read from `pyproject.toml` to prevent accidental exposure in source control. See the other tabs for supported alternatives.

```
FLET_ANDROID_SIGNING_KEY_STORE_PASSWORD="value"
```

### Key password

A password used to access the private key inside the keystore.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--android-signing-key-password`
2. `FLET_ANDROID_SIGNING_KEY_PASSWORD`
3. key store password

**Example**

```
flet build aab --android-signing-key-password value
```

For security reasons, the keystore password is not read from `pyproject.toml` to prevent accidental exposure in source control. See the other tabs for supported alternatives.

```
FLET_ANDROID_SIGNING_KEY_PASSWORD="value"
```

## Android Manifest

The Android Manifest describes essential information about your app to the Android build tools, the Android operating system, and Google Play. The file in which this information is written is `AndroidManifest.xml`, which gets populated with the information you provide.

### Application attributes

You can add or override attributes on the `<application>` element of the `AndroidManifest.xml` file in the build template.

See also:

- `<application>` element

#### Resolution order

Its value is determined in the following order of precedence:

1. `[tool.flet.android.manifest_application]`

#### Example

```toml
[tool.flet.android.manifest_application]
usesCleartextTraffic = "true"
allowBackup = "false"
```

▶ Template translation

### Meta-data

A name-value pair for an item of additional, arbitrary data that can be supplied to the parent component. More information here.

A meta-data item is composed of:

- `name`: A unique name for the item, usually with a Java-style naming convention, for example `"com.sample.project.activity.fred"`.
- `value`: The value of the item. Android supports strings, integers, booleans, and floats. Flet writes values as strings, so pass the literal value you want Android to read (for example `"true"`, `"123"`, `"1.23"`).

See also:

- `<meta-data>` element

**Resolution order**

Its value is determined in the following order of precedence:

1. `--android-meta-data`
2. `[tool.flet.android.meta_data]`

**Example**

```
        flet build apk \
          --android-meta-data
firebase_analytics_collection_enabled=true \
          --android-meta-data default_timeout_seconds=30

        [tool.flet.android.meta_data]
        "firebase_analytics_collection_enabled" = "true"
        "default_timeout_seconds" = "30"
```

▶ Template translation

## Providers

A content provider component that supplies app data to other apps or components. More information here.

Each provider is declared as a TOML table whose key is the provider's `android:name` (the fully-qualified class name). The table's entries become extra `android:<key>="<value>"` attributes on the generated `<provider>` element. A reserved `meta_data` sub-table emits nested `<meta-data>` children inside the provider.

See also:

- `<provider>` element

**Resolution order**

Its value is determined in the following order of precedence:

1. `[tool.flet.android.provider]`

**Supported value forms**

The value for each provider must be a TOML inline table or sub-table of attributes. A value of `false` skips the entry entirely; `true` is not accepted (a `<provider>` with no attributes is meaningless). An empty table `{}` is also treated as `false`.

Attribute values must be strings, booleans, or numbers — they are written verbatim into the manifest. The `name` key is reserved (the `android:name` comes from the table key).

The reserved `meta_data` sub-table emits `<meta-data>` children. Each sub-entry's key is the `android:name`; the value can be either:

- a scalar (string/bool/number) — rendered as `android:value="…"`, or

- an inline table — its entries become `android:<key>="<value>"` attributes on the `<meta-data>` element (useful for `android:resource="@xml/…"`).

**Example**

```
        [tool.flet.android.provider]
        "rikka.shizuku.ShizukuProvider" = { authorities =
"${applicationId}.shizuku", multiprocess = "false", enabled = "true",
exported = "true", permission =
"android.permission.INTERACT_ACROSS_USERS_FULL" }

        [tool.flet.android.provider."com.example.MyProvider"]
        authorities = "${applicationId}.myprovider"
        exported = false
        grantUriPermissions = true

        [tool.flet.android.provider."com.example.MyProvider".meta_data]
        "android.support.FILE_PROVIDER_PATHS" = { resource =
"@xml/file_paths" }
        "some.other.key" = "some-value"
```

▶ Template translation

Flet already declares a built-in `androidx.core.content.FileProvider` with authorities `${applicationId}.provider`. Declaring another provider that uses the same authorities will cause the Android manifest merger to fail — pick a different `android:authorities` value for your custom provider.

## Features

A hardware or software feature that is used by the application. More information here.

- `name`: Specifies a single hardware or software feature used by the application as a descriptor string. Valid attribute values are listed in the Hardware features and Software features sections. These attribute values are case-sensitive.
- `required`: A boolean value (`true` or `false`) that indicates whether the application requires the feature specified by the `name`.

See also:

- `<uses-feature>` element
- Features reference

**Resolution order**

Its value is determined in the following order of precedence:

1. `--android-features`
2. `[tool.flet.android.feature]`
3. Values injected by cross-platform permission bundles, if any.
4. defaults: `android.software.leanback=false`, `android.hardware.touchscreen=false`

#### Supported value forms

Accepts repeated `<name>=<required>` entries. The `<required>` value can be `true` or `false` (case-insensitive). Use boolean values. TOML booleans must be lowercase: `true` or `false`. #### Example

```
flet build apk \
  --android-features android.hardware.camera=true \
  --android-features android.hardware.location.gps=false

[tool.flet.android.feature]
"android.hardware.camera" = true
"android.hardware.location.gps" = false
```

▶ Template translation

## Permissions

Use cross-platform permissions from Permissions when possible, and add Android-specific permissions or features here.

:::danger[Declare only what you use] Google Play rejects apps that request broad media or storage access they do not need. In particular, its Photo and Video Permissions policy rejects apps targeting Android 13 (API 33) or higher that declare `READ_MEDIA_IMAGES` or `READ_MEDIA_VIDEO` when the system pickers are sufficient — and a single active version code carrying them, including on internal and closed testing tracks, is enough to block a release.

`FilePicker` uses the storage access framework and needs no permission at all, and the app's own directories (see `StoragePaths`) are always writable. Reach for `READ_MEDIA_*` or the legacy `*_EXTERNAL_STORAGE` permissions only when your app truly needs library-wide access, and bound the legacy ones with `{ maxSdkVersion = "32" }` so they are not requested on Android 13 or higher. :::

See also:

- `Manifest.permission` constants
- Request app permissions

#### Resolution order

Its value is determined in the following order of precedence:

1. `--android-permissions`
2. `[tool.flet.android.permission]`
3. Values injected by cross-platform permission bundles, if any.
4. defaults: `android.permission.INTERNET=true`

#### Supported value forms

Accepts repeated `<name>=<enabled>` entries. The `<enabled>` value can be `true` or `false` (case-insensitive). Permissions with `false` are actively removed from the merged manifest. Use boolean values. TOML booleans must be lowercase: `true` or `false`.

Permissions set to `false` are emitted with `tools:node="remove"`, which strips them from the merged manifest even when a Flutter plugin declares them — see Removing a permission a plugin adds.

A value can also be a TOML inline table whose entries become extra `android:<key>="<value>"` attributes on the generated `<uses-permission>` element — useful for attributes like `maxSdkVersion` or `usesPermissionFlags`. A non-empty table is always emitted; an empty table `{}` is treated as `false`. #### Example

```
flet build apk \
  --android-permissions android.permission.CAMERA=true \
  --android-permissions android.permission.RECORD_AUDIO=true
```

```
[tool.flet.android.permission]
"android.permission.CAMERA" = true
"android.permission.RECORD_AUDIO" = true
"android.permission.ACCESS_FINE_LOCATION" = { maxSdkVersion =
"30" }
"android.permission.BLUETOOTH_SCAN" = { usesPermissionFlags =
"neverForLocation" }
```

▶ Template translation

**Removing a permission a plugin adds**

Not every permission in your app comes from your own configuration. Gradle's manifest merger folds in the manifest of every Flutter plugin you depend on, and it can also *synthesize* permissions you never asked for.

A real example: `flet-camera` pulls in `camera_android_camerax`, whose manifest declares `WRITE_EXTERNAL_STORAGE` bounded to `maxSdkVersion="28"`. The merger then implies an **unbounded** `READ_EXTERNAL_STORAGE` from it, because on very old Android versions write access implied read access:

```
uses-permission#android.permission.READ_EXTERNAL_STORAGE
IMPLIED from .../AndroidManifest.xml reason:
io.flutter.plugins.camerax requested WRITE_EXTERNAL_STORAGE
```

Neither permission appears anywhere in your `pyproject.toml`, and both show up in the Play Console. Setting them to `false` removes them:

```
[tool.flet.android.permission]
"android.permission.READ_EXTERNAL_STORAGE" = false
"android.permission.WRITE_EXTERNAL_STORAGE" = false
```

which generates:

```
<uses-permission
android:name="android.permission.READ_EXTERNAL_STORAGE"
tools:node="remove" />
<uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"
tools:node="remove" />
```

Removing a permission that nothing declares is harmless — `tools:node="remove"` on an absent element is a no-op.

:::tip[Finding where a permission came from] Gradle writes a report naming the source of every merged manifest element:

`build/flutter/build/app/outputs/logs/manifest-merger-release-report.txt`

Search it for the permission name. Each entry is marked `ADDED from [:some_plugin]`, `MERGED from [some.aar]`, or `IMPLIED from ... reason: ...`, which tells you whether to remove it, drop the dependency, or bound it with `maxSdkVersion`. :::

Only remove a permission the app genuinely does not need. If a plugin declares one because a code path it owns requires it, removing the permission makes that path fail at runtime rather than at build time.

## Minimum SDK version

The minimum Android API level your app can be installed on.

See also:

- `<uses-sdk>` element (`minSdkVersion`)
- Android API level reference

### Resolution order

Its value is determined in the following order of precedence:

1. `[tool.flet.android].min_sdk_version`
2. Flutter default: `flutter.minSdkVersion`

### Example

```toml
[tool.flet.android]
min_sdk_version = 24
```

▶ Template translation

## Target SDK version

The Android API level your app targets for runtime behavior and compatibility.

See also:

- `<uses-sdk>` element (`targetSdkVersion`)
- Target API level requirements and behavior changes

### Resolution order

Its value is determined in the following order of precedence:

1. `[tool.flet.android].target_sdk_version`
2. Flutter default: `flutter.targetSdkVersion`

**Example**

```toml
[tool.flet.android]
target_sdk_version = 35
```

▶ Template translation

### Adaptive icon background

The background color used for the Android adaptive launcher icon.

This value is applied when app icons are generated for Android.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--android-adaptive-icon-background`
2. `[tool.flet.android].adaptive_icon_background`
3. Build template default: `#ffffff`

**Example**

```
flet build apk --android-adaptive-icon-background "#0B6BFF"
```

```toml
[tool.flet.android]
adaptive_icon_background = "#0B6BFF"
```

## ProGuard / R8 rules

Release builds run R8, which removes unused classes and renames the ones that remain. Anything looked up by name at runtime — through reflection or JNI — has to be exempted with a *keep rule*, or the lookup fails against a class that no longer has the name it is being asked for.

Flet generates `android/app/proguard-rules.pro` with these defaults:

```
-keep class com.flet.serious_python_android.** { *; }
-keepnames class * { *; }
```

There are two knobs, because adding and removing rules are different problems:

- **`[tool.flet.android].proguard_rules`** — a list of rules *appended* to the defaults. R8 has no directive that undoes a keep, so a rule you add can only ever widen what is kept.
- **`[tool.flet.android].proguard_default_rules`** — set to `false` to drop the defaults entirely. This is the only way to get rid of `-keepnames class * { *; }`.

Appending is kept separate from replacing on purpose. If the only option were to replace, you would paste today's defaults into your `pyproject.toml` and silently keep them after Flet changes them.

## Dropping the defaults

`-keepnames class * { *; }` keeps every class *and member* name in your app, which switches off obfuscation app-wide and blocks the R8 passes that rely on renaming. It costs real size — on Flet Studio, removing it takes `classes.dex` from 5.9 MB to 3.4 MB (**-43%**).

Dropping the defaults does **not** break Pyjnius's access to the app activity: `serious_python_android` 4.1.0+ ships `-keep class com.flet.serious_python_android.** { *; }` in its own `consumer-rules.pro`, so that rule applies whether or not the template repeats it.

```toml
[tool.flet.android]
proguard_default_rules = false
proguard_rules = [
  # add back names your own code looks up reflectively
  "-keepnames class com.example.myapp.** { *; }",
]
```

Test a release build on a device before shipping this — see the warning below.

## When you need this

The common case is Pyjnius. `autoclass()` resolves Java classes by their fully-qualified name at runtime, so a class R8 renamed can no longer be found:

- **Android framework classes** (`android.os.Build`, `android.bluetooth.BluetoothAdapter`, …) need no rule. They live in the Android runtime, not in your APK, so R8 never touches them.
- **Classes bundled by a Flutter plugin or your own Java/Kotlin** are in your APK and *are* renamed. These need a keep rule.

:::warning A failed lookup crashes the process `autoclass()` on a renamed class does not raise a Python exception you can catch. JNI `FindClass` returns null and the process aborts. On a debuggable build the log shows:

```
JNI DETECTED ERROR IN APPLICATION: obj == null in call to
CallObjectMethodA
Fatal signal 6 (SIGABRT)
```

On a production build there is usually no message at all — just a crash. Because R8 only runs in release builds, this never reproduces in debug. :::

## Example

```toml
[tool.flet.android]
proguard_rules = [
  "-keep class io.flutter.embedding.android.FlutterActivity {
```

```
*; }",
            "-keep class com.example.mylib.** { *; }",
        ]
```

▶ Template translation

# Extract packages

On Android, pure Python code is packaged into stored zip assets. On first launch, Flet copies the standard-library and site-packages zips to app-private storage and imports from those copied zips with `zipimport`. Native extension modules are loaded memory-mapped directly from the APK. This avoids unpacking all site-packages on first launch.

Most packages work from inside the zip. But packages that read bundled **data files** through a real filesystem path — for example with `__file__` or `pkg_resources`, instead of the zip-safe `importlib.resources` — may fail at runtime because their data lives inside `sitepackages.zip`, where plain `open()` cannot read it.

List such packages in `extract_packages` to ship them extracted to the app's files directory instead of inside `sitepackages.zip`. Flet extracts the listed package directories and everything under them, so `__file__`-relative reads work again.

Most packages that bundle data (such as `flet` or `certifi`) read it through `importlib.resources`, which is zip-safe, so they need no entry here. Only add packages that actually fail to find their data when imported from the zip.

## Symptoms

The build succeeds, but the app crashes or errors on the device when the package is imported or first used. The traceback usually contains a path where `sitepackages.zip` or `stdlib.zip` appears as a directory component, for example (`matplotlib`):

```
FileNotFoundError: [Errno 2] No such file or directory:
'/data/user/0/<applicationId>/files/.../sitepackages.zip/matplotlib/mpl-data/matplotlibrc'
```

`NotADirectoryError` or `OSError` with a similar `sitepackages.zip/...` path is also a common sign that the package computed a data path from `__file__` and tried to read it as a regular file.

If this happens with one of your dependencies, add that package to `extract_packages` and consider reporting it in Flet discussions or opening a PR so it can be added to the known packages list.

## Resolution order

1. `--android-extract-packages`
2. `[tool.flet.android].extract_packages`
3. `[tool.flet].extract_packages`

### Example

Each entry is the package's **import name** — its top-level directory under site-packages — not necessarily its PyPI distribution name. For example, use `sklearn`, not `scikit-learn`; use `cv2`, not `opencv-python`.

```
flet build apk --android-extract-packages package1 package2
```

```
[tool.flet.android]
extract_packages = ["package1", "package2"]
```

### Wildcards

An entry is a path relative to site-packages and matches that path and everything under it. Entries may also contain `*` and `?` wildcards, matched against the top-level directory name:

```
[tool.flet.android]
extract_packages = ["somepackage*"]
```

The wildcard form can also extract a sibling `somepackage-<version>.dist-info/` directory. Use it for packages that read their metadata or data files through `pkg_resources`.

### Affected packages

Below are known packages that need to be extracted to work on Android:

| Package (PyPI) | Entry | Reason |
|---|---|---|
| `matplotlib` | `"matplotlib"` | reads `mpl-data` (fonts, `matplotlibrc`) relative to `__file__` |
| `scikit-learn` | `"sklearn"` | loads bundled data files through `__file__`-relative paths |
| `opencv-python` | `"cv2"` | resolves config files and loads its native extension through `__file__`-relative paths |
| `astropy` | `"astropy"` | reads `astropy/CITATION` via `__file__` at import |
| `thinc` | `"thinc"` | reads `thinc/backends/_custom_kernels.cu` via `__file__` at import |
| `spacy` | `"spacy", "thinc"` | imports `thinc` at load and reads its own language data via `__file__`; list both |

# ADB Tips

Android Debug Bridge (adb) is a command-line tool included in the Android SDK that lets you communicate with Android devices and emulators.

If you installed Android Studio on macOS, `adb` is typically located at: `~/Library/Android/sdk/platform-tools/adb`.

See this guide for help installing and using adb on different platforms.

1. To run interactive commands inside an Android simulator or device: `bash adb shell`

2. To overcome “permissions denied” error while trying to browse file system in interactive Android shell: `bash su`

3. To download a file from a device to your local computer: `bash adb pull <device-path> <local-path>`

4. To install an APK on an Android device: `bash adb install <path-to-your.apk>`

   This works for both physical devices and emulators. If more than one device is connected, specify the target device: `bash adb -s <device> install <path-to-your.apk>`

   You can list available devices with: `bash adb devices`

import CrossPlatformPermissions from ‘@site/.crocodocs/cross-platform-permissions.mdx’; import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’;

# Publishing a Flet app

Flet CLI provides the `flet build` command to package a Flet app into a standalone executable or installable package for distribution.

## Prerequisites

### Platform matrix

Use the following matrix to determine on which OS `flet build` can be run in order to target each platform:

| Run on | apk/aab | ipa/ios-simulator | macos | linux | windows | web |
|---|---|---|---|---|---|---|
| macOS | ✅ | ✅ | ✅ | | | ✅ |
| Windows | ✅ | | | ✅ (WSL) | ✅ | ✅ |
| Linux | ✅ | | | ✅ | | ✅ |

### Flutter SDK

Flutter is required to build Flet apps for any platform.

If the minimum required version of the Flutter SDK is not already available in the system `PATH`, it will be automatically downloaded and installed (in the `$HOME/flutter/{version}` directory) during the first build process.

The recommended (minimum required) Flutter SDK version depends on the Flet version installed or in use.

It can be viewed by running one of the following commands:

```
flet --version

uv run python -c "import flet.version;
print(flet.version.flutter_version)"
```

or the below Python code snippet:

```
import flet.version
print(flet.version.flutter_version)
```

# Project structure

The `flet build` command assumes the following minimal Flet project structure:

```
README.md
pyproject.toml # (1)!
src
    assets # (2)!
        icon.png
    main.py # (3)!
```

1. Serves as the main configuration file for your application. It includes metadata, dependencies, and build settings. At a minimum, the `dependencies` section should specify `flet` package.

```
Below is an example of a `pyproject.toml` file:
```toml title="pyproject.toml"
[project]
name = "example"
version = "0.1.0"
description = "An Example."
readme = "README.md"
requires-python = ">=3.14"
authors = [{ name = "Me", email = "me@example.com" }]
dependencies = [
  "flet"
]

[tool.flet]
org = "com.mycompany"
product = "My App"
company = "My Company"
copyright = "Copyright (C) 2025 by My Company"
```
```

```
[tool.flet.app]
path = "src"
```

2. An optional directory that contains application assets (images, sound, text, and other files required by your app) as well as images used for package icons and splash screens.
3. The main entry point of your Flet app. It usually contains the call to `ft.run()`.

To quickly set up a project with the correct structure, use the `flet create` command:

```
flet create <project-name>
```

Where `<project-name>` is the name for your project directory.

:::note[Using `requirements.txt` instead of `pyproject.toml`] Instead of a `pyproject.toml` file, you can also use a `requirements.txt` file to specify dependencies.

In this case, two things to keep in mind:

- if both files are present, `flet build` will ignore `requirements.txt`.
- don't use `pip freeze > requirements.txt` to generate this file or fill it with dependencies, as it may include packages incompatible with the target platform. Instead, hand-pick and include only the direct dependencies required by your app, including `flet`. :::

# Choosing a Python version

`flet build` and `flet publish` bundle a specific Python release into your app. Supported versions and the matching CPython / Pyodide artifacts:

| Short | CPython runtime | Pyodide (web) | Status |
|---|---|---|---|
| 3.14 | 3.14.6 | 314.0.0 | default |
| 3.13 | 3.13.14 | 0.29.4 | stable |
| 3.12 | 3.12.13 | 0.27.7 | stable |

The version is resolved in this order:

1. **`--python-version X.Y`** — explicit override. Must match a supported short version (e.g. `flet build apk --python-version 3.13`).
2. **`[project].requires-python`** in `pyproject.toml` — parsed as a PEP 440 specifier; the **highest** supported short version that satisfies it wins. `requires-python = ">=3.13,<3.14"` resolves to 3.13; `requires-python = ">=3.13"` resolves to 3.14.
3. **Default** — the latest supported version (currently `3.14`).

If neither the CLI flag nor `requires-python` selects a supported version (e.g. `requires-python = ">=3.20"`), the build fails with a clear error listing the versions you can choose from.

For web builds (`flet build web` / `flet publish`), the matching Pyodide runtime is downloaded into the build output and cached under `~/.flet/pyodide/<version>/` on first use, so subsequent builds reuse it. The older `0.27.5` bundle that used to ship inside the build template has been removed in favour of this per-build, versioned download.

:::note[Pre-release Python (3.15 etc.)] To bundle a pre-release Python, name it explicitly — `--python-version 3.15` or `requires-python = "==3.15.*"`. There is **no separate `--pre` flag** for this. The `--pre` flag on `flet publish` is a different, unrelated option for allowing **micropip** to install pre-release Python *packages* at runtime. :::

# How it works

When you run `flet build <target_platform>`, the pipeline is:

1. Create a Flutter project in `{flet_app_directory}/build/flutter` from the template. The Flutter app embeds your packaged Python app in its assets and uses `flet` and `serious_python` to run the app and render the UI. The project is cached and reused across builds for rapid iterations; run `flet clean` to delete the `build` directory and force a rebuild.
2. Copy custom icons and splash images from `assets` into the Flutter project, then generate:
   - Icons for all platforms via `flutter_launcher_icons`.
   - Splash screens for web, iOS, and Android via `flutter_native_splash`.
3. Package the Python app using `serious_python package`:
   - Install app dependencies from pypi.org and pypi.flet.dev for the selected platform, as configured in App dependencies and Source packages.
   - If configured, compile `.py` files to `.pyc`.
   - Add all project files, except those excluded, to the app asset.
4. Run `flutter build` to produce the executable or installable package.
5. Copy build outputs from Step 4 into the output directory.

# Configuration options

Throughout this documentation, the following placeholders are used:

- `<target_platform>` - one of: `apk`, `aab`, `ipa`, `ios-simulator`, `web`, `macos`, `windows`, `linux`.
- `<PLATFORM>` - the config namespace under `[tool.flet.<PLATFORM>]`; one of: `android` (for `apk` and `aab` targets), `ios` (for `ipa` and `ios-simulator` targets), `web`, `macos`, `windows`, `linux`.
- `<python_app_path>` - the path passed to `flet build` (defaults to the current directory).
- `<flet_app_directory>` - the resolved project root for `<python_app_path>`; `pyproject.toml` and `requirements.txt` are read from here.
- `<flet_version>` - the version of Flet in use. You can check with `flet --version` or `uv run python -c "import flet; print(flet.__version__)"`.

:::note[Understanding `pyproject.toml` structure] Flet loads `pyproject.toml` as a nested dictionary and looks up settings using dot-separated paths (for example, `tool.flet.web.base_url`).

The two forms below are equivalent and resolve to the same key-value pair:

- **Form 1** (will be used/preferred throughout this documentation) `toml [tool.flet.section] key = "value"`
- **Form 2** `toml [tool.flet] section.key = "value"`

But they are different or should not be confused with the below ("quoted keys" are literals and do not create nesting):

```toml
[tool.flet]
"section.key" = "value"
```

:::

## App path

Defines the root directory of your Python app within `<python_app_path>`. Flet looks for the entry point, the `assets` directory, and exclude paths relative to this directory.

### Resolution order

Its value is determined in the following order of precedence:

1. `[tool.flet.app].path`
2. `<python_app_path>`

`path` is resolved relative to `<python_app_path>`.

### Example

```toml
[tool.flet.app]
path = "src"
```

## Entry point

This is the Python module that starts your app and contains the call to `flet.run()` or `flet.render()`. Flet uses the module *stem* and looks for `<module>.py` in your app path.

### Resolution order

Its value is determined in the following order of precedence:

1. `--module-name`
2. `[tool.flet.app].module`
3. `"main"` (entry file `main.py`)

Its value can either be `<module>` or `<module>.py`; both resolve to the same Python module.

### Example

```
flet build <target_platform> --module-name app.py
```

```toml
[tool.flet.app]
module = "app.py"
```

## Project name

The project name is the base identifier for bundle IDs and other internal names. The source value is normalized to a safe identifier: lowercased, punctuation and spaces removed or collapsed, and hyphens converted to underscores (for example, `My App` or `my-app` becomes `my_app`).

### Resolution order

Its value is determined in the following order of precedence:

1. `--project`
2. `[project].name`
3. project/app directory name

### Example

```
flet build <target_platform> --project my_app
```

```toml
[project]
name = "my_app"
```

## Product name

The display (user-facing) name shown in window titles, launcher labels, and about dialogs.

It does **not** control the on-disk executable or bundle name. Use the artifact name for artifact naming.

### Resolution order

Its value is determined in the following order of precedence:

1. `--product`
2. `[tool.flet].product`
3. `--project`
4. `[project].name`
5. project/app directory name

### Example

```
flet build <target_platform> --product "My Awesome App"
```

```toml
[tool.flet]
product = "My Awesome App"
```

## Artifact name

The on-disk name for executables and/or app bundles. For example, on Windows it determines the name of the `.exe` file, and on macOS it sets the name of the `.app` bundle.

It does **not** affect bundle IDs or package identifiers.

It can contain spaces or accents, but keep file system restrictions in mind on your target platforms.

### Resolution order

Its value is determined in the following order of precedence:

1. `--artifact`
2. `[tool.flet.<PLATFORM>].artifact`
3. `--project`
4. `[project].name`
5. project/app directory name

### Example

```
flet build <target_platform> --artifact "My Awesome App"
```

```
[tool.flet]
artifact = "My Awesome App"
```

## Organization name

:::notePlatform support Android, iOS, macOS, and Linux only. :::

The organization name in reverse domain name notation, typically in the form `com.mycompany`. It is used as the prefix for the bundle ID and for package identifiers on mobile and desktop targets.

### Resolution order

Its value is determined in the following order of precedence:

1. `--org`
2. `[tool.flet.<PLATFORM>].org`
3. `[tool.flet].org`
4. `"com.flet"`

### Example

```
flet build <target_platform> --org com.mycompany
```

```
[tool.flet]     # or [tool.flet.<PLATFORM>]
org = "com.mycompany"
```

## Bundle ID

:::notePlatform support Android, iOS, macOS, and Linux only. :::

The bundle ID for the application, typically in the form `"com.mycompany.my_app"`.

If not explicitly specified, it is derived from the organization name and the project name used by the build template.

### Resolution order

Its value is determined in the following order of precedence:

1. `--bundle-id`
2. `[tool.flet.<PLATFORM>].bundle_id`
3. `[tool.flet].bundle_id`

### Example

```
flet build <target_platform> --bundle-id com.mycompany.my_app
```

```
[tool.flet] # or [tool.flet.<PLATFORM>]
bundle_id = "com.mycompany.my_app"
```

## Company Name

:::notePlatform support Windows and macOS only. :::

The company name displayed in about app dialogs and metadata (notably on desktop builds).

### Resolution order

Its value is determined in the following order of precedence:

1. `--company`
2. `[tool.flet].company`
3. Build template default (see Template Source)

### Example

```
flet build <target_platform> --company "My Company Inc."
```

```
[tool.flet]
company = "My Company Inc."
```

## Copyright

:::notePlatform support Windows and macOS only. :::

Copyright text displayed in about app dialogs and metadata.

#### Resolution order

Its value is determined in the following order of precedence:

1. `--copyright`
2. `[tool.flet].copyright`
3. Build template default (see Template Source)

#### Example

```
flet build <target_platform> --copyright "Copyright © 2026 My Company Inc."
```

```
[tool.flet]
copyright = "Copyright © 2026 My Company Inc."
```

## Versioning

### Build Number

An integer identifier used internally to distinguish one build from another.

Each new build must have a unique, incrementing number; higher numbers indicate more recent builds.

#### *Resolution order*

Its value is determined in the following order of precedence:

1. `--build-number`
2. `[tool.flet].build_number`
3. Otherwise, the build number from the generated `pubspec.yaml` (see Template Source) will be used.

#### *Example*

```
flet build <target_platform> --build-number 1
```

```
[tool.flet]
build_number = 1
```

### Build Version

A user-facing version string in `x.y.z` format. Increment this for each new release to differentiate it from previous versions.

#### *Resolution order*

Its value is determined in the following order of precedence:

1. `--build-version`
2. `[project].version`
3. `[tool.poetry].version`
4. Otherwise, the build version from the generated `pubspec.yaml` (see Template Source) will be used.

***Example***

```
flet build <target_platform> --build-version 1.0.0
```

```
[project]
version = "1.0.0"
```

## Output directory

The directory where the build output is saved. If the directory already exists, it is deleted and recreated on each build.

For web builds, the app's `assets` directory is copied into the output directory.

### Resolution order

Its value is determined in the following order of precedence:

1. `--output` (or `-o`)
2. `<python_app_path>/build/<target_platform>`

### Example

```
flet build <target_platform> --output <path-to-output-dir>
```

## App dependencies

These are the Python packages that your Flet app depends on to function correctly.

### Resolution order

Its value is determined in the following order of precedence:

- `[tool.poetry].dependencies` if present; otherwise `[project].dependencies` (PEP 621).
- If `[tool.flet.<PLATFORM>].dependencies` is set (where `<PLATFORM>` corresponds to `<target_platform>`), its values are appended to the list above.
- If the result of all above is empty and `requirements.txt` exists in `<python_app_path>`, it is used.
- If the result of all the above is empty, `flet==<flet_version>` is used.

To use a local development version of a dependency during builds, configure `[tool.flet].dev_packages` or `[tool.flet.<PLATFORM>].dev_packages` with a package name to path mapping.

If your app uses Flet extensions (third-party packages), list them in your Python dependencies so they are packaged with the app. Examples of extensions can be found in Built-in extensions.

**Example**

```
        [project]
        dependencies = [
            "flet",
            "requests",
            "flet-extension1",
            "flet-extension2 @ git+https://github.com/account/flet-
extension2.git",   # git repo
            "flet-extension3 @ file:///path/to/flet-extension3",  #
local package
        ]

        [tool.flet.<PLATFORM>]  # will be used/appended only if
<PLATFORM> corresponds to <target_platform>
        dependencies = [
            "dep1",
            "dep2",
        ]
```

## Source packages

:::notePlatform support Android, iOS, and Web only. :::

By default, packaging for mobile and web only installs binary wheels. Use source packages to allow specific dependencies to be installed from source distributions (sdists).

This can be useful for installing - pure Python - dependencies that do not have pre-built wheels for the target platform or an all-platform wheel (`*-py3-none-any.whl`), but instead provide a source distribution (`*.tar.gz`).

For more information on pure vs non-pure Python packages, see our blog post on the topic.

On desktop targets, source installs are already allowed, so this setting is mainly useful for Android, iOS, and Web (`flet build web` only — not `flet publish`).

**Resolution order**

Its value is determined in the following order of precedence:

1. `--source-packages`
2. `[tool.flet.<PLATFORM>].source_packages`
3. `[tool.flet].source_packages`

**Example**

```
flet build <target_platform> --source-packages package1
package2
```

```
[tool.flet]
source_packages = ["package1", "package2"]
```

## Icons

:::notePlatform support Android, iOS, macOS, Windows and Web only. :::

You can customize app icons for all platforms (except Linux) using image files placed in the `assets` directory of your Flet app.

If a platform-specific icon (as in the table below) is not provided, `icon.png` (or any supported format like `.bmp`, `.jpg`, or `.webp`) will be used as fallback. For the iOS platform, transparency (alpha channel) will be automatically removed, if present.

| Platform | File Name | Recommended Size | Notes |
|---|---|---|---|
| iOS | `icon_ios.png` | ≥ 1024×1024 px | Transparency (alpha channel) is not supported and will be automatically removed if present. |
| Android | `icon_android.png` | ≥ 192×192 px | |
| Web | `icon_web.png` | ≥ 512×512 px | |
| Windows | `icon_windows.ico` or `icon_windows.png` | 256×256 px | `.png` file will be internally converted to a 256×256 px `.ico` icon. |
| macOS | `icon_macos.png` | ≥ 1024×1024 px | |

## Splash screen

:::notePlatform support Android, iOS, and Web only. :::

A splash screen is a visual element displayed when an app is launching, typically showing a logo or image while the app loads.

You can customize splash screens for iOS, Android, and Web platforms by placing image files in the `assets` directory of your Flet app.

If platform-specific splash images are not provided, Flet will fall back to `splash.png`. If that is also missing, it will use `icon.png` or any supported format such as `.bmp`, `.jpg`, or `.webp`.

### Splash images

| Platform | Dark Fallback Order | Light Fallback Order |
|---|---|---|
| iOS | `splash_dark_ios.png` → `splash_dark.png` → `splash_ios.png` → `splash.png` → `icon.png` | `splash_ios.png` → `splash.png` → `icon.png` |

| Platform | Dark Fallback Order | Light Fallback Order |
|---|---|---|
| Android | `splash_dark_android.png` → `splash_dark.png` → `splash_android.png` → `splash.png` → `icon.png` | `splash_android.png` → `splash.png` → `icon.png` |
| Web | `splash_dark_web.png` → `splash_dark.png` → `splash_web.png` → `splash.png` → `icon.png` | `splash_web.png` → `splash.png` → `icon.png` |

### Splash Background Colors

You can customize splash background colors using the following options:

- **Splash Color**: Background color for light mode splash screens.
- **Splash Dark Color**: Background color for dark mode splash screens.

#### *Resolution order*

Their values are respectively determined in the following order of precedence:

1. `--splash-color` / `--splash-dark-color`
2. `[tool.flet.<PLATFORM>.splash].color` / `[tool.flet.<PLATFORM>.splash].dark_color`
3. `[tool.flet.splash].color` / `[tool.flet.splash].dark_color`
4. Build template defaults

#### *Example*

```
flet build <target_platform> --splash-color #ffffff --splash-dark-color #333333
```

```toml
[tool.flet.splash]
color = "#ffffff"
dark_color = "#333333"
```

### Disabling Splash Screens

Splash screens are enabled by default but can be disabled.

#### *Resolution order*

Its value is determined in the following order of precedence:

- on Android:
  - `--no-android-splash`
  - `[tool.flet.splash].android`
- on iOS:
  - `--no-ios-splash`
  - `[tool.flet.splash].ios`
- on Web:
  - `--no-web-splash`

- `[tool.flet.splash].web`

***Example***

```
flet build apk --no-android-splash
flet build ipa --no-ios-splash
flet build ios-simulator --no-ios-splash
flet build web --no-web-splash
```

```toml
[tool.flet.splash]
android = false
ios = false
web = false
```

## Boot screen

The boot screen fills the gap between the native splash screen and your app's first frame — that is, while the Flutter app is up but your Flet/Python app is not ready yet. It is told which of two stages it is in:

1. **Preparing** — the packaged app archive (`app.zip`) is being extracted to the app data directory (on first launch or after the app bundle changes). This stage occurs on **Android only**.
2. **Starting up** — the Python runtime and your app are starting, until the first page is shown (all platforms). If startup fails, the error is shown on the boot screen.

A boot screen is **always rendered**, so this gap is a controlled background instead of a bare scaffold. By default the built-in `flet` boot screen shows nothing but a background color — no spinner and no message — until you configure it. You can also replace it entirely with your own widget (see Custom boot screen).

### Selecting a boot screen

A boot screen is addressed by `name`. The default is `flet` (the built-in screen); custom names are provided by extensions.

```toml
[tool.flet.boot_screen]    # or [tool.flet.<PLATFORM>.boot_screen]
name = "flet"
```

Settings under `[tool.flet.<PLATFORM>.boot_screen]` override the global `[tool.flet.boot_screen]` per key.

### Built-in `flet` boot screen

The built-in screen is configured under a table named after it. All options are optional:

```toml
[tool.flet.boot_screen.flet]
theme_mode = "auto"                        # auto (default), light, or dark
bgcolor_light = "#ffffff"
bgcolor_dark = "#000000"
```

```
        spinner_color_light = "blue"
        spinner_color_dark = "yellow"
        spinner_size = 30                          # 0 or absent → no
spinner
        text_color_light = "#000000"
        text_color_dark = "#ffffff"
        prepare_message = "Preparing your app…"    # Android only;
empty/absent → no message
        startup_message = "Starting up…"           # empty/absent → no
message
```

| Option | Description |
|---|---|
| `theme_mode` | Which color set to use: `auto` (follow the device), `light`, or `dark`. Defaults to `auto`. |
| `bgcolor_light` / `bgcolor_dark` | Background color. When omitted, follows Flet's default theme background. |
| `spinner_color_light` / `spinner_color_dark` | Spinner color. When omitted, follows Flet's default theme primary color. |
| `spinner_size` | Spinner diameter in logical pixels. `0` or absent hides the spinner. |
| `text_color_light` / `text_color_dark` | Message text color. When omitted, follows Flet's default theme on-surface color. |
| `prepare_message` | Text shown during the **preparing** stage (Android only). Empty or absent shows no message. |
| `startup_message` | Text shown during the **starting up** stage. Empty or absent shows no message. |
| `fade_out_duration` | Fade-out duration in milliseconds when the app becomes ready. Defaults to `0` (removed instantly); set a value like `300` to fade out. |

Colors accept the same formats as elsewhere in Flet (hex like `#ffffff` or named colors like `blue`).

### Custom boot screen

To take full control of the boot screen — including custom layouts and animations — provide your own Flutter widget from a Flet extension and reference it by `name`. See Boot screen in the extension authoring guide for how to implement one.

```
        [tool.flet.boot_screen]
        name = "my_screen"

        [tool.flet.boot_screen.my_screen]
        # arbitrary options passed to your widget
```

The older `[tool.flet.app.boot_screen]` and `[tool.flet.app.startup_screen]` settings (with `show` / `message`) are deprecated. They are still honored — and mapped onto the built-in `flet` boot screen — but you should migrate to `[tool.flet.boot_screen]`.

## Hidden app window on startup

:::notePlatform support Windows, macOS, and Linux only. :::

A Flet desktop app (Windows, macOS, or Linux) can start with its window hidden. This lets your app perform initial setup (for example, add content, resize or position the window) before showing it to the user.

See this code example.

### Resolution order

Its value is determined in the following order of precedence:

- `[tool.flet.<PLATFORM>.app].hide_window_on_start`, where `<PLATFORM>` can be `windows`, `macos` or `linux`
- `[tool.flet.app].hide_window_on_start`
- `FLET_HIDE_WINDOW_ON_START`

### Example

```toml
[tool.flet.app]    # or [tool.flet.<PLATFORM>.app]
hide_window_on_start = true
```

## Deep linking

:::notePlatform support Android and iOS only. :::

Deep linking allows users to navigate directly to specific content within a mobile app using a URI (Uniform Resource Identifier). Instead of opening the app's homepage, deep links direct users to a specific page, feature, or content within the app, enhancing user experience and engagement.

- **Scheme**: deep linking URL scheme, e.g. `"https"` or `"myapp"`.
- **Host**: deep linking URL host.

See also:

- Flutter deep linking
- Android intents and intent filters
- Defining a custom URL scheme for your app
- Universal Links

### Resolution order

Its value is determined in the following order of precedence:

1. `--deep-linking-scheme` and `--deep-linking-host` (only when both are provided)
2. `[tool.flet.<PLATFORM>.deep_linking].scheme` / `[tool.flet.<PLATFORM>.deep_linking].host`, where `<PLATFORM>` can be android or ios
3. `[tool.flet.deep_linking].scheme` / `[tool.flet.deep_linking].host`

Both scheme and host are required; if either is missing, the deep-linking entries are not added.

**Example**

```
flet build <target_platform> \
  --deep-linking-scheme "https" \
  --deep-linking-host "mydomain.com"
```

```
[tool.flet.deep_linking]    # or [tool.flet.
<PLATFORM>.deep_linking]
scheme = "https"
host = "mydomain.com"
```

▶ Template translation

## Target Architecture

:::notePlatform support Android and macOS only. :::

A target platform can have different CPU architectures, which in turn support different instruction sets.

It is possible to build your app for specific CPU architectures. This is useful for reducing the size of the resulting binary or package, or for targeting specific devices.

For more/complementary information, see the specific platform guides: Android, macOS.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--arch`
2. `[tool.flet.<PLATFORM>].target_arch`, where `<PLATFORM>` can be `android` or `macos`
3. `[tool.flet].target_arch`
4. Platform defaults for the `<target_platform>`

**Example**

```
flet build macos --arch arm64 x86_64
```

```
[tool.flet.macos]     # or [tool.flet]
target_arch = ["arm64", "x86_64"]
```

## Excluding files and directories

Files and/or directories can be excluded from the build process. This can be useful for reducing the size of the resulting binary or package.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--exclude` (can be used multiple times)
2. `[tool.flet.<PLATFORM>.app].exclude` (type: list of strings)
3. `[tool.flet.app].exclude` (type: list of strings)

The files and/or directories specified should be provided as relative paths to the app path directory. Paths are matched exactly (no globs), and directories are excluded recursively.

By default, the `build` directory is always excluded. Additionally, when the target_platform is web, the `assets` directory is always excluded.

**Example**

```
flet build <target_platform> --exclude .git .venv
```

```
[tool.flet.app]    # or [tool.flet.<PLATFORM>.app]
exclude = [".git", ".venv"]
```

## Compilation and cleanup

Flet can compile your app's `.py` files and/or installed packages' `.py` files into `.pyc` files during the packaging process (via `python -m compileall -b`). Cleanup removes known junk files and any additional globs you specify.

1. Compilation:
   - `compile-app`: compile app's `.py` files
   - `compile-packages`: compile site/installed packages' `.py` files
2. Cleanup:
   - `cleanup-app`: remove junk files from the app directory
   - `cleanup-app-files`: additional globs to delete from the app directory (implies `cleanup-app`)
   - `cleanup-package-files`: additional globs to delete from site-packages (implies `cleanup-packages`)
   - `cleanup-packages`: remove junk files from site-packages (defaults to `true`)

By default, Flet **compiles** both your app and the installed packages to `.pyc` during packaging. Shipping bytecode avoids recompiling every module on each cold start — a significant startup win on mobile, where pure Python is imported from a stored zip and cannot cache bytecode back to disk.

Pass `--no-compile-app` / `--no-compile-packages` (or set `[tool.flet.compile].app` / `[tool.flet.compile].packages` to `false`) to disable it — for example to speed up iterative builds, or to keep `.py` source in the bundle so the build still completes with syntax errors present and tracebacks show source lines.

**Resolution order**

The values of `compile-app` and `cleanup-app` are respectively determined in the following order of precedence:

1. `--compile-app` / `--cleanup-app`
2. `[tool.flet.<PLATFORM>.compile].app` / `[tool.flet.<PLATFORM>.cleanup].app`
3. `[tool.flet.compile].app` / `[tool.flet.cleanup].app`
4. `True` / empty list

The values of `compile-packages` and `cleanup-packages` are respectively determined in the following order of precedence:

1. `--compile-packages` / `--cleanup-packages`
2. `[tool.flet.<PLATFORM>.compile].packages` / `[tool.flet.<PLATFORM>.cleanup].packages`
3. `[tool.flet.compile].packages` / `[tool.flet.cleanup].packages`
4. `True` / `True`

The values of `cleanup-app-files` and `cleanup-package-files` are respectively determined in the following order of precedence:

1. `--cleanup-app-files` / `--cleanup-package-files`
2. `[tool.flet.<PLATFORM>.cleanup].app_files` / `[tool.flet.<PLATFORM>.cleanup].package_files`
3. `[tool.flet.cleanup].app_files` / `[tool.flet.cleanup].package_files`
4. `False` / `False`

**Example**

```
flet build <target_platform> \
  --compile-app --compile-packages \
  --cleanup-app-files "**/*.c" "**/*.h" --cleanup-package-files "**/*.pyi"
```

```toml
[tool.flet.compile]     # or [tool.flet.<PLATFORM>.compile]
app = true
packages = true

[tool.flet.cleanup]     # or [tool.flet.<PLATFORM>.cleanup]
app = true
packages = true
app_files = ["**/*.c", "**/*.h"]
package_files = ["**/*.pyi"]
```

## Permissions

:::notePlatform support Android, iOS, and macOS only. :::

`flet build` allows granular control over permissions, features, and entitlements embedded into `AndroidManifest.xml`, `Info.plist` and `.entitlements` files.

See platform guides for setting specific iOS, Android and macOS permissions.

### Predefined cross-platform permission bundles

Cross-platform permissions are named and predefined bundles that apply a baseline set of platform-specific entries required for a feature. Each bundle expands into the corresponding platform-specific equivalents. This is especially useful for beginners who may be unfamiliar with the underlying platform APIs or prefer not to interact with them directly.

Only the bundles you list are applied. If you need different wording or extra entries, set the platform-specific tables directly; those values are merged on top and can override the bundle defaults. The examples below show the exact `pyproject.toml` equivalents for each bundle.

Below is a list of available bundles:

#### *Resolution order*

Its value is determined in the following order of precedence:

1. `--permissions`
2. `[tool.flet].permissions` (type: list of strings)
3. `[]`

#### *Example*

```
flet build <target_platform> --permissions location microphone
```

```
[tool.flet]
permissions = ["location", "microphone"]
```

## Build template

`flet build` creates (and reuses) a Flutter project under `<app_root>/build/flutter` using a cookiecutter template. By default, the template is downloaded as a zip artifact from the matching Flet GitHub Release. The version of the template used is determined by the installed Flet version.

The cached project is refreshed when template inputs change or after you run `flet clean` to delete the `build` directory.

### Template Source

Defines the location of the cookiecutter build-template to be used.

Supported values include:

- A GitHub repository using the `gh:` prefix (e.g., `gh:org/template`)
- A full Git URL (e.g., `https://github.com/org/template.git`)
- A zip URL (e.g., `https://github.com/flet-dev/flet/releases/download/v0.83.0/flet-build-template.zip`)
- A local directory path

**Resolution order**

Its value is determined in the following order of precedence:

1. `--template`
2. `[tool.flet.template].url`
3. The default zip URL from the Flet GitHub Release matching the installed version

**Example**

```
flet build apk --template gh:my-org/my-custom-template
```

```
[tool.flet.template]
url = "gh:my-org/my-custom-template"
```

**Template Reference**

Defines the branch, tag, or commit to check out from the template source.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--template-ref`
2. `[tool.flet.template].ref`
3. `<flet_version>`

**Example**

```
flet build <target_platform> --template-ref main
```

```
[tool.flet.template]
ref = "main"
```

**Template Directory**

Defines the relative path to the cookiecutter template. If template source is set, the path is treated as a subdirectory within its root; otherwise, it is relative to the template root.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--template-dir`
2. `[tool.flet.template].dir`
3. root of the template source

**Example**

```
flet build <target_platform> --template gh:org/template --
template-dir sub/directory
```

```toml
[tool.flet.template]
url = "gh:org/template"
dir = "sub/directory"
```

## Additional `flutter build` Arguments

During the `flet build` process, `flutter build` command gets called internally to package your app for the specified platform. However, not all `flutter build` arguments are exposed or usable through the `flet build` command directly.

For possible `flutter build` arguments, see Flutter docs guide. For most targets, run `flutter build <target_platform> --help`; for `ios-simulator`, run `flutter build ios --simulator --help`.

Passing additional `flutter build` arguments might cause unexpected behavior. Use at your own risk, and only if you fully know what you're doing!

### Resolution order

Its value is determined in the following order of precedence:

1. `--flutter-build-args` (can be used multiple times)
2. `[tool.flet.<PLATFORM>.flutter].build_args`
3. `[tool.flet.flutter].build_args`

### Example

```
flet build apk \
  --flutter-build-args=--obfuscate \
  --flutter-build-args=--export-method=development \
  --flutter-build-args=--dart-
define=API_URL=https://api.example.com
```

```toml
[tool.flet.flutter]     # or [tool.flet.<PLATFORM>.flutter]
build_args = [
  "--obfuscate",
  "--export-method=development",
  "--dart-define=API_URL=https://api.example.com",
]
```

## Flutter dependencies

When you run `flet build`, Flet generates a Flutter shell project and then updates its `pubspec.yaml` using values from `pyproject.toml`.

Use:

- `[tool.flet.flutter.pubspec.dependencies]` for normal package declarations. (Dart docs)
- `[tool.flet.flutter.pubspec.dependency_overrides]` when you must force a version or source, for example, a local path or Git fork. (Dart docs)

Values follow standard Pub dependency syntax, expressed in TOML.

- **Important:** In most cases, you usually do not need to add/override Flutter dependencies. We recommend doing it only if you fully know what you are doing, as it can lead to unexpected behavior.
- If the same package appears in both `pyproject.toml` and the resulting `pubspec.yaml`, the value from `pyproject.toml` wins.
- If you use `{ path = "..." }` under `[tool.flet.flutter.pubspec.dependencies]` or `[tool.flet.flutter.pubspec.dependency_overrides]`, that path is resolved by Flutter from the generated `pubspec.yaml` location: `<flet_app_directory>/build/flutter/pubspec.yaml`. This means relative paths are **not** resolved from your `pyproject.toml` file.

**Example**

```
        [tool.flet.flutter.pubspec.dependencies]    # or
[tool.flet.flutter.pubspec.dependency_overrides]
        # Version
        pkg_1 = "^1.2.3"

        # Local path
        pkg_2 = { path = "../pkg_2" }

        # Git (short form)
        pkg_3 = { git = "https://github.com/org/pkg_3.git" }

        # Git (expanded form: URL + ref + subdirectory)
        pkg_4 = { git = { url =
"https://github.com/org/mono_repo.git", ref = "main", path =
"packages/pkg_4" } }

        # Hosted source
        pkg_5 = { hosted = { name = "pkg_5", url = "https://pub.dev"
}, version = "^1.0.0" }

        # SDK package (dependencies only; typically not used in
dependency_overrides)
        flutter_test = { sdk = "flutter" }
```

## Verbose logging

The `-v` (or `--verbose`) and `-vv` flags enable detailed output from all commands during the flet build process.

Use `-v` for standard/basic verbose logging, or `-vv` for even more detailed output (higher verbosity level). If you need support, we may ask you to share this verbose log.

# Console output

In packaged apps (`flet build` output), all output from your Python code such as `print()` statements, `sys.stdout.write()` calls, and messages from the Python `logging` module is redirected to a `console.log` file. The full path to this file is available via `StoragePaths.get_console_log_filename()` or the `FLET_APP_CONSOLE` environment variable.

Note: `FLET_APP_CONSOLE` is only set in production builds; in development runs, output stays in your terminal.

The log file is written in an unbuffered manner, allowing you to read it at any point in your Python program using:

```
        import os
        import flet as ft

        async def main(page: ft.Page):
            log_file = await
ft.StoragePaths().get_console_log_filename()
            # or
            # log_file = os.getenv("FLET_APP_CONSOLE")

            with open(log_file, "r") as f:
                logs = f.read()
                page.add(ft.Text(logs)) # display on UI

        ft.run(main)
```

If your program calls `sys.exit(100)`, the complete log will automatically be shown in a scrollable window. This is a special “magic” exit code for debugging purposes:

```
        import sys
        sys.exit(100)
```

Calling `sys.exit()` with any other code will terminate the app without displaying the log.

# Continuous Integration/Continuous Deployment (CI/CD)

You can use `flet build` command in your CI/CD pipelines to automate the build and release process of your Flet apps.

## GitHub Actions

You can use GitHub Actions to build your Flet app automatically on every push, pull request, or manual run.

```
        name: Build Flet App # (1)!
```

```yaml
on: # (2)!
  push: # (3)!
  pull_request: # (4)!
  workflow_dispatch: # (5)!

env: # (6)!
  UV_PYTHON: 3.12 # (7)!
  PYTHONUTF8: 1 # (8)!

  # https://flet.dev/docs/reference/environment-variables
  FLET_CLI_NO_RICH_OUTPUT: 1 # (9)!

jobs:
  build:
    name: Build ${{ matrix.name }}
    runs-on: ${{ matrix.runner }}
    strategy: # (10)!
      fail-fast: false
      matrix:
        include:
          # -------- Desktop --------
          - name: linux
            runner: ubuntu-latest
            build_cmd: "flet build linux"
            artifact_path: build/linux
            needs_linux_deps: true

          - name: macos
            runner: macos-26
            build_cmd: "flet build macos"
            artifact_path: build/macos
            needs_linux_deps: false

          - name: windows
            runner: windows-latest
            build_cmd: "flet build windows"
            artifact_path: build/windows
            needs_linux_deps: false

          # -------- Android --------
          - name: aab
            runner: ubuntu-latest
            build_cmd: "flet build aab"
            artifact_path: build/aab
            needs_linux_deps: false

          - name: apk
            runner: ubuntu-latest
            build_cmd: "flet build apk"
            artifact_path: build/apk
            needs_linux_deps: false
```

```
                    # -------- iOS --------
                    - name: ipa
                      runner: macos-26
                      build_cmd: "flet build ipa"
                      artifact_path: build/ipa
                      needs_linux_deps: false

                    - name: ios-simulator
                      runner: macos-26
                      build_cmd: "flet build ios-simulator"
                      artifact_path: build/ios-simulator
                      needs_linux_deps: false

                    # -------- Web --------
                    - name: web
                      runner: ubuntu-latest
                      build_cmd: "flet build web"
                      artifact_path: build/web
                      needs_linux_deps: false

                steps:
                  - name: Checkout repository
                    uses: actions/checkout@v4 # (11)!

                  - name: Setup uv
                    uses: astral-sh/setup-uv@v6 # (12)!

                  - name: Install Linux dependencies # (13)!
                    if: matrix.needs_linux_deps # (14)!
                    shell: bash
                    run: |
                        sudo apt update --allow-releaseinfo-change
                        LINUX_DEPS="$(uv run flet --version --json | jq -r
'.linux_dependencies | join(" ")')"
                        sudo apt-get install -y --no-install-recommends
$LINUX_DEPS
                        sudo apt-get clean

                  - name: Build app # (15)!
                    shell: bash
                    run: |
                      uv run ${{ matrix.build_cmd }} --yes --verbose

                  - name: Upload Artifact
                    uses: actions/upload-artifact@v5.0.0 # (16)!
                    with:
                      name: ${{ matrix.name }}-build-artifact
                      path: ${{ matrix.artifact_path }} # (17)!
                      if-no-files-found: error # (18)!
                      overwrite: false
```

1. Workflow display name shown in the **Actions** tab.
2. Trigger block for automatic and manual workflow runs.
3. Runs this workflow on every push (unless you restrict branches).
4. Runs this workflow when pull requests are opened/updated.
5. Enables manual runs from GitHub UI (**Actions → Run workflow**).
6. Environment variables shared by all jobs and steps.
7. Python version used by `uv`.
8. Forces UTF-8 mode for Python output/IO. Especially useful on Windows builds.
9. Disables rich output from `flet build` for better readability in CI logs.
10. Matrix strategy: each `include` item becomes a parallel build job.
11. Checks out your repository so this workflow can access project files. View its docs here.
12. Installs `uv` on the runner. View its docs here.
13. Installs Linux system packages required by Linux desktop builds.
14. Runs Linux package installation only for matrix entries that need it.
15. Main build step for each target. Executes the target-specific command defined by `matrix.build_cmd`.
16. Uploads build output files as downloadable artifacts. View its docs here.
17. Artifact path expected from each build target.
18. If no files were found to upload, the workflow fails, indicating something went wrong during the build.

The workflow file above builds for all major targets and uploads each build output as an artifact. You can further customize the workflow for your specific needs, for example, restricting the build targets or adding additional steps.

See it in action here.

# Troubleshooting

## Prerelease compatibility

If you are using a prerelease version of the Flet Python package (for example, `0.80.6.devNNNN`) to build an app, the build template may still resolve the latest **stable** `flet` Flutter package, which can lead to version incompatibility issues.

**Why?**: Under normal circumstances, each prerelease of the Flet Python package would require a matching prerelease of the Flutter Flet package to guarantee compatibility. However, we don't publish prerelease versions of the Flutter package to pub.dev. Because of this, the build template resolves the latest **stable** Flutter `flet` release instead.

This creates a version mismatch/incompatibility for apps packaged with `flet build`:

- Your Python code may depend on newly introduced controls or features.
- The packaged Flutter shell may still be using an older stable `flet` version.
- At runtime, the app fails because the Flutter layer does not recognize the new controls/features in your prerelease `flet` package, leading to errors like `Unknown control: <ControlName>`.

**Note**: this issue does not affect the development workflows (ex: running an app with `flet run`), as the `flet` Flutter dependency is only resolved during the `flet build` process.

### Solution

The rule-of-thumb is, if you are using a prerelease Flet Python package, always ensure the Flutter `flet` dependency is aligned with the same development version before building your app:

1. Override the Flutter `flet` dependency to point to the corresponding development Git reference.

`toml [tool.flet.flutter.pubspec.dependency_overrides] flet = { git = { url = "https://github.com/flet-dev/flet.git", ref = "main", path = "packages/flet" } }` 2. Rebuild the app with the build cache cleared (run `flet clean` to delete the `build` directory)

To ensure reproducible builds (ex: in production or CI), prefer using a specific commit SHA, instead of a branch or tag ref.

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs';

Instructions for packaging a Flet app into an Xcode archive and, when signing is configured, an IPA for distribution.

This guide provides detailed on iOS-specific information. Complementary and more general information is available here.

## Prerequisites

### Rosetta 2

Some Flutter tooling and dependencies still ship as Intel binaries. Install Rosetta 2 on Apple Silicon if Flutter or CocoaPods prompts for it:

```
sudo softwareupdate --install-rosetta --agree-to-license
```

### Xcode

Xcode 15 or later to compile native Swift or Objective-C code. Open Xcode once after installation to accept the license and install additional components.

### CocoaPods

CocoaPods 1.16 or later to compile and enable Flutter plugins.

### iOS wheels for binary Python packages

Binary Python packages (vs "pure" Python packages written in Python only) are packages that are partially written in C, Rust, or other languages producing native code. Example packages are `numpy`, `cryptography`, or `pydantic-core`.

Make sure all non-pure (binary) packages used in your Flet app have pre-built wheels for iOS.

## `flet build ipa`

:::caution[Supported host platforms] This command can be run on **macOS only**. :::

Builds an iOS app archive (`.xcarchive`) and, when signing is configured, exports an `.ipa` for testing or distribution.

To generate an `.ipa` for testing on your device or uploading to App Store Connect for distribution, you will need the following:

- Apple Developer Program subscription with access to App Store Connect
- Application Identifier
- Bundle ID configured in your Flet project (must match the App ID)
- Signing Certificate
- Provisioning Profile

## `flet build ios-simulator`

:::caution[Supported host platforms] This command can be run on **macOS only**. :::

Builds an (unsigned) iOS Simulator `.app` bundle, intended for testing on iOS Simulator and does **not** require a signing certificate, provisioning profile, or Apple Developer Program setup.

# Configuration options

### Application Identifier (App ID)

A unique string that identifies your app within the Apple ecosystem. It is required to sign and distribute an iOS app and is used for various services like Push Notifications, App Groups, iCloud, and In-App Purchases.

It consists of two parts:

1. **Team ID**: A unique 10-character string assigned by Apple to your developer account.
2. **Bundle ID**: A reverse domain-style identifier for your app (e.g., `com.example.myapp`).

```
graph TD
    A[App ID: ABCDEFE234.com.example.myapp] --> B[Team ID: ABCDEFE234]
    A --> C[Bundle ID: com.example.myapp]
```

The Bundle ID in your Flet configuration must match the Bundle ID registered in the App ID.

**Creating a new App ID**

1. Visit Apple Developer Portal.
2. Sign in with your **Apple Developer Account**.
3. Click the **"+"** button to add a new identifier.
4. Select **"App IDs"** and click **Continue**.
5. **Enter a Description** – This is just for reference (e.g., "MyApp Identifier").
6. **Choose the App ID Type:**
   - Select **"App"** for a standard iOS/macOS app.
   - If you need an identifier for services like Apple Pay or Passbook, select the appropriate option.
7. **Bundle ID** – Choose:
   - **Explicit Bundle ID** (`com.example.myapp`) – Recommended for most apps.
   - **Wildcard Bundle ID** (`com.example.*`) – Allows multiple apps to use the same identifier (rarely used).
8. **Enable App Services** – Check the boxes for any services your app needs (e.g., Push Notifications, Sign in with Apple, etc.).
9. Click **Continue** and **Register**.

Now you have **Bundle ID** and **Team ID** that will be used to identify your app.

**Team ID**

The developer team ID to include in export options.

***Resolution order***

Its value is determined in the following order of precedence:

1. `--ios-team-id`
2. `[tool.flet.ios].team_id`
3. `[tool.flet.ios.export_methods."EXPORT_METHOD"].team_id`

***Example***

```
flet build ipa --ios-team-id ABCDEFE234
```

```
[tool.flet.ios]
team_id = "ABCDEFE234"
```

### Signing Certificate

The certificate name, SHA-1 hash, or automatic selector to use for signing the iOS app bundle. Automatic selectors allow Xcode to pick the newest installed certificate of a particular type.

The available automatic selectors are:

- `"Apple Development"`
- `"Apple Distribution"`
- `"Developer ID Application"`
- `"iOS Developer"`
- `"iOS Distribution"`

- `"Mac App Distribution"`
- `"Mac Developer"`

**Generating a Certificate Signing Request (CSR)**

Before creating a development or distribution certificate, you need a **CSR (Certificate Signing Request)**.

1. **Open Keychain Access** on your Mac (++cmd+space++, then search for “Keychain Access”).
2. In the top menu, go to **Keychain Access → Certificate Assistant → Request a Certificate From a Certificate Authority…**
3. Fill in:
   - **User Email Address:** Your Apple Developer email.
   - **Common Name:** A descriptive name (e.g., “My App Distribution”).
   - **CA Email Address:** Leave this blank.
   - **Request is for:** Select “**Saved to disk**”.
4. Click **Continue**, choose a location to save the `.certSigningRequest` file, and click **Save**.

**Creating a Certificate in Apple Developer Portal**

1. Go to the Apple Developer Certificates Page.
2. Click the **“+”** button to create a new certificate.
3. Select **“Apple Distribution”** (for App Store & Ad Hoc) or **“Apple Development”** (for development) and click **Continue**.
4. Upload the **CSR file** you created earlier and click **Continue**.
5. Apple will generate the certificate. Click **Download** to get the `.cer` file.
6. Double-click the downloaded `.cer` file to install it in **Keychain Access**.
7. Open the **Keychain Access** app and ensure the certificate is installed under **“login”** keychain. The name of the development certificate usually starts with **“Apple development:”** and the name of the distribution certificate starts with **“Apple distribution:”**.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--ios-signing-certificate`
2. `[tool.flet.ios].signing_certificate`
3. `[tool.flet.ios.export_methods."EXPORT_METHOD"].signing_certificate`

**Example**

```
flet build ipa --ios-signing-certificate "Apple Distribution"
```

```
[tool.flet.ios]
signing_certificate = "Apple Distribution"
```

### Provisioning Profile

A **Provisioning Profile** is a file that allows an iOS app to run on physical devices and be distributed through the App Store or internally. It links your **App ID**, **Developer/Distribution Certificate**, and **Registered Devices**.

There are different types of provisioning profiles:

1. **Development Profile** – Used for testing on physical devices.
2. **Ad Hoc Profile** – Used for distributing an app outside the App Store to specific devices.
3. **App Store Profile** – Used for submitting an app to the App Store.
4. **Enterprise Profile** – Used for internal distribution within an organization.

**Creating a New Provisioning Profile**

Follow these steps to create a provisioning profile via the Apple Developer Portal:

***Step 1: Go to Apple Developer Portal***

- Visit the Apple Developer Portal;
- Sign in with your **Apple Developer Account**

***Step 2: Create a New Provisioning Profile***

- Click the **"+"** button to add a new provisioning profile;
- Choose the **type of profile**:
  - **iOS App Development** – for testing on devices
  - **Ad Hoc** – for distributing to specific devices
  - **App Store** – for submitting an app to the App Store
  - **In-House** – for internal distribution (Enterprise accounts only)
- Click **Continue**

***Step 3: Select an App ID***

- Choose the **App ID** that matches your app;
- Click **Continue**.

***Step 4: Select a Distribution Certificate***

- Choose the appropriate certificate:
  - **iOS Development Certificate** - for **iOS App Development** profile
  - **iOS Distribution Certificate** - for **Ad Hoc** or **App Store** profiles
- Click **Continue**.

***Step 5: Select Registered Devices (for Development & Ad Hoc)***

- If you selected an **iOS App Development** or **Ad Hoc** profile, select the devices to include;
- Click **Continue**.

***Step 6: Name and Generate the Profile***

- Enter a **Profile Name** (e.g., `MyApp Development Profile`);
- Click **Generate**;
- Click **Download** to get the `.mobileprovision` file.

**Installing Provisioning Profile**

Provisioning profiles are stored in `~/Library/MobileDevice/Provisioning Profiles` directory.

To install a downloaded provisioning profile, copy it to `~/Library/MobileDevice/Provisioning\ Profiles` directory with a new `{UUID}.mobileprovision` name.

Run the following command to get profile UUID:

```
        profile_uuid=$(security cms -D -i ~/Downloads/{profile-
name}.mobileprovision | xmllint --xpath
"string(//key[.='UUID']/following-sibling::string[1])" -)
        echo $profile_uuid
```

Run this command to copy profile to `~/Library/MobileDevice/Provisioning Profiles` with a new name `{UUID}.mobileprovision`:

```
        cp ~/Downloads/{profile-name}.mobileprovision
~/Library/MobileDevice/Provisioning\
Profiles/${profile_uuid}.mobileprovision
```

If the copied profile disappears from the `~/Library/MobileDevice/Provisioning Profiles` directory, ensure that the Xcode process is not running in the background.

Finally, you can use the command below to list all installed provisioning profiles, with their names and UUIDs:

```
        for profile in ~/Library/MobileDevice/Provisioning\
Profiles/*.mobileprovision; do security cms -D -i "$profile" | grep -E
-A1 '<key>(Name|UUID)</key>' | sed -n 's/.*<string>\(.*\)
<\/string>/\1/p' | paste -d ' | ' - -; done
```

**Resolution order**

Its value is determined in the following order of precedence:

1. `--ios-provisioning-profile`
2. `[tool.flet.ios].provisioning_profile`
3. `[tool.flet.ios.export_methods."EXPORT_METHOD"].provisioning_profile`

The profile must match your Bundle ID.

**Example**

```
        flet build ipa --ios-provisioning-profile "release-testing
com.mycompany.example-app"
```

```
[tool.flet.ios]
provisioning_profile = "release-testing com.mycompany.example-app"
```

### Export configuration

**Export options**

Additional keys to include in the generated `exportOptions.plist` of the build template.

For a complete list of supported keys and values, run the following command:

```
xcodebuild -help
```

and find the section titled **“Available keys for -exportOptionsPlist”**.

***Resolution order***

Its value is determined in the following order of precedence:

1. `[tool.flet.ios].export_options` (if set, per-method export options are ignored)
2. `[tool.flet.ios.export_methods."EXPORT_METHOD"].export_options` (see export methods)
3. `{}` (no extra keys)

***Supported value forms***

Both simple and complex structures are supported:

- string
- boolean
- dictionary (nested key-value object)
- array of strings
- array of booleans
- array of dictionaries (including dictionaries that contain arrays)

Numbers are not yet supported. ##### Example

```
[tool.flet.ios.export_options]
uploadSymbols = false
compileBitcode = false
thinning = "<none>"

[tool.flet.ios.export_options.manifest]
appURL = "https://example.com/app.ipa"
displayImageURL = "https://example.com/icon57.png"
fullSizeImageURL = "https://example.com/icon512.png"
```

▶ Template translation

**Export method**

Defines how the app should be packaged when exporting the `.ipa` file.

Can be one of the following:

- `debugging` (or deprecated `development`): used for debugging and testing on development devices.
- `release-testing` (or deprecated `ad-hoc`): used for distributing the app outside the App Store to specific registered devices.
- `app-store-connect` (or deprecated `app-store`): used for submitting the app to the App Store.
- `enterprise`: used for internal distribution within an organization (requires an enterprise account).

To configure individual settings for one or more export methods, see export methods.

***Resolution order***

Its value is determined in the following order of precedence:

1. `--ios-export-method`
2. `[tool.flet.ios].export_method`
3. `"debugging"`

***Example***

```
flet build ipa --ios-export-method debugging
```

```
[tool.flet.ios]
export_method = "debugging"
```

#### Export methods

Signing settings can be configured individually per export method.

Per-method values are used only when the corresponding top-level `[tool.flet.ios]` setting is not set. The method key must match the `export_method` value exactly.

Supported keys (same as the top-level settings):

- `provisioning_profile`
- `signing_certificate`
- `export_options`
- `team_id`

***Example***

```
[tool.flet.ios.export_methods."debugging"]
provisioning_profile = "debugging com.mycompany.example-app"
signing_certificate = "Apple Development"
```

```
        [tool.flet.ios.export_methods."release-testing"]
        provisioning_profile = "release-testing com.mycompany.example-
app"
        team_id = "ABCDEFE234"
        signing_certificate = "Apple Distribution"
        export_options = { uploadSymbols = false }

        [tool.flet.ios.export_methods."app-store-connect"]
        provisioning_profile = "app-store-connect
com.mycompany.example-app"
        team_id = "ABCDEFE234"
        signing_certificate = "Apple Distribution"
        export_options = { uploadSymbols = true }
```

### Permissions

iOS permissions are declared through `Info.plist` privacy usage strings. You can also use the cross-platform permission bundles to inject common entries, then override or extend them with platform-specific values.

**Info.plist**

Add or override `Info.plist` entries for iOS builds. These values are written to `ios/Runner/Info.plist` of the build project.

***Resolution order***

Its value is determined in the following order of precedence:

1. `--info-plist`
2. `[tool.flet.ios.info]`
3. Values injected by cross-platform permission bundles, if any.

**Supported value forms**

Accepts repeated `<key>=<value>` entries. The `<value>` can be in one of the following forms:

- `true` or `false` (case-insensitive) for boolean values
- integer and real number literals, for example `32` or `0.5`
- TOML array literals, for example `["myapp", "myapp-beta"]`
- TOML inline tables, for example `{ NSAllowsArbitraryLoads = false }`
- any other value is treated as a string Both simple and complex structures are supported:
- string
- boolean

- integer
- real
- dictionary (nested key-value object)
- array of strings
- array of booleans
- array of integers
- array of reals
- array of dictionaries (including dictionaries that contain arrays) ##### Example

```
        flet build ipa \
          --info-plist NSCameraUsageDescription="This app needs camera
access." \
          --info-plist UIFileSharingEnabled=true \
          --info-plist ExampleInteger=32 \
          --info-plist ExampleReal=0.5 \
          --info-plist 'LSApplicationQueriesSchemes=["myapp", "myapp-
beta"]' \
          --info-plist 'FeatureFlags=[true, false]' \
          --info-plist 'RetryDelays=[1, 2, 3]' \
          --info-plist 'OpacitySteps=[0.25, 0.5, 0.75]' \
          --info-plist 'NSAppTransportSecurity={
NSAllowsArbitraryLoads = false }' \
          --info-plist 'CFBundleURLTypes=[{ CFBundleTypeRole =
"Editor", CFBundleURLName = "example.com", CFBundleURLSchemes =
["myapp"] }, { CFBundleTypeRole = "Viewer", CFBundleURLName =
"example.org", CFBundleURLSchemes = ["myapp-beta"] }]'
```

```
        [tool.flet.ios.info]
        NSCameraUsageDescription = "This app needs camera access."
        UIFileSharingEnabled = true
        ExampleInteger = 32
        ExampleReal = 0.5
        LSApplicationQueriesSchemes = ["myapp", "myapp-beta"]
        FeatureFlags = [true, false]
        RetryDelays = [1, 2, 3]
        OpacitySteps = [0.25, 0.5, 0.75]
        NSAppTransportSecurity = { NSAllowsArbitraryLoads = false }
        CFBundleURLTypes = [
          { CFBundleTypeRole = "Editor", CFBundleURLName =
"example.com", CFBundleURLSchemes = ["myapp"] },
          { CFBundleTypeRole = "Viewer", CFBundleURLName =
"example.org", CFBundleURLSchemes = ["myapp-beta"] },
        ]
```

▶ Template translation

# Deploying an App to an Apple Device for Testing

You can deploy `.ipa` files directly to an iPhone or iPad on macOS—ideal for internal testing without publishing to the App Store.

Follow these steps:

### Install and Launch Apple Configurator

- Install Apple Configurator from App Store;
- Find and open it from your Applications folder.

### Connect Your iOS Device

- Connect your device (iPhone or iPad) to your Mac via USB;
- Unlock the connected device, and tap **Trust This Computer**, if prompted;
- Wait for this device to appear in the Apple Configurator.

### Add the `.ipa` File

- Either drag the `.ipa` file onto the connected device in Apple Configurator;
- Or click **Add → Apps**, then browse and select the `.ipa` file from your Mac.

### Install the App

- Click **Prepare** or **Install** to begin the deployment/installation;
- Apple Configurator will install the app on the connected device.

### Trust the Developer (for Ad Hoc or Enterprise apps)

If your app is signed with an **Ad Hoc** or **Enterprise** provisioning profile, you'll need to manually trust the developer:

- On the iOS device, go to **Settings → General → VPN & Device Management**;
- Under **Developer App**, tap your **Developer Profile**;
- Tap **Trust [Your Developer Name]**, then confirm.

# Uploading app to App Store Connect for distribution

### Install and Sign in to Transporter

- Install and open **Transporter** from the Mac App Store.
- Sign in using your **Apple Developer Account** credentials (the same account used for App Store Connect).

### Prepare Your `.ipa` File

- Build your app and export an `.ipa` file using the **app-store-connect** export method. (**release-testing** is for Ad Hoc device distribution, not App Store Connect.)

**Upload the `.ipa` File in Transporter**

- Drag and drop the `.ipa` file directly into the Transporter window, or click “Add App” and select your `.ipa` file from your Mac.
- Click the “…” button next to “Deliver”, and select “Verify”.
- Wait for Transporter to complete the verification process.
- After successful verification, click “Deliver” to upload your `.ipa` file to App Store Connect.

**Check Upload Status**

- Transporter will display a success message upon completion.
- If errors occur, carefully review the details provided, correct the issues, and repeat the upload process.

**Confirm Upload in App Store Connect**

- Go to App Store Connect.
- Navigate to **Apps → Your App → TestFlight or App Store Version**.
- Your newly uploaded build will initially appear under **Processing** (processing typically takes a few minutes to an hour).
- Once processing completes, your build will become available for submission. You can now **submit the app for review**.

Instructions for packaging a Flet app into a Linux executable.

This guide provides detailed Linux-specific information. Complementary and more general information is available here.

# Prerequisites

Flet uses Flutter to build Linux apps. Compiling the app and its native plugins links against GTK and a number of system libraries, so these must be installed before running `flet build linux`.

On Debian/Ubuntu-based distributions, install the required packages with `apt`:

```
        sudo apt update
        sudo apt install -y \
          binutils clang cmake llvm lld ninja-build pkg-config \
          libgtk-3-dev libsecret-1-0 libsecret-1-dev libunwind-dev \
          gstreamer1.0-alsa gstreamer1.0-gl gstreamer1.0-gtk3
gstreamer1.0-libav \
          gstreamer1.0-plugins-bad gstreamer1.0-plugins-base
gstreamer1.0-plugins-good \
          gstreamer1.0-plugins-ugly gstreamer1.0-pulseaudio
gstreamer1.0-qt5 \
          gstreamer1.0-tools gstreamer1.0-x \
          libasound2-dev libgstreamer1.0-dev \
          libgstreamer-plugins-base1.0-dev libgstreamer-plugins-
```

```
bad1.0-dev \
          libmpv-dev mpv
```

This is the same set of packages Flet uses in its own build environment. A few notes on what they are for:

- **Build toolchain** — `clang`, `cmake`, `ninja-build`, `pkg-config`, `llvm`, `lld`, `binutils` and `libgtk-3-dev` are required to compile and link the app. In particular, the `lld` linker must be present — without it the build fails with a linker error.
- **Secret storage** — `libsecret-1-0` and `libsecret-1-dev` are used for secure storage / keyring access.
- **Audio and video** — the `gstreamer1.0-*`, `libgstreamer*-dev`, `libasound2-dev`, `libmpv-dev` and `mpv` packages are required by the `Audio` service and `Video` control. You can omit them if your app does not play media, but installing the full set above avoids surprises. See those pages for control-specific details.

:::note[Other distributions] Package names differ on non-Debian distributions (e.g. Fedora, Arch). Install the equivalent GTK 3, GStreamer, `mpv`/`libmpv`, `libsecret`, `clang`/`llvm`, `lld`, `cmake` and `ninja` development packages for your distribution. :::

## `flet build linux`

This command can be run on **Linux only** (or WSL).

Builds a Linux executable.

## Window positioning on Wayland

On Linux the **display server** controls window placement, and this differs between X11 and Wayland:

- **X11** lets applications set their own top-level window position.
- **Wayland** (the default session on modern GNOME/Ubuntu) does **not** — by design, a client cannot position its own top-level window; the compositor (e.g. Mutter) decides where windows are placed.

As a result, on a Wayland session the following have **no effect** (window *sizing* still works — only positioning is restricted):

- `Page.window.center()`
- setting `Page.window.left` / `Page.window.top`
- moving the window programmatically

This is a Wayland protocol limitation, not a Flet bug. The same code works as expected on Windows, macOS, Linux X11 sessions, and Wayland sessions running the app through **XWayland**.

To force the X11 backend (XWayland) on a Wayland session and re-enable programmatic positioning, run the app with the `GDK_BACKEND` environment variable:

```
GDK_BACKEND=x11 ./your_app
```

You can check the current session type with:

```
echo $XDG_SESSION_TYPE   # "wayland" or "x11"
```

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs';

Instructions for packaging a Flet app into a macOS application bundle.

This guide provides detailed macOS-specific information. Complementary and more general information is available here.

# Prerequisites

## Rosetta 2

Flutter, which we use for packaging, requires Rosetta 2 on Apple Silicon:

```
sudo softwareupdate --install-rosetta --agree-to-license
```

## Xcode

Xcode 15 or later is required to compile native Swift or Objective-C code.

## CocoaPods

CocoaPods 1.16 or later is required to install and compile Flutter plugins.

# `flet build macos`

This command can be run on **macOS only**.

Builds a macOS application bundle from your Flet app.

# Target architecture

By default, `flet build macos` creates a universal bundle that runs on both Apple Silicon and Intel Macs. Packaging downloads Python wheels for both `arm64` and `x86_64` architectures.

To limit packaging to a specific architecture, see this. This affects which Python wheels are bundled and, in turn, which CPU architectures the app will run on. You will then have to provide your users with the correct build for their Macs.

# Permissions

macOS permissions are declared through `Info.plist` privacy usage strings and app entitlements. You can also use the cross-platform permission bundles to inject common entries, then override or extend them with platform-specific values.

## Info.plist

Add or override `Info.plist` entries for macOS builds. These values are written to `macos/Runner/Info.plist` of the build project.

### Resolution order

Its value is determined in the following order of precedence:

1. `--info-plist`
2. `[tool.flet.macos.info]`
3. Values injected by cross-platform permission bundles, if any.

### Supported value forms

Accepts repeated `<key>=<value>` entries. The `<value>` can be in one of the following forms:

- `true` or `false` (case-insensitive) for boolean values
- integer and real number literals, for example `32` or `0.5`
- TOML array literals, for example `["basic", "advanced"]`
- TOML inline tables, for example `{ NSAllowsArbitraryLoads = false }`
- any other value is treated as a string Both simple and complex structures are supported:
- string
- boolean
- integer
- real
- dictionary (nested key-value object)
- array of strings
- array of booleans
- array of integers
- array of reals
- array of dictionaries (including dictionaries that contain arrays) #### Example

```
        flet build macos \
          --info-plist LSApplicationCategoryType="public.app-
category.utilities" \
          --info-plist NSSupportsSuddenTermination=true \
          --info-plist ExampleInteger=32 \
          --info-plist ExampleReal=0.5 \
          --info-plist 'SupportedModes=["basic", "advanced"]' \
          --info-plist 'FeatureFlags=[true, false]' \
          --info-plist 'RetryDelays=[1, 2, 3]' \
          --info-plist 'OpacitySteps=[0.25, 0.5, 0.75]' \
          --info-plist 'NSAppTransportSecurity={
NSAllowsArbitraryLoads = false }' \
          --info-plist 'CFBundleDocumentTypes=[{ CFBundleTypeName =
"Data File", CFBundleTypeExtensions = ["dat"] }, { CFBundleTypeName =
"JSON File", CFBundleTypeExtensions = ["json"] }]'

        [tool.flet.macos.info]
        LSApplicationCategoryType = "public.app-category.utilities"
        NSSupportsSuddenTermination = true
        ExampleInteger = 32
        ExampleReal = 0.5
        SupportedModes = ["basic", "advanced"]
        FeatureFlags = [true, false]
        RetryDelays = [1, 2, 3]
        OpacitySteps = [0.25, 0.5, 0.75]
        NSAppTransportSecurity = { NSAllowsArbitraryLoads = false }
        CFBundleDocumentTypes = [
          { CFBundleTypeName = "Data File", CFBundleTypeExtensions =
["dat"] },
          { CFBundleTypeName = "JSON File", CFBundleTypeExtensions =
["json"] },
        ]
```

▶ Template translation

## Entitlements

Entitlements are property-list key-value pairs that grant an executable permission to use a service or technology. The supported value type depends on the entitlement key defined in the Apple Developer Entitlements Reference.

Entitlements are written to `macos/Runner/DebugProfile.entitlements` and `macos/Runner/Release.entitlements` in the build template.

### Resolution order

Its value is determined in the following order of precedence:

1. `--macos-entitlements`
2. `[tool.flet.macos.entitlement]`

3. Values injected by cross-platform permission bundles, if any.

4. Defaults:

```toml
[tool.flet.macos.entitlement]
"com.apple.security.app-sandbox" = false
"com.apple.security.cs.allow-jit" = true
"com.apple.security.network.client" = true
"com.apple.security.network.server" = true
"com.apple.security.files.user-selected.read-write" = true
```

**Supported value forms**

Accepts repeated `<key>=<value>` entries. The `<value>` can be in one of the following forms:

- `true` or `false` (case-insensitive) for boolean values
- integer and real number literals, for example `32` or `0.5`
- TOML array literals, for example `["group.example.one", "group.example.two"]`
- TOML inline tables, for example `{ "com.apple.mail" = ["compose"] }`
- any other value is treated as a string Both simple and complex structures are supported:
- string
- boolean
- integer
- real
- dictionary (nested key-value object)
- array of strings
- array of booleans
- array of integers
- array of reals
- array of dictionaries (including dictionaries that contain arrays) #### Example

```
flet build macos \
  --macos-entitlements com.apple.security.network.client=true \
  --macos-entitlements com.apple.developer.ubiquity-kvstore-identifier=ABCDE12345.dev.example.myapp \
  --macos-entitlements ExampleInteger=32 \
  --macos-entitlements ExampleReal=0.5 \
```

```
          --macos-entitlements 'com.apple.security.application-groups=
["group.dev.example.myapp", "group.dev.example.shared"]' \
          --macos-entitlements 'ExampleBooleanArray=[true, false]' \
          --macos-entitlements 'com.apple.security.scripting-targets={
"com.apple.mail" = ["compose", "send"] }' \
          --macos-entitlements 'ExampleArrayOfDictionaries=[{ Name =
"alpha", Enabled = true }, { Name = "beta", Enabled = false }]'
```

```
        [tool.flet.macos.entitlement]
        "com.apple.security.network.client" = true
        "com.apple.developer.ubiquity-kvstore-identifier" =
"ABCDE12345.dev.example.myapp"
        ExampleInteger = 32
        ExampleReal = 0.5
        "com.apple.security.application-groups" = [
          "group.dev.example.myapp",
          "group.dev.example.shared",
        ]
        ExampleBooleanArray = [true, false]
        "com.apple.security.scripting-targets" = { "com.apple.mail" =
["compose", "send"] }
        ExampleArrayOfDictionaries = [
          { Name = "alpha", Enabled = true },
          { Name = "beta", Enabled = false },
        ]
```

▶ Template translation

Fly.io works well for Flet apps: it supports WebSockets, deploys close to users via multiple regions, and has a free tier suitable for small projects.

# Prerequisites

1. Install flyctl.
2. Authenticate:

```
fly auth login
```

# Project files

Place the following files in your app directory.

### `requirements.txt`

At minimum:

```
txt title="requirements.txt" flet
```

**fly.toml**

Fly app configuration:

```toml title="fly.toml"
app = ""

kill_signal = "SIGINT"
kill_timeout = 5
processes = []

[env]
  FLET_SESSION_TIMEOUT = "60"

[http_service]
  internal_port = 8000
  force_https = true
  auto_stop_machines = true
  auto_start_machines = true
  min_machines_running = 0

[http_service.concurrency]
  type = "connections"
  soft_limit = 200
  hard_limit = 250
```

```
- Replace `<your-app-name>` with the app name you want. It will also
be used in the final
    URL of your app, in the form, `https://<your-app-name>.fly.dev`.
- `internal_port` must correspond to the [`FLET_SERVER_PORT`]
(../../../../reference/environment-variables.md#flet_server_port),
which is `8000` by default.
- [`FLET_SESSION_TIMEOUT`](../../../../reference/environment-
variables.md#flet_session_timeout) controls session lifetime
(seconds).
- For other variables, see [environment variables reference]
(../../../../reference/environment-variables.md).

### `Dockerfile`

Create a `Dockerfile` containing the commands to build your
application container,
for example:

```Dockerfile title="Dockerfile"
FROM python:3-alpine

WORKDIR /app

COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "main.py"]
```

`main.py` is the app's entry point. If your file name differs, update accordingly.

Fly.io deploys apps as Docker containers and provides a free remote Docker builder, so you don't need Docker installed locally.
```

# Deploy

From your app directory:

1. Create the Fly app: `bash fly apps create --name <your-app-name>`
2. Deploy: `bash fly deploy`
3. Open app in browser: `bash fly apps open`

Flet apps use WebSockets for real-time partial updates of their UI and sending events back to your program. When choosing a hosting provider for your Flet app you should pay attention to their support of WebSockets. Sometimes WebSockets are not allowed or come as a part of more expensive offering, sometimes there is a proxy that periodically breaks WebSocket connection by a timeout (Flet implements re-connection logic, but it could be unpleasant behavior for users of your app anyway).

Another important factor while choosing a hosting provider for Flet app is latency. Every user action on UI sends a message to Flet app and the app sends updated UI back to user. Make sure your hosting provider has multiple data centers, so you can run your app closer to the majority of your users.

Replit is an online IDE and hosting platform for web apps written in any language. Their free tier allows running any number of apps with some performance limitations.

To run your app on Replit:

- Sign up on Replit.
- Click “New Repl” button, select “Python” template and type the name of your repl, e.g. `my-flet-app`. Alternatively, go to Flet template page and click **Use Template** button. Flet template has everything configured for you, so you can jump to `main.py` and update your program right away.
- Open `.replit` file on the left and add these two options to the root:

```
        # Stops the packager from installing packages when running the
Repl
        disableInstallBeforeRun = true
        # Stops the packager from guessing and auto-installing
packages, but it still runs to install packages when running the Repl
        disableGuessImports = true
```



- On “Tools” pane click “Dependencies” and search for `flet` package and click “Install” button.
- Open `main.py` on “Files” pane and copy-paste your app.
- Modify call to `ft.run()` and include `view=ft.AppView.WEB_BROWSER` parameter:

```
        ft.run(main, view=ft.AppView.WEB_BROWSER)
```

- Run the app. Enjoy.



Host a Flet app on your own server with NGINX.

There are free and inexpensive cloud server tiers available at AWS, Oracle, Linode, and more.

# Setup Flet Environment

### `requirements.txt` and virtualenv

Create `requirements.txt` with a list of application dependencies. At minimum it should contain `flet` module:

```
flet
```

Create a virtualenv and install requirements:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

### Sample Flet app

```
import flet as ft

def main(page: ft.Page):
    page.title = "My Flet app"
    page.add(ft.Text("Hello, world!"))

if __name__ == "__main__":
    ft.run(main)
```

Flet app will be served from the root URL, but you can configure `FLET_WEB_APP_PATH` environment variable to serve beneath the root e.g. `/apps/myapp`.

By default, Flet web app will be running on port `8000`, but you can change that by setting up `FLET_SERVER_PORT` environment variable.

Complete list of environment variables supported by a web app.

# Automatically start Flet app

### Create `systemd` unit file

Automatically start the Flet app using a `systemd` service unit file `flet.service`.

Setup below assumes your Flet app script is defined in `$HOME/flet-app/main.py`. Replace `User`, `Group`, `WorkingDirectory`, etc. as per your setup.

```
[Unit]
Description=Flet App
After=network.target

[Service]
User=ubuntu
Group=ubuntu
WorkingDirectory=/home/ubuntu/flet-app
Environment="PATH=/home/ubuntu/flet-app/.venv/bin"
ExecStart=/home/ubuntu/flet-app/.venv/bin/python
/home/ubuntu/flet-app/main.py

[Install]
WantedBy=multi-user.target
```

### Enable Flet app service

```
cd /etc/systemd/system
sudo ln -s /home/ubuntu/flet-app/flet.service
sudo systemctl start flet
sudo systemctl enable flet
sudo systemctl status flet
```

## NGINX Proxy Setup

NGINX will proxy the Flet app and the websocket.

In your `/etc/nginx/sites-available/*` config file, updating path and port as needed:

```
            location / {
                proxy_pass         http://127.0.0.1:8000/;
                proxy_http_version 1.1;
                proxy_set_header   Upgrade $http_upgrade;
                proxy_set_header   Connection keep-alive;
                proxy_set_header   Host $host;
                proxy_cache_bypass $http_upgrade;
                proxy_set_header   X-Forwarded-For
$proxy_add_x_forwarded_for;
                proxy_set_header   X-Forwarded-Proto $scheme;
            }

            location /ws {
                proxy_pass         http://127.0.0.1:8000/ws;
                proxy_http_version 1.1;
                proxy_set_header   Upgrade $http_upgrade;
                proxy_set_header   Connection "upgrade";
                proxy_set_header   Host $host;
                proxy_cache_bypass $http_upgrade;
                proxy_set_header   X-Forwarded-For
$proxy_add_x_forwarded_for;
```

```
            proxy_set_header   X-Forwarded-Proto $scheme;
        }
```

That's it! Restart NGINX, and open your app in a browser.

Flet runs your app as a FastAPI (ASGI) app to serve a dynamic website.

It uses Uvicorn by default, but any ASGI-compatible server can be used instead.

## Sync and async handlers

In a Flet web app you can mix sync and async handlers.

For example, you can write an app like this:

```
import flet as ft
import time
import asyncio

def main(page: ft.Page):

    def handler(e):
        time.sleep(3)
        page.add(ft.Text("Handler clicked"))

    async def handler_async(e):
        await asyncio.sleep(3)
        page.add(ft.Text("Async handler clicked"))

    page.add(
        ft.Button("Call handler", on_click=handler),
        ft.Button("Call async handler",
on_click=handler_async)
    )

ft.run(main)
```

In the example, a click on one button is handled by a blocking handler while a click on the second button calls an async handler. The first handler runs in a `threading.Thread`, while the second runs as an `asyncio.Task`.

In web apps, threads are a finite resource. A thread pool is usually used and can become a bottleneck as the number of users grows.

If your app is mostly doing I/O (database, web API) and you can use async-ready libraries, prefer async handlers.

Check FastAPI's article about async/await to better understand the differences between concurrency and parallelism.

# Running the app locally

Use `--web` (`-w`) option to start a Flet app as a web app:

```
flet run --web app.py
```

A new browser window/tab will be opened and the app will use a random TCP port.

To run on a fixed port use `--port` (`-p`):

```
flet run --web --port 8000 app.py
```

# Running the app in production

You can run your program directly with `python`:

```
python app.py
```

Uvicorn web server is used by default to host the web app.

If Flet detects a headless Linux environment (such as a Docker container or EC2 VM): 1. Port `8000` will be used to run the app. 2. A browser window will not be opened.

If Flet cannot detect a headless environment, you can force that behavior by defining the following environment variable:

```
FLET_FORCE_WEB_SERVER=true
```

## ASGI web server

You can host your Flet web app with any ASGI-compatible server such as Uvicorn (used by default), Hypercorn or Daphne.

Just tell Flet to export ASGI app:

```python title="main.py"
import flet as ft

def main(page: ft.Page):
    page.add(ft.Text("Hello ASGI!"))

app = ft.run(main, export_asgi_app=True)
```

#### Hypercorn

[Hypercorn](https://github.com/pgjones/hypercorn/) is another ASGI web server inspired by Gunicorn.

To run the app with Hypercorn:

```bash
hypercorn main:app --bind 0.0.0.0:8000
```

### Daphne

Daphne is a HTTP, HTTP2 and WebSocket protocol server for ASGI and ASGI-HTTP, developed to power Django Channels.

To run the app with Daphne:

```
daphne -b 0.0.0.0 -p 8000 main:app
```

### Gunicorn

Gunicorn is popular web server to run Python web applications. While it implements WSGI specification it's possible to run ASGI FastAPI apps with a worker process class provided by Uvicorn:

```
gunicorn --bind 0.0.0.0:8000 -k uvicorn.workers.UvicornWorker
counter:app
```

## Assets

When you open a Flet app in the browser, its `index.html`, Flutter engine, favicon, images and other web app resources are served by a web server. These resources are bundled with `flet` Python package. However, there are situations when you need to modify the contents of certain files to customize the appearance of your app or its behavior, for example:

- Favicon.
- App loading animation.
- `manifest.json` with PWA details.

You can specify `assets_dir` in `flet.run()` to set the location of assets that should be available to the application. `assets_dir` should be relative to your `main.py` directory or an absolute path. Default value for `assets_dir` argument is `assets`.

For example, consider the following program structure:

```
/assets
   /images/my-image.png
main.py
```

You can access your images in the application as follows:

```
import flet as ft

def main(page: ft.Page):
    page.add(ft.Image(src=f"/images/my-image.png"))

ft.run(main, assets_dir="assets")
```

## Customizing web app

### Favicon

To override favicon with your own put `favicon.png` file into the root of assets directory. It should be a PNG image with the size of at least 32x32 pixels.

### Loading animation

To override the Flet animation image, put `icons/loading-animation.png` with your own app logo in the root of the assets directory.

### PWA

Progressive Web Apps, or PWAs, offer a way to turn app-like websites into website-like apps.

Check PWAs Turn Websites Into Apps: Here's How for the PWA introduction.

Browsers that support PWA (installation instructions):

- **Chrome** on all platforms
- **Edge** on all platforms
- **Firefox** on Android
- **Safari** on iOS and iPadOS

The information in this section is based on the following sources (check them out for more details):

- General information about PWAs
- PWA manifests

### Manifest

You can change PWA's name, description, colors and other information in `manifest.json` that must be put in the root of `assets` directory.

Here are the links to the most common manifest items that you'd like to customize:

- `name` - the name of the web application as it is usually displayed to the user.
- `short_name` - the name of the web application displayed to the user if there is not enough space to display `name`.
- `description` - explains what the application does.
- `theme_color` - defines the default theme color for the application.
- `background_color` - defines a placeholder background color for the application page to display before its stylesheet is loaded.

### Icons

Custom icons should be placed in `assets/icons` directory:

- `icon-192.png`, `icon-512.png` - app icons displayed in Windows taskbar.

- `icon-maskable-192.png`, `icon-maskable-512.png` - app icons displayed in Android.
- `apple-touch-icon-192.png` - app icon displayed in iOS.

# Environment variables

Every aspect of web app hosting can be additionally controlled with environment variables.

# Advanced FastAPI scenarios

## Flet FastAPI app

- `flet.fastapi.app()` creates a FastAPI application to handle Flet sessions and mounts the following endpoints at the app root:
  - `/ws` (WS) - WebSocket handler for the Flet app. It calls `main()` when a new WebSocket connection is established and a new app session is created.
- `/upload` (PUT) - file uploads handler.
- `/oauth_callback` (GET) - OAuth flow callback handler.
- `/` (GET) - Flet app static files with SPA catch-all handler.

## Hosting multiple Flet apps under the same domain

```python
import flet as ft
import flet.fastapi as flet_fastapi

async def root_main(page: ft.Page):
    page.add(ft.Text("This is root app!"))

async def sub_main(page: ft.Page):
    page.add(ft.Text("This is sub app!"))

app = flet_fastapi.FastAPI()

app.mount("/sub-app", flet_fastapi.app(sub_main))
app.mount("/", flet_fastapi.app(root_main))
```

Sub-apps must be mapped before the root Flet app as it configures catch-all `index.html` for SPA.

Run the app with `uvicorn` and visit http://127.0.0.1:8000 and then http://127.0.0.1:8000/sub-app/ to see both Flet apps running.

Notice the trailing slash in `/sub-app/` - without it the request will be routed to the root app.

## Adding Flet to the existing FastAPI app

```python
from contextlib import asynccontextmanager

import flet as ft
import flet.fastapi as flet_fastapi
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    await flet_fastapi.app_manager.start()
    yield
    await flet_fastapi.app_manager.shutdown()

app = FastAPI(lifespan=lifespan)

async def main(page: ft.Page):
    page.add(ft.Text("Hello, Flet!"))

app.mount("/flet-app", flet_fastapi.app(main))
```

When adding a Flet app to an existing FastAPI app you need to call `flet_fastapi.app_manager.start()` on app start and `flet_fastapi.app_manager.shutdown()` on shutdown. Use the way that best suits you: lifespan (in the example above) or app events.

`app_manager.start()` method starts background tasks cleaning up expired sessions and OAuth flow states.

`app_manager.shutdown()` method removes any temporary files created by a Flet app.

## Configuring individual Flet endpoints

### Static files

A FastAPI app to serve static Flet app files (index.html, manifest.json, Flutter JS app, etc.) and user assets.

```python
from flet.fastapi import FastAPI, FletStaticFiles

app = FastAPI()

# mount to the root of web app
app.mount(path="/", app=FletStaticFiles())
```

### WebSocket handler

Handles WebSocket connections from the Flet client running in the browser. The WebSocket channel is used to send events from the browser to the backend and receive page updates.

```python
import asyncio
import flet as ft
from flet.fastapi import FletApp, app_manager

async def main(page: ft.Page):
    page.add(ft.Text("Hello, Flet!"))

@app.websocket("/app1/ws")
async def flet_app(websocket: WebSocket):
    await FletApp(
        loop=asyncio.get_running_loop(),
        executor=app_manager.executor,
        main=main,
        before_main=None,
    ).handle(websocket)
```

**Uploads handler**

Handles file uploads by `FilePicker` control. This endpoint is optional - if your app doesn't use `FilePicker`, then it's unnecessary.

```python
from flet.fastapi import FletUpload

@app.put("/upload")
async def flet_uploads(request: Request):
    await
FletUpload("/Users/feodor/Downloads/123").handle(request)
```

**OAuth callback handler**

Handles OAuth flow callback requests. If your app doesn't use authentication, then this endpoint is unnecessary.

```python
from flet.fastapi import FletOAuth

@app.get("/oauth_callback")
async def flet_oauth(request: Request):
    return await FletOAuth().handle(request)
```

# Publish Flet web app

Flet allows publishing your app as a **static** or **dynamic** website.

1. **Static website** - content is delivered as-is and Python runs in the browser (Pyodide).
2. **Dynamic website** - content is generated per user and Python runs on the server.

Here is a table comparing Flet app running as a static vs dynamic website:

| | Static website | Dynamic website |
|---|---|---|
| **Loading time** | ⬇️ Slower - Python runtime (Pyodide) along with app's Python code and all its dependencies must be loaded into the browser. Pyodide initialization takes time too. | ✅ Faster - the app stays and runs on the server. |
| **Python compatibility** | ⬇️ Not every program that works with native Python can be run with Pyodide | ✅ Any Python package can be used. |
| **Responsiveness** | ✅ Zero latency between user-generated events (clicks, text field changes, drags) and page updates. | ⬇️ Non-zero latency - user-generated events are communicated to a server via WebSockets. UI updates are communicated back. |
| **Performance** | ⬇️ Slower - Pyodide is currently 3x-5x slower than native Python because of WASM | ✅ Faster - the code is running on the server by native Python. |
| **Code protection** | ⬇️ Low - app's code is loaded into a web browser and can be inspected by a user. | ✅ High - the app is running on the server. |
| **Hosting** | ✅ Cheap/Free - no code is running on the server and thus the app can be hosted anywhere: GitHub Pages, Cloudflare Pages, Replit, Vercel, a shared hosting or your own VPS. | ⬇️ Paid - the app requires Python code to run on the server and communicate with a web browser via WebSockets. |
| **Python version** | ⬇️ Tied to a specific Pyodide release — pick from the supported set (3.12 / 3.13 / 3.14). See Choosing a Python version. | ✅ Any Python version that runs your server. |

This guide provides the instructions for hosting Flet static website on Cloudflare.

Before we get to the deployment, you will need an account. Get one from here, or simply login if you already have one. After signing up, you will have to verify your email address by clicking the link you will receive in your email. Check the spams too, if you don't get it in your inbox.

In your account, from the side menu, select "Pages" as shown below:



And from there, select the "Create a project" button:



Cloudflare proposes three ways to create a project. Only the first two will be exposed here: - Connect to a git provider - Direct upload



# Connecting to a Git provider

This method works for `flet publish` command only.

For this, you will need to have a GitHub or GitLab account. In this account should be the repository you plan to use. An example could be found in this repo.

Click on the “Connect to Git” blue button:



From there, select the tab with the service containing your repository. Then, connect your account. Select one of the suggested options, then click on “Install & Authorize”.



Choose the repository to be used, and press on the “Begin setup” button.



Before moving on, add a `runtime.txt` file in your repo. It should contain the python version to be used. In the file enter 3.7 which is the latest python version Cloudflare uses at time of writing.

Here is an example from the repo above.

Having this done, we can now move to the next step which will be to configure some build and deployment settings for your site.

Set the name of your project, and the production branch to be used. The production branch is simply the branch of your repo to which any push of a commit will automatically trigger a deployment to your production environment. Pushes to your other branches will trigger deployment instead to your preview environment.

After setting these two, jump down to the “Build settings” section where we will be setting up the build instructions.

Skip the “Framework preset” (allow None) because Flet is neither in the list nor a JavaScript framework :)

The “Build command” depends on your application’s structure. Follow the guide in the sections above to come up with your custom build command.



When the build command is ran by flet, a folder named ‘dist’ is created which will contain all the web files required by Cloudflare pages. Set it as your output directory. Note that this file will not be added to your repository, because Cloudflare only has read access to your code.



You could optionally specify advanced parameters: the root directory (the directory in which Cloudflare runs the build command), and Environment variables (variables to be used during build time). Now, click on the “Save and Deploy” button and let Cloudflare

do the remaining job for you.



Click on the URL that will be shown to move to your deployed site.



If when opening the site you see a Cloudflare error, it means they haven't completely finished the setup. So, simply wait for a minute then refresh the page, and you will see your application running. Test the above deployed site here.

## Direct Upload

This method works for both `flet build web` and `flet publish` commands.

Click on "Upload assets". In step one, give your project a name. Remember that the name you will give will be used to determine the link to which your project will be deployed to.

The second step requires you to upload your project's assets, either as folder or a zip file (with all the assets inside). If you already have one of them, then use the drag-and-drop or select them using the folder picker.

If you don't yet have these assets but already have an app you've built, use the `flet publish` command in the directory containing your app files, and a `dist` folder will be created which you will then upload to Cloudflare pages as mentioned above.

After the upload press on "Deploy site" button at the bottom.



You will then see a success message with a link to your deployed website. Test an example of a deployed site here.



If when opening the site you see a Cloudflare error, it means they haven't completely finished the setup. So, simply wait for a minute then refresh the page, and you will see your application running. You can now click on the "Continue to project" button to monitor your deployments, or create new ones following the same steps above.

## Troubleshooting

When Flet app is running in a web browser all its `print()` statements are displayed in "Console" tab of Developer Tools in a browser. `print()` can be used as a simple debugging tool.

You can also use `logging` module and output messages to Console with different severity.

To enable detailed Flet logging add this to your program:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

This guide shows how to build a Flet static web app and deploy it to GitHub Pages with GitHub Actions.

## Setup

1. Open your repository root.
2. Create this folder if it does not exist: `.github/workflows/`
3. Create a workflow file in that folder, for example: `.github/workflows/build-deploy.yml` - must have the `.yml` or `.yaml` file extension.
4. Copy the workflow configuration below into that file and adjust to your needs.
5. Commit and push to GitHub.
6. Open the **Actions** tab to monitor build/deploy progress.

:::danger[Repository settings] In GitHub, open **Settings → Pages** and make sure deployment source is **GitHub Actions**. :::

## Workflow

```yaml
name: Web Build + Deploy to GitHub Pages # (1)!

on: # (2)!
  push: # (3)!
  pull_request: # (4)!
  workflow_dispatch: # (5)!

concurrency: # (6)!
  group: "pages" # (7)!
  cancel-in-progress: false # (8)!

env: # (9)!
  UV_PYTHON: 3.12 # (10)!

  # https://flet.dev/docs/reference/environment-variables
  FLET_CLI_NO_RICH_OUTPUT: 1 # (11)!

jobs:
  build:
    name: Build web app
    runs-on: ubuntu-latest # (12)!

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4 # (13)!

      - name: Setup uv
        uses: astral-sh/setup-uv@v6 # (14)!
```

```
            - name: Build app
              shell: bash
              run: | # (15)!
                uv run flet build web --yes --verbose --base-url
${GITHUB_REPOSITORY#*/} --route-url-strategy hash

            - name: Upload Pages Artifact
              uses: actions/upload-pages-artifact@v4.0.0 # (16)!
              with:
                path: build/web # (17)!
                name: web-build-artifact # (18)!
                retention-days: 20 # (19)!

        deploy:
          name: Deploy to GitHub Pages
          if: github.event_name == 'push' && github.ref ==
'refs/heads/main' # (20)!
          needs: build # (21)!
          runs-on: ubuntu-latest

          permissions: # (22)!
            pages: write
            id-token: write

          environment: # (23)!
            name: github-pages
            url: ${{ steps.deployment.outputs.page_url }}

          steps:
            - name: Setup Pages
              uses: actions/configure-pages@v5 # (24)!

            - name: Deploy to GitHub Pages
              id: deployment
              uses: actions/deploy-pages@v4.0.5 # (25)!
              with:
                artifact_name: web-build-artifact # (26)!
```

1. Workflow name shown in the Actions list.
2. Trigger block for the workflow.
3. Builds on every push.
4. Builds on pull request updates.
5. Allows manual runs from the Actions tab.
6. Concurrency settings for GitHub Pages deployments.
7. Places deployments into the same `pages` concurrency group.
8. Keeps the currently running deployment instead of canceling it.
9. Environment variables available to all jobs.
10. Python version used by `uv`.
11. Produces cleaner CI logs by disabling rich output.
12. Runs the build job on GitHub-hosted Ubuntu.
13. Checks out repository code. View its docs here.
14. Installs `uv`. View its docs here.

15. Builds static web output and sets:
    - `--base-url ${GITHUB_REPOSITORY#*/}` so project pages deploy under `/<repo>/`. If your repository is `<username>.github.io`, use `--base-url /` instead of repository name. (docs)
    - `--route-url-strategy hash` so routing works on static hosting without server-side rewrites. (docs)
16. Uploads static files as a Pages artifact. View its docs here.
17. Uses `build/web` as artifact source.
18. Artifact name used later by deploy step.
19. Keeps the artifact for 20 days.
20. Deploys only on pushes to `main`. PRs only build but don't deploy. Adjust as needed.
21. Waits for successful build job before deploy.
22. Required permissions for Pages deployment.
23. Connects deployment output URL to the GitHub environment.
24. Configures Pages runtime. View its docs here.
25. Deploys artifact to GitHub Pages. View its docs here.
26. Must match the uploaded artifact name.

See it in action here.

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs';

Instructions for publishing a Flet app as a standalone static website (SPA) that runs entirely in the browser with Pyodide. No Python code runs on the server.

Pyodide is a port of CPython to WebAssembly (WASM) and has some limitations.

:::note[Native Python packages] Native Python packages (vs "pure" Python packages written in Python only) are packages partially written in C, Rust, or other languages producing native code. Example packages are `numpy`, `cryptography`, `lxml`, `pydantic-core`.

Pyodide comes with a list of built-in packages. To use a package from PyPI, it must be pure Python or provide a wheel built for Emscripten. :::

:::note[Async and threading] Static websites run in a single browser thread. You can use sync and async handlers, but long-running CPU work or blocking calls will freeze the UI. Prefer async I/O, or move heavy work to a server and call it via a web API. :::

Pyodide installs packages with micropip. You can use the Micropip API directly in your Flet app:

```python
import sys

if sys.platform == "emscripten":
    import micropip
    await micropip.install("regex")
```

:::note[Pyodide and Python version] The Python version you choose for `flet build web` / `flet publish` pins a specific Pyodide release — see Choosing a Python version for the full matrix and resolution rules. In short:

| Python | Pyodide |
|---|---|
| 3.14 | 314.0.0 |

| Python | Pyodide |
|---|---|
| 3.13 | 0.29.4 |
| 3.12 | 0.27.7 |

The matching Pyodide runtime is downloaded into the build output and cached under `~/.flet/pyodide/<version>/` on first use. The older `0.27.5` bundle that used to ship inside the build template has been removed in favour of this versioned per-build download. :::

# Differences

There are two ways to publish a static website: `flet build web` and `flet publish`. Both produce a static site that runs in the browser via Pyodide. They differ mainly in how and when Python dependencies are installed:

| | `flet publish` | `flet build web` |
|---|---|---|
| Flutter required | No | Yes |
| Dependency install | At runtime, in the browser (`micropip`) | At build time, on your machine (`pip`) |
| Build time | Faster — no Flutter compilation, no local pip install | Slower — Flutter build + local dependency install |
| Initial load time | Slower — wheels are fetched from PyPI on page load | Faster — dependencies are already bundled |
| Pure-Python wheels | ✅ | ✅ |
| Pyodide-built binary wheels | ✅ (from Pyodide CDN) | ✅ (auto-detected from Pyodide registry) |
| Source distributions (sdists) | ❌ `micropip` can't build sdists in the browser | ✅ pure-Python sdists, opt in via `source_packages` |

## Sdist-only dependencies

A common failure mode with `flet publish` is a (transitive) dependency that ships only a source distribution (`.tar.gz`, no wheel) — for example, `docopt`. `micropip` cannot build sdists, so load fails with:

```
ValueError: Can't find a pure Python 3 wheel for '<package>'
```

In such cases, we suggest switching to `flet build web` and adding the package to `source_packages`. **Note** that `source_packages` only works for **pure-Python** sdists. Sdists with C/Rust extensions (e.g. `numpy`, `cryptography`) cannot be built for Pyodide — use Pyodide's built-in packages instead.

## `flet publish`

Does not require Flutter. It packages your app and installs dependencies in the browser at runtime via micropip.

To publish an app, run:

```
flet publish <path-to-app.py>
```

The website is published to `--distpath` (default: `./dist`).

### Testing the site

You can try published Flet app using `flet serve` command:

```
flet serve dist
```

Then, open `http://localhost:8000` in your browser to check the published app.

### Assets

If the assets directory exists (default: `./assets`), its contents are copied to the published site root. Use `--assets` to point to a different folder. Assets are not packaged inside the `app.tar.gz`.

## `flet build web`

Uses Flutter and Pyodide. Dependencies are resolved and installed locally at build time (via `pip`), then bundled into the output archive.

Complementary and more general information is available here.

### Testing the site

`flet serve` serves the default output directory (`./build/web`):

```
flet serve
```

# Configuration options

These settings apply to `flet build web` and `flet publish`, unless noted.

### Base URL

Use a base URL when hosting your app in a subdirectory. Flet normalizes it to `/<value>/` and uses `/` when unset.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--base-url`
2. `[tool.flet.web].base_url`
3. `"/"`

**Example**

```
flet build web --base-url /myapp/
```

```
[tool.flet.web]
base_url = "/myapp/"
```

### Route URL strategy

Controls how routes are represented in the URL:

- `path` - clean paths; requires SPA-capable hosting.
- `hash` - uses the URL hash; works on static hosts without SPA support.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--route-url-strategy`
2. `[tool.flet.web].route_url_strategy`
3. `"path"`

**Example**

```
flet build web --route-url-strategy hash
```

```
[tool.flet.web]
route_url_strategy = "hash"
```

### Web renderer

Selects the Flutter web renderer:

- `canvaskit` (default) - CanvasKit renderer, with the app compiled to JavaScript
- `skwasm` - Skia WebAssembly renderer, with the app compiled to WebAssembly
- `auto` - let Flutter pick the renderer based on the browser

The default is `canvaskit`, not `auto`. With `auto`, Chromium-based browsers pick `skwasm`, where every byte buffer passed between JavaScript and Dart pays a costly WebAssembly boundary conversion. Flet web apps exchange bytes with the Python runtime on every UI update, so `canvaskit` is significantly faster for them.

**Resolution order**

Its value is determined in the following order of precedence:

1. `--web-renderer`
2. `[tool.flet.web].renderer`
3. `"canvaskit"`

**Example**

```
flet build web --web-renderer skwasm
```

```
[tool.flet.web]
renderer = "skwasm"
```

### CDN assets

By default, Pyodide, CanvasKit, and fonts are loaded from CDNs to keep the output small. Disable CDN loading for offline or air-gapped deployments.

**Resolution order**

CDN loading is disabled in the following order of precedence:

1. `--no-cdn`
2. `[tool.flet.web].cdn = false`
3. default: CDN enabled

**Example**

```
flet build web --no-cdn
```

```
[tool.flet.web]
cdn = false
```

### PWA colors

Configure PWA colors used in `manifest.json` and browser UI.

**Resolution order**

For each setting:

1. `--pwa-background-color` / `--pwa-theme-color`
2. `[tool.flet.web].pwa_background_color` / `[tool.flet.web].pwa_theme_color`
3. `#FFFFFF` / `#0175C2`

**Example**

```
flet build web --pwa-background-color "#000000" --pwa-theme-color "#FF0000"
```

```
[tool.flet.web]
pwa_background_color = "#000000"
pwa_theme_color = "#FF0000"
```

### WASM output

By default, `flet build web` enables Flutter's WASM output.

`flet build web` only.

**Resolution order**

The WASM output is disabled in the following order of precedence:

1. `--no-wasm`
2. `[tool.flet.web].wasm = false`
3. default: WASM enabled

**Example**

```
flet build web --no-wasm
```

```
[tool.flet.web]
wasm = false
```

Instructions for packaging a Flet app into a Windows application.

This guide provides detailed Windows-specific information. Complementary and more general information is available here.

# Prerequisites

## Visual Studio

Visual Studio (2022 or 2026) is required with the **Desktop development with C++** workload installed.

Follow this guide for instructions on downloading and installing correct Visual Studio components for Flutter desktop development.

# `flet build windows`

This command can be run on **Windows only**.

Builds a Windows application.

# Troubleshooting

## Developer mode

If you get the below error:

```
Building with plugins requires symlink support.

Please enable Developer Mode in your system settings. Run
  start ms-settings:developers
to open settings.
```

Then, you need to enable Developer Mode as it indicates. Follow this guide on how to do that.

import PypiIndex from '@site/.crocodocs/pypi-index.mdx';

Binary Python packages (as opposed to "pure" Python packages written entirely in Python) contain components written in C, Rust, or other languages that produce native code. Common examples include `numpy`, `cryptography`, and `pydantic`.

Flet provides an alternative package index, pypi.flet.dev, which hosts prebuilt Python binary wheels (`.whl` files used by `pip`) for iOS and Android platforms.

:::danger[Work in progress] New packages are created by adding a recipe to the Mobile Forge project. Currently, we author these recipes for you; once the build process is fully automated, you'll be able to submit a PR and test the compiled package immediately.

If a package is not available on pypi.flet.dev, you can request it in Flet discussions - Packages. Please do not request "pure" Python packages. Check out this guide for more details on the difference between pure and binary packages. :::

Below is the list of packages hosted on pypi.flet.dev and the specific wheel versions currently built and supported:

:::tip[Setting boolean values] To set a boolean `True`, use one of the following string values: `"true"`, `"1"` or `"yes"`. Any other value will be interpreted as `False`. :::

### FLET_APP_CONSOLE

The path to the application's console log file (`console.log`) in the cache storage directory (`FLET_APP_STORAGE_CACHE`).

Its value is set in production mode.

In a running Flet app, the equivalent of this environment variable is [`StoragePaths.get_console_log_filename()`] [flet.StoragePaths.get_console_log_filename].

### FLET_APP_STORAGE_DATA

A directory for **durable** application data — databases, state, config — that is preserved between app updates, backed up by the OS, and never auto-deleted. It is pre-created, app-private, and its location depends on the platform (it maps to the platform's *application support* directory: `%APPDATA%\<company>\<product>\data` on Windows, `~/Library/Application Support/<bundle-id>/data` on macOS, `~/.local/share/<app-id>/data` on Linux, the app-sandboxed support dir on iOS/Android).

In production this directory is also the Python program's **current working directory**, so relative file writes (e.g. `open("app.db", "w")`, `sqlite3.connect("app.db")`) land here. In `flet run` (dev mode) it is `<project>/.flet/storage/data`.

In a running Flet app, this maps to a `data` subdirectory of [`StoragePaths.get_application_support_directory()`] [flet.StoragePaths.get_application_support_directory].

### FLET_APP_STORAGE_CACHE

A directory for **regenerable** cached data. The OS may purge it under storage pressure (and the platform "clear cache" action wipes it), so only store things you can rebuild. Pre-created and app-private; it maps to the platform's *caches* directory (`%LOCALAPPDATA%\<company>\<product>` on Windows, `~/Library/Caches/<bundle-id>` on macOS, `~/.cache/<app-id>` on Linux, the app cache dir on iOS/Android). In `flet run` it is `<project>/.flet/storage/cache`.

In a running Flet app, the equivalent of this environment variable is [`StoragePaths.get_application_cache_directory()`] [flet.StoragePaths.get_application_cache_directory].

### FLET_APP_STORAGE_TEMP

A directory for **throwaway** temporary files — the OS temporary directory (`getTemporaryDirectory()`). It is the most volatile of the three and may be cleared between launches, so don't rely on its contents persisting. (On Android it resolves to the same directory as FLET_APP_STORAGE_CACHE.) In `flet run` it is `<project>/.flet/storage/temp`, and Python's `tempfile` is pointed here too (via `TMPDIR`). Equivalent to Python's `tempfile.gettempdir()`.

In a running Flet app, the equivalent of this environment variable is [`StoragePaths.get_temporary_directory()`] [flet.StoragePaths.get_temporary_directory].

### FLET_APP_USER_MODEL_ID

Windows AppUserModelID used by the desktop client process for taskbar grouping and pinning.

For apps packaged with `flet pack`, this value is set automatically so taskbar pins point to the packaged app executable instead of the cached Flet client executable.

### FLET_ASSETS_DIR

Absolute path to the app's assets directory.

In production apps built with `flet build`, this environment-variable points to the bundled assets absolute location at runtime. Use it when your code needs a filesystem path to bundled files (for example, JSON configs, databases, or model files).

For local runs, it may be unset depending on how the app is started, so use a fallback:

```
        import os
        from pathlib import Path
        import flet as ft

        default_assets_dir = Path(__file__).parent / "assets"
        assets_dir = Path(os.environ.get("FLET_ASSETS_DIR",
str(default_assets_dir))).resolve()

        def main(page: ft.Page):
            ...

        ft.run(main, assets_dir="assets")
```

For control properties like `Image.src`, continue using paths relative to the `ft.run(assets_dir=...)`, as described in the assets cookbook.

### FLET_ANDROID_SIGNING_KEY_ALIAS

Android signing key alias used by `flet build` for Android app signing.

It is used only when a keystore is configured.

### FLET_ANDROID_SIGNING_KEY_PASSWORD

Android signing key password used by `flet build` for Android app signing.

If `FLET_ANDROID_SIGNING_KEY_STORE_PASSWORD` is set but this variable is not, the keystore password is reused as the key password.

### FLET_ANDROID_SIGNING_KEY_STORE

Path to the Android upload keystore (`.jks`) used by `flet build` for Android app signing.

### FLET_ANDROID_SIGNING_KEY_STORE_PASSWORD

Android signing keystore password used by `flet build` for Android app signing.

If `FLET_ANDROID_SIGNING_KEY_PASSWORD` is set but this variable is not, the key password is reused as the keystore password.

### FLET_CLI_NO_RICH_OUTPUT

Whether to disable rich output in the console.

Defaults to `"false"`.

### FLET_PLATFORM

The platform on which the application is running. Its value is one of the following: `"android"`, `"ios"`, `"linux"`, `"macos"`, `"windows"` or `"fuchsia"`.

### FLET_CLI_SKIP_FLUTTER_DOCTOR

Whether to skip running `flutter doctor` when a build fails.

Defaults to `False`.

### FLET_HIDE_WINDOW_ON_START

Set to `true` to start app with the main window hidden.

Defaults to `False`.

### FLET_FORCE_WEB_SERVER

Set to `true` to force running app as a web app. Automatically set on headless Linux hosts.

### FLET_OAUTH_CALLBACK_HANDLER_ENDPOINT

Custom path for OAuth handler.

Defaults to `"/oauth_callback"`.

### FLET_OAUTH_STATE_TIMEOUT

Maximum allowed time (in seconds) to complete OAuth web flow.

Defaults to `600`.

### FLET_MAX_UPLOAD_SIZE

Maximum allowed size (in bytes) of uploaded files.

Default is unlimited.

### FLET_SECRET_KEY

A secret key to sign temporary upload URLs.

### FLET_SERVER_IP

IP address to listen web app on, e.g. `"127.0.0.1"`.

Defaults to `0.0.0.0` - bound to all server IPs.

### FLET_SERVER_PORT

TCP port to run app on.

`8000` if the program is running on a Linux server or `FLET_FORCE_WEB_SERVER` is set; otherwise random port.

### FLET_SERVER_UDS_PATH

The Unix Domain Socket (UDS) path for the Flet server. It enables inter-process communication on Unix-based systems, with its value being a socket file path in the format `flet_<pid>.sock`.

### FLET_SESSION_TIMEOUT

Session lifetime in seconds.

Defaults to `3600`.

### FLET_UPLOAD_DIR

Absolute path to app “upload” directory.

### FLET_UPLOAD_HANDLER_ENDPOINT

Custom path for upload handler.

Defaults to `"/upload"`.

### FLET_WEB_APP_PATH

A URL path after domain name to host web app under, e.g. `"/apps/myapp"`.

Defaults to `"/"` - host app in the root.

### FLET_WEB_NO_CDN

Set to `true` to avoid loading CanvasKit, Pyodide, and fonts from CDNs.

### FLET_WEBSOCKET_HANDLER_ENDPOINT

Custom path for WebSocket handler.

Defaults to `"/ws"`.

### FLET_WEB_RENDERER

Web rendering mode: `"canvaskit"` (default), `"skwasm"` or `"auto"`.

### `FLET_WEB_USE_COLOR_EMOJI`

Set to `True`, `true` or `1` to load web font with colorful emojis.

### `FLET_WEB_ROUTE_URL_STRATEGY`

The URL strategy of the web application. Its value can be either `"path"` (default) or `"hash"`.

# API Reference

- Controls - UI building blocks with properties, events, and usage examples.
- Services - Device and platform capabilities such as sensors, storage, and permissions.
- CLI - `flet` commands for creating, running, packaging, and debugging apps.
- Types - Core types, enums, events, exceptions, and utilities shared across the SDK.
- Built-in binary Python packages for Android and iOS - Prebuilt wheels available from `https://pypi.flet.dev` for mobile targets.
- Environment Variables - Runtime configuration toggles for apps and the CLI.

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/accelerometer/main.py’} language=“python” />

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’; import {ClassAll, CodeExample} from ‘@site/src/components/crocodocs’;

# Audio

Allows playing audio in Flet apps.

## Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

## Usage

To use `Audio` control add `flet-audio` package to your project dependencies:

```
uv add flet-audio

pip install flet-audio  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. :::note[Linux requirements] To play audio on Linux (or WSL) you need to install `GStreamer` library.

To install the minimal set of GStreamer libs on Ubuntu/Debian, run:

```
sudo apt install libgtk-3-dev libgstreamer1.0-dev
libgstreamer-plugins-base1.0-dev
```

To install the full set:

```
sudo apt install \
  libgstreamer1.0-dev libgstreamer-plugins-base1.0-dev
libgstreamer-plugins-bad1.0-dev \
  gstreamer1.0-plugins-base gstreamer1.0-plugins-good
gstreamer1.0-plugins-bad \
  gstreamer1.0-plugins-ugly gstreamer1.0-libav gstreamer1.0-
doc gstreamer1.0-tools \
  gstreamer1.0-x gstreamer1.0-alsa gstreamer1.0-gl
gstreamer1.0-gtk3 \
  gstreamer1.0-qt5 gstreamer1.0-pulseaudio
```

If you receive `error while loading shared libraries: libgstapp-1.0.so.0`, it means `GStreamer` is not installed in your WSL environment. Install the full set of GStreamer libs, as shown above.

See this guide for installing on other Linux distributions. :::

## Examples

<CodeExample path={frontMatter.examples + '/audio_playback_controls/main.py'} language="python" />

## Description

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

# Audio Recorder

Allows recording audio in Flet apps.

It is based on the record Flutter package.

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

# Usage

To use `AudioRecorder` service add `flet-audio-recorder` package to your project dependencies:

```
uv add flet-audio-recorder

pip install flet-audio-recorder  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Requirements

The below sections show the required configurations for each platform.

## Android

Configuration to be made to access the microphone:

- `android.permission.RECORD_AUDIO`: Allows audio recording.
- `android.permission.MODIFY_AUDIO_SETTINGS` (optional): Allows using bluetooth telephony device like headset/earbuds.

```
flet build apk \
  --android-permissions android.permission.RECORD_AUDIO=true \
  --android-permissions
android.permission.MODIFY_AUDIO_SETTINGS=true
```

```
[tool.flet.android.permission]
"android.permission.RECORD_AUDIO" = true
"android.permission.MODIFY_AUDIO_SETTINGS" = true
```

:::note No storage permission is needed to save recordings: writing to the app's own directories (see `StoragePaths`) is always allowed, and saving elsewhere should go through `FilePicker.save_file()`, which uses the system picker. Requesting `WRITE_EXTERNAL_STORAGE` or `READ_EXTERNAL_STORAGE` instead can get your app rejected from Google Play. ::: See also:

- setting Android permissions
- Audio formats sample rate hints

## iOS

Configuration to be made to access the microphone:

- `NSMicrophoneUsageDescription`: Required for recording audio.

```
flet build ipa \
  --info-plist NSMicrophoneUsageDescription="Some message to
describe why you need this permission..."
```

```
[tool.flet.ios.info]
NSMicrophoneUsageDescription = "Some message to describe why
you need this permission..."
```

See also:

- setting iOS permissions.

### macOS

Configuration to be made to access the microphone:

- `NSMicrophoneUsageDescription`: Allows recording audio.
- `com.apple.security.device.audio-input`: Allows recording audio using the built-in microphone and accessing audio input using Core Audio.

```
flet build macos \
  --info-plist NSMicrophoneUsageDescription="Some message to
describe why you need this permission..." \
  --macos-entitlements com.apple.security.device.audio-
input=true
```

```
[tool.flet.macos.info]
NSMicrophoneUsageDescription = "Some message to describe why
you need this permission..."

[tool.flet.macos.entitlement]
"com.apple.security.device.audio-input" = true
```

See also:

- setting macOS permissions
- setting macOS entitlements

### Linux

The following dependencies are required (and widely available on your system):

- `parecord`: Used for audio input.
- `pactl`: Used for utility methods like getting available devices.
- `ffmpeg`: Used for encoding and output.

On Ubuntu 24.04.3 LTS, you can install them using:

```
sudo apt install pulseaudio-utils ffmpeg
```

### Cross-platform

Additionally/alternatively, you can make use of our predefined cross-platform `microphone` permission bundle:

```
flet build <target_platform> --permissions microphone
```

```
[tool.flet]
permissions = ["microphone"]
```

## Examples

<CodeExample path={frontMatter.examples + '/audio_recorder/main.py'} language="python" />

### Stream chunks and save/download {#stream-chunks-and-save-download}

On web, [`AudioRecorder.stop_recording()`] [flet_audio_recorder.AudioRecorder.stop_recording] returns a browser-local Blob URL. Use streaming when your app needs access to the recorded bytes.

Set [`AudioRecorderConfiguration.encoder`] [flet_audio_recorder.AudioRecorderConfiguration.encoder] to [`AudioEncoder.PCM16BITS`][flet_audio_recorder.AudioEncoder.PCM16BITS] and handle [`AudioRecorder.on_stream`][flet_audio_recorder.AudioRecorder.on_stream] to receive [`AudioRecorderStreamEvent.chunk`] [flet_audio_recorder.AudioRecorderStreamEvent.chunk] bytes in Python.

[`AudioEncoder.PCM16BITS`][flet_audio_recorder.AudioEncoder.PCM16BITS] encoded streams are supported on all platforms. Stream chunks are raw PCM16 bytes and are not directly playable as an audio file. Wrap the bytes in a container such as WAV in Python when the destination needs a directly playable recording.

The example below collects the streamed chunks, wraps the PCM16 bytes in a WAV container, and passes the resulting bytes to [`FilePicker.save_file()`] [flet.FilePicker.save_file] so users can save or download the recording.

<CodeExample path={frontMatter.examples + '/stream_and_download/main.py'} language="python" />

### Streaming upload

Pass [`AudioRecorderUploadSettings`] [flet_audio_recorder.AudioRecorderUploadSettings] to [`AudioRecorder.start_recording()`] [flet_audio_recorder.AudioRecorder.start_recording] to upload [`AudioEncoder.PCM16BITS`][flet_audio_recorder.AudioEncoder.PCM16BITS] recording bytes directly while recording.

The uploaded file contains raw PCM16 bytes, so a `.pcm` extension is intentional. See the stream chunks and save/download example for inspiration, if you need to create a playable WAV file.

Built-in upload URLs from [`Page.get_upload_url()`][flet.Page.get_upload_url] require upload storage and a signing key. Run with `upload_dir` and set `FLET_SECRET_KEY`.

<CodeExample path={frontMatter.examples + '/upload/main.py'} language="python" />

# Description

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/barometer/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/battery/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/browser_context_menu/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/text/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/images/main.py’} language=“python” />

<CodeExample path={frontMatter.examples + ‘/files/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/connectivity/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample, Image} from ‘@site/src/components/crocodocs’;

## Usage

Create an instance of `FilePicker`:

```
import flet as ft

file_picker = ft.FilePicker()
```

To open the file picker dialog call one of these three methods: `pick_files()`, `save_file()` or `get_directory_path()`, depending on the use case.

In most cases you can use a lambda function for that:

```
        ft.Button(
            content="Pick files,
            on_click=lambda _:
file_picker.pick_files(allow_multiple=True)
        )
```

### Uploading files

To upload one or more files, call `FilePicker.pick_files()` to let the user select files, then pass the returned list to `FilePicker.upload()` to perform the upload.

:::note[Separate uploads per user] If you need to separate uploads for each user you can specify a filename prepended with any number of directories in `page.get_upload_url()` call, for example:

```
        upload_url =
page.get_upload_url(f"/{username}/pictures/{f.name}", 600)
```

`/{username}/pictures` directories will be automatically created inside `upload_dir` if non-existent. :::

### Upload storage

Notice the usage of `page.get_upload_url()` method – it generates a presigned upload URL for Flet's internal upload storage.

:::note[Use any storage for file uploads] You can generate presigned upload URL for AWS S3 storage using boto3 library.

The same technique should work for Wasabi, Backblaze, MinIO and any other storage providers with S3-compatible API. :::

To enable Flet saving uploaded files to a directory provide a full or relative path to that directory in `flet.run()` call:

```
        ft.run(main, upload_dir="uploads")
```

You can even put uploads inside assets directory, so uploaded files, e.g. pictures, docs, or other media, can be accessed from a Flet client right away:

```
        ft.run(main, assets_dir="assets", upload_dir="assets/uploads")
```

and in your app you can display the uploaded picture with:

```
        ft.Image(src="/uploads/<some-uploaded-picture.png>")
```

# Examples

<CodeExample path={frontMatter.examples + '/pick_save_and_get_directory_path/main.py'} language="python" />

<Image src={frontMatter.example_images + ‘/pick_save_and_get_directory_path.png’} width=“55%” />

### Pick and upload files

The following example demonstrates multi-file pick and upload app.

<CodeExample path={frontMatter.examples + ‘/pick_and_upload/main.py’} language=“python” />

<Image src={frontMatter.example_images + ‘/pick_and_upload.png’} width=“55%” />

### Pick text content and save/download it

Use `pick_files()` with `with_data=True` when you need file contents directly, such as in web apps where `FilePickerFile.path` is not available.

<CodeExample path={frontMatter.examples + ‘/pick_and_save_text_content/main.py’} language=“python” />

import TabItem from ‘@theme/TabItem’; import Tabs from ‘@theme/Tabs’; import {ClassAll, CodeExample} from ‘@site/src/components/crocodocs’;

Control the device torch/flashlight in your Flet app via the `flet-flashlight` extension, built on top of Flutter’s `flashlight` package.

## Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ❌ | ❌ | ❌ | ✅ | ✅ | ❌ |

## Usage

Add `flet-flashlight` to your project dependencies:

```
uv add flet-flashlight

pip install flet-flashlight  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Example

<CodeExample path={frontMatter.examples + ‘/flashlight/main.py’} language=“python” />

## Description

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

Access device location services in your Flet app using the `flet-geolocator` extension. The control wraps Flutter's `geolocator` package and exposes async helpers for permission checks and position streams.

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

# Usage

Add `flet-geolocator` to your project dependencies:

```
uv add flet-geolocator

pip install flet-geolocator  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Requirements

The below sections show the required configurations for each platform.

## Android

Configuration to be made to access the device's location:

- `ACCESS_FINE_LOCATION`: Allows access precise location. Will be preferred over `ACCESS_COARSE_LOCATION`, if both are set.
- `ACCESS_COARSE_LOCATION`: Allows access approximate location.
- `ACCESS_BACKGROUND_LOCATION` (optional): Allows access to location even when the app is in the background. Effective as from Android 10 (API level 29).
- `FOREGROUND_SERVICE_LOCATION` (optional): Allows access to location even when the app is in the foreground. Effective as from Android 14 (API level 34).

- At least one of `ACCESS_FINE_LOCATION` or `ACCESS_COARSE_LOCATION` permission is **required** to get location updates, with the former being preferred if both are set.
- Specifying the `ACCESS_COARSE_LOCATION` permission results in location updates with accuracy approximately equivalent to a city block. It might take a long time (minutes) before you will get your first locations fix as `ACCESS_COARSE_LOCATION` will only use the network services to calculate the position of the device. More information here.

```
flet build apk \
  --android-permissions
android.permission.ACCESS_FINE_LOCATION=true \
  --android-permissions
android.permission.ACCESS_COARSE_LOCATION=true \
```

```
          --android-permissions
android.permission.ACCESS_BACKGROUND_LOCATION=true \
          --android-permissions
android.permission.FOREGROUND_SERVICE_LOCATION=true
```

```
        [tool.flet.android.permission]
        "android.permission.ACCESS_FINE_LOCATION" = true
        "android.permission.ACCESS_COARSE_LOCATION" = true
        "android.permission.ACCESS_BACKGROUND_LOCATION" = true
        "android.permission.FOREGROUND_SERVICE_LOCATION" = true
```

See also:

- setting Android permissions

### iOS

Configuration to be made to access the device's location:

```
        flet build ipa \
          --info-plist NSLocationWhenInUseUsageDescription="Some
message to describe why you need this permission..."
```

```
        [tool.flet.ios.info]
        NSLocationWhenInUseUsageDescription = "Some message to
describe why you need this permission..."
```

See also:

- setting iOS Info.plist entries

### macOS

Configuration to be made to access the device's location:

```
        flet build macos \
          --info-plist NSLocationUsageDescription="Some message to
describe why you need this permission..." \
          --macos-entitlements com.apple.security.personal-
information.location=true
```

```
        [tool.flet.macos.info]
        NSLocationUsageDescription = "Some message to describe why you
need this permission..."

        [tool.flet.macos.entitlement]
        "com.apple.security.personal-information.location" = true
```

See also:

- macOS permissions
- macOS entitlements

### Cross-platform

Additionally/alternatively, you can make use of our predefined cross-platform `location` permission bundle:

```
flet build <target_platform> --permissions location
```

```
[tool.flet]
permissions = ["location"]
```

## Web: cached vs. fresh positions

On the web, `Geolocator.get_current_position` accepts a position from the browser's cache up to **5 minutes old by default**, otherwise it asks the browser for a fresh fix and times out after 30 seconds if none arrives. This default exists because desktop browsers' on-demand location pipeline is slow and frequently fails on stationary devices.

If you need stricter freshness (or a longer cache window), pass a `GeolocatorWebConfiguration` with an explicit `maximum_age` and/or `time_limit`:

```
geo = ftg.Geolocator(
    configuration=ftg.GeolocatorWebConfiguration(
        accuracy=ftg.GeolocatorPositionAccuracy.HIGH,
        maximum_age=ft.Duration(seconds=10),
        time_limit=ft.Duration(seconds=15),
    ),
)
```

# Troubleshooting

### macOS: web browser returns `POSITION_UNAVAILABLE` or never resolves

When running a Flet app in `--web` mode on macOS, the browser may fail to obtain a position even though Location Services is on and the browser is allowed in *System Settings → Privacy & Security → Location Services*. Symptoms include:

- `getCurrentPosition` hanging or returning a `Location request timed out.` error.
- The position stream emitting `Failed to obtain position: Position update is unavailable` (this is the W3C `POSITION_UNAVAILABLE` / error code 2 from the browser).

This is a known issue with macOS' `locationd` daemon getting into a stuck state where it reports the browser as authorized but never delivers a fix. Restart it:

```
sudo killall locationd
```

The daemon respawns automatically. You can verify the browser side independently by visiting browserleaks.com/geo — if that page also fails to obtain a position, the problem is with the OS, not your Flet app.

The native (non-web) macOS Flet app is unaffected because it uses CoreLocation directly through the platform plugin rather than going through the browser's Geolocation API.

## Example

<CodeExample path={frontMatter.examples + '/geolocator/main.py'} language="python" />

## Description

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/gyroscope/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/haptic_feedback/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/magnetometer/main.py'} language="python" />

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

Helps manage runtime permissions in your Flet apps.

It is powered by the Flutter `permission_handler` package.

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ❌ | ❌ | ✅ | ✅ | ✅ |

# Usage

Add `flet-permission-handler` to your project dependencies:

```
uv add flet-permission-handler

pip install flet-permission-handler  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. :::note[Note] On mobile platforms you must also declare permissions in the native project files. See Flet publish docs. :::

# Requirements

While the permissions are being requested during runtime, you'll still need to tell the OS which permissions your app might potentially use.

## Android

See:

- full list of Android permissions
- `Permission` enum, which lists all the supported permissions
- setting Android permissions

## iOS

See:

- `Permission` enum, which lists all the supported permissions and their `Info.plist` keys
- setting iOS permissions

## Example

<CodeExample path={frontMatter.examples + '/permission_handler/main.py'} language="python" />

## Description

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/screen_brightness/main.py'} language="python" />

import TabItem from '@theme/TabItem'; import Tabs from '@theme/Tabs'; import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

# Secure Storage

A service for safely storing sensitive key–value data using the platform's native secure storage mechanisms—Keychain on iOS/macOS, Windows Credential Manager, libsecret on Linux, and Keystore on Android.

Powered by Flutter's `flutter_secure_storage` package.

You need `libsecret-1-dev` on your machine to build the project, and `libsecret-1-0` to run the application (add it as a dependency after packaging your app). If you using snapcraft to build the project use the following.

Apart from `libsecret` you also need a keyring service, for that you need either `gnome-keyring` (for Gnome users) or `kwalletmanager` (for KDE users) or other light provider like `secret-service`.

```
sudo apt-get install libsecret-1-dev libsecret-1-0
```

# Platform Support

| Platform | Windows | macOS | Linux | iOS | Android | Web |
|---|---|---|---|---|---|---|
| Supported | ✅ | ✅ | ✅ | ✅ | ✅ | ✅ |

# Usage

Add `flet-secure-storage` to your project dependencies:

```
uv add flet-secure-storage

pip install flet-secure-storage  # (1)!
```

1. After this, you will have to manually add this package to your `requirements.txt` or `pyproject.toml`. ## Example

<CodeExample path={frontMatter.examples + '/secure_storage/main.py'} language="python" />

# Description

import {ClassAll} from '@site/src/components/crocodocs';

# Usage example

Require biometrics OR device passcode:

```
options = IOSOptions(
    access_control_flags=[
        AccessControlFlag.BIOMETRY_ANY,
        AccessControlFlag.OR,
        AccessControlFlag.DEVICE_PASSCODE
    ]
)
```

import {ClassAll} from '@site/src/components/crocodocs';

# Disabling Auto Backup

By default Android backups data on Google Drive. It can cause exception `java.security.InvalidKeyException: Failed to unwrap key`. You need to:

- Disable autobackup, details
- Exclude sharedprefs used by `SecureStorage`

Add the following to your `pyproject.toml`:

```
[tool.flet.android.manifest_application]
"allowBackup" = "false"
"fullBackupContent" = "false"
```

# Encryption Options

## Default

```
AndroidOptions()
```

- **Key Cipher:** RSA/ECB/OAEPWithSHA-256AndMGF1Padding
- **Storage Cipher:** AES/GCM/NoPadding
- **Biometric Support:** No
- **Description:** Standard secure storage with RSA OAEP key wrapping. Strong authenticated encryption without biometrics. Recommended for most use cases.

## Optional Biometrics

```
AndroidOptions(
    enforce_biometrics=False,

key_cipher_algorithm=KeyCipherAlgorithm.AES_GCM_NO_PADDING,
)
```

- **Key Cipher:** AES/GCM/NoPadding
- **Storage Cipher:** AES/GCM/NoPadding
- **Biometric Support:** Optional
- **Description:** KeyStore-based with optional biometric authentication. Gracefully degrades if biometrics unavailable.

## Required Biometrics

```
AndroidOptions(
    enforce_biometrics=True,

key_cipher_algorithm=KeyCipherAlgorithm.AES_GCM_NO_PADDING,
)
```

- **Key Cipher:** AES/GCM/NoPadding
- **Storage Cipher:** AES/GCM/NoPadding
- **Biometric Support:** Required (API 28+)
- **Description:** KeyStore-based requiring biometric/PIN authentication. Throws error if device security not available.

# Custom Cipher Combinations

For advanced users, all combinations below are supported using the `AndroidOptions()` constructor with custom parameters:

| Key Cipher Algorithm | Storage Cipher Algorithm | Imp |
|---|---|---|
| RSA_ECB_PKCS1_PADDING | AES_CBC_PKCS7_PADDING | RSA AES |
| RSA_ECB_PKCS1_PADDING | AES_GCM_NO_PADDING | RSA AES |
| RSA_ECB_OAEP_WITH_SHA256_AND_MGF1_PADDING | AES_CBC_PKCS7_PADDING | RSA AES |
| RSA_ECB_OAEP_WITH_SHA256_AND_MGF1_PADDING | AES_GCM_NO_PADDING | RSA AES |
| AES_GCM_NO_PADDING | AES_CBC_PKCS7_PADDING | Key |
| AES_GCM_NO_PADDING | AES_GCM_NO_PADDING | Key |

# Biometric Authentication

Secure Storage supports biometric authentication (fingerprint, face recognition, etc.) on Android API 23+.

## Required Permissions

To use biometric authentication on Android, you need to grant the necessary permissions (`USE_BIOMETRIC` and optionally `USE_FINGERPRINT`) in your project.

For configure permissions in your `pyproject.toml` or when building the app using `flet build`.

See the official Flet documentation for details: Android Permissions in Flet

Example configuration in `pyproject.toml`:

```
[tool.flet.android.permission]
"android.permission.USE_BIOMETRIC" = true
"android.permission.USE_FINGERPRINT" = true
```

You can also pass permissions when building your Android app:

```
flet build \
  --android-permissions android.permission.USE_BIOMETRIC=True
\
  android.permission.USE_FINGERPRINT=True
```

This ensures that biometric authentication works correctly on all supported Android devices.

### Using Biometric Authentication

You can enable biometric authentication:

```
        # Optional biometric authentication (graceful degradation)
        storage = SecureStorage(
          android_options=AndroidOptions(
            enforce_biometrics=False, # Default - works without
biometrics
            biometric_prompt_title='Unlock to access your data',
            biometric_prompt_subtitle='Use fingerprint or face
unlock',
          ),
        )

        # Strict biometric enforcement (requires device security)
        storage = SecureStorage(
          android_options=AndroidOptions(
            enforce_biometrics=True, # Requires biometric/PIN/pattern
            biometric_prompt_title: 'Biometric authentication
required',
          ),
        )
```

### Requirements

- **API Level**: Android 6.0 (API 23) minimum for basic encryption
- **API Level**: Android 9.0 (API 28) minimum for enforced biometric authentication
- **Device Security**: Device must have a PIN, pattern, password, or biometric enrolled (when using `enforce_biometrics = True`)
- **Permissions**: `USE_BIOMETRIC` permission in pyproject.toml

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

# Usage Example

### Usage with accessibility control

```
        from flet_secure_storage import SecureStorage
        from flet_secure_storage.types import IOSOptions,
KeychainAccessibility

        storage = SecureStorage(
            ios_options=IOSOptions(
                accessibility=KeychainAccessibility.FIRST_UNLOCK
            )
        )
```

```
        await storage.set(key="token", value="secret_value")
```

### Biometric authentication:

```
        from flet_secure_storage.types import IOSOptions,
AccessControlFlag

        options = IOSOptions(
            access_control_flags=[
                AccessControlFlag.BIOMETRY_ANY,
                AccessControlFlag.OR,
                AccessControlFlag.DEVICE_PASSCODE
            ]
        )

        await storage.set(key="secure_key", value="secure_value",
ios=options)
```

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

# Important Security Considerations

SecureStorage uses an experimental implementation using WebCrypto API. Use at your own risk. The browser creates the private key, and encrypted strings in localStorage are not portable to other browsers or machines and will only work on the same domain.

You MUST have HTTP Strict Forward Secrecy enabled and proper headers applied to your responses, or you could be subject to JavaScript hijacking.

Required security measures:

- Enable HSTS (HTTP Strict Transport Security)
- Use proper security headers

References:

- https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
- https://www.netsparker.com/blog/web-security/http-security-headers/

# Application-Specific Key Wrapping

On web, all keys are stored in LocalStorage. You can wrap this stored key with an application-specific key to make it more difficult to analyze:

```
storage = SecureStorage(
    web_options=WebOptions(
        wrap_key='your_application_specific_key',
        wrap_key_iv='your_application_specific_iv',
    ),
)
```

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/accessibility_features/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/shake_detector/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/shared_preferences/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/share/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

:::note Relationship to the `FLET_APP_STORAGE_*` environment variables `StoragePaths` exposes the raw platform directories. The pre-created `FLET_APP_STORAGE_*` env vars map onto them: `FLET_APP_STORAGE_CACHE` = `get_application_cache_directory()`, `FLET_APP_STORAGE_TEMP` = `get_temporary_directory()`, and `FLET_APP_STORAGE_DATA` = a flet-owned **`data` subdirectory** of `get_application_support_directory()` (which is also the app's current working directory). Prefer the env vars for app storage; use `StoragePaths` when you need a different platform directory (e.g. Documents or Downloads). :::

## Examples

<CodeExample path={frontMatter.examples + '/storage_paths/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/url_launcher/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/user_accelerometer/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/wakelock/main.py'} language="python" />

# Flet Studio

Flet Studio is a browser-based editor for writing, running, saving and sharing Flet apps. Your Python code runs entirely in the browser via Pyodide — there is no Flet account required to try it, and no local install required to run something someone else built and shared with you.

# Why it exists

Flet Studio started as a tool we wanted for ourselves and kept being useful for things we hadn't planned. In rough order:

- **A playground that saves your work.** Unlike a transient REPL or a paste-and-run sandbox, projects in Studio persist across sessions and across devices once you sign in.
- **A way to share small apps via a link.** Send a colleague or someone on the Flet team a URL and they can open, run, and read the code without setting anything up. Handy for bug reports, design sketches, and "look at this control".
- **A gallery of Flet examples that you can actually edit.** Browse examples, open one, tweak it, fork it into your own project — same editor, same runtime.
- **A zero-commitment way to try Flet.** Before `pip install flet`, before reading the getting-started docs, you can write a few lines and see them run.
- **A dogfooding project.** Studio's UI is itself a Flet app. Building it surfaces gaps in Flet and ideas for improvements, which feed back into the framework.
- **The base UI for future Flet services.** Cloud packaging and publishing are on the roadmap — Studio is the surface those will plug into. (Today: editor only.)

# How it works

- **Your code runs in the browser.** Flet Studio uses Pyodide — CPython compiled to WebAssembly — to execute your Python directly in the browser tab. Nothing you write is sent to a server to be run.
- **There is a Flet Studio backend, but it's only for the IDE itself** — file storage, authentication, accounts, project metadata. Your app's Python code never executes on our servers; it executes client-side, in your browser, in Pyodide.
- **The rendering engine is the same one `flet publish` and `flet build web` produce.** What you see in Studio is what you'd get from a published Flet static site — same controls, same layout, same web runtime.

# Limitations

Studio inherits the constraints of running Python in the browser. Most of these are covered in more detail in the static website publishing docs:

- **Pure-Python packages and Pyodide-built wheels only.** Most of the Python ecosystem works, but packages with C/Rust extensions need a Pyodide build. See the Pyodide built-in packages list. Source-only sdists (no wheel) cannot be installed.
- **Single browser thread.** No real threads. Long blocking work freezes the UI — prefer async I/O.
- **Browser sandbox.** No access to your local filesystem, no shell, no access to local hardware that isn't exposed by the browser.
- **Performance.** Pyodide is roughly 3×–5× slower than CPython for CPU-bound code.
- **Cold start.** The first load fetches the Python runtime and any wheels you import, so initial load is slower than a native install. Subsequent loads are cached.

## Does it replace local Flet development?

No.

Studio is for trying Flet, prototyping small apps, sharing snippets, learning, and showing off ideas. For everything else — desktop and mobile builds, native packages, real multi-file projects, your favorite IDE and debugger, the full Python ecosystem without the Pyodide constraints above — install Flet locally and use `flet run` and `flet build`. See installation when you're ready to make the jump.

# Flet Studio Privacy Policy

**Effective date:** July 14, 2026

This Privacy Policy explains how **Appveyor Systems Inc.** ("AppVeyor", "we", "us", or "our") collects, uses, shares, and protects personal information in connection with the Flet Studio service ("Flet Studio", "the Service"). Flet Studio is a hosted development environment and runtime for building applications with the Flet framework.

This policy applies to information we collect from users of Flet Studio at https://studio.flet.dev (and any related domains we operate for the Service) and through the Flet Studio web application.

---

## 1. Introduction

### 1.1 Who we are

Appveyor Systems Inc. is the operator of Flet Studio. Appveyor Systems Inc. also operates the AppVeyor CI/CD service, which is governed by a separate privacy policy at https://www.appveyor.com/privacy-policy/. This document applies only to Flet Studio.

### 1.2 Our role under GDPR

With respect to the personal information described in this policy — your account profile, sign-in metadata, usage data, and the content you create or upload to Flet Studio — **Appveyor Systems Inc. is the data controller**. We determine the purposes and means by which this information is processed.

Separately, when you build a Flet application using Flet Studio and that application collects or processes personal information about *your* own users, you are the controller for that personal information and Appveyor Systems Inc. may act as a processor on your behalf. That relationship is governed by a Data Processing Agreement (DPA), not by this Privacy Policy. Contact us at privacy@flet.dev to discuss a DPA if you require one.

### 1.3 Scope

This policy covers personal information processed by Flet Studio as currently implemented, including paid plans, AI credit purchases, AI-assisted development, and email communications. If we add features that materially change how personal information is processed, we will update this policy as described in Section 12.

---

## 2. Information We Collect

### 2.1 Information you provide

**Account profile.** When you create an account, we store the information needed to identify you and authenticate your sessions. This is provided through the identity provider you sign in with (GitHub, Google, or Microsoft) and includes:

- Your email address
- Your full name (if available from the provider)
- A unique user identifier issued by the provider
- Your username or handle (where the provider exposes one)
- Your profile avatar URL

**Account and team information.** For each account, we store the account name, a URL-safe slug, the account type (personal or team), the plan tier, and — for team accounts — the membership and role of each member.

**Content you create or upload.** When you use Flet Studio, we store the app source code, files, and versions that you create, edit, or upload to the Service, along with their metadata (file path, size, timestamps, app name, description, dependencies, tags). This content is stored in object storage hosted by our infrastructure provider, Railway.

**AI conversations.** When you use the AI agent, we store your prompts, the AI's responses (including generated code and a record of the actions the AI took in your project), and any ratings you give on responses. Conversations are stored in our database, per app and per account, until you delete the conversation or the associated app, until your account is deleted, or until we remove them under a data-retention policy we may adopt.

**Billing information.** When you purchase a paid plan or AI credits, Stripe collects your payment card details and billing address at checkout. We store your Stripe customer and subscription identifiers, your plan status, and records of your purchases and credit grants, and we retain billing addresses and purchase records for bookkeeping in QuickBooks Online.

**Newsletter subscription.** When you create an account, your email address and name are added to the Flet Studio newsletter mailing list, operated by Mailgun. Your subscription and unsubscribe status is maintained by Mailgun, not in our database. You can unsubscribe at any time using the toggle in account settings, the unsubscribe link in every newsletter email, or your mail client's unsubscribe function.

### 2.2 Information collected automatically

**Sign-in and security data.** When you sign in, we record the timestamp of your most recent successful sign-in and the IP address from which it occurred. We also maintain an internal token version counter that is used to invalidate active sessions when you sign out or change credentials. We use this information to maintain account security, detect suspicious activity, and provide an audit trail.

**Usage data.** To enforce per-account quotas and prevent abuse, we maintain daily counters of certain actions (for example, how many gallery clones or archive uploads you have performed in the last 24 hours) and a running total of the storage your account is consuming. These counters are tied to your account, not to individual sign-in sessions.

**AI usage metering.** For each AI agent run, we record the model used, its token counts, and the resulting credit cost in a usage ledger tied to your account. We use these records to bill AI usage against your credits and to enforce quotas.

**Server-side telemetry.** When telemetry is enabled, our server software emits operational traces and logs to a server-side observability tool (Pydantic Logfire) to help us diagnose errors, investigate incidents, and monitor performance. These traces may incidentally include account or request identifiers; we apply a PII-scrubbing rule to redact sensitive fields. Telemetry is configurable and is not enabled in all environments.

We do **not** use client-side analytics services. We do not load Google Analytics, Mixpanel, Segment, advertising pixels, or any similar product into the Flet Studio web application.

### 2.3 Information from third parties

When you authenticate via GitHub, Google, or Microsoft, we receive the profile fields listed in Section 2.1 from the provider you choose. We do not receive your password for that provider, and we do not request access to your private repositories or files unless you explicitly grant a scope that permits it.

When you make a purchase, we receive payment and subscription status events from Stripe (for example, that a payment succeeded, a subscription renewed, or a subscription was cancelled). These events reference Stripe identifiers.

---

## 3. How We Use Information

We use personal information to:

- Provide the Service: authenticate you, render your accounts and projects, store and serve your content
- Operate AI-assisted development features: your prompts, conversation history, and the project context needed to generate a response are sent to our AI model providers (see Section 5.1)
- Improve the Service: we may review AI conversations to diagnose problems with the AI features and improve their quality (for example, refining prompts, tooling, and agent behavior)
- Process payments, maintain billing and credit-usage records, and calculate applicable taxes

- Send the Flet Studio newsletter with product news and updates — you can opt out at any time (see Section 2.1)
- Enforce per-account limits and prevent abuse of the Service
- Maintain the security and integrity of the Service, including investigating fraud, abuse, or unauthorized access
- Diagnose, debug, and improve the Service through server-side telemetry
- Communicate with you about your account or service-related matters when you contact us
- Comply with legal obligations

We do **not** use your personal information or User Content for advertising, for profiling, or for automated decision-making that produces legal or similarly significant effects (for example, suspending an account by algorithm with no human review), and we do **not** use them to train machine-learning models — nor do we permit our AI model providers to do so (see Section 5.1).

---

# 4. Legal Bases for Processing (GDPR)

If you are located in the European Economic Area, the United Kingdom, or Switzerland, we rely on the following legal bases under the GDPR (or its UK equivalent) when we process your personal information:

- **Performance of a contract** — to provide the Service you have signed up for. This covers account creation, authentication, storage of your content, quota enforcement, paid subscriptions and credit purchases, and providing the AI features you invoke.
- **Legitimate interests** — to keep the Service secure, prevent abuse, debug and improve the Service, maintain operational telemetry, and send the Flet Studio newsletter to existing account holders with product news and updates (with an easy opt-out at any time and the right to object described in Section 10).
- **Compliance with a legal obligation** — when we are required to retain, disclose, or process personal information to comply with applicable law (for example, retaining billing records for tax purposes).
- **Consent** — where we ask for it explicitly. If we introduce features that require consent, we will request it separately and you will be able to withdraw it at any time.

---

# 5. How We Share Information

We do not sell your personal information. We do not share your personal information with third parties for advertising or for cross-context behavioral advertising of any kind.

We share personal information only with the following categories of recipients:

## 5.1 Service providers (subprocessors)

We rely on a small number of third-party service providers to operate Flet Studio. Each acts as a processor on our behalf and is bound by appropriate contractual obligations:

| Service provider | Purpose | Data shared |
|---|---|---|
| **GitHub, Google, Microsoft** | OAuth identity providers used to authenticate accounts | Authentication exchange; we receive the profile fields listed in Section 2.1 from the provider you choose |
| **Railway** | Hosting for our application servers, our managed PostgreSQL database, and the object storage that holds user-uploaded content | All personal information stored by the Service is hosted on Railway infrastructure |
| **xAI** | AI model provider for the AI agent | Your prompts, conversation history, app source code and files (including images), and your name, email address, and username. Sent only when you use AI features; **not used to train models** |
| **OpenAI** | AI model provider for the AI agent | Same as xAI; **not used to train models** |
| **Stripe** | Payment processing, subscription management, and tax calculation | Email address, account identifier, plan, and — collected by Stripe directly at checkout — your billing address and payment card details; your purchase history |
| **Mailgun** | Newsletter and email delivery | Email address, full name, and newsletter subscription/suppression status |
| **Pydantic Logfire** | Server-side observability and tracing | Request traces, query traces, and error spans; PII-scrubbed before transmission |
| **Intuit QuickBooks Online** | Accounting and bookkeeping | Billing address, purchase and invoice records |

Our agreements with our AI model providers do not permit them to use your data to train their models.

## 5.2 Legal and safety disclosures

We may disclose personal information when we believe in good faith that disclosure is required by law or by a valid legal process (such as a subpoena or court order).

### 5.3 Business transfers

If Appveyor Systems Inc. is involved in a merger, acquisition, reorganization, or sale of assets, personal information may be transferred as part of that transaction. We will notify you of any change in ownership or material change in the use of your personal information.

---

## 6. Cookies and Similar Technologies

Flet Studio uses only what is strictly necessary to keep you signed in. Specifically:

- A single authentication cookie named `flet_studio_auth`, transmitted only over HTTPS in production environments
- A JSON Web Token stored in your browser's `localStorage` (via the Flet framework's shared-preferences mechanism), used to maintain your session

We do **not** set any advertising cookies, analytics cookies, social-media cookies, or other tracking technologies. Because we use only strictly-necessary technologies, we do not display a cookie consent banner.

---

## 7. Data Retention

- **Account data** (profile fields, account/team records) is retained for as long as your account is active.
- **User content** (apps, files, versions) is retained for as long as your account is active or until you delete it.
- **Sign-in metadata** (last sign-in timestamp, last sign-in IP) is overwritten on each new sign-in; older values are not kept.
- **Usage counters** are kept per day and are not retained beyond what is needed for quota enforcement and operational analysis.
- **AI conversations** are retained until you delete the conversation or the associated app, or until your account is deleted.
- **Billing records** (purchase and credit-usage ledgers, invoices) are retained for as long as required by applicable tax and accounting law, generally seven years in Canada, even after account deletion.
- **Newsletter list membership** is maintained by Mailgun until you unsubscribe; when we process an account-deletion request, we also remove your address from the mailing list.
- **Server-side telemetry** (Logfire traces and logs) is retained for a limited period configured with our observability provider.
- We may adopt shorter retention periods for specific data categories in the future; if we do, we will update this policy.
- **Account deletion**: when you request deletion of your account, we delete or anonymize your account profile, content, and associated records within 30 days, except where we are required to retain certain information to comply with legal obligations or to resolve disputes.

To request deletion, email privacy@flet.dev from the email address associated with your account.

## 8. Data Security

We protect personal information using a combination of technical and organizational measures:

- All traffic between your browser and Flet Studio is encrypted in transit using TLS.
- Personal information is stored on infrastructure operated by Railway, which provides encryption at rest for managed databases and object storage.
- Access to production systems is limited to personnel who require it for their role, authenticated using strong credentials.
- Authentication tokens carry a version claim that allows us to revoke all of your active sessions immediately when needed.

No method of transmission or storage is perfectly secure, but we work to maintain protections appropriate to the sensitivity of the information we handle.

---

## 9. International Data Transfers

Appveyor Systems Inc. and our infrastructure providers may store and process personal information in jurisdictions other than the one in which you reside. Where personal information of users located in the European Economic Area, the United Kingdom, or Switzerland is transferred to a country that has not been recognized as providing an adequate level of data protection, we rely on appropriate safeguards, including the European Commission's Standard Contractual Clauses or equivalent mechanisms.

---

## 10. Your Rights (EEA, UK, and Swiss Residents)

If you are located in the European Economic Area, the United Kingdom, or Switzerland, you have the following rights with respect to your personal information under applicable data-protection law:

- **Right of access** — to obtain confirmation as to whether we process personal information about you, and to receive a copy of that information
- **Right to rectification** — to have inaccurate personal information corrected and incomplete information completed
- **Right to erasure ("right to be forgotten")** — to have your personal information deleted in the circumstances described in the GDPR
- **Right to restriction of processing** — to have processing limited in certain circumstances
- **Right to data portability** — to receive your personal information in a structured, commonly-used, machine-readable format and to transmit it to another controller
- **Right to object** — to object to processing carried out on the basis of our legitimate interests
- **Right to lodge a complaint** — with the data-protection supervisory authority in the country where you live or work, or where the alleged infringement took place

To exercise any of these rights, email **privacy@flet.dev** from the email address associated with your account. We will respond to your request within the time limits prescribed by applicable law (generally one month). We will not discriminate against you for exercising any of your rights.

---

## 11. Children's Privacy

Flet Studio is not directed to children under the age of 16, and we do not knowingly collect personal information from children under 16. If we become aware that we have collected personal information from a child under 16 without verified parental consent, we will take steps to delete that information. If you believe a child has provided us with personal information, please contact us at privacy@flet.dev.

---

## 12. Changes to This Policy

We may update this Privacy Policy from time to time to reflect changes to the Service or to our legal obligations. When we make material changes, we will update the "Effective date" at the top of this policy and, where appropriate, notify you through the Service or by email before the changes take effect. Your continued use of Flet Studio after a change takes effect constitutes acceptance of the revised policy.

---

## 13. Contact Us

If you have questions about this Privacy Policy or about how Flet Studio handles your personal information, please contact us:

**Appveyor Systems Inc.** Email: privacy@flet.dev

# Flet Studio Terms of Service

**Effective date:** July 14, 2026

These Terms of Service ("Terms") form a binding agreement between you and **Appveyor Systems Inc.** ("AppVeyor", "we", "us", or "our") and govern your access to and use of the Flet Studio service ("Flet Studio" or the "Service"). Flet Studio is a hosted development environment and runtime for building applications with the Flet framework.

Please read these Terms carefully. By accessing or using Flet Studio, you agree to be bound by these Terms and by our Privacy Policy. If you do not agree, you must not use the Service.

---

# 1. Acceptance of Terms

By creating an account, signing in, or otherwise using Flet Studio, you confirm that you have read, understood, and agree to be bound by these Terms. If you are accepting these Terms on behalf of a company, organization, or other legal entity, you represent and warrant that you have the authority to bind that entity to these Terms, in which case “you” refers to that entity.

We may update these Terms from time to time as described in Section 14.4. Your continued use of the Service after an update takes effect constitutes acceptance of the updated Terms.

---

# 2. The Service

Flet Studio provides a hosted environment that allows you to create, edit, store, run, and share applications written in Python using the Flet framework. The Service includes account management, project storage, code editing, AI-assisted development, application execution, sharing and publishing, and related features. Flet Studio is currently delivered through the browser; we may also offer desktop or mobile applications in the future.

We may add, modify, or discontinue features at any time. Where a change materially and adversely affects the way you use the Service, we will use commercially reasonable efforts to provide advance notice through the Service or by email.

---

# 3. Accounts and Eligibility

## 3.1 Eligibility

You must be at least 16 years old to use Flet Studio. If you are using the Service on behalf of an entity, you must have authority to act on its behalf. The Service is not directed to children under 16, and we do not knowingly permit them to create accounts.

## 3.2 Account creation

You can create an account by signing in with a supported identity provider (currently GitHub, Google, or Microsoft) or, where available, by creating a local account with an email and password. You must provide accurate and complete information, and you must keep it accurate over time.

## 3.3 Account security

You are responsible for safeguarding your credentials and for all activity that occurs under your account. You must notify us promptly at legal@flet.dev if you suspect unauthorized access to your account.

### 3.4 One account per person

You may not create multiple personal accounts to circumvent quotas, restrictions, or suspensions, except where we expressly permit it (for example, separate personal and team accounts).

### 3.5 Privacy

Information you provide when creating and using an account is processed in accordance with our Privacy Policy.

### 3.6 Communications

We send service and transactional emails about your account and the Service (for example, security notices and billing communications); you cannot opt out of these while you hold an account. Account holders are also subscribed to the Flet Studio newsletter with product news and updates. You can unsubscribe from the newsletter at any time using the toggle in your account settings or the unsubscribe link included in every newsletter email.

---

## 4. Acceptable Use

You agree not to use Flet Studio to do any of the following, or to permit or encourage others to do so:

- Upload, create, transmit, or distribute content that is unlawful, infringing, defamatory, harassing, threatening, hateful, obscene, sexually explicit, or that promotes violence
- Send spam, conduct phishing, distribute malware, or engage in fraud
- Violate the intellectual-property, privacy, publicity, or other rights of any person or entity
- Impersonate any person or entity or misrepresent your affiliation with one
- Probe, scan, or test the vulnerability of the Service or circumvent any authentication, access control, or rate-limiting mechanism
- Reverse engineer, decompile, or disassemble any part of the Service, except to the extent applicable law expressly permits
- Use automated means (bots, crawlers, scrapers) to access the Service in ways that are not expressly authorized, or to extract data at a scale or rate that interferes with normal operation
- Exceed your assigned quotas, evade rate limits, or otherwise consume disproportionate resources
- Run cryptocurrency miners, denial-of-service tooling, botnet clients, or any workload whose purpose is to abuse compute, storage, or network resources
- Use the Service in a manner that violates any applicable law, regulation, or third-party agreement, or that violates export-control or sanctions laws

We may, but are not obligated to, investigate suspected violations and may take any action we consider appropriate, including limiting features, removing content, or suspending or terminating your account.

---

# 5. User Content and License Grant

## 5.1 Ownership

“User Content” means the source code, files, project metadata, and any other materials you create, upload, or store using Flet Studio. **You retain all ownership rights in your User Content.** Nothing in these Terms transfers ownership of your User Content to us.

## 5.2 License to AppVeyor

You grant AppVeyor a worldwide, non-exclusive, royalty-free, fully paid license to host, store, transmit, display, copy, and execute your User Content **solely as needed to operate and provide the Service to you**, including making backups, performing maintenance, and enabling features you have requested. The license is limited to these operational purposes; it is not a license to publish, monetize, or repurpose your User Content.

This license terminates when you delete the relevant User Content or close your account, except that we may retain copies in routine backups for a limited period and as required by law, as described in our Privacy Policy.

## 5.3 No publication or sharing without your instruction

**AppVeyor will not publish, share, or redistribute your User Content, or make it publicly available, except as you explicitly direct us to do (for example, by using a feature that publishes content at your request).** This commitment applies regardless of any other clause in these Terms.

## 5.4 Your responsibility

You are solely responsible for your User Content, including its legality and your right to use it. You represent and warrant that you have all rights necessary to grant the license in Section 5.2, and that your User Content does not violate the Acceptable Use restrictions in Section 4.

## 5.5 Copyright complaints

If you believe content on Flet Studio infringes your copyright, send a notice to **legal@flet.dev** that includes:

1. A physical or electronic signature of the rights holder or authorized representative
2. Identification of the work claimed to have been infringed
3. Identification of the allegedly infringing material with enough detail to locate it
4. Your contact information (address, telephone number, email)
5. A statement that you have a good-faith belief that the use is not authorized
6. A statement, under penalty of perjury, that the information in the notice is accurate and that you are authorized to act for the rights holder

We will respond to valid notices in a manner consistent with applicable copyright law and may remove allegedly infringing material and terminate the accounts of repeat infringers.

---

# 6. AI Features

Flet Studio includes an AI agent that can read, generate, and modify your app code based on your instructions.

**Third-party model providers.** To generate responses, your prompts, conversation history, app file contents (including images), and basic profile information (your name, email address, and username) are sent to third-party AI model providers, as described in our Privacy Policy.

**AI output.** AI output is generated automatically. It **may be inaccurate or incomplete, and generated code may not work or may contain security issues**. You are responsible for reviewing and testing AI output before relying on it. AI output stored in your apps is treated as your User Content under Section 5.

**AI credits.** AI usage consumes AI credits, metered by model token usage at the rates displayed within the Service. Your plan includes a monthly credit allowance, and additional credits can be purchased (see Section 7).

Your use of AI features is subject to Section 4 (Acceptable Use); you may not use them to generate unlawful or malicious content or attempt to circumvent usage metering.

---

# 7. Plans, Billing, and Payments

## 7.1 Plans

Flet Studio is offered under a free plan and one or more paid plans. The current plans, their features and limits, and their prices — together with the price of AI credit packs — are listed on our pricing page. We may change plan limits and pricing with reasonable advance notice; where we materially reduce a limit, we will provide notice through the Service or by email. A price change for a subscription takes effect at the start of a subsequent billing period.

## 7.2 AI credit packs

AI credit packs are one-time purchases. Purchased credits do not expire while your account remains active. Monthly plan credits reset each billing cycle and do not roll over. AI credits have no cash value and are not transferable.

## 7.3 Payment processing

Payments are processed by our third-party payment processor, **Stripe**. By making a purchase you agree to Stripe’s terms applicable to the payment. Your card details are collected by Stripe directly and are not received or stored by AppVeyor. Paid plans are billed in advance and **renew automatically each month until cancelled**. You can manage or cancel your subscription at any time through the billing portal in your account settings; cancellation takes effect at the end of the current billing period, and you retain paid-plan features until then. When a subscription ends, your account reverts to the free plan and its limits.

### 7.4 Taxes

Prices shown do not include taxes. Applicable sales tax will be calculated based on your billing address and itemized at checkout.

### 7.5 Refunds

All fees and credit purchases are **non-refundable except where required by applicable law**. We do not provide credits or prorated refunds for partial billing periods, and unused AI credits are not redeemable for cash.

### 7.6 Non-payment

If a payment for your subscription fails and cannot be collected after our payment processor's retries, we may downgrade your account to the free plan or suspend paid features.

---

## 8. Service Availability and Modifications

We strive to keep Flet Studio available and performant, but the Service is provided **without any uptime or availability guarantee**. We may perform scheduled or emergency maintenance, suspend access to all or part of the Service, or modify, deprecate, or discontinue features at any time.

Where a change materially and adversely affects how you use the Service, we will use commercially reasonable efforts to provide advance notice through the Service or by email.

---

## 9. Suspension and Termination

### 9.1 Termination by you

You may stop using Flet Studio at any time. You may delete your account by emailing legal@flet.dev from the email address associated with your account.

### 9.2 Suspension or termination by AppVeyor

We may suspend or terminate your access to all or part of the Service:

- Immediately, if we reasonably believe you have breached these Terms, your account presents a security risk, you are abusing the Service, or we are required to do so by law
- With reasonable notice, for any other reason

### 9.3 Effect of termination

Upon termination, your right to use the Service ends immediately. We will handle your account data and User Content in accordance with our Privacy Policy, which provides that account data and User Content are deleted or anonymized within 30 days of a deletion request, subject to backups and legal retention requirements.

### 9.4 Survival

The following sections survive termination of these Terms: Section 5.1 (Ownership), Section 5.4 (Your responsibility), Section 7.5 (Refunds), Section 10 (Disclaimers), Section 11 (Limitation of Liability), Section 12 (Indemnification), Section 13 (Governing Law and Dispute Resolution), and Section 14 (General Provisions), together with any other provision that by its nature should survive.

---

## 10. Disclaimers

THE SERVICE IS PROVIDED **“AS IS” AND “AS AVAILABLE”** WITHOUT WARRANTIES OF ANY KIND, WHETHER EXPRESS, IMPLIED, OR STATUTORY. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, APPVEYOR DISCLAIMS ALL WARRANTIES, INCLUDING WITHOUT LIMITATION ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, ACCURACY, QUIET ENJOYMENT, AND ANY WARRANTIES ARISING FROM COURSE OF DEALING OR USAGE OF TRADE.

WE DO NOT WARRANT THAT THE SERVICE WILL BE UNINTERRUPTED, ERROR-FREE, SECURE, OR FREE OF HARMFUL COMPONENTS, OR THAT ANY CONTENT (INCLUDING YOUR USER CONTENT) WILL BE PRESERVED WITHOUT LOSS. YOU ARE SOLELY RESPONSIBLE FOR THE CODE AND APPLICATIONS YOU CREATE WITH FLET STUDIO AND FOR THE CONSEQUENCES OF RUNNING THEM.

Some jurisdictions do not allow the exclusion of certain warranties, so some of the above exclusions may not apply to you.

---

## 11. Limitation of Liability

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW:

- **No indirect damages.** APPVEYOR WILL NOT BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXEMPLARY, OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF PROFITS, REVENUE, GOODWILL, OR DATA, ARISING OUT OF OR IN CONNECTION WITH THESE TERMS OR THE SERVICE, EVEN IF APPVEYOR HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
- **Aggregate cap.** APPVEYOR’S TOTAL AGGREGATE LIABILITY ARISING OUT OF OR IN CONNECTION WITH THESE TERMS OR THE SERVICE WILL NOT EXCEED THE GREATER OF (a) THE AMOUNTS YOU PAID TO APPVEYOR FOR THE SERVICE IN THE TWELVE (12) MONTHS PRECEDING THE EVENT

GIVING RISE TO THE CLAIM, OR (b) ONE HUNDRED CANADIAN DOLLARS (CA$100).
- **Data-loss exclusion.** APPVEYOR IS NOT LIABLE FOR ANY LOSS, ALTERATION, OR DESTRUCTION OF YOUR USER CONTENT. YOU ARE RESPONSIBLE FOR MAINTAINING YOUR OWN INDEPENDENT BACKUPS.

These limitations apply regardless of the legal theory on which a claim is based (contract, tort, statute, or otherwise) and apply even if a remedy fails of its essential purpose. Some jurisdictions do not allow the exclusion or limitation of certain damages; in those jurisdictions, the above limitations apply to the maximum extent permitted by law.

---

# 12. Indemnification

You agree to defend, indemnify, and hold harmless Appveyor Systems Inc., its affiliates, and their respective officers, directors, employees, and agents from and against any and all claims, damages, losses, liabilities, costs, and expenses (including reasonable legal fees) arising out of or related to:

- Your User Content
- Your use of or access to the Service
- Your breach of these Terms
- Your violation of any applicable law or any rights of a third party

We may, at our option, assume the exclusive defense and control of any matter subject to indemnification by you, in which case you agree to cooperate with our defense.

---

# 13. Governing Law and Dispute Resolution

These Terms, and any dispute or claim arising out of or in connection with them or their subject matter (including non-contractual disputes or claims), are governed by **the laws of the Province of British Columbia and the federal laws of Canada applicable therein**, without regard to conflict-of-laws principles.

You and AppVeyor each consent to the **exclusive jurisdiction of the courts located in the Province of British Columbia, Canada** for the resolution of any dispute arising out of or relating to these Terms or the Service, and you waive any objection based on inconvenient forum.

These Terms do not require mandatory arbitration, and nothing in these Terms is intended to waive your right to participate in a class action where applicable law would otherwise permit one.

---

# 14. General Provisions

## 14.1 Entire agreement

These Terms, together with the Privacy Policy and any additional terms you accept at the time of purchasing a paid feature, constitute the entire agreement between you and AppVeyor concerning the Service and supersede all prior or contemporaneous agreements on the same subject.

## 14.2 Severability

If any provision of these Terms is held to be invalid or unenforceable, that provision will be enforced to the maximum extent permissible, and the remaining provisions will remain in full force and effect.

## 14.3 No waiver

Our failure to enforce any right or provision of these Terms will not be deemed a waiver of that right or provision.

## 14.4 Changes to these Terms

We may modify these Terms from time to time. When we make material changes, we will update the “Effective date” above and, where appropriate, notify you through the Service or by email before the changes take effect. Your continued use of the Service after the new Terms take effect constitutes acceptance.

## 14.5 Assignment

You may not assign or transfer these Terms or your account without our prior written consent. We may assign these Terms to an affiliate or in connection with a merger, acquisition, reorganization, or sale of assets, on notice to you.

## 14.6 Relationship of the parties

Nothing in these Terms creates a partnership, joint venture, agency, employment, or fiduciary relationship between you and AppVeyor.

## 14.7 Force majeure

Neither party will be liable for failure or delay in performance caused by events beyond its reasonable control, including acts of God, war, terrorism, civil unrest, labor disputes, internet or telecommunications outages, governmental actions, or epidemics.

### 14.8 Notices

We may give notices to you through the Service, by posting on our website, or by email to the address associated with your account. You must give notices to us in writing at **legal@flet.dev**.

### 14.9 Contact

**Appveyor Systems Inc.** Email: legal@flet.dev

# What’s new in Flet Studio

New features, improvements, and bug fixes in Flet Studio, listed in reverse-chronological order. Flet Studio ships independently of the Flet SDK — see Flet release notes for SDK changes.

## July 14, 2026

- New AI agent that helps you build and modify Flet apps with natural-language instructions.
- Paid Creator plan and AI credit packs, with billing powered by Stripe.
- Flet Studio newsletter — manage your subscription in account settings.
- Updated Terms of Service and Privacy Policy to cover AI features, paid plans, and the newsletter.

## May 29, 2026

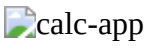
- New sign in options: Google and Microsoft.

## May 26, 2026

- The initial public release of Flet Studio.

import {CodeExample, Image} from ‘@site/src/components/crocodocs’;

In this tutorial you will learn, step-by-step, how to create a Calculator app in Python using Flet framework and publish it as a desktop, mobile or web app. The app is a simple console program, yet it is a multi-platform application with similar to iPhone calculator app UI:




You can find a live demo here.

In this tutorial, we will cover all of the basic concepts for creating a Flet app: building a page layout, adding controls, making reusable UI components, handling events, and publishing options.

# Getting started with Flet

To create a multi-platform app in Python with Flet, you don’t need to know HTML, CSS or JavaScript, but you do need a basic knowledge of Python and object-oriented programming.

Before you can create your first Flet app, you need to setup your development environment, which requires Python 3.10 or above and `flet` package.

Once you have Flet installed, let’s create a simple hello-world app.

Create `hello.py` with the following contents:

```python title=“hello.py” import flet as ft

def main(page: ft.Page): page.add(ft.Text(value=“Hello, world!”))

ft.run(main)


Run this app and you will see a new window with a greeting:

<Image src="examples/tutorials/todo/media/hello-world.png" alt="hello-world" width="55%" />

## Adding page controls

Now you are ready to create a calculator app.

To start, you'll need a [`Text`](../controls/text.md) control for showing the result of calculation,
and a few [`Button`](../controls/button.md)s with all the numbers and actions on them.

Create `calc.py` with the following contents:

<CodeExample path={frontMatter.examples + '/calc1.py'} language="python" />

Run the app and you should see a page like this:

<Image src={frontMatter.example_images + '/calc1.png'} alt="calc1" width="50%" />

## Building page layout

Now let's arrange the text and buttons in 6 horizontal [`Row`](../controls/row.md)s.
```

````
Replace `calc.py` contents with the following:

<CodeExample path={frontMatter.examples + '/calc2.py'}
language="python" />

Run the app and you should see a page like this:

<Image src={frontMatter.example_images + '/calc2.png'} alt="calc2"
width="50%" />

### Using Container for decoration

To add a black background with rounded border around the calculator,
we will be using [`Container`](../controls/container.md) control.
Container may decorate only one control, so we will need to wrap all
the 6 rows into a single vertical
[`Column`](../controls/column.md) that will be used as the container's
[`content`](../controls/container.md#flet.Container.content):

<Image src="examples/tutorials/calculator/media/container-layout.svg"
alt="container-layout" width="55%" />

Here is the code for adding the container to the page:

```python
    page.add(
        ft.Container(
            width=350,
            bgcolor=ft.Colors.BLACK,
            border_radius=ft.BorderRadius.all(20),
            padding=20,
            content=ft.Column(
                controls= [], # (1)!
            )
        )
    )
````

1. This controls list (of the `Column`) will contain the six rows of buttons.

## Styled Controls

To complete the UI portion of the program, we need to update style for result text and buttons to look similar to iPhone calculator app.

For the result text, let’s specify its `color` and `size` properties:

```
        result = ft.Text(value="0", color=ft.Colors.WHITE, size=20)
```

For the buttons, if we look again at the UI we are aiming to achieve, there are 3 types of buttons:
```

1. **Digit Buttons**. They have dark grey background color and white text, size is the same for all except `0` button which is twice as large.

2. **Action Buttons**. They have orange background color and white text, size is the same for all.

3. **Extra action buttons**. They have light grey background color and dark text, size is the same for all.

The buttons will be used multiple time in the program, so we will be creating custom Styled Controls to reuse the code.

Since all those types should inherit from `Button` class and have common `text` and `expand` properties, let's create a parent `CalcButton` class:

```
@ft.control
class CalcButton(ft.Button):
    expand: int = 1
```

Now let's create child classes for all three types of buttons:

```
@ft.control
class DigitButton(CalcButton):
    bgcolor: ft.Colors = ft.Colors.WHITE_24
    color: ft.Colors = ft.Colors.WHITE

@ft.control
class ActionButton(CalcButton):
    bgcolor: ft.Colors = ft.Colors.ORANGE
    color: ft.Colors = ft.Colors.WHITE

@ft.control
class ExtraActionButton(CalcButton):
    bgcolor: ft.Colors = ft.Colors.BLUE_GREY_100
    color: ft.Colors = ft.Colors.BLACK
```

We will be using these new classes now to create rows of buttons in the Container:

```
content = ft.Column(
            controls=[
                ft.Row(
                    controls=[result],
                    alignment=ft.MainAxisAlignment.END,
                ),
                ft.Row(
                    controls=[
                        ExtraActionButton(content="AC"),
                        ExtraActionButton(content="+/-"),
                        ExtraActionButton(content="%"),
                        ActionButton(content="/"),
                    ]
                ),
                ft.Row(
```

```
                    controls=[
                        DigitButton(content="7"),
                        DigitButton(content="8"),
                        DigitButton(content="9"),
                        ActionButton(content="*"),
                    ]
                ),
                ft.Row(
                    controls=[
                        DigitButton(content="4"),
                        DigitButton(content="5"),
                        DigitButton(content="6"),
                        ActionButton(content="-"),
                    ]
                ),
                ft.Row(
                    controls=[
                        DigitButton(content="1"),
                        DigitButton(content="2"),
                        DigitButton(content="3"),
                        ActionButton(content="+"),
                    ]
                ),
                ft.Row(
                    controls=[
                        DigitButton(content="0", expand=2),
                        DigitButton(content="."),
                        ActionButton(content="="),
                    ]
                ),
            ]
        )
```

▶ Full code

<Image src={frontMatter.example_images + ‘/calc3.png’} alt=“calc3” width=“50%” />

Just what we wanted!

# Reusable UI components

While you can continue writing your app in the `main` function, the best practice would be to create a reusable UI component.

Imagine you are working on an app header, a side menu, or UI that will be a part of a larger project (for example, at Flet we will be using this Calculator app in a bigger “Gallery” app that will show all the examples for Flet framework).

Even if you can’t think of such uses right now, we still recommend creating all your Flet apps with composability and reusability in mind.

To make a reusable Calc app component, we are going to encapsulate its state and presentation logic in a separate `CalculatorApp` class.

Copy the entire code for this step from here.

:::note[Try something] Try adding two `CalculatorApp` components to the page:

```
# create application instance
calc1 = CalculatorApp()
calc2 = CalculatorApp()

# add application's root control to the page
page.add(calc1, calc2)
```

:::

## Handling events

Now let's make the calculator do its job. We will be using the same event handler for all the buttons and use `content` property to differentiate between the actions depending on the button clicked.

We will define `button_click` method in `CalculatorClass` and pass it to each button. Below is `on_click` event handler that will reset the Text value when "AC" button is clicked:

```
def button_clicked(self, e):
    data = e.control.content
    print(f"Button clicked with data = {data}")
    if data == "AC":
        self.result.value = "0"
```

With similar approach, `button_click` method will handle different calculator actions depending on `content` property for each button. Copy the entire code for this step from here.

Run the app and see it in the action: calc-app2

## Publishing your app

Congratulations! You have created your Calculator app with Flet, and it looks awesome! Now it's time to share your app with the world!

Flet Python app and all its dependencies can be packaged into a standalone executable a package for distribution using `flet build` command.

Follow these instructions to package your Calculator app into a desktop executable, mobile app bundle or web app.

# Summary

In this tutorial you have learned how to:

- Create a simple Flet app;
- Work with Reusable UI components;
- Design UI layout using `Column`, `Row` and `Container` controls;
- Handle events;
- Publish your Flet app to multiple platforms;

For further reading you can explore controls and examples.

import {CodeExample, Image} from '@site/src/components/crocodocs';

In this tutorial we are going to create a trivial in-memory Chat app that will help you understand Flet framework basics. This app could be a good starting point to creating your own more complex and useful projects.

In this tutorial you will learn how to:

- Create your first Flet app
- Add page controls and handle events
- Broadcast messages using built-in PubSub library
- Use AlertDialog control for accepting user name
- Enhance user interface with reusable controls
- Deploy the app as a web app

The complete application will look like this:

 chat

You can try the live demo here.

# Getting started with Flet

It's a tradition to start with "Hello, world!" app!

To create a multi-platform app in Python with Flet, you don't need to know HTML, CSS or JavaScript, but you do need a basic knowledge of Python and object-oriented programming.

Before you can create your first Flet app, you need to setup your development environment.

Once you have Flet installed, let's create a hello-world app.

Create `hello.py` with the following contents:

```python title="hello.py"
import flet as ft

def main(page: ft.Page):
    page.add(ft.Text(value="Hello, world!"))

ft.run(main)
```

Run this app you will see a new window with a greeting:

<Image src="examples/tutorials/todo/media/hello-world.png" alt="hello-world" width="55%" />

## Adding page controls and handling events

To start, we want to be able to take user input (chat message) and show messages history on the screen.
The layout for this step could look like this:

<Image src="examples/tutorials/chat/media/layout-1.svg" alt="chat-layout-1" width="55%" />

To implement this layout, we will be using these Flet controls:

* [`Column`](../controls/column.md) - a container to display chat messages (`Text` controls) vertically.
* [`Text`](../controls/text.md) - chat message displayed in the chat Column.
* [`Row`](../controls/row.md) - a container to display `TextField` and `Button` horizontally.
* [`TextField`](../controls/textfield.md) - input control used for taking new message input from the user.
* [`Button`](../controls/button.md) - "Send" button that will add new message to the chat Column.

Create `chat.py` with the following contents:

<CodeExample path={frontMatter.examples + '/chat_1.py'} language="python" />

When user clicks on the "Send" button, it triggers [`on_click`](../controls/button.md#flet.Button.on_click) event which calls `send_click` method. `send_click`
then adds new [`Text`](../controls/text.md) control to the list of [`Column.controls`](../controls/column.md#flet.Column.controls) and clears `new_message` text field value.

:::note[Note]
After any properties of a control are updated in event handler, there is no need to call an `update()` method as the update will be done automatically, but it should be called in case properties are updated outside of the event.
:::

Chat app now looks like this:
<Image src="examples/tutorials/chat/media/chat-1.png" alt="chat-1" width="55%" />

```
## Broadcasting chat messages

In the previous step we have created a simple app that takes input
from the user and displays chats messages on the screen.

If you open this app in two web browser tabs, it will create two app
sessions. Each session will have its own list of messages.

:::tip[Tip]
To open your app in two web browser tabs locally, run the following
command:
```

flet run –web

```
Once it is opened, copy the url and paste it into a new tab.
:::

To build a realtime chat app, you need to somehow pass the messages
between chat app sessions.
When a user sends a message, it should be broadcasted to all other app
sessions and displayed on their pages.

Flet provides a simple built-in [PubSub](../cookbook/pub-sub.md)
mechanism for asynchronous communication between page sessions.

First, we need subscribe the user to receive broadcast messages:
```python
    page.pubsub.subscribe(on_message)
```

`pubsub.subscribe()` method will add current app session to the list of subscribers. It accepts `handler` as an argument, that will later be called at the moment a publisher calls `pubsub.send_all()` method.

In the `handler` we will be adding new message (`Text`) to the list of chat `controls`:

```
            def on_message(message: Message):
                chat.controls.append(ft.Text(f"{message.user}:
{message.text}"))
                page.update()
```

Finally, you need to call `pubsub.send_all()` method when the user clicks on “Send” button:

```
            def send_click(e):
                page.pubsub.send_all(Message(user=page.session.index,
text=new_message.value))
                new_message.value = ""
                page.update()

            page.add(chat, ft.Row([new_message, ft.Button("Send",
on_click=send_click)]))
```
```

`pubsub.send_all()` will call the `on_message()` and pass on the Message object down to it.

Here is the full code for this step:

<CodeExample path={frontMatter.examples + '/chat_2.py'} language="python" />

 chat-2

# User name dialog

Chat app that you have created in the previous step has basic functionality needed to exchange messages between user sessions. It is not very user-friendly though, since it shows `session.id` that sent a message, which doesn't tell much about who you are communicating with.

Let's improve our app to show user name instead of `session.id` for each message. To capture user name we will be using `AlertDialog` control. Let's add it to the page:

```
            user_name = ft.TextField(label="Enter your name")

            page.show_dialog(
                ft.AlertDialog(
                    open=True,
                    modal=True,
                    title=ft.Text("Welcome!"),
                    content=ft.Column([user_name], tight=True),
                    actions=[ft.Button(content="Join chat",
on_click=join_click)],
                    actions_alignment=ft.MainAxisAlignment.END,
                )
            )
```

A dialog will be opened on the start of the program since we have set its `open` property to `True`.

 username-dialog

When the user clicks on "Join chat" button, it will call `join_click` method that should send a message to all subscribers, informing them that the user has joined the chat. This message should look different from the regular chat message, for example, like this:

 chat-4

Let's add `message_type` property to the `Message` class to differentiate between login and chat messages:

```
        @dataclass
        class Message:
            user: str
            text: str
            message_type: str
```

We will be checking `message_type` in `on_message` method:

```
        def on_message(message: Message):
            if message.message_type == "chat_message":
                chat.controls.append(ft.Text(f"{message.user}:
{message.text}"))
            elif message.message_type == "login_message":
                chat.controls.append(
                    ft.Text(message.text, italic=True,
color=ft.Colors.BLACK_45, size=12)
                )
            page.update()
```

Messages of “login_message” and “chat_message” types will now be sent on two events: when user joins the chat and when user sends a message.

Let’s create `join_click` method:

```
        def join_click(e):
            if not user_name.value:
                user_name.error = "Name cannot be blank!"
                user_name.update()
            else:
                page.session.store.set("user_name", user_name.value)
                page.pop_dialog()
                page.pubsub.send_all(Message(user=user_name.value,
text=f"{user_name.value} has joined the chat.",
message_type="login_message"))
```

We used page session storage to store user_name for its future use in `send_click` method to send chat messages.

User name dialog will close as soon as we call `page.pop_dialog()` method.

Finally, let’s update `send_click` method to use `user_name` that we previously saved using `page.session.store`:

```
        def send_click(e):

page.pubsub.send_all(Message(user=page.session.store.get('user_name'),
text=new_message.value, message_type="chat_message"))
            new_message.value = ""
```

▶ Code




# Enhancing user interface

Chat app that you have created in the previous step already serves its purpose of exchanging messages between users with basic login functionality.

Before moving on to deploying your app, we suggest adding some extra features to it that will improve user experience and make the app look more professional.

## Reusable user controls

You may want to show messages in a different format, like this:

 chat-layout-chatmessage

Chat message will now be a `Row` containing `CircleAvatar` with username initials and `Column` that contains two `Text` controls: user name and message text.

We will need to show quite a few chat messages in the chat app, so it makes sense to create your own reusable control. Lets create a new `ChatMessage` dataclass that will inherit from `Row`.

When creating an instance of `ChatMessage` class, we will pass a `Message` object as an argument and then `ChatMessage` will display itself based on `message.user_name` and `message.text`:

```
        @ft.control
        class ChatMessage(ft.Row):
            def __init__(self, message: Message):
                super().__init__()
                self.message = message
                self.vertical_alignment = ft.CrossAxisAlignment.START
                self.controls = [
                    ft.CircleAvatar(

content=ft.Text(self.get_initials(self.message.user_name)),
                        color=ft.Colors.WHITE,

bgcolor=self.get_avatar_color(self.message.user_name),
                    ),
                    ft.Column(
                        tight=True,
                        spacing=5,
                        controls=[
                            ft.Text(self.message.user_name,
weight=ft.FontWeight.BOLD),
                            ft.Text(self.message.text,
selectable=True),
                        ],
                    ),
                ]

            def get_initials(self, user_name: str):
                if user_name:
                    return user_name[:1].capitalize()
                else:
                    return "Unknown"  # or any default value you
prefer
```

```
            def get_avatar_color(self, user_name: str):
                colors_lookup = [
                    ft.Colors.AMBER,
                    ft.Colors.BLUE,
                    ft.Colors.BROWN,
                    ft.Colors.CYAN,
                    ft.Colors.GREEN,
                    ft.Colors.INDIGO,
                    ft.Colors.LIME,
                    ft.Colors.ORANGE,
                    ft.Colors.PINK,
                    ft.Colors.PURPLE,
                    ft.Colors.RED,
                    ft.Colors.TEAL,
                    ft.Colors.YELLOW,
                ]
                return colors_lookup[hash(user_name) %
len(colors_lookup)]
```

`ChatMessage` control extracts initials and algorithmically derives avatar color from a username. Later, if you decide to improve control layout or its logic, it won't affect the rest of the program - that's the power of encapsulation!

## Laying out controls

Now you can use your brand new `ChatMessage` to build a better layout for the chat app:

 chat-layout-2

Instances of `ChatMessage` will be created instead of plain `Text` chat messages in `on_message` method:

```
            def on_message(message: Message):
                if message.message_type == "chat_message":
                    m = ChatMessage(message)
                elif message.message_type == "login_message":
                    m = ft.Text(message.text, italic=True,
color=ft.Colors.BLACK_45, size=12)
                chat.controls.append(m)
                page.update()
```

Other improvements suggested with the new layout are:

- `ListView` instead of `Column` for displaying messages, to be able to scroll through the messages later
- `Container` for displaying border around `ListView`
- `IconButton` instead of `Button` to send messages
- Use of `expand` property for controls to fill available space

Here is how you can implement this layout:

```
    # Chat messages
    chat = ft.ListView(
        expand=True,
        spacing=10,
        auto_scroll=True,
    )

    # A new message entry form
    new_message = ft.TextField(
        hint_text="Write a message...",
        autofocus=True,
        shift_enter=True,
        min_lines=1,
        max_lines=5,
        filled=True,
        expand=True,
        on_submit=send_message_click,
    )

    # Add everything to the page
    page.add(
        ft.Container(
            content=chat,
            border=ft.Border.all(1, ft.Colors.OUTLINE),
            border_radius=5,
            padding=10,
            expand=True,
        ),
        ft.Row(
            [
                new_message,
                ft.IconButton(
                    icon=ft.Icons.SEND_ROUNDED,
                    tooltip="Send message",
                    on_click=send_message_click,
                ),
            ]
        ),
    )
```

▶ Full code

This is the final version of the chat app for the purpose of this tutorial. Below you can read more about the enhancements that we have made.

## Keyboard support

### Focusing input controls

All data entry controls have `autofocus` property which when set to `True` moves initial focus to the control. If there is more than one control on a page with `autofocus` set, then the first one added to the page will get focus.

We set `autofocus=True` on a username TextField inside a dialog and then on a TextField for entering chat message to set initial focus on it when the dialog is closed.

When a user click “Send” button or presses Enter to submit a chat message, TextField loses focus. To programmatically set control focus we used `TextField.focus()` method.

### Submitting forms on `Enter`

It’s so tempting to submit forms with just pushing `Enter` button on the keyboard! Type your name in the dialog, hit `Enter`, type a new message, hit `Enter`, type another, hit `Enter` - no mouse involved at all! 🚀

Flet has support for that by providing `TextField.on_submit` event handler which fires when a user press `Enter` button while the focus is on the TextField.

### Entering multiline messages

What about multiline TextFields where `Enter` must advance a cursor to the next line? We’ve got that covered too! `TextField` control has `shift_enter` property which when set to `True` enables Discord-like behavior: to get to a new line user presses `Shift+Enter` while hitting just `Enter` submits a form.

## Animated scrolling to the last message

Noticed a nice animation of scrolling to the last message in a chat window? It could be enabled by setting `ListView.auto_scroll` property to `True`. The top most `Page` class, being a scrollable container itself, also supports `auto_scroll`.

## Page title

Final touch - page title that could be changed as simply as:

```
page.title = "Flet Chat"
page.update()
```

# Deploying the app

Congratulations! You have created your Chat app in Python with Flet, and it looks awesome!

Now it’s time to share your app with the world!

Follow these instructions to deploy your Flet app as a web app to Fly.io or Replit.

## What's next

There are plenty of features we could implement to improve this chat app:

- Disconnect, reconnect, session timeout
- Upload/download images
- Authentication, avatars
- Using database for the storage
- Chat channels, topics
- Full-text search
- Emojis, markdown
- Bots
- Mobile app

Please let us know if you would like to contribute to the app/tutorial and share it with other Flet developers.

## Summary

In this tutorial, you have learnt how to:

- Create a simple Flet app;
- Add page controls and handle events;
- Use built in PubSub library;
- User AlertDialog for entering user name;
- Build page layout with reusable controls;
- Deploy your Flet app to the web;

For further reading you can explore controls and examples.

import {Image} from '@site/src/components/crocodocs';

In this tutorial we will show you step-by-step creation of a famous Klondike solitaire game in Python with Flet. For an inspiration, we looked at this online game: https://www.solitr.com/

This tutorial is aimed at beginner/intermediate level Python developers who have basic knowledge of Python and object oriented programming.

You can find a live demo here.

 solitaire

We have broken down the game implementation into the following steps:

- Getting started with Flet
- Proof of concept app for draggable cards
- Fanned card piles
- Solitaire setup
- Solitaire rules

- Winning the game
- Deploying the app

In the Part 2 (will be covered in the next tutorial) we'll be adding Appbar with options to start new game, view game rules and change game settings.

# Getting started with Flet

To create a multi-platform app in Python with Flet, you don't need to know HTML, CSS or JavaScript, but you do need a basic knowledge of Python and object-oriented programming.

Before you can create your first Flet app, you need to setup your development environment, which requires Python 3.9 or above and `flet` package.

Once you have Flet installed, let's create a simple hello-world app.

Create `hello.py` with the following contents:

```python title="hello.py"
import flet as ft

def main(page: ft.Page):
    page.add(ft.Text(value="Hello, world!"))

ft.run(main)
```

```
Run this app and you will see a new window with a greeting:

<Image src="examples/tutorials/todo/media/hello-world.png" alt="hello-
world" width="55%" />

## Proof of concept app for draggable cards

For the proof of concept, we will only be using three types of
controls:

* [`Stack`](../controls/stack.md) - will be used as a parent control
for absolute positioning of slots and cards
* [`GestureDetector`](../controls/gesturedetector.md) - the card that
will be moved within the [`Stack`](../controls/stack.md)
* [`Container`](../controls/container.md) - the slot where the card
will be dropped. Will also be used as `content` for the
[`GestureDetector`](../controls/gesturedetector.md).

We have broken down the proof of concept app into four easy steps, so
that after each step you have
a complete short program to run and test.

###  Step 1: Drag the card around

In this step we will create a [`Stack`](../controls/stack.md)
(Solitaire game field)
```

```
and a [`GestureDetector`](../controls/gesturedetector.md) (Solitaire
card).
The card will then be added to the list of the [`Stack.controls`]
(../controls/stack.md#flet.Stack.controls).
[`top`](../controls/gesturedetector.md) and [`left`]
(../controls/gesturedetector.md) properties of the
`GestureDetector` are used for absolute positioning of the card in the
`Stack`.

```python
import flet as ft

def main(page: ft.Page):
    def drag(e: ft.DragUpdateEvent):
        ...

    card = ft.GestureDetector(
        on_pan_update=drag,
        left=0,
        top=0,
        content=ft.Container(bgcolor=ft.Colors.GREEN, width=70,
height=100),
    )

    page.add(ft.Stack(controls=[card], width=1000, height=500))

ft.run(main)
```

Run the app to see the card added to the stack:


drag_and_drop1

To be able to move the card, we'll update the `drag` method that is called in `on_pan_update` event of `GestureDetector` which happens every `drag_interval` while the user drags the card with their mouse.

To show the card's movement, we'll be updating the card's `top` and `left` properties in the `drag` callback each time the `on_pan_update` event happens.

Below is the simplest code for dragging `GestureDetector` in `Stack`:

```
        import flet as ft

        # Use of GestureDetector with on_pan_update event for dragging
card
        # Absolute positioning of controls within stack

        def main(page: ft.Page):
            def drag(e: ft.DragUpdateEvent):
                e.control.top = max(0, e.control.top +
e.local_delta.y)
                e.control.left = max(0, e.control.left +
e.local_delta.x)
```
```

```
                e.control.update()

            card = ft.GestureDetector(
                mouse_cursor=ft.MouseCursor.MOVE,
                drag_interval=5,
                on_pan_update=drag,
                left=0,
                top=0,
                content=ft.Container(bgcolor=ft.Colors.GREEN,
width=70, height=100),
            )

            page.add(ft.Stack(controls=[card], width=1000,
height=500))

        ft.run(main)
```

Now you can see the card moving:

 drag_and_drop2

After any properties of a control are updated, an `update()` method of the control (or its parent control) should be called for the update to take effect.

## Step 2: Drop the card in the slot or bounce it back

The goal of this step is to be able to drop a card into a slot if it is close enough and bounce it back if it's not.

 drag-and-drop3

Let's create a `Container` control that will represent a slot to which we'll be dropping the card:

```
        slot = ft.Container(
            width=70,
            height=100,
            left=200,
            top=0,
            border=ft.Border.all(1)
        )
        page.add(ft.Stack(controls = [slot, card], width=1000,
height=500))
```

`on_pan_end` event of the card is called when the card is dropped:

```
        card = ft.GestureDetector(
            mouse_cursor=ft.MouseCursor.MOVE,
            drag_interval=5,
            on_pan_update=drag,
            on_pan_end=drop,
            left=0,
```

```
            top=0,
            content=ft.Container(bgcolor=ft.Colors.GREEN, width=70,
height=100),
        )
```

On this event, we'll call `drop` method to check if the card is close enough to the slot (let's say it's closer than 20px to the slot), and `place` it there:

```
    def drop(e: ft.DragEndEvent):
        if (
            abs(e.control.top - slot.top) < 20
            and abs(e.control.left - slot.left) < 20
        ):
            place(e.control, slot)
        e.control.update()

    def place(card, slot):
        """place card to the slot"""
        card.top = slot.top
        card.left = slot.left
        page.update()
```

Now, if the card is not close enough, we need to bounce it back to its original position. Unfortunately, we don't know the original position coordinates, since the card's `top` and `left` properties were changed on `on_pan_update` event.

To solve this problem, let's create a `Solitaire` class object to keep track of the original position of the card when `on_pan_start` event of the card is called:

```
    class Solitaire:
        def __init__(self):
            self.start_top = 0
            self.start_left = 0

    solitaire = Solitaire()

    def start_drag(e: ft.DragStartEvent):
        solitaire.start_top = e.control.top
        solitaire.start_left = e.control.left
        e.control.update()
```

Now let's update `on_pan_end` event with the option to bounce card back:

```
    def bounce_back(game, card):
        """return card to its original position"""
        card.top = game.start_top
        card.left = game.start_left
        page.update()

    def drop(e: ft.DragEndEvent):
        if (
            abs(e.control.top - slot.top) < 20
            and abs(e.control.left - slot.left) < 20
```

```
            ):
                place(e.control, slot)
            else:
                bounce_back(solitaire, e.control)

            e.control.update()
```

The full code for this step can be found here.

## Step 3: Adding a second card

Eventually, we'll need 52 cards to play the game. For our proof of concept, let's add a second card:

```
            card2 = ft.GestureDetector(
                mouse_cursor=ft.MouseCursor.MOVE,
                drag_interval=5,
                on_pan_start=start_drag,
                on_pan_update=drag,
                on_pan_end=drop,
                left=100,
                top=0,
                content=ft.Container(bgcolor=ft.Colors.YELLOW,
width=70, height=100),
            )

            controls = [slot, card1, card2]
            page.add(ft.Stack(controls=controls, width=1000,
height=500))
```

Now, if you run the app with the two cards, you will notice that when you move the cards around, the yellow card (`card2`) is moving as expected but the green the card (`card1`) is moving under the yellow card.

 drag_and_drop4.gif

It happens because `card2` is added to the list of stack's `controls` after `card1`. To fix this problem, we need to move the draggable card to the top of the list of controls on `on_pan_start` event:

```
        def move_on_top(card, controls):
            """Moves draggable card to the top of the stack"""
            controls.remove(card)
            controls.append(card)
            page.update()

        def start_drag(e: ft.DragStartEvent):
            move_on_top(e.control, controls)
            solitaire.start_top = e.control.top
            solitaire.start_left = e.control.left
```

Now the two cards can be dragged without issues:



The full code for this step can be found here.

## Step 4: Adding more slots

As a final step for the proof of concept app, let's create two more slots:

```
        slot0 = ft.Container(
            width=70, height=100, left=0, top=0,
border=ft.Border.all(1)
        )

        slot1 = ft.Container(
            width=70, height=100, left=200, top=0,
border=ft.Border.all(1)
        )

        slot2 = ft.Container(
            width=70, height=100, left=300, top=0,
border=ft.Border.all(1)
        )

        slots = [slot0, slot1, slot2]
```

When creating new cards, we will not specify their `top` and `left` position now, but instead, will place them to the `slot0`:

```
        # deal cards
        place(card1, slot0)
        place(card2, slot0)
```

`on_pan_end` event, where we check if a card is close to a slot, we will now go through the list of slots to find where the card should be dropped:

```
        def drop(e: ft.DragEndEvent):
            for slot in slots:
                if (
                    abs(e.control.top - slot.top) < 20
                and abs(e.control.left - slot.left) < 20
                ):
                    place(e.control, slot)
                    e.control.update()
                    return

            bounce_back(solitaire, e.control)
            e.control.update()
```

As a result, the two cards can be dragged between the three slots:



The full code for this step can be found here.

Congratulations on completing the proof of concept app for the Solitaire game! Now you can work with `GestureDetector` to move cards inside `Stack` and place them to certain `Containers`, which is a great part of the game to begin with.

# Fanned card piles

In the proof of concept app you have accomplished the task of dropping a card into a slot in proximity or bounce it back. If there is already a card in that slot, the new card is placed on top of it, covering it completely.

In the actual Solitaire game, if there is already a card in a *tableau* slot, you want to place the draggable card a bit lower, so that you can see the previous card too, and if there are two cards, even lower. Those are called “fanned piles”.

Then, we want to be able to pick a card from the fanned pile that is not the top card of the pile and drag the card together with all the cards below it:

 fanned_piles1.gif

To be able to do that, it would be useful to have the information about the pile of cards in the slot from which the card is dragged, as well as in the slot to which it is being dropped. Let’s restructure our program and get it ready for the implementation of the fanned piles.

## Slot, Card and Solitaire classes

A slot could have a `pile` property that would hold a list of cards that were placed there. Now the slot is a `Container` control object, and we can’t add any new properties to it. Let’s create a new `Slot` class that will inherit from `Container` and add a `pile` property to it:

```
SLOT_WIDTH = 70
SLOT_HEIGHT = 100

import flet as ft

class Slot(ft.Container):
    def __init__(self, top, left):
        super().__init__()
        self.pile=[]
        self.width=SLOT_WIDTH
        self.height=SLOT_HEIGHT
        self.left=left
        self.top=top
        self.border=ft.Border.all(1)
```

Similarly to `Slot` class, let’s create a new `Card` class with `slot` property to remember in which slot it resides. It will inherit from `GestureDetector` and we’ll move all card-related methods to it:

```
            CARD_WIDTH = 70
            CARD_HEIGHT = 100
            DROP_PROXIMITY = 20

            import flet as ft

            class Card(ft.GestureDetector):
                def __init__(self, solitaire, color):
                    super().__init__()
                    self.slot = None
                    self.mouse_cursor=ft.MouseCursor.MOVE
                    self.drag_interval=5
                    self.on_pan_start=self.start_drag
                    self.on_pan_update=self.drag
                    self.on_pan_end=self.drop
                    self.left=None
                    self.top=None
                    self.solitaire = solitaire
                    self.color = color
                    self.content=ft.Container(bgcolor=self.color,
    width=CARD_WIDTH, height=CARD_HEIGHT)

                def move_on_top(self):
                    """Moves draggable card to the top of the stack"""
                    self.solitaire.controls.remove(self)
                    self.solitaire.controls.append(self)
                    self.solitaire.update()

                def bounce_back(self):
                    """Returns card to its original position"""
                    self.top = self.slot.top
                    self.left = self.slot.left
                    self.update()

                def place(self, slot):
                    """Place card to the slot"""
                    self.top = slot.top
                    self.left = slot.left
                    self.slot = slot

                def start_drag(self, e: ft.DragStartEvent):
                    self.move_on_top()
                    self.update()

                def drag(self, e: ft.DragUpdateEvent):
                    self.top = max(0, self.top + e.local_delta.y)
                    self.left = max(0, self.left + e.local_delta.x)
                    self.update()

                def drop(self, e: ft.DragEndEvent):
                    for slot in self.solitaire.slots:
```

```
                    if (
                        abs(self.top - slot.top) < DROP_PROXIMITY
                        and abs(self.left - slot.left) <
DROP_PROXIMITY
                    ):
                        self.place(slot)
                        self.update()
                        return

                self.bounce_back()
                self.update()
```

Since each card has `slot` property now, there is no need to remember `start_left` and `start_top` position of the draggable card in Solitaire class anymore, because we can just bounce it back to its slot.

Let's update `Solitaire` class to inherit from `Stack`, and move the creation of cards and slots there:

```
SOLITAIRE_WIDTH = 1000
SOLITAIRE_HEIGHT = 500

import flet as ft
from slot import Slot
from card import Card

class Solitaire(ft.Stack):
    def __init__(self):
        super().__init__()
        self.controls = []
        self.slots = []
        self.cards = []
        self.width = SOLITAIRE_WIDTH
        self.height = SOLITAIRE_HEIGHT

    def did_mount(self):
        self.create_card_deck()
        self.create_slots()
        self.deal_cards()

    def create_card_deck(self):
        card1 = Card(self, color="GREEN")
        card2 = Card(self, color="YELLOW")
        self.cards = [card1, card2]

    def create_slots(self):
        self.slots.append(Slot(top=0, left=0))
        self.slots.append(Slot(top=0, left=200))
        self.slots.append(Slot(top=0, left=300))
        self.controls.extend(self.slots)
        self.update()
```

```
    def deal_cards(self):
        self.controls.extend(self.cards)
        for card in self.cards:
            card.place(self.slots[0])
        self.update()
```

If you try to call `create_slots()` and `create_card_deck()` and `deal_cards()` methods `__init__()` method of the Solitaire class, it will cause an error “Control must be added to the page first”. To fix this, we create slots and cards inside the `did_mount()` method, which happens immediately after the stack is added to the page.

Our main program will be very simple now:

```
import flet as ft
from solitaire import Solitaire

def main(page: ft.Page):

    solitaire = Solitaire()

    page.add(solitaire)

ft.run(main)
```

You can find the full source code for this step here. It works exactly the same way as the proof of concept app, but re-written with the new classes to be ready for adding more complex functionality to it.

## Placing card with offset

When the card is being placed to a slot in the `card.place()` method, we need to do three things: * Remove the card from its original slot, if it exists * Change card’s slot to the new slot * Add the card to the new slot’s pile

```
def place(self, slot):
    self.top = slot.top
    self.left = slot.left

    # remove card from it's original slot, if exists
    if self.slot is not None:
        self.slot.pile.remove(self)

    # change card's slot to a new slot
    self.slot = slot

    # add card to the new slot's pile
    slot.pile.append(self)
```

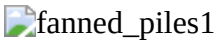
When updating card’s `top` and `left` position, `left` should remain the same, but `top` will depend on the length of the new slot’s pile and `CARD_OFFSET`:

```
        CARD_OFFSET = 20

        ...

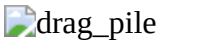
        def place(self, slot):
            self.top = slot.top + len(slot.pile) * CARD_OFFSET
            self.left = slot.left
            ...
```

Now the cards are placed with offset which gives us the fanned pile look:



## Drag pile of cards

If you try to drag the card from the bottom of the pile now, it will look like this:



To fix this problem, we need to update all the methods in `card.py` that work with the draggable card to work with the draggable pile instead.

Let's create `get_draggable_pile()` method that will get the list of cards that need to be dragged together, starting with the card you picked:

```
            def get_draggable_pile(self):
                """returns list of cards that will be dragged
together, starting with the current card"""
                if self.slot is not None:
                    self.draggable_pile =
self.slot.pile[self.slot.pile.index(self) :]
                else:  # slot == None when the cards are dealt and
need to be place in slot for the first time
                    self.draggable_pile = [self]
```

Since `self.draggable_pile` is a new `Card` attribute, Let's add it to `__init__()`:

```
        class Card(ft.GestureDetector):
            def __init__(self, solitaire, color):
                super().__init__()
                ...
                self.draggable_pile = [self]
```

Now we need to update `start_drag()` method to get the draggable pile each time a drag starts:

```
            def start_drag(self, e: ft.DragStartEvent):
                self.get_draggable_pile()
                self.move_on_top()
                self.solitaire.update()
```

Then, we'll update `move_on_top()` method:

```
            def move_on_top(self):
                """Brings draggable card pile to the top of the
stack"""

                # for card in self.get_draggable_pile():
                for card in self.draggable_pile:
                    self.solitaire.controls.remove(card)
                    self.solitaire.controls.append(card)
                self.solitaire.update()
```

Additionally, we need to update `drag()` method to go through the draggable pile and update positions of all the cards being dragged:

```
            def drag(self, e: ft.DragUpdateEvent):
                for card in self.draggable_pile:
                    card.top = (
                        max(0, self.top + e.local_delta.y)
                        + self.draggable_pile.index(card) *
CARD_OFFSET
                    )
                    card.left = max(0, self.left + e.local_delta.x)
                    self.solitaire.update()
```

Also, we need to update `place()` method to place the draggable pile to the slot:

```
            def place(self, slot):
                """Place draggable pile to the slot"""
                for card in self.draggable_pile:
                    card.top = slot.top + len(slot.pile) * CARD_OFFSET
                    card.left = slot.left

                    # remove card from it's original slot, if it
exists
                    if card.slot is not None:
                        card.slot.pile.remove(card)

                    # change card's slot to a new slot
                    card.slot = slot

                    # add card to the new slot's pile
                    slot.pile.append(card)

                self.solitaire.update()
```

Finally, if no slot in proximity is found, we need to bounce the whole pile back to its original position:

```
            def bounce_back(self):
                """Returns draggable pile to its original position"""
                for card in self.draggable_pile:
                    card.top = card.slot.top +
card.slot.pile.index(card) * CARD_OFFSET
```

```
                card.left = card.slot.left
            self.solitaire.update()
```

The full source code of this step can be found here. Now we can drag and drop cards in fanned piles, which means we are ready for the real deal!

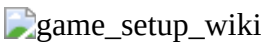
# Solitaire setup

Let's take a look at the wikipedia article about Klondike (solitaire):

> Klondike is played with a standard 52-card deck.

> After shuffling, a tableau of seven fanned piles of cards is laid from left to right. From left to right, each pile contains one more card than the last. The first and left-most pile contains a single upturned card, the second pile contains two cards, the third pile contains three cards, the fourth pile contains four cards, the fifth pile contains five cards, the sixth pile contains six cards, and the seventh pile contains seven cards. The topmost card of each pile is turned face up. The remaining cards form the stock and are placed facedown at the upper left of the layout.

> The four foundations (light rectangles in the upper right of the figure) are built up by suit from Ace (low in this game) to King, and the tableau piles can be built down by alternate colors.




We will now work on this setup step by step.

## Create card deck

The first step is to create a full deck of cards in Solitaire class. Each card should have a `suit` property (hearts, diamonds, clubs and spades) and a `rank` property (from Ace to King). For the suit, its `color` is important, because tableau piles are built by alternate colors.

For the rank, its `value` is important, because foundations are built from the lowest (Ace) to the highest (King) rank value.

In solitaire.py, create `Suite` and `Rank` classes:

```
class Suite:
    def __init__(self, suite_name, suite_color):
        self.name = suite_name
        self.color = suite_color

class Rank:
    def __init__(self, card_name, card_value):
        self.name = card_name
        self.value = card_value
```

Now, in the `Card` class, instead of accepting the color as an argument, we'll be accepting `suite` and `rank` in `__init__()`. Additionally, we'll add `face_up` property to the card and the Container will now has image of the back of the card as its `content`:

```
class Card(ft.GestureDetector):
    def __init__(self, solitaire, suite, rank):
        super().__init__()
        self.mouse_cursor=ft.MouseCursor.MOVE
        self.drag_interval=5
        self.on_pan_start=self.start_drag
        self.on_pan_update=self.drag
        self.on_pan_end=self.drop
        self.suite=suite
        self.rank=rank
        self.face_up=False
        self.top=None
        self.left=None
        self.solitaire = solitaire
        self.slot = None
        self.content=ft.Container(
            width=CARD_WIDTH,
            height=CARD_HEIGHT,
            border_radius = ft.BorderRadius.all(6),
            content=ft.Image(src="images/card_back.png"))
```

All the images for the face up cards, as well as card back are stored in the "assets/images" folder in the same directory as main.py.

You could specify the assets folder, see Assets guide for more information and examples.

Finally, in `solitaire.create_card_deck()` we'll create lists of suites and ranks and then the 52-card deck:

```
def create_card_deck(self):
    suites = [
        Suite("hearts", "RED"),
        Suite("diamonds", "RED"),
        Suite("clubs", "BLACK"),
        Suite("spades", "BLACK"),
    ]
    ranks = [
        Rank("Ace", 1),
        Rank("2", 2),
        Rank("3", 3),
        Rank("4", 4),
        Rank("5", 5),
        Rank("6", 6),
        Rank("7", 7),
        Rank("8", 8),
        Rank("9", 9),
        Rank("10", 10),
        Rank("Jack", 11),
```

```
                Rank("Queen", 12),
                Rank("King", 13),
            ]

            self.cards = []

            for suite in suites:
                for rank in ranks:
                    self.cards.append(Card(solitaire=self,
suite=suite, rank=rank))
```

The card deck is ready to be dealt, and now we need to create the layout for it.

## Create slots

Klondike solitaire game layout should look like this:

 solitaire-layout

Now, in the `Slot` class, we'll be accepting `solitaire`, `border` and `border_radius` as arguments in `__init__()`:

```
        class Slot(ft.Container):
            def __init__(self, top, left, border):
                super().__init__()
                ...
                self.solitaire = solitaire
                self.border = border
                self.border_radius = ft.BorderRadius.all(6)
```

Let's create all those slots in `solitaire.create_slots()`:

```
        def create_slots(self):

            self.stock = Slot(self, top=0, left=0,
border=ft.Border.all(1))
            self.waste = Slot(self, top=0, left=100, border=None)

            self.foundations = []
            x = 300
            for i in range(4):
                self.foundations.append(Slot(self, top=0, left=x,
border=ft.Border.all(1, "outline")))
                x += 100

            self.tableau = []
            x = 0
            for i in range(7):
                self.tableau.append(Slot(self, top=150, left=x,
border=None))
                x += 100
```

```
        self.controls.append(self.stock)
        self.controls.append(self.waste)
        self.controls.extend(self.foundations)
        self.controls.extend(self.tableau)
        self.update()
```

Note: some slots should have visible border and some shouldn't, so we added border to the list of arguments for the creation of `Slot` objects.

If you try to run the app, it will cause an error "IndexError: list index out of range". To fix this, update `deal_cards()` method as described below.

## Deal cards

Let's start with shuffling the cards and adding them to the list of controls:

```
import random

...

def deal_cards(self):
    random.shuffle(self.cards)
    self.controls.extend(self.cards)
    self.update()
```

Then we'll deal the cards to the tableau piles from left to right so that each pile contains one more card than the last, and place the remaining cards to the stock pile:

```
def deal_cards(self):
    random.shuffle(self.cards)
    self.controls.extend(self.cards)

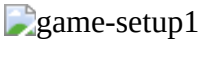
    # deal to tableau
    first_slot = 0
    remaining_cards = self.cards

    while first_slot < len(self.tableau):
        for slot in self.tableau[first_slot:]:
            top_card = remaining_cards[0]
            top_card.place(slot)
            remaining_cards.remove(top_card)
        first_slot +=1

    # place remaining cards to stock pile
    for card in remaining_cards:
        card.place(self.stock)

    self.update()
```

Let's run the program and see where we are at now:



Cards in stock were placed in a fanned pile in the same manner as to the tableau, but they should have been placed to a regular pile instead. To fix this problem, let's add this condition to the `card.place()` method:

```
        def place(self, slot):
            """Place draggable pile to the slot"""
            for card in self.draggable_pile:
                if slot in self.solitaire.tableau:
                    card.top = slot.top + len(slot.pile) * CARD_OFFSET
                else:
                    card.top = slot.top
            self.left = slot.left
            ...
```

Now cards are only placed in fanned piles to tableau:




If you try moving the cards around now, the program won't work. The reason for this is that in the `card.drop()` method iterates through list of slots which we don't have now.

Let's update the method to go separately through foundations and tableau:

```
        def drop(self, e: ft.DragEndEvent):
            for slot in self.solitaire.tableau:
                if (
                    abs(self.top - (slot.top + len(slot.pile) *
CARD_OFFSET)) < DROP_PROXIMITY
                and abs(self.left - slot.left) < DROP_PROXIMITY
                ):
                    self.place(slot)
                    self.solitaire.update()
                    return

            for slot in self.solitaire.foundations:
                if (
                    abs(self.top - slot.top) < DROP_PROXIMITY
                and abs(self.left - slot.left) < DROP_PROXIMITY
                ):
                    self.place(slot)
                    self.solitaire.update()
                    return

            self.bounce_back()
```

## Reveal top cards in tableau piles

Now we have the correct game setup and as a last touch we need to reveal the topmost cards in tableau piles.

In `Slot` class, create a `get_top_card()` method:

```
        def get_top_card(self):
            if len(self.pile) > 0:
                return self.pile[-1]
```

In `Card` class, create `turn_face_up()` method:

```
        def turn_face_up(self):
            self.face_up = True

self.content.content.src=f"images/{self.rank.name}_{self.suite.name}.svg"
            self.solitaire.update()
```

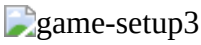
Finally, reveal the topmost cards at the end of `solitaire.deal_cards()`:

```
        def deal_cards(self):
            ...
            self.update()

            for slot in self.tableau:
                slot.get_top_card().turn_face_up()

            self.update()
```

Let's see how it looks now:

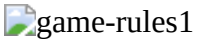



The full source code for this step can be found here.

Congratulations on completing the Solitaire game setup! You've created a full 52-card deck, built layout with stock, waste, foundations and tableau piles, dealt the cards and revealed the top cards in tableau. Let's move on to the next item on our todo list, which is Solitaire Rules.

# Solitaire rules

If you run your current version of Solitaire, you'll notice that you can do some crazy things with your cards:




Now it is time to implement some rules.

## General rules

Currently, we can move any card, but only face-up cards should be moved. Let's add this check in `start_drag`, `drag` and `drop` methods of the card:

```
        def start_drag(self, e: ft.DragStartEvent):
            if self.face_up:
                self.move_on_top()
                self.solitaire.update()
```

```
        def drag(self, e: ft.DragUpdateEvent):
            if self.face_up:
                for card in self.draggable_pile:
                    card.top = max(0, self.top + e.local_delta.y) +
self.draggable_pile.index(card) * CARD_OFFSET
                    card.left = max(0, self.left + e.local_delta.x)
                    card.solitaire.update()

        def drop(self, e: ft.DragEndEvent):
            if self.face_up:
                for slot in self.solitaire.tableau:
                    if (
                        abs(self.top - (slot.top + len(slot.pile) *
CARD_OFFSET)) < DROP_PROXIMITY
                        and abs(self.left - slot.left) <
DROP_PROXIMITY
                    ):
                        self.place(slot)
                        return

                for slot in self.solitaire.foundations:
                    if (
                        abs(self.top - slot.top) < DROP_PROXIMITY
                        and abs(self.left - slot.left) <
DROP_PROXIMITY
                    ):
                        self.place(slot)
                        return

            self.bounce_back()
```

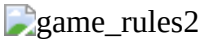
Now let's specify `click` method for the `on_tap` event of the card to reveal the card if you click on a faced-down top card in a tableau pile:

```
        class Card(ft.GestureDetector):
            def __init__(self, solitaire, suite, rank):
                ...
                self.on_tap = self.click
                ...

            ...

            def click(self, e):
                if self.slot in self.solitaire.tableau:
                    if not self.face_up and self ==
self.slot.get_top_card():
                        self.turn_face_up()
                        self.solitaire.update()
```

Let's check how it works:



## Foundations rules

At the moment we can place fanned piles to foundations, which shouldn't be allowed. Let's check the draggable pile length to fix it:

```
        def drop(self, e: ft.DragEndEvent):
            if self.face_up:
                for slot in self.solitaire.tableau:
                    if (
                        abs(self.top - (slot.top + len(slot.pile) *
CARD_OFFSET)) < DROP_PROXIMITY
                        and abs(self.left - slot.left) <
DROP_PROXIMITY
                    ):
                        self.place(slot)
                        return

                if len(self.draggable_pile) == 1:
                    for slot in self.solitaire.foundations:
                        if (
                            abs(self.top - slot.top) < DROP_PROXIMITY
                        and abs(self.left - slot.left) <
DROP_PROXIMITY
                        ):
                            self.place(slot)
                            return

            self.bounce_back()
```

Then, of course, not any card can be placed to a foundation. According to the rules, a foundation should start with an Ace and then the cards of the same suite can be placed on top of it to build a pile form Ace to King.

Let's add this rule to Solitaire class:

```
        def check_foundations_rules(self, card, slot):
            top_card = slot.get_top_card()
            if top_card is not None:
                return (
                    card.suite.name == top_card.suite.name
                    and card.rank.value - top_card.rank.value == 1
                )
            else:
                return card.rank.name == "Ace"
```

We'll check this rule in `drop()` method of the card before placing a card to a foundation slot:

```
        def drop(self, e: ft.DragEndEvent):
            if self.face_up:
```

```
                for slot in self.solitaire.tableau:
                    if (
                        abs(self.top - (slot.top + len(slot.pile) *
CARD_OFFSET)) < DROP_PROXIMITY
                        and abs(self.left - slot.left) <
DROP_PROXIMITY
                    ):
                        self.place(slot)
                        return

                if len(self.draggable_pile) == 1:
                    for slot in self.solitaire.foundations:
                        if (
                            abs(self.top - slot.top) < DROP_PROXIMITY
                            and abs(self.left - slot.left) <
DROP_PROXIMITY
                        ) and
self.solitaire.check_foundations_rules(self, slot):
                            self.place(slot)
                            return

                self.bounce_back()
```

As a final touch for foundations rules, let's implement `doubleclick` method for `on_double_tap` event of a card. It will be checking if the faced-up card fits into any of the foundations and place it there:

```
        class Card(ft.GestureDetector):
            def __init__(self, solitaire, suite, rank):
                ...
                self.on_double_tap = self.doubleclick
                ...

            ...

            def doubleclick(self, e):
                self.get_draggable_pile()
                if self.face_up and len(self.draggable_pile) == 1:
                    self.move_on_top()
                    for slot in self.solitaire.foundations:
                        if
self.solitaire.check_foundations_rules(self, slot):
                            self.place(slot)
                            return
```

## Tableau rules

Finally, let's implement the rules to build tableau piles down from King to Ace by alternating suite color. Additionally, only King can be placed to an empty tableau slot.

Let's add these rules for Solitaire class:

```
        def check_tableau_rules(self, card, slot):
            top_card = slot.get_top_card()
            if top_card is not None:
                return (
                    card.suite.color != top_card.suite.color
                    and top_card.rank.value - card.rank.value == 1
                    and top_card.face_up
                )
            else:
                return card.rank.name == "King"
```

Similarly to the foundations rules, we'll check tableau rules before placing a card to a tableau pile:

```
        def drop(self, e: ft.DragEndEvent):
            if self.face_up:
                for slot in self.solitaire.tableau:
                    if (
                        abs(self.top - (slot.top + len(slot.pile) *
CARD_OFFSET)) < DROP_PROXIMITY
                        and abs(self.left - slot.left) <
DROP_PROXIMITY
                    ) and self.solitaire.check_tableau_rules(self,
slot):
                        self.place(slot)
                        return

                if len(self.draggable_pile) == 1:
                    for slot in self.solitaire.foundations:
                        if (
                            abs(self.top - slot.top) < DROP_PROXIMITY
                            and abs(self.left - slot.left) <
DROP_PROXIMITY
                        ) and
self.solitaire.check_foundations_rules(self, slot):
                            self.place(slot)
                            return

                self.bounce_back()
```

## Stock and waste

To properly play Solitaire game right now we are missing the remaining cards that are piled in the stock.

Let's update `click()` method of the card to go through the stock pile and place the cards to waste as we go:

```
        def click(self, e):
            if self.slot in self.solitaire.tableau:
                if not self.face_up and self ==
self.slot.get_top_card():
```

```
                self.turn_face_up()
            elif self.slot == self.solitaire.stock:
                self.move_on_top()
                self.place(self.solitaire.waste)
                self.turn_face_up()
```

Now, in `Card` class, create `turn_face_down()` method:

```
def turn_face_down(self):
    """Hides card"""
    self.face_up = False
    self.content.content.src = "/images/card_back.png"
    self.solitaire.update()
```

That's it! Now you can properly play solitaire, but it very difficult to win the game if you cannot pass though the waste again. Let's implement `click()` for `on_click` event of the stock Slot to go thought the stock pile again:

```
        class Slot(ft.Container):
            def __init__(self, solitaire, top, left, border):
                super().__init__()
                self.pile=[]
                self.width=SLOT_WIDTH
                self.height=SLOT_HEIGHT
                self.left=left
                self.top=top
                self.on_click=self.click
                self.solitaire=solitaire
                self.border=border
                self.border_radius = ft.BorderRadius.all(6)

            def click(self, e):
                if self == self.solitaire.stock:
                    self.solitaire.restart_stock()
```

`restart_stock()` method in `Solitaire` class will place all cards from waste to stock again:

```
        def restart_stock(self):
            while len(self.waste.pile) > 0:
                card = self.waste.get_top_card()
                card.turn_face_down()
                card.move_on_top()
                card.place(self.stock)
```

For `card.place()` method to work properly with cards from Stock and Waste, we've added a condition to `card.get_draggable_pile()`, so that it returns the top card only and not the whole pile:

```
            def get_draggable_pile(self):
                """returns list of cards that will be dragged
together, starting with the current card"""
```

```
                if (
                    self.slot is not None
                    and self.slot != self.solitaire.stock
                    and self.slot != self.solitaire.waste
                ):
                    self.draggable_pile =
self.slot.pile[self.slot.pile.index(self) :]
                else:  # slot == None when the cards are dealt and
need to be place in slot for the first time
                    self.draggable_pile = [self]
```

All done! The full source code for this step can be found here.

Let's move on to the last step of the game itself - detecting the situation when you have won.

## Winning the game

According to wikipedia, some suggest the chances of winning the game as being 1 in 30 games.

Knowing that the chances of winning are quite low, we should plan on showing the user something exciting when that finally happens.

First, let's add a check for the winning condition to `Solitaire` class. If all four foundations contain total of 52 cards, then you have won:

```
    def check_win(self):
        cards_num = 0
        for slot in self.foundations:
            cards_num += len(slot.pile)
        if cards_num == 52:
            return True
        return False
```

We'll be checking if this condition is true each time a card is placed to a foundation:

```
    def place(self, slot):
        """Place draggable pile to the slot"""

        for card in self.draggable_pile:
            if slot in self.solitaire.tableau:
                card.top = slot.top + len(slot.pile) * CARD_OFFSET
            else:
                card.top = slot.top
            card.left = slot.left

            # remove card from it's original slot, if exists
            if card.slot is not None:
                card.slot.pile.remove(card)

            # change card's slot to a new slot
```

```
                card.slot = slot

                # add card to the new slot's pile
                slot.pile.append(card)

            if self.solitaire.check_win():
                self.solitaire.winning_sequence()

            self.solitaire.update()
```

Finally, if the winning condition is met, it will trigger a winning sequence involving position animation:

```
        def winning_sequence(self):
            for slot in self.foundations:
                for card in slot.pile:
                    card.animate_position=1000
                    card.move_on_top()
                    card.top = random.randint(0, SOLITAIRE_HEIGHT)
                    card.left = random.randint(0, SOLITAIRE_WIDTH)
                    self.update()

self.controls.append(ft.AlertDialog(title=ft.Text("Congratulations!
You won!"), open=True))
```

 winning_the_game

Wow! We did it. You can find the full source code for the Solitaire game here.

Now, as we have a decent desktop version of the game, let's deploy it as a web app to share with your friends and colleagues.

# Deploying the app

Congratulations! You have created your Solitaire game app in Python with Flet, and it looks awesome!

Now it's time to share your app with the world!

Follow these instructions to deploy your Flet app as a web app to Fly.io or Replit.

# Summary

In this tutorial, you have learnt how to:

- Create a simple Flet app;
- Drag and drop cards with `GestureDetector`;
- Create your own classes that inherit from Flet controls;
- Design UI layout using absolute positioning of controls in `Stack`;
- Implement implicit animations;
- Deploy your Flet app to the web;

For further reading you can explore controls and examples.

We would love to hear your feedback! Please drop us an email, join the discussion on Discord.

import {CodeExample, Image} from '@site/src/components/crocodocs';

In this tutorial we will show you, step-by-step, how to create a To-Do app in Python using Flet framework and then publish it as a desktop, mobile or web app. The app is a single-file console program of just 164 lines (formatted!) of Python code, yet it is a multi-platform application with rich, responsive UI:



You can see the live demo here.

We chose a To-Do app for the tutorial, because it covers all of the basic concepts you would need to create a Flet app: building a page layout, adding controls, handling events, displaying and editing lists, making reusable UI components, and publishing options.

The tutorial consists of the following steps:

- Getting started with Flet
- Adding page controls and handling events
- View, edit and delete list items
- Filtering list items
- Final touches
- Publishing the app

# Getting started with Flet

To create a multi-platform app in Python with Flet, you don't need to know HTML, CSS or JavaScript, but you do need a basic knowledge of Python and object-oriented programming.

Before you can create your first Flet app, you need to setup your development environment, which requires Python 3.10 or above and `flet` package.

Once you have Flet installed, let's create a simple hello-world app.

Create `hello.py` with the following contents:

<CodeExample path={frontMatter.examples + '/hello.py'} language="python" title="hello.py" />

Run this app and you will see a new window with a greeting:



# Adding page controls and handling events

To start, we'll need a `TextField` for entering a task name, and "+" `FloatingActionButton` with an event handler that will display a `Checkbox` with a new task.

Create `todo.py` with the following contents:

<CodeExample path={frontMatter.examples + '/step_1.py'} language="python" title="todo.py" />

Run the app and you should see a page like this:



## Page layout

Now let's make the app look nice! We want the entire app to be at the top center of the page, taking up 600 px width. The TextField and the "+" button should be aligned horizontally, and take up full app width:



`Row` is a control that is used to lay its children controls out horizontally on a page. `Column` is a control that is used to lay its children controls out vertically on a page.

Replace `todo.py` contents with the following:

<CodeExample path={frontMatter.examples + '/step_2.py'} language="python" title="hello.py" />

Run the app and you should see a page like this:



## Reusable UI components

While we could continue writing our app in the `main` function, the best practice would be to create a reusable UI component. Imagine you are working on an app header, a side menu, or UI that will be a part of a larger project. Even if you can't think of such uses right now, we still recommend creating all your Flet apps with composability and reusability in mind.

To make a reusable To-Do app component, we are going to encapsulate its state and presentation logic in a separate class:

<CodeExample path={frontMatter.examples + '/step_3.py'} language="python" title="todo.py" />

▶ Try this out!

# View, edit and delete list items

In the previous step, we created a basic To-Do app with task items shown as checkboxes. Let's improve the app by adding "Edit" and "Delete" buttons next to a task name. The "Edit" button will switch a task item to edit mode.



Each task item is represented by two rows: `display_view` row with Checkbox, "Edit" and "Delete" buttons and `edit_view` row with TextField and "Save" button. `view` column serves as a container for both `display_view` and `edit_view` rows.

To encapsulate task item views and actions, we introduced a new `Task` class.

Additionally, we changed `TodoApp` class to create and hold `Task` instances when the "Add" button is clicked.

For "Delete" task operation, we implemented `task_delete()` method in `TodoApp` class which accepts task control instance as a parameter.

Then, we passed a reference to `task_delete` method into Task constructor and called it on "Delete" button event handler.

<CodeExample path={frontMatter.examples + '/step_4.py'} language="python" title="todo.py" />

Run the app and try to edit and delete tasks:



# Filtering list items

We already have a functional To-Do app where we can create, edit, and delete tasks. To be even more productive, we want to be able to filter tasks by their status.

Copy the entire code for this step from here. Below we will explain the changes we've done to implement filtering.

`Tabs` control is used to display filter:

```python title="todo.py" # …

class TodoApp(ft.Column): # application's root control is a Column containing all other controls def init(self): self.new_task = ft.TextField(hint_text="Whats needs to be done?", expand=True) self.tasks = ft.Column()

    self.filter = ft.TabBar(
        scrollable=False,
        tabs=[
            ft.Tab(label="all"),
            ft.Tab(label="active"),
            ft.Tab(label="completed"),
        ],
```

```
        )

        self.filter_tabs = ft.Tabs(
            length=3,
            selected_index=0,
            on_change=lambda e: self.update(),
            content=self.filter,
        )

    # ...
```

To display different lists of tasks depending on their statuses, we could maintain
three lists with "All", "Active" and "Completed" tasks. We, however, chose an easier
approach where we maintain the same list and only change a task's visibility depending on its status.

In `TodoApp` class we overrode [`before_update()`](../cookbook/custom-controls.md#before_update) method called every time when
the control is being updated. It iterates through all the tasks and updates their
`visible` property depending on the status of the task:

```python title="todo.py"
class TodoApp(ft.Column):

    # ...

    def before_update(self):
        status = self.filter.tabs[self.filter.selected_index].text
        for task in self.tasks.controls:
            task.visible = (
                status == "all"
                or (status == "active" and task.completed == False)
                or (status == "completed" and task.completed)
            )
```

Filtering should occur when we click on a tab or change a task status.
`TodoApp.before_update()` method is called when Tabs selected value is changed or Task item checkbox is clicked:

```python title="todo.py" class TodoApp(ft.Column):

# ...

def tabs_changed(self, e):
    self.update()

def task_status_change(self, e):
    self.update()
```

```
def add_clicked(self, e):
    task = Task(
        task_name=self.new_task.value,
        on_status_change=self.task_status_change,
        on_delete=self.task_delete,
    )
    self.tasks.controls.append(task)
    self.new_task.value = ""
    self.update()
# ...
```

class Task(ft.Column): task_name: str = “” on_status_change: Callable[[], None] = field(default=lambda: None) on_delete: Callable[[“Task”], None] = field(default=lambda task: None)

```
# ...

def status_changed(self, e):
    self.completed = self.display_task.value
    self.task_status_change()
```

```
Run the app and try filtering tasks by clicking on the tabs:

<Image src="examples/tutorials/todo/media/filtering.gif" width="55%"
/>

## Final touches

Our Todo app is almost complete now. As a final touch, we will add a
footer (`Column` control) displaying the number of incomplete tasks
(`Text` control) and a "Clear completed" button.

<details>
<summary>Full code</summary>

<CodeExample path="tutorials/todo/todo.py" language="python" />
</details>

<Image src="examples/tutorials/todo/media/app-4.png" alt="app-4"
width="55%" />

## Publishing the app

Congratulations! You have created your first Python app with Flet, and
it looks awesome!

Now it's time to share your app with the world!

[Follow these instructions](../publish/index.md) to publish your Flet
app as a mobile, desktop or web app.
```

```
## Summary

In this tutorial, you have learnt how to:

* Create a simple Flet app;
* Work with [Reusable UI components](../cookbook/custom-controls.md);
* Design UI layout using [`Column`](../controls/column.md) and [`Row`]
(../controls/row.md) controls;
* Work with lists: view, edit and delete items, filtering;
* [Publish](../publish/index.md) your Flet app to multiple platforms;

For further reading you can explore [controls](/docs/controls) and
[examples](https://github.com/flet-
dev/flet/tree/main/sdk/python/examples).

---
class_name: "flet.AccelerometerReadingEvent"
title: "AccelerometerReadingEvent"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name={frontMatter.class_name} />

---
title: "Aliases"
---

import {ClassSummary} from '@site/src/components/crocodocs';

<ClassSummary name="flet.AppCallable" showRootHeading={true} />
<ClassSummary name="flet.AnimationValue" showRootHeading={true} />
<ClassSummary name="flet.BadgeValue" showRootHeading={true} />
<ClassSummary name="flet.BlurValue" showRootHeading={true} />
<ClassSummary name="flet.BorderRadiusValue" showRootHeading={true} />
<ClassSummary name="flet.BorderSideStrokeAlignValue" showRootHeading=
{true} />
<ClassSummary name="flet.BoxShadowValue" showRootHeading={true} />
<ClassSummary name="flet.ColorValue" showRootHeading={true} />
<ClassSummary name="flet.context" showRootHeading={true} />
<ClassSummary name="flet.ControlEvent" showRootHeading={true} />
<ClassSummary name="flet.ControlEventHandler" showRootHeading={true}
/>
<ClassSummary name="flet.ControlStateValue" showRootHeading={true} />
<ClassSummary name="flet.DateTimeValue" showRootHeading={true} />
<ClassSummary name="flet.DurationValue" showRootHeading={true} />
<ClassSummary name="flet.EventControlType" showRootHeading={true} />
<ClassSummary name="flet.EventHandler" showRootHeading={true} />
<ClassSummary name="flet.IconData" showRootHeading={true} />
<ClassSummary name="flet.IconDataOrControl" showRootHeading={true} />
```

```
<ClassSummary name="flet.KeyValue" showRootHeading={true} />
<ClassSummary name="flet.MarginValue" showRootHeading={true} />
<ClassSummary name="flet.Number" showRootHeading={true} />
<ClassSummary name="flet.OffsetValue" showRootHeading={true} />
<ClassSummary name="flet.PaddingValue" showRootHeading={true} />
<ClassSummary name="flet.ResponsiveNumber" showRootHeading={true} />
<ClassSummary name="flet.RotateValue" showRootHeading={true} />
<ClassSummary name="flet.ScaleValue" showRootHeading={true} />
<ClassSummary name="flet.StrOrControl" showRootHeading={true} />
<ClassSummary name="flet.TooltipValue" showRootHeading={true} />

---
examples: "controls/core/types/alignment"
example_media: "examples/controls/core/types/alignment/media"
title: "Alignment"
---

import {ClassAll, CodeExample, Image} from
'@site/src/components/crocodocs';

<ClassAll name="flet.Alignment" membersOrder="source" image=
{frontMatter.example_media + '/overview.png'} imageCaption="Alignment
explained" />

## Examples

<CodeExample path={frontMatter.examples + '/container/main.py'}
language="python" />

<Image src={frontMatter.example_media + '/container.png'} width="55%"
/>

---
title: "AndroidBuildVersion"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AndroidBuildVersion" />

---
title: "AndroidDeviceInfo"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AndroidDeviceInfo" />

---
class_name: "flet.AnimatedSwitcherTransition"
examples: "controls/core/types/animated_switcher_transition"
```

```
title: "AnimatedSwitcherTransition"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
class_name: "flet.AnimationCurve"
examples: "controls/core/types/animation_curve"
title: "AnimationCurve"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
title: "Animation"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.Animation" />

---
title: "AnimationStyle"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AnimationStyle" />
```

```
---
title: "AppBarTheme"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AppBarTheme" />

---
title: "AppLifecycleStateChangeEvent"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AppLifecycleStateChangeEvent" />

---
class_name: "flet.AppLifecycleState"
examples: "controls/core/types/app_lifecycle_state"
title: "AppLifecycleState"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
title: "AppView"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AppView" separateSignature={false} />

---
class_name: "flet.Assertiveness"
examples: "controls/core/types/assertiveness"
title: "Assertiveness"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';
```

```
<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
title: "Auth0OAuthProvider"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.Auth0OAuthProvider" />

---
title: "Authorization"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.Authorization" />

---
title: "AuthorizationService"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.AuthorizationService" />

---
title: "AzureOAuthProvider"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.providers.AzureOAuthProvider" />

---
title: "GitHubOAuthProvider"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.providers.GitHubOAuthProvider" />
```

```
---
title: "GoogleOAuthProvider"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.providers.GoogleOAuthProvider" />

---
title: "Group"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.Group" />

---
title: "OAuthProvider"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.OAuthProvider" />

---
title: "OAuthToken"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.OAuthToken" />

---
title: "User"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.auth.User" />

---
title: "AutoCompleteSelectEvent"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AutoCompleteSelectEvent" />

---
title: "AutoCompleteSuggestion"
---
```

```
import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AutoCompleteSuggestion" />

---
title: "AutoFillGroupDisposeAction"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AutofillGroupDisposeAction" separateSignature=
{false} />

---
title: "AutoFillHint"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AutofillHint" separateSignature={false} />

---
title: "AutomaticNotchShape"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.AutomaticNotchShape" />

---
class_name: "flet.Axis"
examples: "controls/core/types/axis"
title: "Axis"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
class_name: "flet.Badge"
```

```
examples: "controls/material/badge"
example_images: "test-
images/examples/controls/material/golden/macos/badge"
title: "Badge"
---

import {ClassMembers, ClassSummary, CodeExample, Image} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} image=
{frontMatter.example_images + '/image_for_docs.png'}
imageCaption="Badge" imageWidth="10%"/>

## Examples

<CodeExample path={frontMatter.examples + '/badge/main.py'}
language="python" />

<Image src={frontMatter.example_images + '/basic.png'} width="40%" />

<ClassMembers name={frontMatter.class_name} />

---
title: "BadgeTheme"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.BadgeTheme" />

---
title: "BannerTheme"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.BannerTheme" />

---
class_name: "flet.BarometerReadingEvent"
title: "BarometerReadingEvent"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name={frontMatter.class_name} />

---
title: "BatteryStateChangeEvent"
---
```

```
import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.BatteryStateChangeEvent" separateSignature=
{false} />

---
title: "BatteryState"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.BatteryState" separateSignature={false} />

---
title: "BeveledRectangleBorder"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.BeveledRectangleBorder" />

---
class_name: "flet.BlendMode"
examples: "controls/core/types/blend_mode"
title: "BlendMode"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
title: "Blur"
---

import {ClassAll, CodeExample, Image} from
'@site/src/components/crocodocs';

<ClassAll name="flet.Blur" />

## Examples
```

```
### Example 1

<CodeExample path="controls/core/types/blur/container/main.py"
language="python" />

<Image src="examples/controls/core/types/blur/media/container.gif"
alt="container" width="55%" />

---
class_name: "flet.BlurStyle"
examples: "controls/core/types/blur_style"
title: "BlurStyle"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
class_name: "flet.BlurTileMode"
examples: "controls/core/types/blur_tile_mode"
title: "BlurTileMode"
---

import {ClassMembers, ClassSummary, CodeExample} from
'@site/src/components/crocodocs';

<ClassSummary name={frontMatter.class_name} />

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'}
language="python" />

<ClassMembers name={frontMatter.class_name} separateSignature={false}
/>

---
title: "Border"
---

import {ClassAll, CodeExample} from '@site/src/components/crocodocs';
```

```
<ClassAll name="flet.Border" />

## Examples

### Example 1

<CodeExample path="controls/core/types/border/container/main.py"
language="python" />

---
title: "BorderRadius"
---

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.BorderRadius" />

## Usage example

```python
container_1.border_radius= ft.BorderRadius.all(30)
```

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from
‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python”
/>

import {ClassMembers, ClassSummary, CodeExample} from
‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python”
/>

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from
‘@site/src/components/crocodocs’;
```

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

## Example 1

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

# Usage example

You can configure a different shape, background color for a hovered state and configure fallback values for all other states.

To configure style attribute for all Material states set its value to a literal (or class instance). For example, if you set `color` property to a literal the value will be applied to all button states:

```
ButtonStyle(
    color=ft.Colors.WHITE
)
```

To configure style attribute for specific Material states set its value to a dictionary where the key is state name. For example, to configure different background colors for `HOVERED` and `FOCUSED` states and another colors for all other states:

```
ButtonStyle(
    color={
        ft.ControlState.HOVERED: ft.Colors.WHITE,
        ft.ControlState.FOCUSED: ft.Colors.BLUE,
        ft.ControlState.DEFAULT: ft.Colors.BLACK,
    }
)
```

## Various button shapes example



```
import flet as ft

def main(page: ft.Page):
    page.padding = 30
    page.spacing = 30
    page.add(
        ft.FilledButton(
            "Stadium",
            style=ft.ButtonStyle(
                shape=ft.StadiumBorder(),
            ),
        ),
        ft.FilledButton(
            "Rounded rectangle",
            style=ft.ButtonStyle(
                shape=ft.RoundedRectangleBorder(radius=10),
            ),
        ),
        ft.FilledButton(
            "Continuous rectangle",
            style=ft.ButtonStyle(
                shape=ft.ContinuousRectangleBorder(radius=30),
            ),
        ),
        ft.FilledButton(
            "Beveled rectangle",
            style=ft.ButtonStyle(
                shape=ft.BeveledRectangleBorder(radius=10),
            ),
        ),
        ft.FilledButton(
            "Circle",
            style=ft.ButtonStyle(shape=ft.CircleBorder(),
padding=30),
        ),
    )

ft.run(main)
```

## Styled button example

Check the following example:



```
import flet as ft
```

```
        def main(page: ft.Page):

            page.add(
                ft.Button(
                    "Styled button 1",
                    style=ft.ButtonStyle(
                        color={
                            ft.ControlState.HOVERED: ft.Colors.WHITE,
                            ft.ControlState.FOCUSED: ft.Colors.BLUE,
                            ft.ControlState.DEFAULT: ft.Colors.BLACK,
                        },
                        bgcolor={ft.ControlState.FOCUSED:
ft.Colors.PINK_200, "": ft.Colors.YELLOW},
                        padding={ft.ControlState.HOVERED: 20},
                        overlay_color=ft.Colors.TRANSPARENT,
                        elevation={"pressed": 0, "": 1},
                        animation_duration=500,
                        side={
                            ft.ControlState.DEFAULT: ft.BorderSide(1,
ft.Colors.BLUE),
                            ft.ControlState.HOVERED: ft.BorderSide(2,
ft.Colors.BLUE),
                        },
                        shape={
                            ft.ControlState.HOVERED:
ft.RoundedRectangleBorder(radius=20),
                            ft.ControlState.DEFAULT:
ft.RoundedRectangleBorder(radius=2),
                        },
                    ),
                )
            )

        ft.run(main)
```

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

<ClassAll name=“flet.ColorScheme” extra={{ show_class_docstring: true, show_children: true, events: [] }} />

import {ClassAll} from ‘@site/src/components/crocodocs’;

<ClassAll name=“flet.Colors” separateSignature={false} membersOrder=“source” groupByCategory={false} summary={{ attributes: false, functions: false }} />

import {ClassAll} from ‘@site/src/components/crocodocs’;

# @component

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from '@site/src/components/crocodocs';

# @control

import {ClassAll} from '@site/src/components/crocodocs';

## Usage Example

Configuring `fill_color` can be done in two ways:

### 1. Single Value for All States

If you want to use the same color across all Material states, simply assign `fill_color` to a single color:

```
ft.Radio(fill_color=ft.Colors.GREEN)
```

### 2. Specific Values for Each State

For more control, you can provide a dictionary where the key is the state name and the value is the corresponding color.

#### Ordering Matters

- The order in which states appear in the dictionary will determine their priority, allowing for flexibility and customization.
- The position of `DEFAULT` does not affect behavior—it always has the least priority and can be placed anywhere in the dictionary.

#### Example

To configure different fill colors of a `Radio` button for `ControlState.HOVERED` and `ControlState.FOCUSED`, with a fallback color for all other states:

```
ft.Radio(
    fill_color={
        ft.ControlState.HOVERED: ft.Colors.GREEN,
        ft.ControlState.FOCUSED: ft.Colors.RED,
        ft.ControlState.DEFAULT: ft.Colors.BLACK,
    }
)
```

This setup ensures that `HOVERED` and `FOCUSED` states take precedence, while all unspecified states default to `BLACK`.

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

<ClassAll name=“flet.CupertinoColors” separateSignature={false} membersOrder=“source” groupByCategory={false} summary={{ attributes: false, functions: false }} />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

<ClassAll name=“flet.CupertinoIcons” separateSignature={false} membersOrder=“source” groupByCategory={false} summary={{ attributes: false, functions: false }} />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary} from ‘@site/src/components/crocodocs’;

# Examples

## Mobile device orientation configuration

See the Page mobile device orientation example, which demonstrates how to use `DeviceOrientation` with `Page.set_allowed_device_orientations()`.

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll, CodeExample} from '@site/src/components/crocodocs';

### Examples

### Containers with gradients

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

<ClassAll name="flet.Icons" separateSignature={false} membersOrder="source" groupByCategory={false} summary={{ attributes: false, functions: false }} />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

# Types

Explore the shared building blocks that power Flet apps: aliases, base controls, reusable classes, enums, events, and exceptions.

- Aliases - shorthand names and conveniences for common patterns.
- Base Controls - foundational control types to extend and compose.
- Classes - core data structures (layout, animation, theming, platform info, and more).
- Decorators - helpers for defining components, controls, and observables.
- Enums - constant sets that define options for controls and services.
- Events - callback payloads emitted by controls and services.
- Exceptions - Flet-specific error types for diagnostics and handling.
- Methods - utilities for lifecycle, hooks, and component state.

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

## Predefined filters

## Usage example

```
        ft.CupertinoTextField(
            placeholder_text="Only numbers are allowed",
            input_filter=ft.InputFilter(allow=True,
regex_string=r"^[0-9]*$", replacement_string="")
        )

        ft.TextField(
            label="Only letters are allowed",
            input_filter=ft.TextOnlyInputFilter()
        )
```

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll, CodeExample, Image} from '@site/src/components/crocodocs';

# Examples

## Container with linear gradient


container

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

# Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

## Example 1

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

# @observable

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

## Example 1


container

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

## Canvas paint

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

## Canvas paint




import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

## Canvas paint




import {ClassAll, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

## Canvas paint




import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

## Container with radial gradient




import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

### Container with sweep gradient


container

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

# Examples

## Example 1

```
        page.theme = ft.Theme(
            tabs_theme=ft.TabBarTheme(
                divider_color=ft.Colors.BLUE,
                indicator_color=ft.Colors.RED,
                indicator_tab_size=True,
                label_color=ft.Colors.GREEN,
                unselected_label_color=ft.Colors.AMBER,
                overlay_color={
                    ft.MaterialState.FOCUSED:
ft.Colors.with_opacity(0.2, ft.Colors.GREEN),
                    ft.MaterialState.DEFAULT:
ft.Colors.with_opacity(0.2, ft.Colors.PINK),
                },
            )
        )
```

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

## Usage Example

The enum is a flag, so multiple decorations can be combined together as follows:

```
        style = ft.TextStyle(decoration=ft.TextDecoration.UNDERLINE |
ft.TextDecoration.OVERLINE)
```

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

## Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassMembers, ClassSummary, CodeExample} from ‘@site/src/components/crocodocs’;

# Examples

<CodeExample path={frontMatter.examples + ‘/showcase/main.py’} language=“python” />

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll, CodeExample, Image} from ‘@site/src/components/crocodocs’;

# Examples

### Tooltip with decoration


with-decoration

import {ClassAll} from ‘@site/src/components/crocodocs’;

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

# @value

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

## Usage Example

You can explicitly set what renderer to use when running a Flet program in web mode using the `web_renderer` parameter of the `ft.run`.

```
import flet as ft

def main(page: ft.Page):
    ...

ft.run(main, view=ft.AppView.WEB_BROWSER, web_renderer=ft.WebRenderer.CANVAS_KIT)
```

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassMembers, ClassSummary, CodeExample} from '@site/src/components/crocodocs';

## Examples

<CodeExample path={frontMatter.examples + '/showcase/main.py'} language="python" />

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

import {ClassAll} from '@site/src/components/crocodocs';

# Breaking changes and deprecations

Breaking changes and deprecations are release changes that can require code, dependency, or configuration updates when you upgrade. Removed APIs are listed here as breaking changes. Deprecated APIs still work for now, but they are scheduled for removal.

These breaking change and deprecation guides are accurate as of the release that introduced the change. Later releases might add new APIs or additional migration paths.

This page lists the guides created for each release.

# By release

The following guides are available. They're sorted by release, with the most recent release first. Each guide explains the change, the reason for it, and how to migrate your code.

## Released in Flet 0.86.0

### Breaking changes

- App files ship unpacked in a read-only bundle; storage dirs reworked
- Android: site-packages ship zipped; some packages need `extract_packages`
- Android: `x86` removed from target architectures
- Default bundled Python version is now 3.14
- App and packages are compiled to `.pyc` by default
- `flet.version.pyodide_version` and `PYODIDE_VERSION` removed
- Flet protocol framing upgraded for DataChannel support

### Deprecations

- `flet build --clear-cache` flag deprecated

## Released in Flet 0.85.0

### Breaking changes

- Deprecated spacing and border helper functions removed

### Deprecations

- `DragTargetEvent` coordinate fields deprecated
- `Video` control APIs deprecated

# `DragTargetEvent` coordinate fields deprecated

This guide is accurate as of Flet 0.85.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

Flet 0.85.0 deprecated [`DragTargetEvent.x`][flet.DragTargetEvent.x], [`DragTargetEvent.y`][flet.DragTargetEvent.y], and [`DragTargetEvent.offset`][flet.DragTargetEvent.offset].

Use [`DragTargetEvent.local_position`][flet.DragTargetEvent.local_position] for target-relative coordinates, or [`DragTargetEvent.global_position`][flet.DragTargetEvent.global_position] for page-level coordinates.

## Context

The old `x`, `y`, and `offset` fields did not make their coordinate space explicit. The new position fields separate target-relative coordinates from global coordinates, which makes drag and drop code easier to read and less ambiguous.

## Migration guide

Code before migration:

```
def on_accept(event: ft.DragTargetEvent):
    target_x = event.x
    target_y = event.y
    target_position = event.offset
```

Code after migration:

```
def on_accept(event: ft.DragTargetEvent):
    target_x = event.local_position.x
    target_y = event.local_position.y
    target_position = event.local_position
```

If your code needs page-level coordinates instead, use `event.global_position.x`, `event.global_position.y`, or `event.global_position`.

## Timeline

- Deprecated in: `0.85.0`
- Removal in: `0.88.0`

## References

- API documentation: [`DragTargetEvent`][flet.DragTargetEvent]
- Issues and PRs: #6387, #6401
- Release notes: Flet 0.85.0

# `Video` control APIs deprecated

This guide is accurate as of Flet 0.85.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

Flet 0.85.0 deprecated [`Video.show_controls`][flet_video.Video.show_controls], [`Video.playlist_add()`][flet_video.Video.playlist_add], and [`Video.playlist_remove()`][flet_video.Video.playlist_remove].

Use [`Video.controls`][flet_video.Video.controls] to configure or hide video controls. Mutate [`Video.playlist`][flet_video.Video.playlist] directly with standard list methods.

## Context

`Video.controls` provides a single configuration point for built-in and custom video controls. It also supports the old hide-controls behavior by setting the value to `None`.

The playlist is now the source of truth for video items. Mutating the list directly keeps the API aligned with other list-backed Flet controls.

## Migration guide

### Hiding controls

Code before migration:

```
video = ft.Video(
    playlist=[media],
    show_controls=False,
)
```

Code after migration:

```
video = ft.Video(
    playlist=[media],
```

```
        controls=None,
    )
```

### Updating the playlist

Code before migration:

```
video.playlist_add(media)
video.playlist_remove(media_index)
```

Code after migration:

```
video.playlist.append(media)
video.playlist.pop(media_index)
video.update()
```

Call `video.update()` when the mutation happens after the control has already been added to the page.

## Timeline

- Deprecated in: `0.85.0`
- Removal in: `0.88.0`

## References

- API documentation: [`Video`][flet_video.Video]
- Issues and PRs: PR #6463
- Release notes: Flet 0.85.0

# Deprecated spacing and border helper functions removed

This guide is accurate as of Flet 0.85.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

Flet 0.85.0 removed deprecated module-level helper functions from `ft.margin`, `ft.padding`, `ft.border`, and `ft.border_radius`. Replace module-level helper calls with class methods:

| Removed helper | Replacement |
|---|---|
| `margin.all(...)` | `Margin.all(...)` |
| `margin.symmetric(...)` | `Margin.symmetric(...)` |
| `margin.only(...)` | `Margin.only(...)` |
| `padding.all(...)` | `Padding.all(...)` |
| `padding.symmetric(...)` | `Padding.symmetric(...)` |
| `padding.only(...)` | `Padding.only(...)` |
| `border.all(...)` | `Border.all(...)` |
| `border.symmetric(...)` | `Border.symmetric(...)` |
| `border.only(...)` | `Border.only(...)` |
| `border_radius.all(...)` | `BorderRadius.all(...)` |
| `border_radius.horizontal(...)` | `BorderRadius.horizontal(...)` |
| `border_radius.vertical(...)` | `BorderRadius.vertical(...)` |
| `border_radius.only(...)` | `BorderRadius.only(...)` |

# Background

The helper functions were replaced by class methods to keep the public API consistent with the concrete value types they create. For example, `Padding.all(...)` now clearly returns a [`Padding`][flet.Padding] value, and `BorderRadius.only(...)` returns a [`BorderRadius`][flet.BorderRadius] value.

# Migration guide

Code before migration:

```
import flet as ft

card = ft.Container(
    padding=ft.padding.all(12),
    border=ft.border.all(1, ft.Colors.OUTLINE),
    border_radius=ft.border_radius.all(8),
)
```

Code after migration:

```
import flet as ft

card = ft.Container(
    padding=ft.Padding.all(12),
    border=ft.Border.all(1, ft.Colors.OUTLINE),
    border_radius=ft.BorderRadius.all(8),
)
```

## Timeline

- Removed in: `0.85.0`

## References

- API documentation: [`Border`][flet.Border], [`BorderRadius`][flet.BorderRadius], [`Padding`][flet.Padding]
- Issues and PRs: #6425
- Release notes: Flet 0.85.0

# Android: site-packages ship zipped; some packages need `extract_packages`

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

Flet 0.86.0 changes how Python code is packaged into Android apps (`flet build apk` / `aab`):

- **Native extension modules** (`.so` files) are loaded **memory-mapped directly from the APK** — no extraction to disk.
- **Pure Python code** — the standard library and your site-packages — ships in *stored* zip assets (`stdlib.zip`, `sitepackages.zip`) and is imported **in place** via `zipimport`, so it is no longer duplicated per ABI or unpacked on first launch.

This makes APKs significantly smaller and removes the need for `useLegacyPackaging` / `keepDebugSymbols` workarounds. Importing from a zip is transparent to most packages.

Some packages still locate bundled data files through a real filesystem path, usually with `__file__` or `pkg_resources`, instead of the zip-safe `importlib.resources`. Those packages must be listed in **`extract_packages`** so they ship extracted to disk.

This affects **Android only**. On macOS, iOS, Windows, and Linux, site-packages ship unpacked; web (Pyodide) is unchanged.

For the maintained feature documentation, including examples, wildcard behavior, and the current list of known affected packages, see Android packaging: extract packages.

## Symptoms

The build succeeds, but the app crashes or errors on the device when the package is imported or first used. The traceback usually contains a path where `sitepackages.zip` or `stdlib.zip` appears as a directory component, for example:

```
FileNotFoundError: [Errno 2] No such file or directory:
'/data/user/0/<applicationId>/files/.../sitepackages.zip/matplotlib/mpl-data/matplotlibrc'
```

`NotADirectoryError` or `OSError` with a similar `sitepackages.zip/...` path is also a common sign that the package computed a data path from `__file__` and tried to read it as a regular file.

## Migration guide

Add the failing package's import name to your `pyproject.toml`:

```toml
[tool.flet.android]
extract_packages = ["matplotlib", "sklearn"]
```

or pass them on the command line:

```
flet build apk --android-extract-packages matplotlib sklearn
```

An entry is the package's **import name** — its top-level directory under site-packages — not the PyPI distribution name: `sklearn`, not `scikit-learn`; `cv2`, not `opencv-python`.

Flet extracts the listed package directories and everything under them to the app's files directory, so `__file__`-relative reads work again.

### No action needed for

- Apps targeting only desktop, iOS, or web.
- Android apps whose dependencies are all zip-safe (the common case) — the change is then purely a size win.

## Timeline

- Changed in: `0.86.0`

## References

- Android packaging: extract packages
- `flet build` CLI reference
- Release notes: Flet 0.86.0

# Android: x86 removed from target architectures

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

The 32-bit `x86` ABI is no longer a valid Android target architecture. Starting with Flet 0.86.0, `flet build apk` / `aab` validates the requested architectures upfront, and a build that includes `x86` — via `--arch`, `[tool.flet.android].target_arch`, or `[tool.flet].target_arch` — fails immediately with:

```
Invalid Android architecture(s): x86.
Supported: armeabi-v7a, arm64-v8a, x86_64.
```

In earlier Flet versions `x86` was listed as a supported value and accepted without error — but it never produced a working app for x86 devices: Flutter has no x86 release (AOT) support and has since removed the target entirely, and the bundled CPython distributions were never built for x86. The value was silently passed through and effectively ignored.

## Background

Flet 0.86.0 reworked the Android build pipeline (see the `extract_packages` guide): requested ABIs are now explicitly mapped to Flutter `--target-platform` values and to bundled Python distributions, so an ABI that neither Flutter nor Python supports is rejected at the start of the build instead of silently doing nothing. `x86` has also been removed from the build templates and documentation throughout.

## Migration guide

Remove `x86` from your architecture lists:

```toml
[tool.flet.android]
target_arch = ["arm64-v8a", "armeabi-v7a", "x86_64"]  # drop "x86"
```

or from the command line:

```
flet build apk --arch arm64-v8a x86_64
```

Nothing is lost by dropping it — previous builds never shipped a working x86 app. If you were targeting x86 for Android *emulators*, use an `x86_64` (or `arm64-v8a`) emulator system image instead; x86 images are only published for long-obsolete API levels.

All three supported ABIs — `arm64-v8a`, `x86_64`, and `armeabi-v7a` — are available for every bundled Python version; see supported target architectures.

## Timeline

- Changed in: `0.86.0`

## References

- Android: supported target architectures
- Flutter Android deployment docs
- Release notes: Flet 0.86.0

# App files ship unpacked in a read-only bundle; storage dirs reworked

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

Flet 0.86.0 (with `serious_python` 4.0.0) changes how a packaged app's Python files are shipped and where the app reads and writes data:

- **App files ship unpacked inside the app bundle**, next to the Python standard library and site-packages, on macOS / iOS / Windows / Linux. There is no first-launch `app.zip` extraction. On **Android** the app ships as a *stored* `app.zip` asset inside the APK and is unpacked once (version-keyed) to the app's files directory on the first launch after an install/update. Web (Pyodide) is unchanged.
- The app directory is now **read-only** (signed `.app`, iOS bundle, `Program Files`, the APK).
- The Python program's **current working directory** moved from the app directory to a writable, app-private data directory, exposed as `FLET_APP_STORAGE_DATA`.
- `FLET_APP_STORAGE_DATA` now maps to the OS *application support* directory (a `data` subdir), **not** the user's Documents folder.
- The storage env vars are now a clean three-tier set: `FLET_APP_STORAGE_DATA` (durable), the new **`FLET_APP_STORAGE_CACHE`** (regenerable), and `FLET_APP_STORAGE_TEMP` (now the OS temp dir, previously the cache dir).
- `flet run` (dev mode) now sets the working directory to a hidden, git-ignored `<project>/.flet/storage/data`, mirroring a built app.

# Background

Previously your Python sources were zipped into `app/app.zip`, shipped as a Flutter asset, and extracted at runtime on first launch — while the stdlib and site-packages already shipped unpacked inside the bundle. Lifting app-file handling into `serious_python` removes the first-launch unzip and the hash bookkeeping, and puts your code in the same place as the stdlib. Because that place is read-only, the working directory has to move somewhere writable — and the natural home for app data is the OS *application support* directory, which is app-private on every platform (no Documents clutter, no manual scoping).

# Migration guide

## Write data to the storage dirs, not next to your code

The app directory is read-only. Relative writes resolve against the **current working directory**, which is now `FLET_APP_STORAGE_DATA` (a writable, durable, app-private dir). `open("app.db", "w")` and `sqlite3.connect("app.db")` keep working — the file just lands in the data dir instead of next to your `.py` files, and now persists across app updates.

## Read shipped files via `__file__`, not relative paths

Relative **reads** of files you bundle with your app (e.g. `open("seed.json")`) no longer resolve against your app folder. Read bundled data relative to your module instead:

```
from pathlib import Path

SEED = Path(__file__).parent / "seed.json"
data = SEED.read_text()
```

or use `importlib.resources`, or place the file under your Flet `assets/` directory and resolve it via `FLET_ASSETS_DIR`.

## Storage environment variables

| Variable | Meaning | Lifetime |
|---|---|---|
| `FLET_APP_STORAGE_DATA` | Durable app data (= the working directory) | Never auto-deleted; backed up |
| `FLET_APP_STORAGE_CACHE` | Regenerable cache (**new**) | OS may purge under storage pressure |
| `FLET_APP_STORAGE_TEMP` | OS temp / scratch (**changed** — was the cache dir) | Most volatile; may vanish between launches |

If you relied on `FLET_APP_STORAGE_TEMP` for data that should survive (it used to be the cache dir), switch to `FLET_APP_STORAGE_CACHE` (or `FLET_APP_STORAGE_DATA` for must-keep data).

## Physical locations

`FLET_APP_STORAGE_DATA` (= the working directory):

| Platform | Location |
|---|---|
| Windows | `%APPDATA%\<company>\<product>\data` (e.g. `C:\Users\<User>\AppData\Roaming\<company>\<product>\data`) |
| macOS | `~/Library/Application Support/<bundle-id>/data` (sandboxed apps: redirected into the app container) |
| Linux | `${XDG_DATA_HOME:-~/.local/share}/<app-id>/data` |
| iOS | `<app-sandbox>/Library/Application Support/<bundle-id>/data` |
| Android | `/data/data/<package>/files/data` |
| `flet run` (dev) | `<project>/.flet/storage/data` |

`FLET_APP_STORAGE_CACHE` and `FLET_APP_STORAGE_TEMP`:

| Platform | `FLET_APP_STORAGE_CACHE` | `FLET_APP_STORAGE_TEMP` |
|---|---|---|
| Windows | `%LOCALAPPDATA%\<company>\<product>` | `%TEMP%` (…\AppData\Local\Tem |
| macOS | `~/Library/Caches/<bundle-id>` | `$TMPDIR` (e.g. `/var/folders/.../T/`) |
| Linux | `${XDG_CACHE_HOME:-~/.cache}/<app-id>` | system temp (`$TMPDIR`, fallback `/tmp`) |
| iOS | `<app-sandbox>/Library/Caches/<bundle-id>` | `<app-sandbox>/tmp` |
| Android | `getCacheDir()` (`/data/data/<package>/cache`) | same as `CACHE` (`getCacheDir()`) |
| `flet run` (dev) | `<project>/.flet/storage/cache` | `<project>/.flet/storage/te` |

## `flet run` dev storage

`flet run` now creates a hidden `<project>/.flet/` directory (with `storage/{data,cache,temp}`) and runs your app with the working directory set to `.flet/storage/data`, so dev behaves like a built app (including the read-shipped-files change above). The directory writes its own `.gitignore` and is ignored by Git automatically; the old visible `storage/` directory is no longer used.

## Timeline

- Changed in: `0.86.0`

## References

- Environment variables: `FLET_APP_STORAGE_DATA`, `FLET_APP_STORAGE_CACHE`, `FLET_APP_STORAGE_TEMP`
- Cookbook: Read and write files
- Release notes: Flet 0.86.0

# App and packages are compiled to `.pyc` by default

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

`flet build` now **compile your app and the installed packages to `.pyc` by default** (via `python -m compileall -b`, with the original `.py` files removed from the bundle). Previously compilation was off unless you opted in with `--compile-app` / `--compile-packages` or `[tool.flet.compile]` in `pyproject.toml`.

Shipping bytecode removes per-launch recompilation. The win is largest on **mobile**, where pure Python is imported from a stored, in-place zip (`zipimport`) that cannot cache bytecode back to disk — so without precompiled `.pyc`, every module is recompiled from source on **every** cold start. On a mid-range Android device this costs roughly 1–2 seconds of startup, scaling with the number of modules imported.

## Background

Compilation has always been available behind the `--compile-app` / `--compile-packages` flags; it just defaulted off. With the move to in-place `zipimport` packaging on Android, leaving it off meant a recurring cold-start tax on every launch, so the default was flipped on.

Compiled web builds were verified to load correctly in Pyodide: the standalone CPython used to compile and the Pyodide runtime share the same minor version (e.g. 3.14), and CPython keeps the `.pyc` magic number stable across patch releases of a minor version, so the bytecode is accepted.

## Migration guide

Most apps need no changes — builds simply start faster.

If you relied on uncompiled builds (to keep `.py` source in the bundle for debugging, to let a build complete despite syntax errors, or to speed up iterative builds), opt out explicitly.

### Opt out via the CLI flag

The flags are now `argparse.BooleanOptionalAction`, so each gets a `--no-` form (the original `--compile-app` / `--compile-packages` still work):

```
flet build apk --no-compile-app --no-compile-packages
```

### Opt out in `pyproject.toml`

```
[tool.flet.compile]
app = false
packages = false
```

Per-platform overrides also work, e.g. `[tool.flet.android.compile]`. The resolution order is CLI flag → `[tool.flet.<platform>.compile]` → `[tool.flet.compile]` → default (`true`).

## Timeline

- Changed in: `0.86.0`

## References

- API documentation: Compilation and cleanup
- Issues and PRs: #6598
- Release notes: Flet 0.86.0

# Flet protocol framing upgraded for DataChannel support

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

# Summary

Flet 0.86.0 introduces dedicated **data channels** for widgets that need to move bulk binary data (image frames, audio buffers, ML tensors) between Dart and Python without going through the MsgPack control protocol.

Enabling this required a one-time upgrade of the Flet protocol's wire format:

- **Stream-oriented transports** (`flet run` dev mode over UDS / TCP) now use **4-byte little-endian length-prefixed framing**. The previous streaming `msgpack.Unpacker.feed` decode is gone.
- **Every transport** (sockets, WebSocket, `dart_bridge` FFI, Pyodide `postMessage`) now puts a **1-byte type discriminator** at the head of each packet:
  - `0x00` — MsgPack-encoded Flet control frame (widget state, events) — same contents as before, just with a type byte in front.
  - `0x01` — raw DataChannel frame (`[channel_id:u32 LE][payload]`).

The new format is **not backwards-compatible**: a Flet 0.85 client cannot talk to a Flet 0.86 server, and vice versa. The `flet-cli` dev server and the in-process Python runtime were upgraded together in this release.

# Background

Flet's previous wire format on UDS / TCP relied on `msgpack.Unpacker.feed` to re-assemble messages from arbitrary byte-stream chunks. That works for MsgPack-only traffic but doesn't compose with a second logical stream of raw bytes — you can't tell a partial MsgPack value from the start of a raw frame without an out-of-band marker.

Adding the 1-byte type byte + 4-byte length prefix solves this in the smallest possible way:

- Receivers read `[length][type][...]`, fan out by type byte, no streaming decoder state to maintain.
- Message-oriented transports (WebSocket, `postMessage`, `dart_bridge`) drop the length prefix since each message is already one packet; they keep the type byte.
- Per-frame overhead is 5 bytes (message-oriented) or 9 bytes (stream-oriented) — under 1% at any payload size that motivates a data channel.

# Migration guide

## Most users — nothing to change

If you only use `flet build` artifacts or run `flet run` with the matching `flet` package version (the default install pulls both at once), there's nothing to do. The CLI and the runtime are version-locked.

### Users running `flet-cli` and the `flet` Python package from different installs

Make sure both come from the same release. The common gotcha is a global `flet` install plus a project-local `flet` in `.venv` at different versions — upgrade or pin both to ≥ `0.86.0` (or both to ≤ `0.85.x`).

A version mismatch will surface as a connection failure during `flet run` with a parse error in the dev-server log.

### Users with custom backends or sidecars speaking the Flet protocol

The wire format changed. Update your encoder/decoder:

- **Inbound on a stream transport** (UDS / TCP): read 4 bytes (length, u32 LE), read that many bytes, the first byte is the type discriminator, the rest is the payload.
- **Inbound on a message transport** (WebSocket, `postMessage`): each message's first byte is the type discriminator, the rest is the payload.
- **Outbound**: same shape, mirrored.

Treat `0x01` (raw DataChannel) frames as opaque if you don't use data channels — forward them along or drop them, never feed them to your MsgPack decoder.

### Code that depended on `StreamingMsgpackDeserializer`

The Dart-side class `package:flet/src/transport/streaming_msgpack_deserializer.dart` is removed. There are no public consumers — it was an internal helper for stream-transport framing that's no longer needed now that every packet is length-delimited. If you imported it (you shouldn't have), decode each inbound buffer with one-shot `msgpack.deserialize(bytes)` instead.

### Custom widgets that override `MatplotlibChartCanvas`

`MatplotlibChartCanvas` now transports its `apply_full` / `apply_diff` / `clear` frames through a [DataChannel] rather than `_invoke_method` arguments. The Python-facing method signatures are unchanged (`apply_full(image_bytes: bytes)`, etc.), and they remain the documented API. But if you subclassed the canvas and overrode the Dart-side `_invokeMethod` handler, that override no longer fires — the Dart side now consumes a 1-byte opcode + PNG payload from the channel directly.

## Timeline

- Changed in: `0.86.0`

## References

- API: `flet.DataChannel` (Python) and `FletBackend.openDataChannel` (Dart) — see the module docstrings in `flet/data_channel.py`; dedicated reference pages

will be added in a follow-up.
- Release notes: Flet 0.86.0

# Default bundled Python version is now 3.14

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

Flet 0.86.0 introduces multi-version bundled CPython support to `flet build` and `flet publish`. The bundled Python interpreter is now selected per build (see Choosing a Python version) and the **default is the latest supported stable** — currently CPython 3.14. Earlier Flet releases implicitly bundled CPython 3.12 via the single-version `serious_python` 1.x.

Apps that depend on Python packages whose pre-built wheels aren't yet available for 3.14 (typically packages with C/Rust extensions that haven't caught up to the new ABI) need to pin a previous Python version.

## Background

Multi-version Python support landed in #6577 and is tracked by the central registry in `flet_cli/utils/python_versions.py`. The supported short versions are `3.12`, `3.13`, and `3.14` (each pinned to a specific CPython patch + Pyodide release on the registry side).

The default flips to the latest stable so new projects get the most recent Python by default. Existing projects without a pin start picking it up too on their next build.

## Migration guide

You can pin a different Python version in three equivalent ways. Use whichever fits your project layout.

### Pin in `pyproject.toml` (recommended)

Add or update `[project].requires-python`:

```toml
[project]
requires-python = ">=3.12,<3.13"
```

`flet build` parses the specifier and picks the **highest supported short version** that satisfies it. So `>=3.12,<3.13` resolves to 3.12, `>=3.13,<3.14` resolves to 3.13, and `>=3.14` resolves to 3.14 (the default).

### Pin via the CLI flag

Pass `--python-version` on every invocation:

```
flet build apk --python-version 3.12
flet publish --python-version 3.12
```

The CLI flag overrides anything in `pyproject.toml`.

### Pin via environment variable

Export `SERIOUS_PYTHON_VERSION` in the shell that runs `flet build`:

```
export SERIOUS_PYTHON_VERSION=3.12
flet build apk
```

Useful in CI pipelines where you don't want to thread the flag through every job.

## Timeline

- Changed in: `0.86.0`

## References

- API documentation: Choosing a Python version
- Issues and PRs: #6577
- Release notes: Flet 0.86.0

# `flet build --clear-cache` flag deprecated

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

## Summary

Flet 0.86.0 deprecated the `--clear-cache` flag of `flet build` and `flet debug`.

Use the new `flet clean` command instead.

## Context

`--clear-cache` only deleted the Flutter bootstrap project (`build/flutter`), and only when the build template hash had changed — so it was a no-op in the common case and never reset the rest of the `build` directory.

The new `flet clean` command deletes the entire `build` directory (the Flutter bootstrap project, cached artifacts, hash stamps, and generated output) unconditionally, giving a predictable, full reset before the next build.

## Migration guide

Before:

```
flet build web --clear-cache
```

After:

```
flet clean
flet build web
```

`flet clean` accepts an optional path to the app directory (defaults to the current directory), so you can target another project with `flet clean path/to/app`.

## Timeline

- Deprecated in: `0.86.0`
- Removal in: `0.89.0`

## References

- CLI documentation: `flet clean`, `flet build`
- Issues and PRs: #6233
- Release notes: Flet 0.86.0

# `flet.version.pyodide_version` and `PYODIDE_VERSION` removed

This guide is accurate as of Flet 0.86.0. Later releases might add new APIs or additional migration paths.

The breaking changes and deprecations index lists the guides created for each release.

# Summary

Flet 0.86.0 removed the module-level `flet.version.pyodide_version` attribute and the matching `PYODIDE_VERSION` constant. The single fixed-string export no longer makes sense now that Pyodide is selected **per build** as part of the multi-version bundled Python support — the bundled release depends on which Python version your app pins, not a global default.

# Background

Earlier Flet releases used a single Pyodide pin in `flet.version` to drive the `flet --version` output and let tooling read which Pyodide release `flet build web` would bundle. With multi-version Python support the mapping is now Python-version-aware (`3.12 → 0.27.7`, `3.13 → 0.29.4`, `3.14 → 314.0.0`). It is loaded on demand from python-build's date-keyed `manifest.json` — the single source of truth shared with serious_python — via `flet_cli/utils/python_versions.py`, so it is no longer hand-maintained.

# Migration guide

If you imported the constant somewhere — typically from `flet.version` — switch to looking the release up from the new registry.

Code before migration:

```
import flet.version

print(f"Bundled Pyodide: {flet.version.pyodide_version}")
```

Code after migration:

```
from flet_cli.utils.python_versions import
get_supported_python_versions

for release in get_supported_python_versions():
    print(f"  {release.short} → Pyodide {release.pyodide}")
```

If you only need the version that `flet build` is about to bundle, resolve the Python release the same way the CLI does:

```
from flet_cli.utils.python_versions import (
    get_default_python_version,
    get_release,
)

release = get_release(get_default_python_version())
print(f"Default Pyodide: {release.pyodide}")
```

## Timeline

- Removed in: `0.86.0`

## References

- API documentation: Choosing a Python version
- Issues and PRs: #6577
- Release notes: Flet 0.86.0

# Compatibility policy

Flet tries to balance API stability with the need to keep improving the framework, fixing bugs, and simplifying APIs before and after Flet 1.0.

## Breaking changes

A breaking change is a change that can require users to update app code, configuration, dependencies, or build settings when upgrading Flet.

Examples include:

- removing a public API;
- changing a default behavior in a way that affects existing apps;
- changing event payloads or method return values;
- changing minimum supported Python, Flutter, platform, or dependency versions;
- changing build, packaging, or publishing requirements.

When a breaking change needs migration guidance, it should be listed in Breaking changes and deprecations and linked from Release notes.

## Deprecation policy

When possible, Flet deprecates APIs before removing them. Deprecated APIs remain available during the deprecation period, emit runtime warnings where supported, and appear with deprecation labels in API docs.

By default, deprecated APIs are removed after three minor releases. For example, an API deprecated in `0.85.0` is normally scheduled for removal in `0.88.0`.

Exceptions can happen when a deprecation needs more migration time or when compatibility would block an important fix. Any exception should be called out in the related release notes or migration guide.

## Where changes are documented

- Release notes list each release and link to announcements, changelogs, and migration guides.
- Breaking changes and deprecations explain changes that require migration, grouped by release.
- API reference pages show deprecation details next to affected APIs.

# Stay up to date

Use this section to track Flet releases, breaking changes, deprecations, and compatibility policy.

- Release notes - release announcements and changelog links.
- Breaking changes and deprecations - migration guides for changes that can require code or configuration updates.
- Compatibility policy - how Flet handles breaking changes and deprecations.

# Release notes

This page links release announcements, changelogs, and migration notes for Flet releases.

## Stable releases

### 0.86.x

- 0.86.0: Announcement, Changelog, Breaking changes and deprecations

### 0.85.x

- 0.85.3: Changelog
- 0.85.2: Changelog
- 0.85.1: Changelog
- 0.85.0: Announcement, Changelog, Breaking changes and deprecations

### 0.84.x

- 0.84.0: Announcement, Changelog

### 0.83.x

- 0.83.1: Changelog
- 0.83.0: Announcement, Changelog

### 0.82.x

- 0.82.2: Changelog
- 0.82.0: Changelog

### 0.81.x

- 0.81.0: Announcement, Changelog

### 0.80.x

- 0.80.5: Changelog
- 0.80.4: Changelog
- 0.80.3: Changelog
- 0.80.2: Changelog
- 0.80.1: Changelog
- 0.80.0: Changelog

### 0.70.x

- 0.70.0: Changelog

### 0.28.x

- 0.28.3: Changelog
- 0.28.2: Changelog
- 0.28.0: Changelog

### 0.27.x

- 0.27.6: Changelog
- 0.27.5: Changelog
- 0.27.4: Changelog
- 0.27.3: Changelog
- 0.27.2: Changelog
- 0.27.1: Changelog
- 0.27.0: Announcement, Changelog

### 0.26.x

- 0.26.0: Announcement, Changelog

### 0.25.x

- 0.25.2: Changelog
- 0.25.1: Changelog
- 0.25.0: Announcement, Changelog

### 0.24.x

- 0.24.1: Changelog
- 0.24.0: Announcement, Changelog

### 0.23.x

- 0.23.2: Changelog
- 0.23.1: Changelog
- 0.23.0: Announcement, Changelog

### 0.22.x

- 0.22.1: Changelog
- 0.22.0: Changelog

### 0.21.x

- 0.21.2: Changelog
- 0.21.1: Changelog
- 0.21.0: Changelog

### 0.20.x

- 0.20.2: Changelog
- 0.20.1: Changelog
- 0.20.0: Changelog

### 0.19.x

- 0.19.0: Changelog

### 0.18.x

- 0.18.0: Changelog

### 0.17.x

- 0.17.0: Changelog

### 0.16.x

- 0.16.0: Changelog

### 0.15.x

- 0.15.0: Changelog

### 0.14.x

- 0.14.0: Changelog

### 0.13.x

- 0.13.0: Changelog

### 0.12.x

- 0.12.2: Changelog
- 0.12.1: Changelog
- 0.12.0: Changelog

### 0.11.x

- 0.11.0: Changelog

### 0.10.x

- 0.10.3: Changelog
- 0.10.2: Changelog
- 0.10.1: Changelog
- 0.10.0: Changelog

### 0.9.x

- 0.9.0: Changelog

### 0.8.x

- 0.8.4: Changelog
- 0.8.3: Changelog
- 0.8.2: Changelog
- 0.8.1: Changelog
- 0.8.0: Changelog

### 0.7.x

- 0.7.4: Changelog
- 0.7.3: Changelog
- 0.7.2: Changelog
- 0.7.1: Changelog
- 0.7.0: Changelog

### 0.6.x

- 0.6.2: Changelog

- 0.6.1: Changelog
- 0.6.0: Changelog

### 0.5.x

- 0.5.2: Changelog
- 0.5.1: Changelog
- 0.5.0: Changelog

### 0.4.x

- 0.4.2: Changelog
- 0.4.1: Changelog
- 0.4.0: Changelog

### 0.3.x

- 0.3.2: Changelog
- 0.3.1: Changelog
- 0.3.0: Changelog

### 0.2.x

- 0.2.4: Changelog
- 0.2.2: Changelog

### 0.1.x

- 0.1.65: Changelog
- 0.1.64: Changelog
- 0.1.63: Changelog
- 0.1.62: Changelog
- 0.1.61: Changelog
- 0.1.60: Changelog
- 0.1.59: Changelog
- 0.1.58: Changelog
- 0.1.57: Changelog
- 0.1.56: Changelog
- 0.1.55: Changelog
- 0.1.54: Changelog
- 0.1.53: Changelog
- 0.1.52: Changelog
- 0.1.51: Changelog
- 0.1.50: Changelog
- 0.1.49: Changelog
- 0.1.48: Changelog
- 0.1.47: Changelog
- 0.1.46: Changelog
- 0.1.43: Changelog
- 0.1.42: Changelog

- 0.1.41: Changelog
- 0.1.40: Changelog
- 0.1.39: Changelog
- 0.1.38: Changelog
- 0.1.37: Changelog
- 0.1.36: Changelog
- 0.1.35: Changelog
- 0.1.34: Changelog
- 0.1.33: Changelog
- 0.1.32: Changelog
- 0.1.31: Changelog
- 0.1.30: Changelog
- 0.1.29: Changelog
- 0.1.28: Changelog
- 0.1.27: Changelog
- 0.1.26: Changelog
- 0.1.25: Changelog
- 0.1.24: Changelog
- 0.1.23: Changelog
- 0.1.22: Changelog
- 0.1.20: Changelog
- 0.1.19: Changelog
- 0.1.18: Changelog
- 0.1.17: Changelog
- 0.1.16: Changelog
- 0.1.14: Changelog
- 0.1.13: Changelog
- 0.1.12: Changelog
- 0.1.11: Changelog
- 0.1.10: Changelog
- 0.1.7: Changelog
- 0.1.5: Changelog
- 0.1.4: Changelog